

Ingénierie des exigences

Notes de cours



Thierry Van den Berghe

Table des matières

Table des matières	ii
Avant-propos.....	5
Quel est l'objectif du cours ?	5
Comment ces notes sont-elles organisées ?	5
Quelles sont les principales sources de ces notes ?	6
1 Introduction à la business analyse	7
1.1 Définition et enjeux	7
1.2 Les missions de la business analyse.....	10
1.3 Le métier de business analyst.....	11
1.4 Zoom sur la résolution de problèmes.....	14
1.5 Le processus de la business analyse	17
1.5.1 Business Context.....	18
1.5.2 BA Planning.....	18
1.5.3 Strategy Analysis	20
1.5.4 Requirement analysis	22
1.5.5 Solution Design.....	22
1.5.6 Solution Evaluation.....	22
1.6 Green IT et business analyse	23
1.7 Exercices	24
2 Ingénierie des exigences	27
2.1 Contexte de l'ingénierie logicielle	27
2.2 Définitions	30
2.3 Complémentarité des disciplines.....	31
2.4 Taxonomie des exigences	33
2.5 Enjeux et défis de l'ingénierie des exigences	36
3 Cycle de gestion des exigences.....	38
3.1 Compréhension du domaine.....	38
3.1.1 Organisation.....	39
3.1.2 Environnement.....	41
3.1.3 Acteurs	42
3.2 Élicitation des exigences.....	44
3.2.1 Bonnes pratiques de l'élicitation des exigences	44
3.2.2 Techniques d'élicitation.....	46
3.2.3 Zoom sur les interviews	50

3.3	Analyse des exigences	53
3.4	Représentation des exigences	58
3.4.1	Qualité d'une exigence	61
3.4.2	Exigences en langage naturel.....	61
3.4.3	Zoom sur les user stories.....	63
3.4.4	Modélisation graphique des exigences	67
3.5	Organisation des exigences	70
3.5.1	SRS (System Requirement Specification)	70
3.5.2	Validation de la qualité	72
3.5.3	Contexte Agile	73
3.5.4	Cahier des charges	74
3.6	Gestion des exigences	75
3.7	Estimation des coûts	77
3.7.1	Modèles d'estimation paramétriques	78
3.7.2	Estimations d'experts.....	86
3.8	Exercices	87
4	Diagramme de cas d'utilisation	91
4.1	Cas d'utilisation.....	92
4.2	Acteur.....	92
4.3	Structuration des cas d'utilisation	94
4.4	Spécification des cas d'utilisation	95
4.5	Démarche de construction	97
4.6	Erreurs fréquentes	99
4.7	Exemple.....	100
4.8	Exercices	104
5	Diagramme d'activité	109
5.1	Action	109
5.2	Arc entre des actions	110
5.3	Nœud	111
5.4	Partition	114
5.5	Données et persistance	116
5.6	Événement, poids de flux et décomposition	118
5.7	Exception et zone interruptible	120
5.8	Zone d'expansion	122
5.9	Erreurs fréquentes	122
5.10	Exemple.....	123

5.11	Exercices	126
6	<i>Réingénierie des processus avec BPM</i>	133
6.1	Processus d'affaires (business process ou processus métier)	133
6.2	Cycle BPM	134
6.3	Exercice	138
7	<i>Modélisation des processus avec BPMN</i>	139
7.1	Activité	140
7.2	Séquencement	141
7.3	Données	142
7.4	Ressources	143
7.5	Compléments sur les événements	143
7.6	Exercices	144
8	<i>Tests</i>	147
8.1	Définition et enjeux	147
8.2	Phasage des tests	149
8.3	Types de tests	150
8.4	Techniques de test	152
9	<i>IA pour l'ingénierie des exigences</i>	154
9.1	Opportunités de l'IA	154
9.2	Risques de l'IA	157
9.3	Démarche de mise en œuvre de l'IA	157
10	<i>Annexes</i>	159
10.1	Exemple de Solution Design Document	159
10.2	Introduction à ArchiMate	163
10.3	Guide d'organisation de réunions (Podeswa)	166
10.4	Exemples de questions relatives à l'éllicitation des exigences non fonctionnelles (Podeswa) ..	176
	<i>Bibliographie</i>	178

Avant-propos

La transformation digitale des entreprises implique que les collaborateurs sont demandeurs de davantage d'automatisation des processus d'affaires. Par conséquent, les systèmes informatiques sont très étendus au sein des organisations, et leur développement ainsi que leur maintenance mobilisent de gros moyens (jusqu'à 6 % du chiffre d'affaires annuel selon le Gartner Group (Gartner, 2010)). Cette informatisation touche l'ensemble des composantes de l'organisation.

Cependant, la transformation digitale n'est pas la même pour tous. Chaque entreprise a ses exigences en termes de métier, de processus d'affaires et de données à gérer. Le système informatique doit intégrer ces exigences et répondre aux besoins particuliers de l'ensemble des collaborateurs. Son élaboration implique donc un transfert de la connaissance métier des managers vers les informaticiens, ce qui est indispensable pour permettre à ces derniers de développer un système adapté. À cet égard, une bonne collaboration entre managers et informaticiens est un facteur clé de succès dans la construction de tout système informatique.

L'ingénierie des exigences adresse justement les problèmes de rédaction d'un ensemble cohérent et réaliste d'exigences qui vont guider le développement d'un système d'information. Le business analyst joue un rôle clé dans la co-construction de ces exigences avec le business, et fait office d'interface entre les utilisateurs et les informaticiens.

Quel est l'objectif du cours ?

Ce cours vise globalement à former les futurs business analysts aux techniques et méthodes de traitement des exigences.

Les objectifs plus spécifiques sont :

- présenter la business analyse et sa place dans l'ingénierie logicielle ;
- maîtriser les mécanismes de gestion des exigences, de la compréhension du domaine d'application à la validation du système ;
- maîtriser les techniques de représentation des exigences ;
- interagir avec les parties prenantes.

Comment ces notes sont-elles organisées ?

La Section 1 introduit le lecteur à la business analyse et au rôle du business analyst.

Les Sections 2 et 3 présentent la discipline de l'ingénierie des exigences et ses principales tâches. Elles discutent de la formulation des exigences en langage naturel, et notamment sous forme de user stories.

Les Sections 4 et 5 sont dédiées à deux langages graphiques pour représenter les exigences : les diagrammes de cas d'utilisation et les diagrammes d'activité.

Les Sections 6 et 7 donnent une brève introduction à l'approche de réingénierie des processus BPM et au langage graphique associé (BPMN) pour représenter les processus. BPM et BPMN ont largement inspiré la business analyse et les diagrammes d'activité. S'ils font figure de pionniers historiques, ils restent très populaires dans les entreprises.

La Section 8 est consacrée aux tests. Elle complète des discussions des sections précédentes sur la validation et la gestion de la qualité.

Quelles sont les principales sources de ces notes ?

(van Lamsweerde, 2009) constitue la principale référence du cours. L'ouvrage propose une démarche très complète pour le travail des exigences.

(International Institute of Business Analysis, (IIBA), 2015), cité souvent comme le BABOK (Business Analysis Body of Knowledge) est un référentiel très populaire dans le monde de la business analyse. Le lecteur y trouvera des nombreuses définitions, démarches et pratiques. Mais l'ouvrage n'a pas une visée éducative et s'adresse davantage aux professionnels qu'aux étudiants. (Milani, 2019) et (Laplante, 2018) traitent de manière assez complète de la business analyse et complètent avantageusement la référence précédente, sur laquelle ils sont basés, mais avec davantage de pédagogie.

(Paul & Cadle, 2020) donne une description complète et pédagogique de la business analyse, toujours selon le BABOK. Une référence incontournable...

(Alexander & Beus-Dukic, 2009) propose des méthodes et des conseils très pratiques, surtout pour rassembler et formuler les exigences à travers des interactions organisées avec les utilisateurs.

(Seidl, Scholz, Huemer, & Kappel, 2015) est support pédagogique rigoureux, synthétique mais complet pour appréhender le langage UML (Unified Modelling Language).

(Dumas, La Rosa, Mendling Hajo, & Reijers, 2013) est une référence incontournable pour la démarche et les notations BPMN. Le livre propose de nombreux exercices et illustrations.

(ISTQB) est édité par l'International Software Testing Qualification Board et prépare à une des certifications de cette même organisation. On y introduit la notion de test logiciel et une démarche générique de test.

1 Introduction à la business analyse

Les TIC (Technologie de l'information et de la Communication) sont omniprésentes dans tous les aspects de notre vie professionnelle ou privée. C'est aujourd'hui un lieu commun. En effet, les organisations de toutes tailles et de tous secteurs sont engagées dans une digitalisation massive de leurs processus d'affaires. Le modèle d'affaires de bon nombre d'entre elles repose principalement sur les TIC, comme l'industrie de la musique en ligne.

Les gains en performance et les bénéfices de la digitalisation des processus d'affaires ne font plus débat : ils sont évidents, même si les technologies sont sources de risques, comme ceux liés à la sécurité ou la protection des données.

Cependant, le potentiel d'apport de valeur des TIC est conditionné à leur alignement avec les besoins des utilisateurs : au départ, les TIC sont transversales, i.e., elles n'ont pas été conçues pour servir une entreprise ou un métier particulier. Il faut donc les exploiter au mieux dans des solutions spécifiques qui vont réellement apporter de la valeur aux utilisateurs en répondant à leurs besoins propres. La business analyse va précisément participer à la mise en place de solutions digitales qui sont alignées sur les besoins.

La discipline de la business analyse, ou analyse métier, et la profession de business analyst sont relativement récentes. Plus exactement, elles existaient sous différentes formes et appellations depuis l'utilisation de l'informatique dans les organisations, mais elles ont été récemment cadrées suite à la prolifération massive des systèmes d'information dans l'entreprise et la reconnaissance de l'impérieuse nécessité d'articuler business et IT.

Origines de la business analyse (Paul & Cadle, 2020). Le développement relativement récent des TIC a permis :

- l'émergence de nouveaux modèles d'affaires et de nouveaux services ;
- une relation directe et individualisée avec le client ;
- l'externalisation de fonctions autres que celles liées au cœur de métier, avec une interopérabilité forte entre l'entreprise et ses partenaires auprès desquels elle délègue des fonctions comme la logistique, la paie des collaborateurs ou le développement de logiciels.

Pourtant les apports espérés des TIC n'ont pas été tout de suite au rendez-vous, et de nombreux projets ont mis en évidence des problèmes de dépassement des coûts et des délais, des solutions imparfaites, et surtout une communication problématique entre l'IT et le business.

Ces soucis de communication trouvent leur origine dans l'externalisation de la fonction IT, compliquant le transfert de connaissances métier du business vers l'IT. La reconnaissance de la fonction de business analyst palie en bonne partie ce problème : un business analyst interne à l'entreprise va assurer une excellente connaissance du métier et sera moins coûteux qu'une ressource externe. Il agit comme consultant interne dans des projets de développement et contribue :

- à la bonne compréhension et la bonne formulation des exigences ;
- au pilotage des développements des systèmes TIC par le business ;
- à l'accompagnement de la mise en œuvre de ces systèmes par une gestion du changement au niveau du business.

1.1 Définition et enjeux

(International Institute of Business Analysis, (IIBA), 2015) définit la business analyse comme suit.

Définition 1 : la business analyse est une pratique d'activation du changement en définissant les besoins business et en recommandant des solutions qui apportent de la valeur aux parties prenantes.

Cette définition peut surprendre à première vue, mais elle a le mérite de présenter la business analyse comme moteur du changement : à partir d'un bilan sur le fonctionnement actuel d'une organisation (as is), on fixe un horizon à atteindre (to be) à travers la spécification des solutions les plus pertinentes, et on détermine un chemin pour déployer ces solutions (gap analysis). La discipline va donc au-delà de la « simple » analyse : elle propose des solutions qui répondent aux besoins du business, qu'il s'agisse de problèmes existants ou d'opportunités à saisir.

Souvent, les solutions préconisées par la business analyse reposent sur les TIC, mais pas seulement. Par exemple, ces solutions recommanderont des révisions des processus purement manuels, comme la mise en place de réunions quotidiennes pour mieux organiser le travail, ou des changements d'attributions de tâches entre les différents acteurs qui prennent en charge l'exécution des processus, comme demander à un étudiant de construire son PAE plutôt que de confier cette tâche à un secrétariat des étudiants.

Les business analysts interviennent dans des types très variés de projets, tels que :

- la résolution de problèmes, comme des dysfonctionnements de processus nécessitant leur réingénierie ;
- la saisie d'une opportunité, comme des nouveaux produits (digitaux), de nouveaux marchés ou de nouvelles collaborations portées par les TIC. On pense par exemple à des opportunités de logistique intégrée ou à la vente de musique en streaming ;
- la réduction des coûts, typiquement grâce à l'automatisation de tâches manuelles ou à l'externalisation des tâches, comme dans Web banking, qui permet à une banque d'externaliser des tâches d'encodage de virements et autres ;
- le déploiement de progiciels intégrés (ERP, CRM, etc.), où le business analyst va s'assurer de l'alignement des processus de l'entreprise avec ceux du progiciel et inversement ;
- l'analyse des données, où des solutions sont mises en œuvre pour générer de la valeur à partir de l'analyse de gros volumes de données.

Pour bien cerner la business analyse, le BABOK (International Institute of Business Analysis, (IIBA), 2015) introduit les différents concepts clés de la discipline dans le « Core Concept Model » de la Figure 1 :

- **Changement** : action de transformation pour répondre à un besoin ;
- **Besoin** : problème ou opportunité à traiter ;
- **Solution** : manière spécifique de répondre à un besoin dans un contexte donné ;
- **Partie prenante** : individus ou groupes impliqués dans le changement, les besoins ou la solution ;
- **Valeur** : apport, importance ou utilité de quelque chose pour une partie prenante dans un contexte donné. La valeur est tangible ou intangible ;
- **Contexte** : circonstances qui influencent/sont influencées/exploquent le changement, comme des régulations ou la culture d'entreprise.

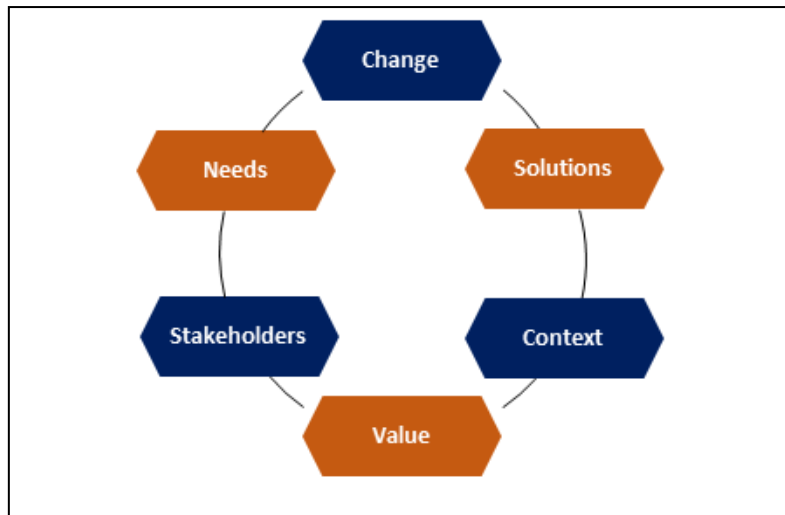


Figure 1: Core Concept Model de la business analyse selon le BABOK.

Le lecteur identifiera les relations entre ces éléments.

Spectre de l'analyse (Paul & Cadle, 2020). Plusieurs facettes de l'analyse rentrent dans la définition de la business analyse, comme le montre la Figure 2. Ces facettes se superposent partiellement et se distribuent entre une approche de haut niveau (niveau stratégique) et une approche de bas niveau (le système TIC). Un business analyst peut être confronté à chacune de ces facettes en fonction des projets ou de son expérience.

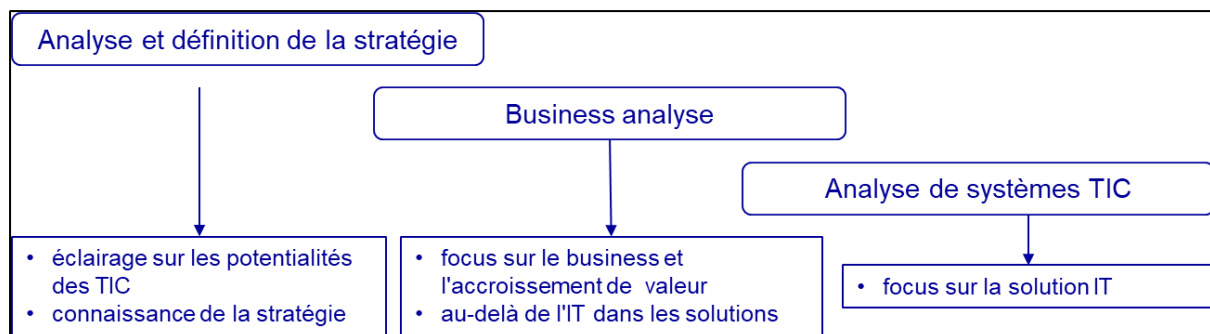


Figure 2 : 3 variations de l'analyse.

L'analyse et la définition de la stratégie relèvent typiquement du top management et ne sont pas directement du ressort de la business analyse. Cependant, le business analyst peut renseigner les dirigeants sur les opportunités que présentent les technologies digitales, voir les conseiller sur le meilleur usage de ces technologies pour mener à bien des changements. Par ailleurs, le business analyst doit être parfaitement conscient des choix stratégiques pour recommander, concevoir et mettre en œuvre les tactiques et solutions qui réalisent la stratégie.

L'analyse de systèmes TIC est à l'autre extrémité du spectre de l'analyse et concerne la réalisation très concrète du système TIC. Le rôle consacré de « system analyst » désigne la responsabilité d'analyser et de spécifier les exigences avec un détails suffisant pour permettre à l'équipe de développement de construire un système opérationnel.

La business analyse se place entre ces extrêmes et met l'amélioration du business avec (ou parfois sans) les TIC. accent sur les résultats. Certaines organisations font clairement la distinction entre le business analyst et le system analyst, d'autres pas.

1.2 Les missions de la business analyse

Nous avons déjà évoqué l'étendue de la business analyse, et cette section répertorie six missions typiques de la discipline (Paul & Cadle, 2020).

Investigation de la situation et analyse du problème. Cette première mission consiste à « découvrir les causes racine des problèmes, à identifier où des besoins émergent, et à définir les contours des projets qui adresse ces besoins » (Paul & Cadle, 2020).

Souvent, le business analyst est interpellé par des problèmes informels, mal définis et mal compris. Mais avant de pouvoir les résoudre, il devra clairement définir et décrire les problèmes et leurs sources. Ce travail d'investigation devra concilier les perceptions multiples et parfois conflictuelles d'un problème qu'ont les différentes parties prenantes, puis identifier les actions à entreprendre pour améliorer le fonctionnement du business.

Différents langages, comme Archimate (Section 10.2) permettent de représenter l'organisation, les parties prenantes, les processus, les données ou les composants techniques. Ces représentations aident à comprendre le problème et à identifier des pistes d'amélioration.

Étude de faisabilité et business case. Cette mission « évalue les options pour répondre aux besoins du business et analyse l'opportunité d'un changement » (Paul & Cadle, 2020).

Plus précisément, les différentes solutions pour résoudre un problème vont être identifiées et évaluées selon leur valeur, leur coût, leur faisabilité, leur niveau de risque, etc. À ce stade, les solutions sont décrites avec un niveau de détails suffisant pour poser un choix, sans plus.

Amélioration des processus d'affaires. « Il s'agit de rechercher, d'analyser et de définir les processus d'affaires existants (as is) les processus souhaités (to be), et d'analyser les écarts (gap analysis) afin d'identifier les actions requises pour implémenter les processus révisés. » (Paul & Cadle, 2020).

Cela implique souvent une analyse critique de l'existant (situation as is), la définition d'un état futur à atteindre (situation to be), et une étude des écarts et des actions pour réaliser l'état to be (gap analysis). Le trio as is – to be – gap analysis est mené ici au niveau du détail des processus (le même trio est aussi d'application lors de l'investigation de la situation et l'analyse du problème, mais plutôt à un niveau macroscopique, i.e., au niveau de l'entreprise).

Définition des exigences. Cela concerne « l'élicitation, l'analyse et la définition des exigences pour le business et les initiatives de changement IT » (Paul & Cadle, 2020).

L'ingénierie des exigences est précisément centrée sur ce travail sur les exigences, et est largement discutée dans la suite de ces notes. Notons cependant que les exigences poursuivent un but d'optimisation du fonctionnement du business, que cela implique ou non des changements des systèmes IT.

Tests d'acceptance business. Cette mission vise à « soutenir les acteurs business dans le test des nouveaux processus et des systèmes IT pour confirmer leur acceptation par le business » (Paul & Cadle, 2020).

Cette mission va donc au-delà du test logiciel, puisqu'elle couvre également l'évaluation des adaptations de processus qui n'impliquent pas l'IT, comme des simplifications ou des changements de responsabilités au niveau des acteurs.

Le business analyst ne doit pas vouloir se substituer aux acteurs business pour ces tests mais les épauler, en leur faisant comprendre l'importance du testing, et en les aidant dans la conception et l'exécution de tests, et dans la documentation des résultats.

Gestion du changement. Cette mission porte sur « le support au déploiement du changement au niveau business et IT afin d’assurer une transition en douceur » (Paul & Cadle, 2020).

Les changements produits par la solution sont censés apporter des bénéfices. Le business analyst va accompagner ces changements notamment en planifiant la transition vers les nouveaux processus et systèmes IT, en s’assurant de l’adhésion des parties prenantes face aux changements et en organisant des formations.

1.3 Le métier de business analyst

Cœur du métier. Les responsabilités du business analyst découlent naturellement des fondements de la business analyse.

Son premier rôle est de capter et de comprendre les besoins des utilisateurs, puis d’identifier les solutions les mieux alignées sur ces besoins¹. Comme le montre la Figure 3, le business analyst doit beaucoup communiquer avec l’ensemble des parties prenantes, et tant avec les utilisateurs qu’avec les informaticiens. D’ailleurs, le business analyst est apprécié dans son rôle d’interface entre ces deux communautés, qui ont trop souvent du mal à interagir.

Ce rôle d’interface implique de la créativité, des capacités d’analyse poussées et une pensée critique. En effet, le business analyst ne doit se borner à retranscrire les exigences du business pour les transmettre directement à l’IT, mais il doit comprendre en détails le problème et ses causes racines, challenger les processus en place et proposer des solutions innovantes coconstruites avec le business et l’IT.



Figure 3 : le rôle d'interface du business analyst.

¹ Selon le BABOK, "Business analysts are responsible for discovering, synthesizing, and analyzing information from a variety of sources within an enterprise, including tools, processes, documentation, and stakeholders. The business analyst is responsible for eliciting the actual needs of stakeholders—which frequently involves investigating and clarifying their expressed desires—in order to determine underlying issues and causes.

Business analysts play a role in aligning the designed and delivered solutions with the needs of stakeholders.

In many cases, the business analyst will also work to facilitate communication between organizational units. In particular, business analysts often play a central role in aligning the needs of business units with the capabilities delivered by information technology and may serve as a “translator” between those groups."

Le business analyst intervient dans des projets de développement de solutions informatiques :

- en amont du développement de la solution :
 - alignement : il s'assure que la stratégie de l'organisation est en phase avec son environnement. Il investigue les problèmes et les opportunités ;
 - définition : il analyse les causes à la racine des problèmes et enclenche les projets de changement correspondants. Il dégage des solutions, évalue leur faisabilité et leurs risques, et s'implique dans le choix de la meilleure solution. Il conduit l'ingénierie des exigences pour les projets et solutions retenus ;
- pendant le développement de la solution :
 - conception : le business analyst complète les exigences si nécessaire, et facilite la communication entre le business et l'IT ;
 - développement : il répond aux demandes d'éclaircissement au niveau des exigences. Il contribue à la priorisation des exigences. Il maintient la cohérence de la solution apportée par rapport aux exigences ;
 - test : il fixe les critères d'acceptation et accompagne le business dans l'évaluation de la solution produite tant au niveau des nouveaux processus qu'au niveau du logiciel délivré ;
 - implémentation : il participe à la planification et à la préparation de la mise en œuvre des solutions. Il s'assure que le business est suffisamment préparé pour accepter et activer les nouvelles manières de travailler. Il accompagne le changement induit par les solutions ;
- après le développement : il s'assure que les bénéfices attendus sont bien au rendez-vous, et identifie les éventuelles actions à entreprendre pour produire les résultats.

Déclinaisons du métier du business analyst. La discipline de la business analyse a commencé à émerger dans les années 80 et le métier du business analyst s'est étendu et complexifié depuis. Initialement centré sur la business analyse et le rôle d'intermédiaire entre business IT, le business analyst endosse maintenant de nouvelles responsabilités, comme participer à la définition de la stratégie.

Selon (Paul & Cadle, 2020), les déclinaisons typiques du business analyst sont les suivantes :

- « Business » business analyst /entreprise business analyst : cette déclinaison met l'accent sur la compréhension des situations d'affaires à améliorer au niveau de l'entreprise dans sa globalité, les solutions business à développer et les besoins du business à rencontrer ;
- Digital business analyst : cette variante du métier se concentre sur les opportunités que les technologies digitales peuvent amener à l'organisation, tant à travers l'amélioration des processus d'affaires que l'amélioration du service et de l'expérience client. Ce profil implique une bonne connaissance des TIC émergentes et de la créativité.
- Technical business analyst : ici, le travail du BA consiste à analyser les exigences détaillées dans le cadre d'un développement logiciel ;
- Proxy business analyst : ce rôle correspond à celui de product owner dans le contexte Agile, où le BA gère un product backlog, représente les utilisateurs pendant le développement et prend des décisions sur le produit final, comme l'établissement de priorités dans les exigences.

Niveaux d'expertise dans la fonction du business analyst. Au-delà de son cœur de métier, le business analyst va endosser des responsabilités croissantes avec l'expérience. Voici trois stades schématiques de la carrière de business analyst proposés par (Milani, 2019).

Stade 1 : business analyst junior. Sans surprise, ce stade englobe les responsabilités de base du business analyst, comme comprendre et formaliser les besoins et les communiquer à l'IT, ou évaluer la solution. Le business analyst junior améliore un système en place, sans traiter en détails la motivation ou le cadrage du projet.

Stade 2 : business analyst expérimenté. Il s'agit ici de comprendre les motivations du changement (business drivers), ce qui implique le cadrage, la motivation et la mise en évidence des opportunités du projet. Typiquement, le business analyst expérimenté va travailler à l'amélioration des processus en place selon une démarche structurée

de réingénierie des processus. Généralement, il va établir un bilan de l'existant (as is), décider d'un état à atteindre par le changement (to be) et fixer un chemin pour arriver à cet état (gap analysis).

Stade 3 : business analyst senior. Il va comprendre les objectifs business de haut niveau à atteindre et décider des composants à adapter, évaluer des alternatives et des choix en termes de projets. Ce niveau inclut la gestion d'un portefeuille de projets et l'établissement de priorités en fonction notamment d'un alignement stratégique des projets TIC. Le business analyst senior intervient donc à un haut niveau de décision. Il intègre le management stratégique et vise l'amélioration du business dans son ensemble.

La Table 1 propose des correspondances typiques entre les déclinaisons du métier de BA et les niveaux d'expertise cités ci-dessus. Cependant, les réalités de terrain révèlent davantage de variétés dans les déclinaisons de la profession, les niveaux d'expertise et leurs correspondances.

	<i>business analyst junior</i>	<i>business analyst expérimenté</i>	<i>business analyst senior</i>
Enterprise business analyst			*
Digital business analyst		*	
Technical business analyst	*		
Proxy business analyst	*	*	

Table 1 : correspondance entre déclinaisons du métier de BA et expertise.

Chaque stade requiert de cumuler différentes compétences, comme l'illustre la Figure 4 et comme détaillé dans la suite.

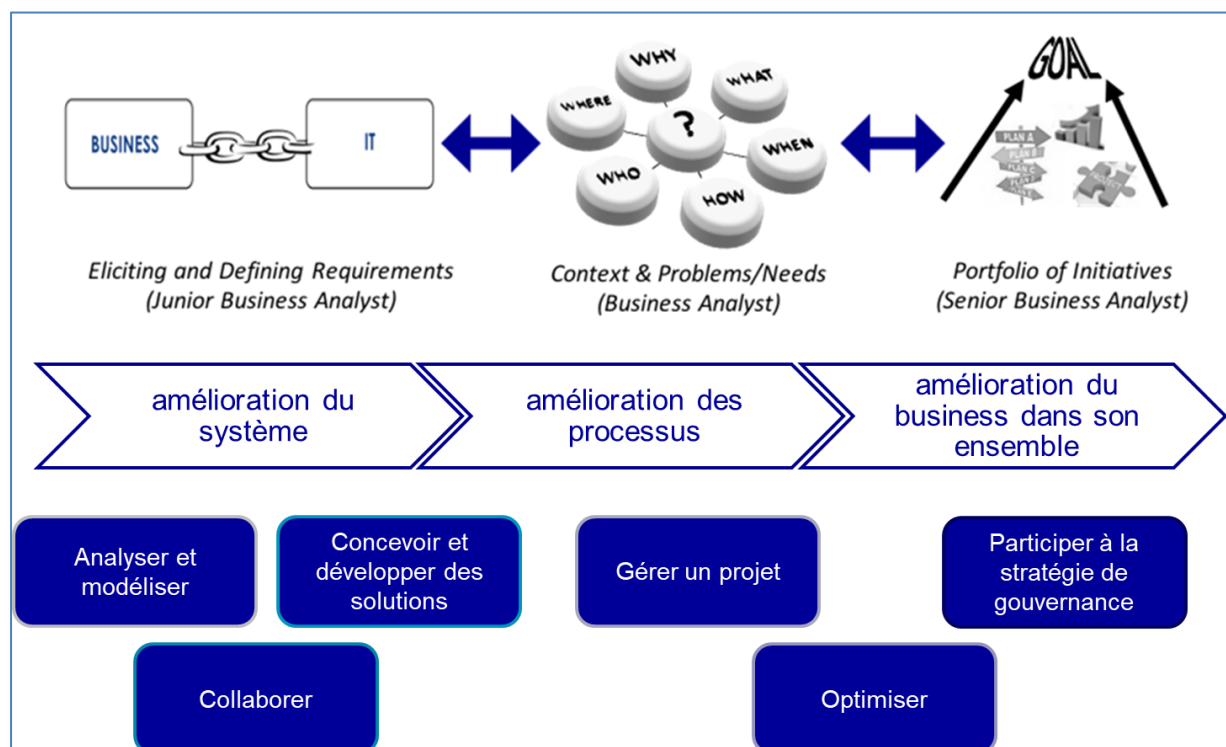


Figure 4 : les 3 paliers d'expertises du business analyst et les compétences mobilisées.

Compétences du business analyst. Le business analyst correspond à un profil généraliste et est amené à réaliser de nombreuses tâches différentes. En amont des compétences discutées ci-dessous, le business analyst doit être familier avec les méthodes de résolution de problèmes (Section 1.4).

Son rôle d'intermédiaire entre le business et l'IT exige de très bonnes compétences relationnelles (soft skills), mais aussi une bonne connaissance de l'entreprise et de son environnement, ainsi qu'une maîtrise minimale des TIC.

Naturellement, le business analyst doit être un expert dans les techniques qu'il applique au quotidien (le BABOK en cite plusieurs dizaines !).

La Figure 5 (inspirée de (Paul & Cadle, 2020)) donne un aperçu des compétences essentielles du business analyst.

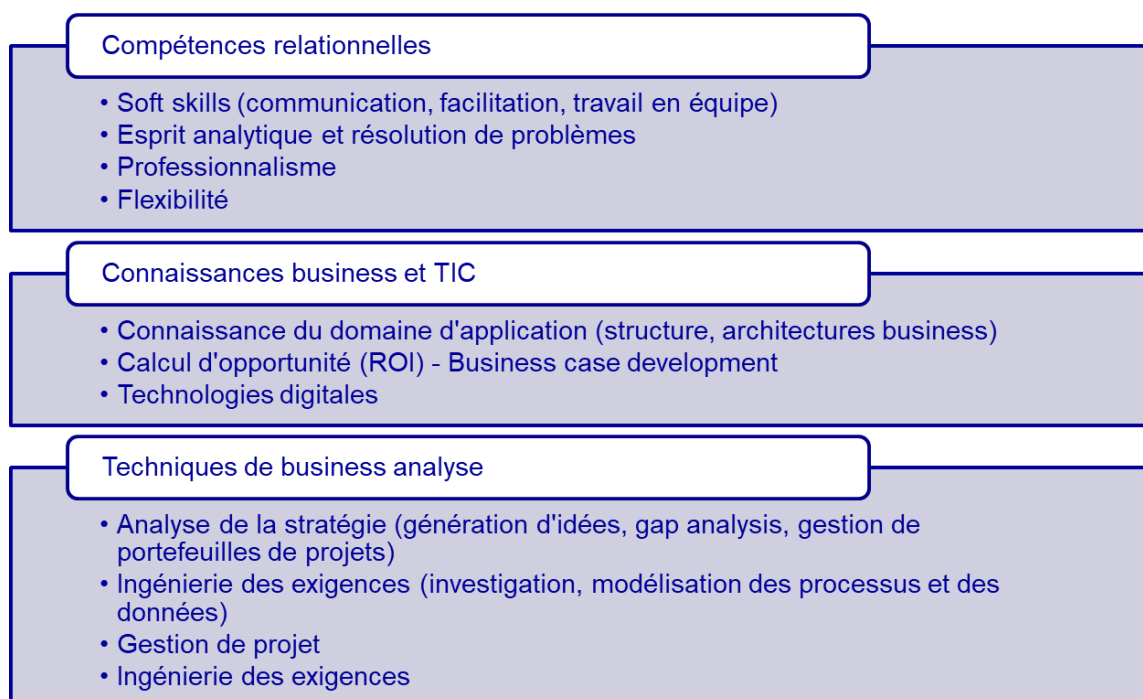


Figure 5 : compétences clés du BA.

1.4 Zoom sur la résolution de problèmes

La plupart des modèles de résolution de problèmes suggère a minima une phase de **compréhension du problème**, une phase de définition des **solutions** et une phase de **mise en œuvre** des solutions. Une multitude de techniques existent pour soutenir chacune de ces phases. Voyons cela en détails.

Compréhension du problème.

1. Une première étape consiste à appréhender le problème et son contexte de manière globale et informelle. En effet, les situations en entreprise sont souvent complexes et les problèmes sont mal définis et mal compris. Le business analyst va interagir à bâtons rompus avec les parties prenantes pour décrire progressivement le problème et son contexte, et va le formuler avec des techniques peu formelles comme les cartes mentales (Mind Mapping).

Mind Mapping aide à structurer des idées, des sujets, des acteurs, etc. dans une arborescence décrivant un concept racine, par exemple une entreprise. Les branches détaillent un thème lié au concept racine, comme un acteur ou un processus. Chaque branche peut être elle-même décomposée par des branches de plus bas niveau, comme dans la Figure 6.

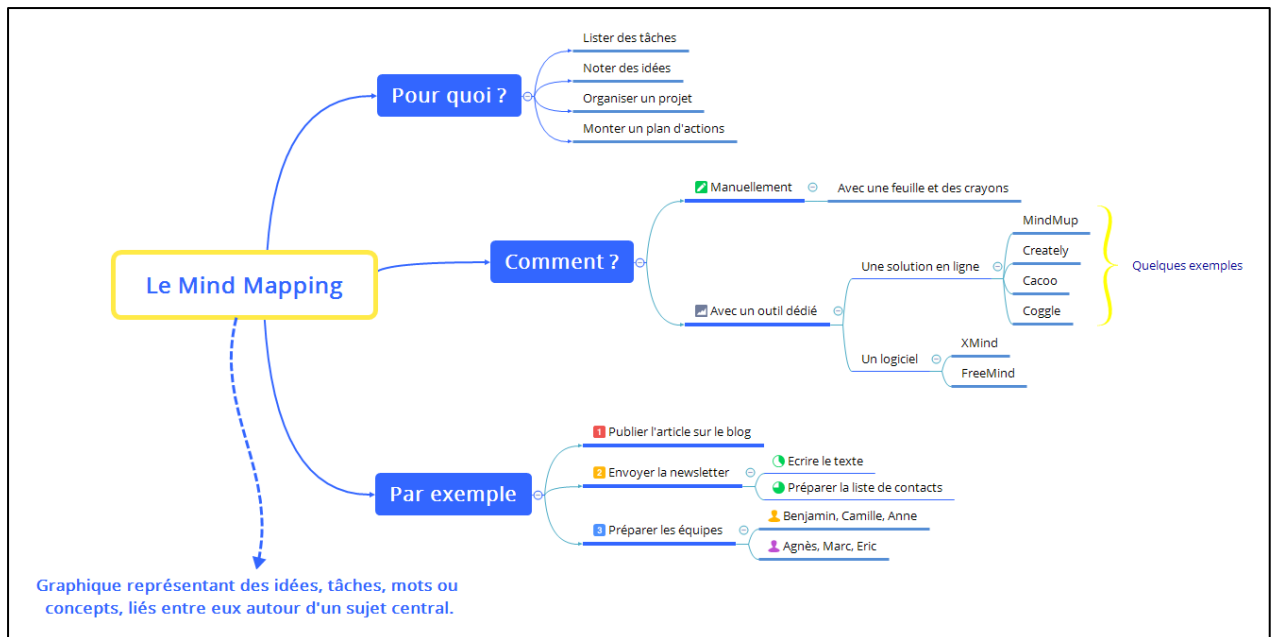


Figure 6 : exemple de Mind Mapping.

2. Une seconde étape consiste à analyser les résultats de l'étape précédente et à les compléter avec des données quantitatives et qualitatives collectées par exemple via des enquêtes, des entretiens ou des observations, comme celles de la Table 2.

Enquête réalisée auprès de 150 étudiants sur leurs usages du mind mapping	
Prendre des notes lors d'un cours et résumer les points les plus importants	60%
Montrer les liens et les relations entre les idées principales d'un travail	50%
Faire un brainstorming de tout ce que vous savez déjà sur une question de recherche	45%

Table 2 : exemple de données quantitatives.

3. À ce stade, la complexité de la situation est décrite et comprise et objectivée par des données quantifiables. Une troisième étape permet finalement de spécifier le problème et ses causes (et pas uniquement ses symptômes).

Une technique populaire pour cela est l'analyse des causes et effets (diagramme d'Ishikawa, discuté dans la Section 6.2) pour identifier les facteurs contributifs. Les facteurs causaux sont classés en catégories choisies en fonction de la nature du problème à analyser. Le diagramme prend la forme d'un squelette de poisson, dont la tête représente le problème et les arêtes les catégories de facteurs causaux, comme le montre la Figure 7.

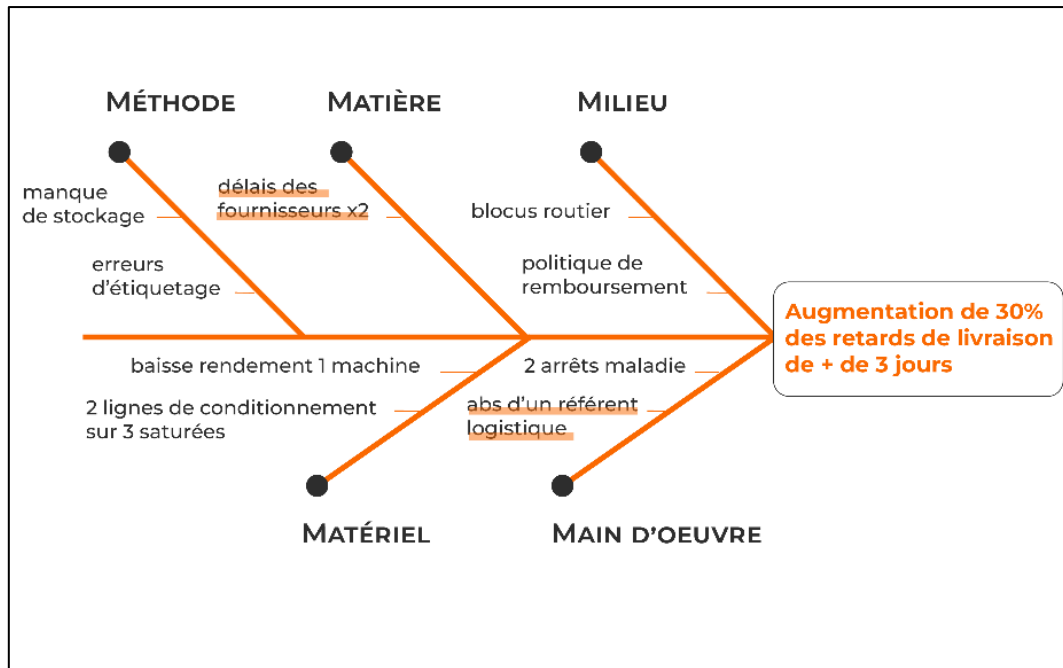


Figure 7 : exemple de diagramme d'Ishikawa.

Maintenant que le problème est correctement cerné, il est temps d'aborder les solutions.

Définition de la solution.

1. Cette phase commence souvent par une génération d'idées, par exemple avec une séance de brainstorming.

Un brainstorming est une session en groupe où chacun peut proposer des idées sur un sujet donné. Les idées sont directement consignées sur un tableau visible de tous pour stimuler d'autres suggestions (Figure 8).



Figure 8 : exemple de support d'une séance de brainstorming.

2. Une fois que toutes les idées ont émergé, elles sont examinées en détails. Les idées les plus pertinentes sont conservées et débouchent sur des solutions.

3. Finalement, les solutions sont évaluées selon différents critères choisis en fonction du problème et du contexte. Les critères typiques concernent le coût, le délai de réalisation, la contribution à la résolution de problème ou le niveau de risque. La Figure 9 montre un exemple de comparaison de différents modes de transport pour adresser un problème de mobilité.

Échelle de positionnement : 0 = nul, 1 = pas bon, 2 = moyen, 3 = bien, 4 = très bien.

CRITÈRES	Pondé- ration	SOLUTIONS							
		TGV 1re classe	Total	AVION	Total	VOITURE	Total	AUTO STOP	Total
Rapidité	4	3	12	3	12	1	4	0	0
Confort	3	3	9	2	6	0	0	1	3
Sécurité	2	3	6	3	6	1	2	0	0
Coût	1	2	2	0	0	0	0	3	3
TOTAL			29		24		6		6

Figure 9 : exemple de comparaison multicritère.

Les solutions sont maintenant identifiées et énoncées, mais sans beaucoup de détails.

Mise en œuvre. Cette phase détaille finement la mise en œuvre des solutions.

Souvent en business analyse, la résolution de problèmes suit l’approche de la trilogie as is - to be – gap analysis, d’abord à un niveau macroscopique pour identifier des solutions globales, puis à un niveau plus microscopique liés à aux détails de mise en œuvre d’une solution, comme expliqué en détails dans la Section 1.5

1.5 Le processus de la business analyse

Une mission de business analyse adopte un processus² structuré pour organiser toutes les tâches à réaliser.

Cette section présente un processus générique qui balise le travail de la business analyse. Cependant, il n’existe pas de recette universelle pour tous les projets, et ce processus doit être adapté aux particularités du problème à résoudre.

Le processus de la Figure 10 montre les étapes typiques de la business analyse. Ces étapes ne sont pas systématiquement exécutées en séquence, mais avec des allers-retours, des itérations et du parallélisme. Ce processus est basé sur les recommandations du BABOK (International Institute of Business Analysis, (IIBA), 2015).

² Un processus est un ensemble de tâches coordonnées et orientées vers un but.

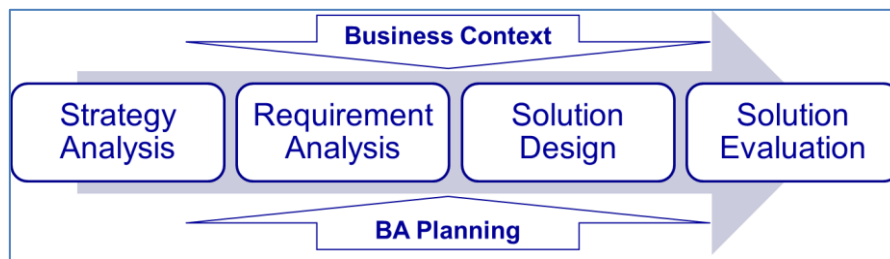


Figure 10 : processus générique de la business analyse.

1.5.1 Business Context

Avant de détailler les différentes étapes, rappelons que les business analysts doivent être conscients du contexte dans lequel ils opèrent car il impacte toutes les étapes de la business analyse. Nous savons que le contexte recouvre les circonstances qui influencent/sont influencées/explicitent le changement, comme le business model de l'organisation, ses valeurs, sa stratégie, les réglementations à respecter ou encore sa structure organisationnelle.

Par exemple, dans le contexte d'un business model basé sur l'excellence de la relation client, la qualité des processus client prime sur leur coût d'exécution. Dans ce contexte, les processus devront être plutôt flexibles et personnalisables en fonction du client final.

En revanche, dans le contexte d'un business model basé sur de gros volumes de transactions et de clients, les coûts seront les plus déterminants, et les processus devront être standards pour tous les clients et économes en ressources.

1.5.2 BA Planning

La planification fixe l'agenda des tâches la business analyse, mais pas seulement. Elle consiste à :

- cadrer le projet, notamment en précisant le problème/opportunité adressé par le projet, ses objectifs à atteindre et sa valeur attendue ;
- choisir les outils, les méthodes et les processus les plus pertinents pour adresser le problème posé ;
- identifier et planifier des tâches et la production de livrables, et décider de l'attribution des tâches aux parties prenantes.

Le BABOK suggère de produire un « Business Analysis Plan ». Un exemple d'un tel plan est repris dans le Tableau 1 :

• Cadrage – description du besoin (problème/opportunité) • ex. : un parc d'attractions a vu sa fréquentation augmenter fortement ces 5 dernières années, et les solutions en place pour gérer l'afflux des visiteurs et l'espace disponible sont dénoncées par plusieurs plaintes client. Une meilleure solution doit être mise en place pour améliorer l'expérience client et ainsi augmenter la satisfaction client et poursuivre la croissance du parc. – objectifs du travail de business analyse • identifier les options de solutions et fournir assez de données pour la prise de décision sans trop détailler les options • ex. de solutions pour améliorer l'expérience des visiteurs : agrandissement du parc, contrôle de nombre d'entrées par période via un système de réservation, instauration d'une tarification dynamique – valeur attendue, effets désirés • ex. : meilleure satisfaction client, accroissement du nombre de visiteurs
• Choix d'outils et de méthodes – typiquement : approche prédictive (classique) vs approche adaptative (Agile) • recommandation en fonction du type de projet (taille, complexité, etc.), de la volatilité des exigences, etc. • ex. : choix de la méthode Agile Scrum
• Identification et de planification des tâches et livrables – quelles sont les tâches et les livrables pour déployer la solution ? Quelle est la planification globale du projet ? Quelles sont les estimations des ressources et des coûts ? – ex. de tâches : • définition du "as-is" • définition du "to-be" • analyse de risques • choix de la solution • design de la solution choisie • développement de la solution et évaluation

Tableau 1 : exemple de Business Analysis Plan.

1.5.3 Strategy Analysis

Cette première étape de processus de business analyse se décompose elle-même en plusieurs tâches, comme le montre la Figure 11.

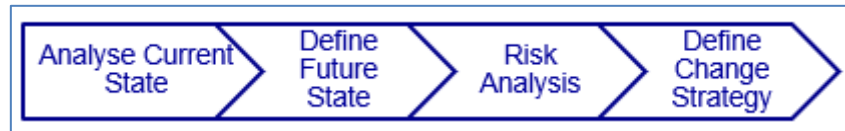


Figure 11 : décomposition de l'étape Strategy Analysis.

La tâche *Analyse Current State* vise à décrire l'état actuel de l'organisation (état « as is »), en précisant par exemple son métier, ses capacités humaines et matérielles, sa culture d'entreprise, ses ressources, ou encore son infrastructure. À partir de là, on va identifier des besoins business qui prennent la forme de problèmes à résoudre, comme un accroissement des plaintes de clients, une diminution de revenu, etc., ou d'opportunités à saisir (notamment celles portées par les TIC) comme de nouveaux marchés ou produits. Les sciences de gestion proposent plusieurs outils pour cerner cet état actuel, comme des cadres d'analyse de la stratégie d'entreprise tels que le modèle « business canevas » ou le modèle PESTEL (Politique Économique Social Technologique Environnemental et Légal).

Ensuite, les business analysts vont décrire l'état futur souhaité qui justifie le changement (état "to be"). Selon le BABOK, définir l'état futur revient à déterminer l'ensemble des conditions nécessaires pour satisfaire le besoin du business. Ces conditions peuvent prendre la forme de processus revisités et/ou de nouveaux objectifs à atteindre. Plusieurs livrables sont cités ici, comme :

- Business Objectives : orientation souhaitée pour atteindre l'état futur ;
- Future State Description : description des composants à adapter (processus, capacités, ressources, infrastructure, etc.) pour atteindre l'état futur ;
- Potential Value : valeur espérée grâce à l'implémentation de l'état futur.

La tâche *Risk Analysis* traite des risques liés à la réalisation de l'état futur. Au-delà de l'habituelle identification et quantification des risques, des stratégies et des contremesures vont être proposées pour mitiger ces risques en diminuant leur probabilité de survenance ou leurs impacts.

Finalement, la tâche *Define Change Strategy* va proposer et décrire les solutions alternatives possibles pour activer le changement. En d'autres termes, comment réaliser l'état futur, par exemple grâce à un développement logiciel sur mesure, ou par le déploiement d'un progiciel, en encore en adaptant les processus. Les business analysts parlent souvent de Gap Analysis pour adresser les questions suivantes :

- comment passer de l'état "as is" à l'état "to be" ?
- quelles sont les solutions possibles ?
- quelle est la portée de chaque solution alternative ?

Il restera alors à évaluer les solutions alternatives et à sélectionner la meilleure sur base de critères comme l'impact de la solution (bénéfices), le respect des contraintes (délais, coûts, etc.), l'ampleur du changement ou encore la faisabilité de la solution.

Le Tableau 2 illustre la démarche « as is – to be – gap analysis » :

- Analyse as is : - contexte : - une administration régionale gère des dossiers de demandes de primes à la rénovation durable déposées par les citoyens - à faire : description des processus en place (soumission d'une demande, examen d'une demande, réponse à une demande) - mesure de la performance du système en place - ex. de KPI : délai moyen du traitement d'une demande, satisfaction des citoyens - problèmes - délai trop long (6-12 mois) dans la prise de décision du montant éligible et l'octroi des primes - mécontentement des parties prenantes (taux de satisfaction citoyen de 35%, agents débordés) → l'administration souhaite un changement pour résoudre ce problème				
- Analyse to be : - fixation d'objectifs afin de résoudre le problème identifié en amont - ex. : réduction du temps moyen du traitement à 3 mois - ex. : augmentation du niveau de satisfaction citoyen de 20 % - révision des processus et adaptation des ressources de l'administration				
Gap analysis : - identification des facteurs limitants - ex. : manque d'automatisation des processus - ex. : nombre d'intervenants trop nombreux - ex. : motivation et formation du personnel irrégulières - identification des solutions possibles (<i>bridging the gap</i>) - automatisation de tâches encore manuelles - optimisation des processus en place (suppression des tâches peu porteuses de valeur, réduction du nombre d'intervenants, etc.) - à faire : description des processus futurs (soumission d'une demande, examen d'une demande, réponse à une demande) - plan de formation et de motivation du personnel				
Exemple d'évaluation des solutions possibles				
	Solution	Impact sur le délai	Impact sur la satisfaction	Coût
	automatisation	***	**	*
				Niveau de risque
				**

	optimisation des processus	**	**	**	*	
	formation et motivation	*	***	***	**	

Tableau 2 : exemple de Strategy Analysis.

1.5.4 Requirement analysis

Cette étape est le cœur de l'ingénierie des exigences. Elle sera largement détaillée dans la suite de ces notes. En résumé, son objectif est de constituer un ensemble d'exigences complet, cohérent et porteur de valeur pour la solution retenue. Notons que les étapes précédentes (as is, to be, gap analysis) se cantonnent à une description des exigences à un niveau macro, alors que cette étape vise à cerner des exigences détaillées.

1.5.5 Solution Design

Ici encore, on rentre dans un niveau de détail très fin puisque la solution proposée va être complètement définie et précisée. Cela implique :

- une compréhension fine des besoins : analyse approfondie de la situation actuelle sur base de la phase requirement analysis et des points d'amélioration nécessaires ;
- une présentation de la solution proposée : comment la solution va adresser les besoins identifiés, par exemple en montrant comment améliorer l'efficacité opérationnelle et répondre aux besoins spécifiques ;
- la portée du projet : quels sont les fonctionnalités et les processus couverts par la solution ?
- des mécanismes de validation de la solution à travers des tests et une assurance qualité.

Cette étape représente l'aboutissement de l'ensemble du processus de business analyse. D'ailleurs, les livrables des étapes précédentes sont largement repris et détaillés dans un document appelé Solution Design Document, qui matérialise le travail de la phase de solution design.

Un exemple de Solution Design Document est proposé en annexe (Section 10.1).

1.5.6 Solution Evaluation

Cette étape vérifie que la solution atteint les objectifs fixés et répond aux exigences des parties prenantes. Comme détaillé dans la Section 8, les tâches clés sont la planification, la création et la réalisation de tests.

1.6 Green IT et business analyse

Impact environnemental du numérique. Plusieurs études³ font état de l'augmentation de l'empreinte numérique sur les ressources planétaires. Cela s'explique bien évidemment par des usages toujours plus nombreux pour les individus et les organisations, la diminution du coût (financier mais pas écologique) du matériel et des services, le suréquipement, etc.

Par exemple, la consommation électrique totale du digital est estimée à 10% de la production mondiale d'électricité (<https://www.connaissancedesenergies.org/overview-electricity-used-global-digital-ecosystem-220218>).

En France, chaque personne possède en moyenne 15 équipements connectés, comme des smartphones, des ordinateurs, des tablettes, des voitures, etc. (<https://www.leprogres.fr/environnement/2022/01/19/chaque-francais-possede-en-moyenne-15-equipements-connectes>). La fabrication de la plupart de ces équipements demande beaucoup de ressources et d'énergie (<https://www.youtube.com/watch?v=1ltDaaoi8TU>).

Réponse de la business analyse. Ces différents constats alarmants ont notamment motivé l'émergence du Green IT, qui regroupe un ensemble de pratiques pour réduire l'empreinte environnementale du digital.

Dans ce contexte, les business analysts ont un rôle à jouer pour intégrer des préoccupations environnementales dans les projets et les solutions qu'ils proposent, par exemple :

- en considérant l'impact environnemental dans le choix d'une solution ;
- intégrer la sobriété numérique, par exemple en réfléchissant à des processus légers ou limitant l'automatisation à ce qui est vraiment nécessaire ;
- encourager l'équipe de développement à concevoir des services numériques éco-responsables (<https://ecoresponsable.numerique.gouv.fr/publications/referentiel-general-ecoconception/#specifications>).

³ Voir par exemple les rapports de greenIT.fr ou theshiftproject.org

1.7 Exercices

IEX01 : Proposez des relations entre les éléments du Core Concept Model de la Figure 1.

IEX02 : Proposer des correspondances entre domaines de connaissances cités par le BABOK et celle visées par le master en business analyst (Figure 12).

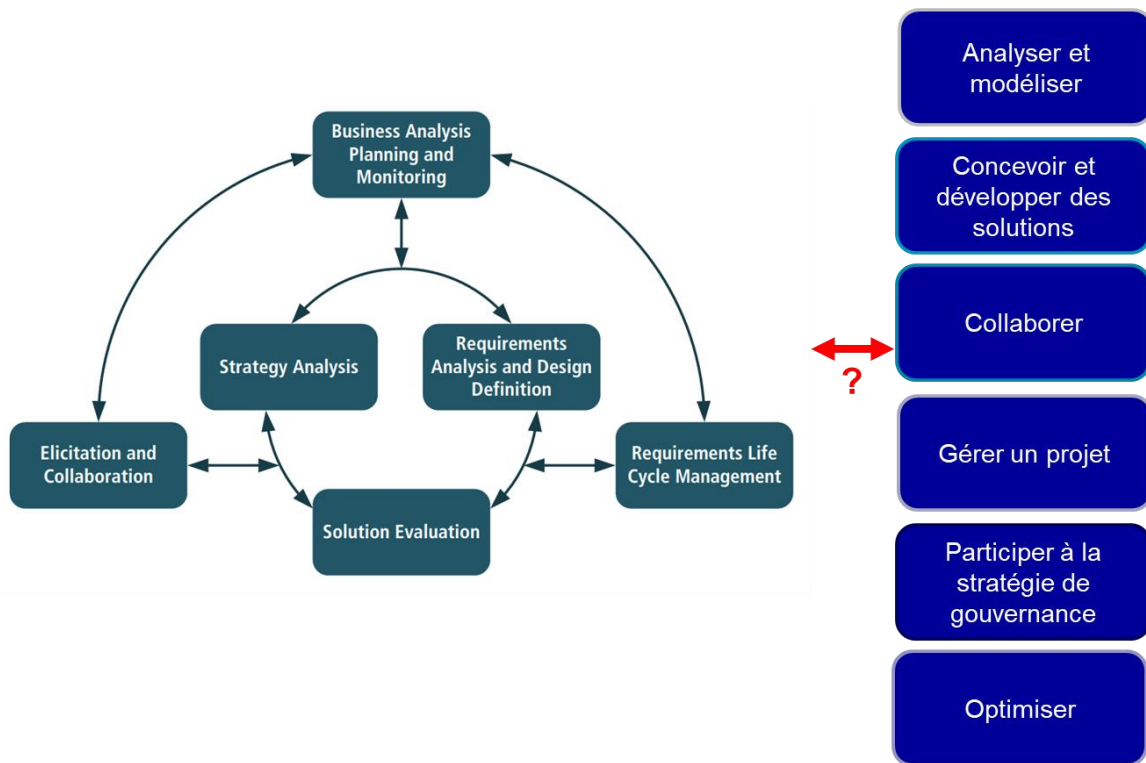


Figure 12 : compétences du BABOK vs. compétences du master en BA.

IEX03 : Appliquez une approche de résolution de problème dans le contexte d'une administration régionale qui gère des dossiers de demandes de primes à la rénovation durable déposées par les citoyens.

On vous demande :

- Un mind mapping pour décrire le problème de satisfaction des citoyens et sa situation
- Des données sur la performance (choisissez des indicateurs et trouvez des données relatives aux administrations sur le Web)
- Un diagramme d'Ishikawa pour identifier les causes du problème
- La tenue d'une séance de brainstorming
- Une liste de solutions prioritaires

Interview d'un citoyen (adapté de Copilot 20/12/2024)

Interviewer : Bonjour, merci de prendre le temps de nous parler aujourd'hui. Pouvez-vous nous expliquer votre expérience avec l'administration communale concernant votre demande de prime à la rénovation durable ?

Citoyen mécontent : Bonjour. Oui, bien sûr. Franchement, c'est un vrai parcours du combattant. J'ai **déposé ma demande il y a plus de six mois** et je n'ai toujours **pas de réponse définitive**. Les **délais sont interminables** et chaque fois que je contacte l'administration, on me dit que mon dossier est en **cours de traitement**.

Interviewer : Cela doit être très frustrant. Pouvez-vous nous en dire plus sur les difficultés que vous avez rencontrées ?

Citoyen mécontent : Absolument. D'abord, les formulaires sont extrêmement compliqués et toujours en format papier : quasi rien n'est en ligne. Il y a tellement de documents à fournir et chaque fois que je pense avoir tout complété, on me demande encore autre chose. Ensuite, il y a le manque de communication. J'ai envoyé plusieurs emails et passé des appels, mais les réponses sont souvent vagues ou inexistantes, et de la part de personnes différentes qui ne semblent pas connaître l'historique du dossier. On a l'impression que personne ne sait vraiment ce qui se passe avec mon dossier.

Interviewer : Avez-vous reçu des explications sur les raisons de ces délais et de cette complexité ?

Citoyen mécontent : Pas vraiment. On m'a dit que c'était dû à un grand nombre de demandes et à des ressources limitées, mais cela ne justifie pas un tel manque d'efficacité. J'ai l'impression que l'administration n'est pas bien organisée et que les citoyens en paient le prix.

Interviewer : Que souhaiteriez-vous voir changer dans ce processus ?

Citoyen mécontent : J'aimerais que l'administration soit plus transparente et réactive. Il faudrait simplifier les formulaires, améliorer la communication avec les citoyens et mettre tout ça en ligne. Un suivi plus rigoureux des dossiers serait aussi nécessaire pour éviter ces longs délais. En somme, il faut rendre ce processus plus accessible et moins stressant pour les gens qui veulent simplement améliorer leur habitat de manière durable.

Interviewer : Merci beaucoup pour votre témoignage. Nous espérons que votre situation s'améliorera bientôt.

Citoyen mécontent : Merci à vous. J'espère aussi que les choses changeront pour le mieux.

Interview d'un agent (adapté de Copilot 20/12/2024)

Interviewer : Bonjour, merci de nous accorder cet entretien. Pouvez-vous nous parler de votre expérience en tant qu'agent chargé des dossiers de demandes de primes à la rénovation durable ?

Agent administratif : Bonjour. Eh bien, pour être honnête, c'est assez difficile en ce moment. Nous sommes débordés par le nombre de demandes et la procédure est extrêmement complexe. Je dois avouer que je ne me sens pas suffisamment formé pour gérer tout cela efficacement.

Interviewer : Cela doit être très stressant. Quelles sont les principales difficultés que vous rencontrez ?

Agent administratif : Oui, c'est vraiment stressant. La principale difficulté est la complexité des procédures. Il y a tellement de règles et de documents à vérifier que cela devient vite écrasant. De plus, nous n'avons pas reçu une formation adéquate pour bien comprendre toutes les étapes du processus. Cela entraîne des erreurs et des retards, ce qui est frustrant pour les citoyens et pour nous. La plupart des traitements sont manuels car l'informatique ne prend pas le temps de nous développer un programme.

Sans parler des changements fréquents de la procédure, dès que les politiques s'en mêlent...

Interviewer : Comment cette situation affecte-t-elle votre motivation au travail ?

Agent administratif : Honnêtement, c'est démotivant. On a l'impression de ne jamais pouvoir rattraper le retard et de toujours être sous pression. Cela affecte aussi notre moral, car nous voulons aider les citoyens, mais nous sommes limités par le système et le manque de ressources. Plusieurs de mes collègues sont absents, et me refilent des dossiers que je ne connais pas bien.

Interviewer : Que pensez-vous qu'il faudrait changer pour améliorer la situation ?

Agent administratif : Il faudrait simplifier les procédures et offrir une meilleure formation aux agents. Un renforcement des effectifs serait également nécessaire pour gérer le volume de demandes. Enfin, une meilleure communication interne et avec les citoyens pourrait aider à réduire les frustrations et à rendre le processus plus fluide.

Interviewer : Merci beaucoup pour votre témoignage. Nous espérons que des améliorations seront apportées rapidement.

Agent administratif : Merci à vous. J'espère aussi que les choses changeront pour le mieux, pour nous et pour les citoyens.

IEX04 : Développez un modèle cible (To-Be) et de mettez en évidence les écarts (Gap Analysis) en fonction de l'analyse de la solution existante (As-is) de l'entreprise OptiLog ci-dessous.

L'entreprise OptiLog, une PME spécialisée dans la logistique urbaine, gère les livraisons pour des commerces de proximité (épiceries, librairies, petites boutiques). L'entreprise connaît une croissance rapide, mais ses processus internes n'ont pas suivi. Le dirigeant souhaite améliorer la performance et réduire les coûts d'exploitation.

Processus actuels

- Prise de commande : Les clients appellent ou envoient un e-mail. Un employé saisit manuellement la commande dans un fichier Excel.
- Planification des tournées : Les chauffeurs reçoivent chaque matin une feuille papier avec leur liste de livraisons. L'itinéraire est organisé à la main par le responsable logistique.
- Suivi des livraisons : Les chauffeurs appellent le standard en cas de problème. Les clients n'ont pas de suivi en temps réel.
- Facturation : Les factures sont générées manuellement en fin de mois à partir des fichiers Excel.

Problèmes identifiés

- Perte de temps liée à la saisie manuelle et aux erreurs fréquentes.
- Itinéraires sous-optimisés → coûts élevés en carburant.
- Manque de visibilité pour les clients (pas de suivi en ligne).
- Retards fréquents dans la facturation.
- Difficulté à gérer l'augmentation du volume de commandes.

Votre solution inclura :

- Les options possibles pour répondre au problème posé et le choix de l'option que vous préconisez.
- Une brève description des processus que vous envisager pour le to-be et les bénéfices attendus.
- Les actions recommandées pour réaliser le to be (gap analysis)

2 Ingénierie des exigences

2.1 Contexte de l'ingénierie logicielle

L'ingénierie des exigences est souvent perçue comme une discipline connexe de l'ingénierie logicielle. Si l'ingénierie concerne le développement de solutions à des problèmes pratiques sur base de connaissances scientifiques, l'IEEE (Institute of Electrical and Electronics Engineers - www.ieee.org) définit l'ingénierie logicielle comme suit :

Définition 2 : « l'ingénierie logicielle consiste en l'application d'approches systématiques, rigoureuses, quantifiables pour le développement, la mise en œuvre et la maintenance d'un logiciel. »

Il s'agit donc de développer du logiciel de manière organisée et professionnelle. L'ingénierie logicielle propose des approches ou processus de développement pour cadrer le travail de production de solutions logicielles. Ces processus peuvent être utilisés comme des guides ou « livres de recettes » pour mener à bien des projets de développement de logiciels.

La littérature propose plusieurs processus de développement auxquels les informaticiens peuvent se référer pour identifier les tâches d'un projet logiciel et les planifier, mais aussi pour déterminer les livrables à produire et les rôles à assumer.

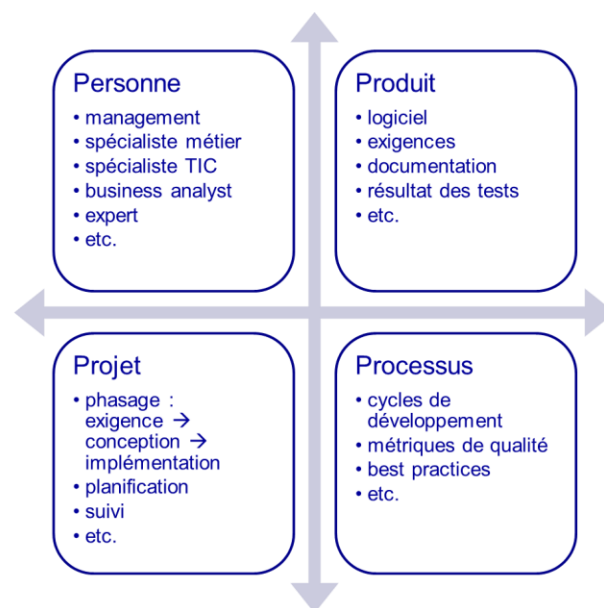


Figure 13 : les quatre P d'un projet informatique.

La nature d'un projet informatique s'explique assez bien à travers les quatre P de la Figure 13 :

- **Personne** : un projet informatique fait intervenir de nombreux acteurs différents, dont les représentants du business et des utilisateurs, et les spécialistes des TIC, le business analyst, etc.
- **Produit** (ou livrable) : la finalité première du projet est de délivrer un système qui répond aux exigences métier, mais pas seulement. Les livrables du projet reprennent aussi la liste des exigences, les résultats de tests, la documentation utilisateur, etc.
- **Projet** : le projet doit être planifié avant son lancement, puis différents aspects de son déroulement doivent être suivis, comme la qualité ou le respect du budget.

- **Processus** : un projet se structure selon un processus qui guide l'équipe de développement notamment à travers une décomposition du projet en phases et en activités. Globalement, la plupart des projets démarrent par une phase dédiée aux des exigences, puis enchaînent une phase seconde phase de conception du futur logiciel et se terminent par une phase d'implémentation. En réalité, le phasage est plus morcelé et moins linéaire que cela, comme expliqué ci-dessous.

Gestion de la complexité. En pratique, les projets de développement logiciels sont assez conséquents, car les besoins du business sont assez larges. Les solutions logicielles sont elles aussi vastes et complexes, ce qui suggère d'utiliser des stratégies de phasage et de décomposition pour un développement progressif du logiciel.

L'organisation d'un projet en différentes *phases* en facilite la gestion (planification, attribution, budget, etc.) et le contrôle (monitoring et ajustement). Une phase reprend un ensemble *d'activités* qui contribuent à produire les livrables du projet (Figure 14).

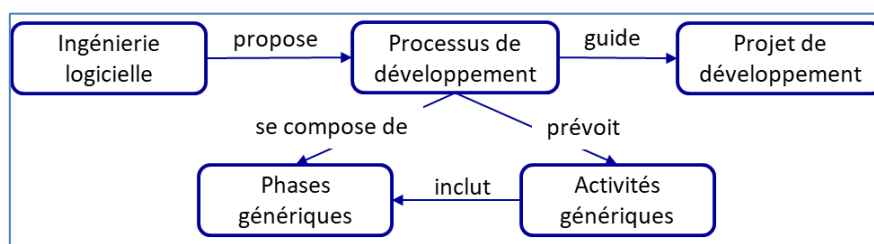


Figure 14 : processus et projets de développement.

Cycles de développement. Le cycle de développement d'un processus précise les interrelations entre les phases et la localisation des activités. Il répond au moins aux questions suivantes :

- dans quel ordre et à quelle fréquence exécuter les phases ?
- quels sont les critères pour passer d'une phase à l'autre ?
- quels sont des livrables de chaque phase ?
- quelles sont les activités à réaliser lors de chaque phase ?

Il existe plusieurs processus et de cycles de développement qui combinent différemment ses phases et leurs activités. Certains cycles imposent une succession strictement séquentielle des phases du projet, alors que d'autres encouragent leur répétition à travers des itérations successives, comme le montre le Tableau 3.

La méthode Scrum est emblématique du courant méthodologique Agile, qui encourage entre autres :

- des interactions entre les personnes plutôt que des processus et outils ;
- des collaborations plutôt que des contrats ;
- du logiciel plutôt que de la documentation ;
- de l'adaptation plutôt qu'un plan figé.

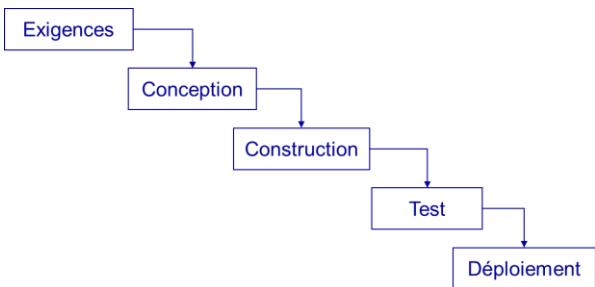
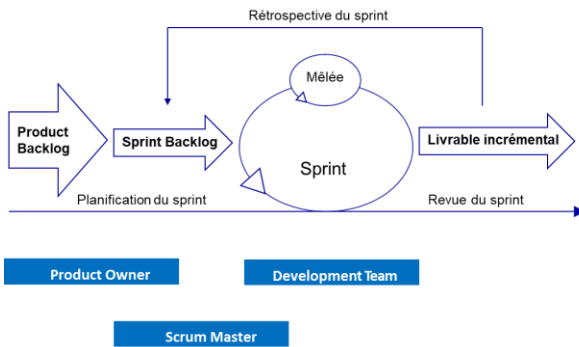
Cycle en cascade Enchaînement séquentiel des phases Les exigences sont censées être complètes et figées au début du cycle Pas de mécanisme de gestion de la complexité ou du changement, lourdeur des tests concentrés sur une seule phase	 <pre>graph TD; A[Exigences] --> B[Conception]; B --> C[Construction]; C --> D[Test]; D --> E[Déploiement]</pre>																																																																																
Unified Process 4 phases séquentielles mais avec des parcours itératifs au sein des phases Distribution des activités dans les différentes phases Itérations délivrant des incréments du futur système	Iterative Development Business value is delivered incrementally in time-boxed cross-discipline iterations. <table border="1" data-bbox="718 660 1380 949"><thead><tr><th></th><th>Inception</th><th colspan="2">Elaboration</th><th colspan="4">Construction</th><th colspan="2">Transition</th></tr><tr><th></th><th>I1</th><th>E1</th><th>E2</th><th>C1</th><th>C2</th><th>C3</th><th>C4</th><th>T1</th><th>T2</th></tr></thead><tbody><tr><td>Business Modeling</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td></tr><tr><td>Requirements</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td></tr><tr><td>Analysis & Design</td><td>Low</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td></tr><tr><td>Implementation</td><td>Low</td><td>Low</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td></tr><tr><td>Test</td><td>Low</td><td>Low</td><td>Low</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td><td>High</td></tr><tr><td>Deployment</td><td>Low</td><td>Low</td><td>Low</td><td>Low</td><td>Low</td><td>Low</td><td>High</td><td>High</td><td>High</td></tr></tbody></table> Time →		Inception	Elaboration		Construction				Transition			I1	E1	E2	C1	C2	C3	C4	T1	T2	Business Modeling	High	High	High	High	High	High	High	High	High	Requirements	High	High	High	High	High	High	High	High	High	Analysis & Design	Low	High	High	High	High	High	High	High	High	Implementation	Low	Low	High	High	High	High	High	High	High	Test	Low	Low	Low	High	High	High	High	High	High	Deployment	Low	Low	Low	Low	Low	Low	High	High	High
	Inception	Elaboration		Construction				Transition																																																																									
	I1	E1	E2	C1	C2	C3	C4	T1	T2																																																																								
Business Modeling	High	High	High	High	High	High	High	High	High																																																																								
Requirements	High	High	High	High	High	High	High	High	High																																																																								
Analysis & Design	Low	High	High	High	High	High	High	High	High																																																																								
Implementation	Low	Low	High	High	High	High	High	High	High																																																																								
Test	Low	Low	Low	High	High	High	High	High	High																																																																								
Deployment	Low	Low	Low	Low	Low	Low	High	High	High																																																																								
Scrum Découpe en itérations appelées sprints	 <pre>graph LR; PB[Product Backlog] --> SB[Sprint Backlog]; SB --> S((Sprint)); S --> LI[Livable incrémental]; S --> RS[Revue du sprint]; RS --> S; S --> RSprint[Rétrospective du sprint]; RSprint --> S; S --> MS((Mêlée)); MS --> S</pre> Product Owner Development Team Scrum Master																																																																																

Tableau 3 : exemples de cycles de développement.

Le choix d'un processus/cycle de développement conditionne le rôle du business analyst et la manière de traiter les exigences. Par exemple, dans un cycle en cascade, le business analyst va produire un ensemble complet et détaillé d'exigences **avant** le développement de la solution. Par contre en Agile, les exigences évoluent et se précisent **pendant** le développement.

2.2 Définitions

Ingénierie des exigences. Nous avons vu que :

- la business analyse recommande des solutions pour un besoin business (problème ou opportunité). Ces solutions impliquent bien souvent le développement de systèmes informatiques ;
- l'ingénierie logicielle balise le travail développement de ces systèmes.

L'ingénierie des exigences se focalise, elle, sur la compréhension des besoins et sur ce qui est attendu du système, à avoir le problème fonctionnel que le système va adresser.

- **Définition 3** : l'ingénierie des exigences est la discipline qui propose des processus de découverte, de compréhension, d'analyse et de formulation du problème à résoudre par le système informatique à construire

Le domaine d'application est par exemple une entreprise, un département ou un métier. Le système informatique va traiter d'un problème posé par le domaine d'application et agir au profit de celui-ci.

- **Définition 4** : le domaine d'application est la partie de monde qui interagit avec le système informatique

L'ingénierie des exigences se concentre sur l'intersection entre le système et le domaine d'application. Elle décrit la partie visible du système selon la perspective de l'utilisateur. Il ne s'agit donc pas ici de décrire comment le système fonctionne, mais bien d'identifier la manière dont il sert le domaine d'application, comme l'illustre la Figure 15.

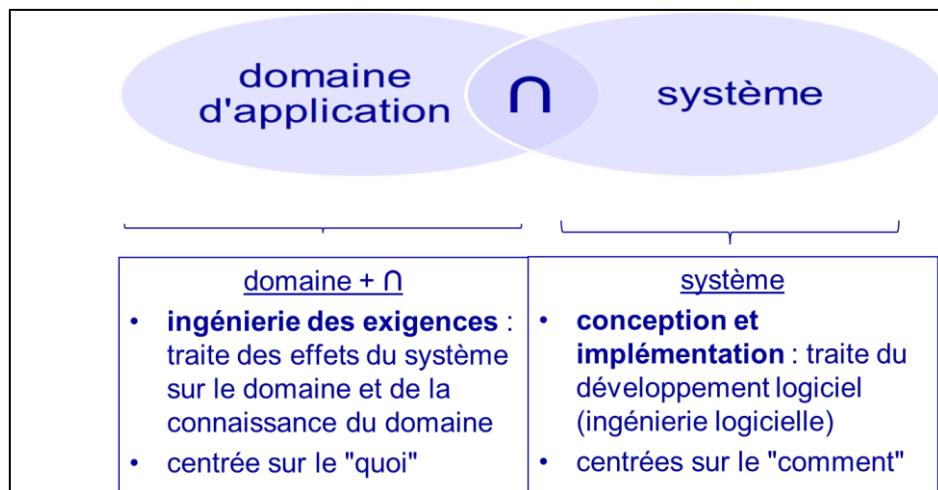


Figure 15 : rôle de l'ingénierie des exigences.

Exigence utilisateur. Le business analyst va décrire le comportement attendu du système du point de vue de l'utilisateur, sans rentrer dans les détails techniques liés à sa réalisation. L'exigence utilisateur est l'unité de description de ce comportement. Voici quelques définitions d'une exigence proposées par les auteurs :

- « énoncé prescriptif sur les *fonctionnalités*, les *qualités* et les *contraintes* de développement d'un logiciel » (van Lamsweerde, 2009) ;
- « représentation *utilisable* d'un besoin. Une exigence est centrée sur le type de *valeur* qui peut être apportée » (International Institute of Business Analysis, (IIBA), 2015) ;
- "(1) A condition or capability needed by a user to solve a problem or achieve an objective. (2) A condition or capability that must be met/possessed by a system or system component to satisfy a contract, standard, specification or other formally imposed documents. (3) A documented representation of a condition or a capability as in (1) or (2) above" (IEEE).

La suite de ces notes se rapportera à la définition suivante :

Définition 5 : une exigence utilisateur est l'énoncé d'une capacité que doit posséder le système à construire pour répondre à un objectif donné, ou exprime une contrainte que le système doit respecter.

Par exemple, les utilisateurs pourraient exiger d'un système qu'il soit capable de calculer les frais de port d'une commande en fonction du pays du client, ou qu'il respecte une contrainte de confidentialité des données personnelles du client.

Notons encore qu'une exigence utilisateur ne doit pas être confondue avec un besoin, qui concerne davantage une attente du business qu'une capacité précise du futur système.

De plus, une exigence utilisateur :

- doit être documentée et utilisable pour guider le développement de la solution. Elle sera donc suffisamment précise et complète pour que l'équipe de développement puis y répondre ;
- concerne le logiciel mais correspond à un objectif utilisateur. Par conséquent, une exigence a une motivation en amont et doit porter une réelle valeur au business en aval ;
- sous-entend une implémentation possible avec les TIC disponibles, moyennant des coûts raisonnables. Le business analyst peut modérer l'appétit parfois insatiable des utilisateurs en termes d'automatisation à tous crins, car il est censé connaître les capacités réelles des TIC.

2.3 Complémentarité des disciplines

Ce n'est pas toujours évident de clairement délimiter différentes disciplines complémentaires dont les historiques, les communautés et les définitions se chevauchent. Voici une tentative de cadrage :

- Business analyse :
 - centrée sur l'accroissement de valeur par le changement ;
 - traite des motivations stratégiques du système (*le pourquoi*) ;
 - cible l'ensemble des responsabilités du business analyst ;
- Ingénierie des exigences :
 - centrée sur la définition et la gestion des exigences qui apportent de la valeur au business (*le quoi*) ;
 - détaille les activités de la business analyse liées aux traitements des exigences ;
 - cible le cœur de métier du business analyst ;
- Ingénierie logicielle :
 - centrée sur le développement d'un logiciel fonctionnel de qualité (*le comment*) ;

- cible les responsabilités de l'équipe de développement et du chef de projet ;
- Gestion de projet :
 - centrée sur les aspects managériaux du projet de développement ;
 - cible les responsabilités du chef de projet.

Un projet de développement implique une collaboration étroite entre le business analyst et l'équipe technique :

- le business analyst va définir les exigences (« le quoi ») répondant à une problème ou une opportunité (« le pourquoi ») ;
- l'équipe IT va alors concevoir, développer et déployer la solution (« le comment ») ;
- le business analyst va valider la solution et accompagner son intégration au niveau du business.

(Paul & Cadle, 2020) montre bien la complémentarité du business analyst et de l'architecte de la solution (équipe IT) dans un projet de changement (Figure 16).

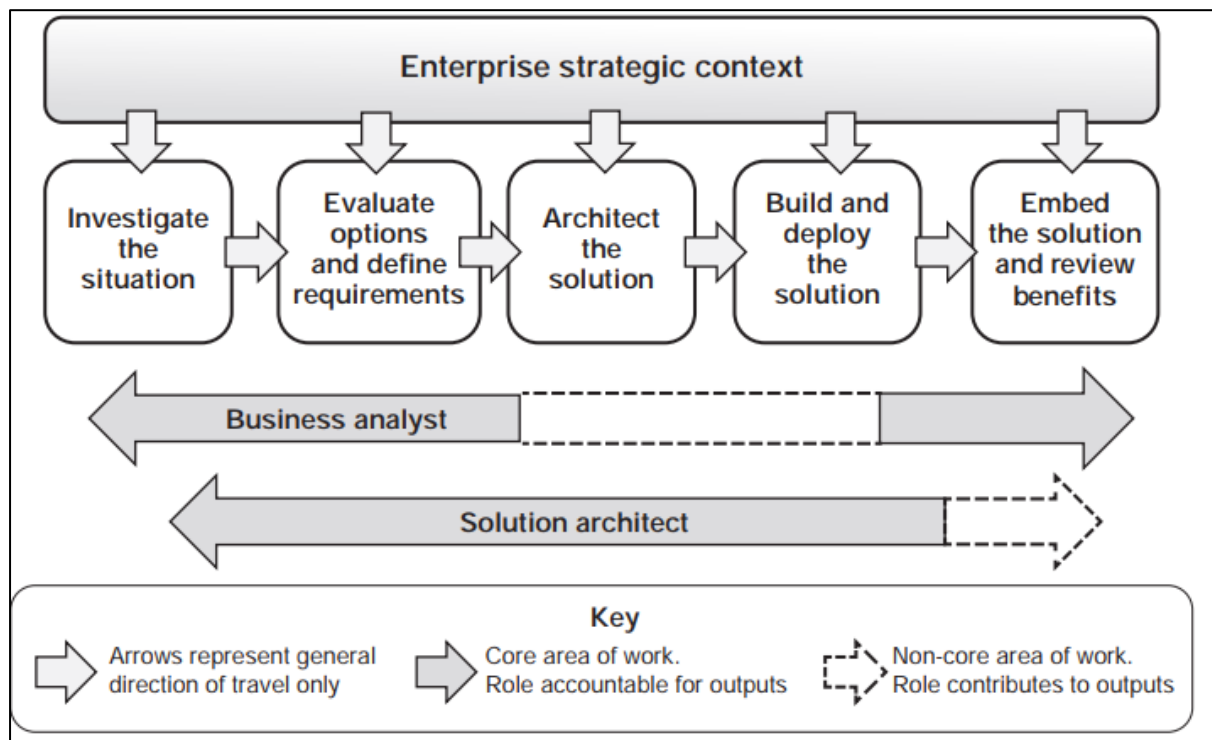


Figure 16 : rôles de business analyst et de solution architect dans un projet de développement (Paul & Cadle, 2020).

2.4 Taxonomie des exigences

Le concept d'exigence connaît plusieurs déclinaisons.

Exigence utilisateur fonctionnelle et non fonctionnelle. Nous savons qu'une exigence utilisateur décrit le comportement attendu système tel que perçu par l'utilisateur. On distingue :

- l'exigence utilisateur fonctionnelle, qui porte sur une fonctionnalité directement liée à l'automatisation des processus d'affaires ;
- l'exigence utilisateur non fonctionnelle, qui porte sur une propriété ou qualité désirée du système dans son ensemble, mais sans lien direct avec les processus d'affaires. Une telle exigence impose une contrainte sur la manière dont le système réalise les fonctionnalités, par exemple en termes de rapidité d'exécution ou de sécurité. La Figure 17 reprend des catégories d'exigences non fonctionnelles.

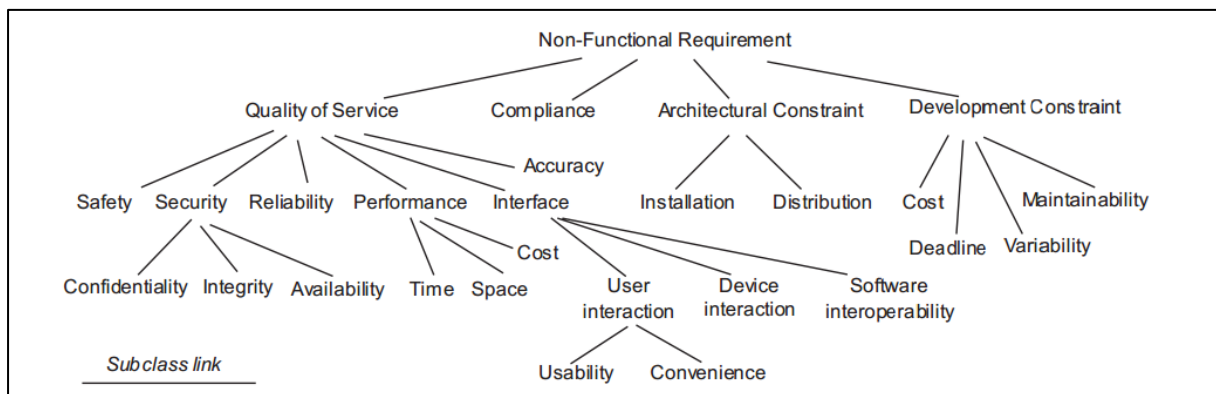


Figure 17 : typologie des exigences non fonctionnelles (van Lamsweerde, 2009)

Selon (Laplante, 2018), l'importance des exigences non fonctionnelles est trop souvent sous-estimée, et leur prise en charge est parfois postposée après la mise en production, i.e., en dehors du processus d'ingénierie des exigences. Cette situation s'explique par la pression mise sur les équipes pour produire du logiciel le plus vite possible, quitte à négliger le non fonctionnel. Les conséquences de telles omissions sont :

- les dépassements de coûts et de délais, par exemple quand un logiciel peu performant doit être revu ou quand le matériel doit être renforcé ;
- des périodes d'interruption, par exemple lors de cyberattaques portant sur un logiciel mal sécurisé ;
- une mauvaise expérience utilisateurs, par exemple lorsque des exigences non fonctionnelles en termes d'ergonomie n'ont gérées.

Exigences du domaine et exigences système. Si une exigence utilisateur spécifie la manière dont le système sert le domaine d'application, et donc décrit l'intersection système-domaine, d'autres types d'exigences ciblent plutôt le domaine d'application ou le système lui-même (Figure 18).

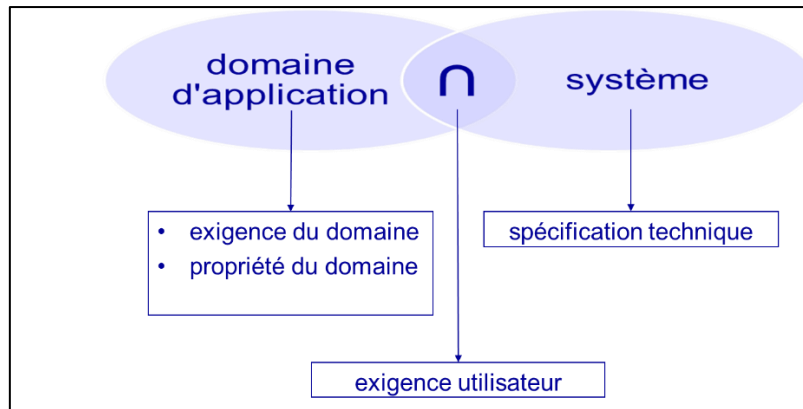


Figure 18 : types d'exigences.

Une *exigence du domaine* est un énoncé à propos des objectifs et des besoins du domaine. Ces exigences peuvent être génériques et partiellement définies, comme une vision, des objectifs stratégiques, etc. Elles ne sont pas directement exploitables pour guider le développement du système, mais elles sont opérationnalisées par les exigences utilisateurs qui, elles, proposeront une solution fonctionnelle opérable. Par exemple, l'exigence du domaine suivante : « la productivité des vendeurs doit augmenter de 10% grâce à une meilleure automatisation des processus de vente » exprime un objectif stratégique de haut niveau qui cadrera les exigences utilisateurs correspondantes, comme « le système doit proposer une tarification dynamique en fonction de la demande ».

Une exigence du domaine est prescriptive, car elle guide la manière dont le système devra être construit. Par contre, une *propriété du domaine* exprime un énoncé à propos du domaine indépendamment du comportement du système. Ces propriétés sont descriptives car elles visent à appréhender le domaine d'application par exemple à travers sa structure, ses valeurs, etc. Si ces propriétés ne décrivent pas les processus, elles impactent les exigences utilisateur et le développement du système. Par exemple, imaginons la propriété suivante : « l'entreprise veille à l'équilibre entre vie professionnelle et vie privée pour chaque collaborateur ». Une telle propriété conditionnera des exigences utilisateur non fonctionnelles comme « le système d'emails n'est pas accessible entre 18h et 8h ».

Finalement, une *spécification technique* est un énoncé prescriptif à propos du fonctionnement interne du système. Ces spécifications relèvent de la solution technique (« comment » plutôt que le « quoi ») mais réalisent les exigences utilisateur avec les TIC. Elles sont souvent exprimées sous forme de schémas techniques qui décrivent les structures internes du logiciel, des structures de données ou des architectures matérielles.

Selon (International Institute of Business Analysis, (IIBA), 2015), les exigences du domaine (ou Business Requirements) motivent un projet de développement (le « pourquoi »), et conditionnent les exigences utilisateurs (Stakeholder Requirements) qui reflètent ce qui est attendu du système (le « quoi »). Les exigences système (Solution Requirement) sont déduites des exigences utilisateur et exprime comment la solution est élaborée (le « comment »).

Finalement, la Figure 19 donne une perspective plus complète sur les niveaux d'abstraction des exigences. Les exigences du domaine déterminent les exigences du business cible du projet. Les parties prenantes désignent tous les acteurs impliqués dans le business. Une partie de ces parties prenantes sont les utilisateurs du futur système, qui expriment des exigences sur le fonctionnement attendu du système. Ces exigences utilisateurs déterminent les exigences techniques du système.

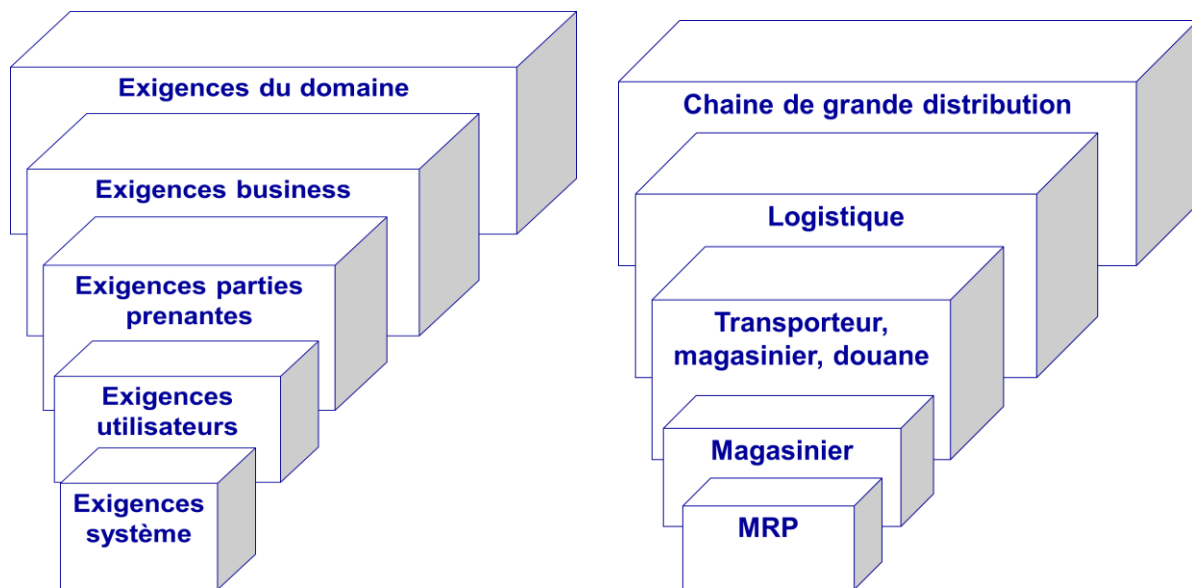


Figure 19 : correspondances entre les niveaux d'exigences, avec un exemple de portée des exigences.

Traçabilité. Les exigences de différents types évoqués ci-dessus sont corrélées entre elles, puisque, finalement, elles expriment toutes les attentes du domaine à différents niveaux d'abstraction (Figure 20). D'ailleurs, on veillera à maintenir des liens des traçabilités entre ces niveaux. La traçabilité facilite la propagation des modifications des exigences d'un niveau vers les autres niveaux. Le plus souvent, c'est une mise à jour des exigences du domaine qui se produit, et qui est répercutée en cascade sur les exigences utilisateur puis sur les spécifications techniques.

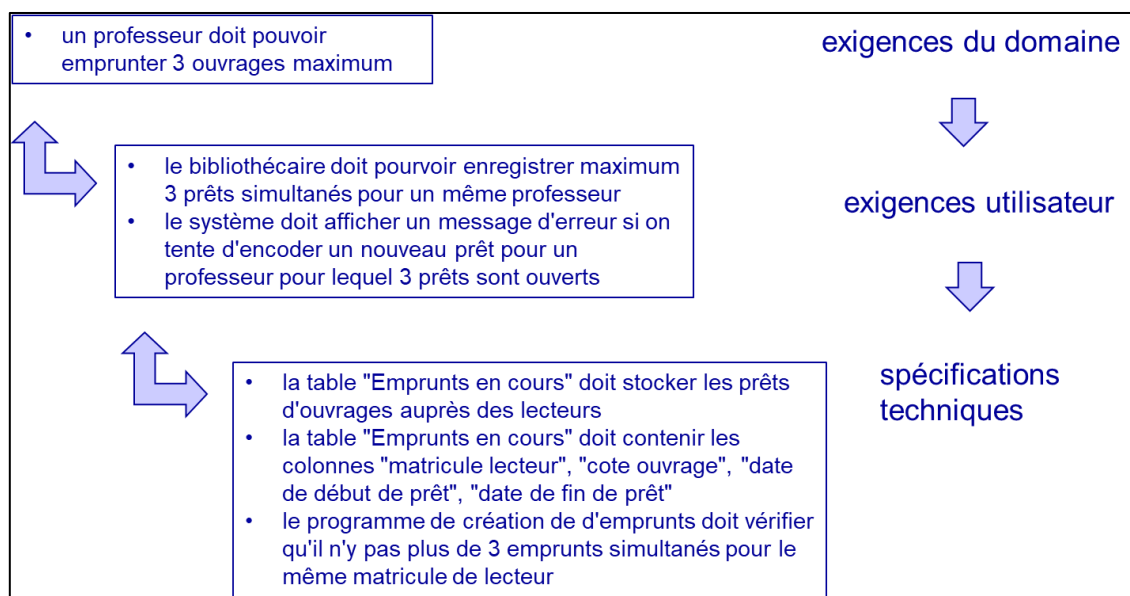


Figure 20 : correspondances entre des exigences de différents niveaux.

2.5 Enjeux et défis de l'ingénierie des exigences

Enjeux. La bonne conduite l'ingénierie des exigences est déterminante pour la réussite d'un projet de développement. La reconnaissance et la formalisation relativement récentes de la discipline démontrent d'ailleurs bien son enjeu, et plusieurs études confirment que le bon traitement des exigences est un facteur clé de succès des projets TIC⁴. Ainsi, plus de 80% des erreurs présentes dans une solution IT sont introduites lors de l'analyse des exigences, contre 10% lors de son développement, alors que, paradoxalement, moins de 12% du temps est alloué au traitement des exigences (Paul & Cadle, 2020).

Identifier et formuler correctement les exigences permet de bien poser le problème à résoudre, et donc de produire une solution adaptée qui satisfait les parties prenantes.

Des manquements au niveau des exigences entraînent de nombreuses difficultés potentielles :

- dépassement des coûts et des délais, voire abandon du projet de développement, dû à des solutions qui ne conviennent pas et qui doivent être retravaillées. Un logiciel qui comporte des erreurs est coûteux à corriger... et bien plus que de définir les exigences qui le déterminent. Certaines études (IBM) affirment que corriger un logiciel en production peut coûter jusqu'à 1000 fois plus que la correction des exigences correspondantes, notamment à cause des effets de bord.
La littérature documente plusieurs exemples d'erreurs logicielles dont les effets ont été importants, comme le décès de patients causés par un logiciel de dispatching d'ambulances de Londres dans les années 90, (https://www.researchgate.net/publication/3792694_Disaster_in_London_The_LAS_case_study), ou la perte d'un satellite suite à un défaut du logiciel de guidage du lanceur Ariane 5 (https://fr.wikipedia.org/wiki/Vol_501_d%27Ariane_5) ;
- logiciel de piètre qualité, dont la couverture fonctionnelle est insuffisante, ou dont certaines facilités sont peu porteuses de valeur ;
- manque d'adhésion des utilisateurs, qui peuvent se sentir mal compris ou mal écoutés, et qui se lassent de tester des versions successives de la solution qui ne les satisfont pas.

Défis. L'ingénierie des exigences recouvre des tâches difficiles et peut encore progresser : selon (Laplante, 2018), seuls 37% des praticiens se déclarent satisfaits des pratiques de l'ingénierie des exigences. Pourtant, il s'agit d'une activité passionnante mais pleine de défis, comme ceux liés à la formulation des exigences par l'utilisateur :

- l'émergence de besoins nouveaux exigeant des solutions novatrices sans réelle expérience antérieure ;
- l'étendue, la complexité et le manque de précision des besoins du business ;
- la nature immatérielle des systèmes d'information qui complique leur description par l'utilisateur ;
- l'impératif de précision et de complétude dans la formulation des exigences dans un environnement changeant et incertain.

Le business analyst doit donc soutenir l'utilisateur pour faire émerger progressivement les exigences à travers des échanges constructifs et durables.

Ensuite, le business analyst doit combiner de nombreuses compétences très diverses, comme les hard skills (compétences analytiques et techniques, comme la modélisation) et les soft skills (compétences relationnelles et organisationnelles, comme la gestion d'équipe, la communication ou la négociation). Il doit aussi maîtriser une large palette de méthodes, de techniques et d'outils... à choisir en fonction de chaque situation. Ainsi, (International Institute of Business Analysis, (IIBA), 2015) répertorie plusieurs dizaines de techniques utiles pour la business analyse.

⁴ Le Standish Group, notamment, relate que 56% des failles dans les exigences sont à la source de 56 % des problèmes de retard ou d'insatisfaction des utilisateurs.

Finalement, un projet de développement nécessite la collaboration harmonieuse entre des parties prenantes parfois très différentes au niveau de leur métier, leur culture, leurs intérêts, leurs compétences, leur jargon, etc. Le business analyst doit mettre en place d'un dialogue constructif entre plusieurs groupes d'acteurs, et notamment entre les spécialistes TIC et les acteurs du business.

3 Cycle de gestion des exigences

Les tâches de développement des exigences peuvent être regroupées dans quatre grandes phases ou groupes d'activités :

- la compréhension du domaine et l'élicitation des exigences, dont l'objectif est d'appréhender le business dans son ensemble et de capter les exigences des utilisateurs ;
- l'analyse des exigences, dont l'objectif est d'améliorer la cohérence des exigences, de les prioriser et de choisir les meilleures alternatives ;
- la spécification des exigences, dont l'objectif est de rédiger les exigences sous la forme la plus appropriée au contexte ;
- la validation des exigences, dont l'objectif est de vérifier la qualité des exigences au niveau de leur formulation, de leur valeur et de leur cohérence.

Comme le montre la Figure 21 (partie gauche), ces quatre phases visent à développer les exigences pour produire un livrable consignant toutes les exigences selon un canevas établi. Ces phases sont interdépendantes car la sortie de l'une peut constituer l'entrée de l'autre. Les enchainements entre phases sont complexes et comportent des itérations et des allers-retours, d'où la spirale dans la figure de gauche. La nature de leurs interdépendances est illustrée par la Figure 21 (partie droite).

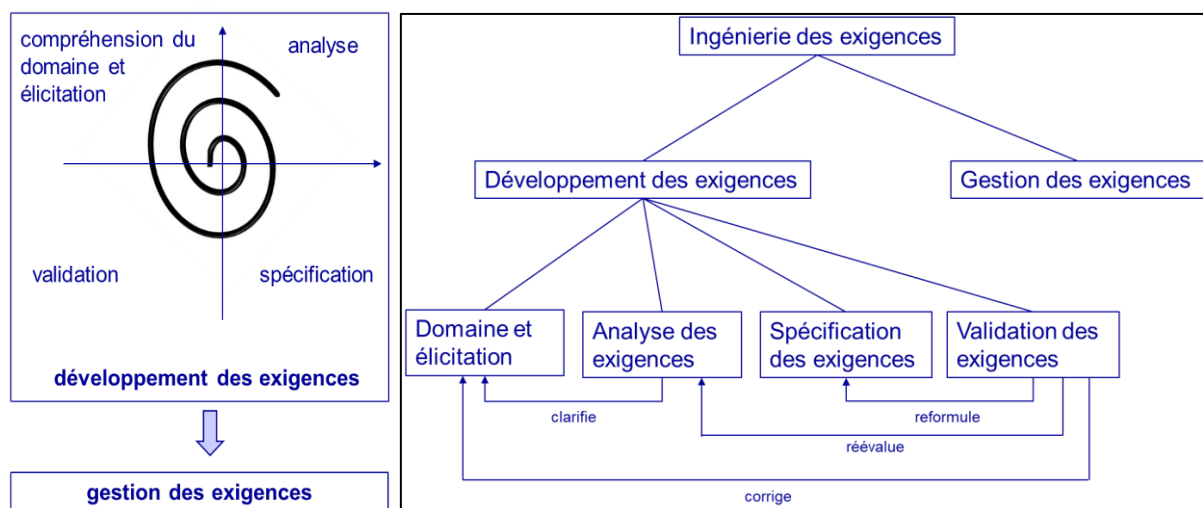


Figure 21 : phases de l'ingénierie des exigences et leurs interdépendances (van Lamsweerde, 2009).

Au-delà de leur développement, les exigences doivent être gérées, i.e., elles doivent être adaptées et complétées à la lumière des changements dans le business, qui sont fréquents !

La suite de cette section passe en revue chacun de ces différentes phases.

3.1 Compréhension du domaine

Le Section 1.5.1 a déjà expliqué qu'une bonne compréhension globale du domaine d'application (Business Context) est importante car les particularités du domaine impactent le travail de business analyse. Selon le BABOK, « Business analysis involves understanding how organizations function to accomplish their purposes, and defining the capabilities an organization requires to provide products and services to external stakeholders. It includes the definition of organizational goals, how those goals connect to specific objectives, determining the

courses of action that an organization has to undertake to achieve those goals and objectives, and defining how the various organizational units and stakeholders within and outside of that organization interact ».

La compréhension du domaine implique donc de cerner l'organisation elle-même, sa stratégie⁵, les parties prenantes au projet, etc.

Plus spécifiquement, la démarche d'optimisation des processus d'affaires (as is -> to be) menée pour activer le changement est cadrée par une bonne compréhension des activités de l'entreprise dans son ensemble : les processus d'affaires se déclinent dans une hiérarchie à trois niveaux :

- le niveau entreprise, qui décrit les activités de haut niveau pour fournir un service ou un produit au client. Ce niveau est modélisé par des schémas de chaînes de valeur ;
- le niveau processus, qui concerne les processus d'affaires détaillés qui répondent à l'occurrence d'un événement, comme l'arrivée d'une commande client. Ces processus sont représentés avec des diagrammes d'activité UML ou des diagrammes BPMN ;
- le niveau acteur, qui décrivent les actions réalisées par un même acteur à un moment donné et à un endroit donné. Les actions sont décrites en texte.

Établir des correspondances entre ces 3 niveaux permet de comprendre la place de chaque processus dans la production de valeur, et de détecter des dysfonctionnements ou des inefficiences à corriger.

L'environnement de l'organisation a aussi un impact sur la business analyse. Par exemple, des réglementations comme le RGPD amènent leurs contraintes et leurs exigences fonctionnelles et non fonctionnelles, comme l'adaptation de la politique de cookies des sites Web, ou la limitation du temps de stockage des données personnelles.

3.1.1 Organisation

Plusieurs approches permettent de comprendre comment les organisations fonctionnent pour atteindre leurs buts. Elles impliquent décrire leur stratégie, leur capacités ou leur structure organisationnelle.

Cette section se limite à présenter deux techniques pour décrire les processus au niveau entreprise : la chaîne de valeur et l'architecture d'entreprise. Le lecteur se tournera vers la littérature des sciences de gestion pour d'autres aspects.

Chaîne de valeur. La notion de chaîne de valeur a été proposée par M. Porter en 1985 et reste très utilisée pour organiser les différents processus par rapport à la production de valeur. La chaîne de valeur catégorise les activités de l'organisation comme suit :

- activités primaires : ces activités sont prises en charge par des processus qui produisent directement de la valeur. On distingue les activités suivantes :
 - logistique entrante, comme les l'enregistrement des clients ou le stockage des matières premières ;
 - opérations, comme la fabrication de produits ;
 - logistique sortante, comme la distribution de produit ou la fourniture de service ;
 - marketing et vente ;
 - service, comme le support avant-vente ou après-vente au client ;
- activité de support : ces activités sont réalisées par des processus qui soutiennent les activités primaires, comme ceux-ci :

⁵ Par exemple, la stratégie de l'entreprise oriente le type de solutions proposées par le business analyst. Prenons le cas d'une entreprise qui adopte une stratégie de prix bas, comme une compagnie aérienne low cost. Les solutions proposées seront centrées sur la réduction de coût et non sur la qualité du service ou la flexibilité, comme une solution massivement automatisée et autonome pour gérer la relation client sans implication humaine de la part de la compagnie.

- infrastructure, par exemple la maintenance des bâtiments ;
- gestion des ressources humaines ;
- développement technologique, par exemple le développement IT ;
- achat, comme la gestion des fournisseurs.

La Figure 22 montre la chaîne de valeur d’une entreprise commerciale. Chaque rectangle représente une activité et les processus qui la réalisent, ce qui matérialise les dépendances entre les niveaux entreprise et processus.

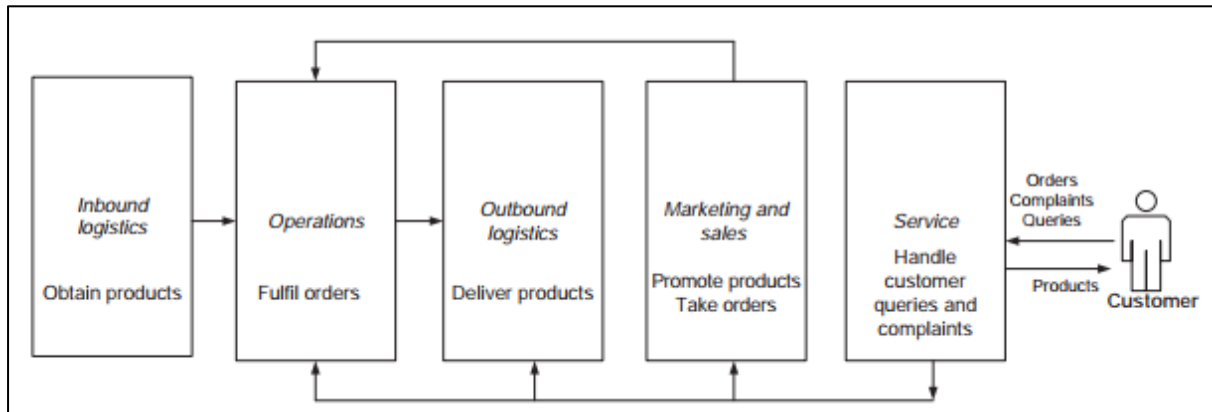


Figure 22 : exemple de chaîne de valeur (Paul & Cadle, 2020).

Architecture d’entreprise. Ce modèle articule les processus d’affaires avec l’infrastructure IT. Le schéma bloc de la Figure 23 montre les composants d’une architecture d’entreprise :

- l’architecture business détaille les capacités, les processus d’affaires, etc. de l’organisation (une capacité est une activité de l’entreprise qui apporte de la valeur) ;
- l’architecture des applications reprend le portefeuille d’application informatiques et leur alignement sur l’architecture business ;
- l’architecture data répertorie toutes les données de l’organisation, leur alignement sur l’architecture business et le stockage en lien avec les applications ;
- l’architecture d’infrastructure décrit le matériel, les logiciels de base, les composants réseau, etc., mais aussi où les applications sont exécutées et où les données sont stockées.

La couche conformité (compliance) identifie les référentiels, normes et règlements auxquels l’organisation doit se conformer, et la couche sécurité reprend les composants qui assurent la sécurité de l’information.

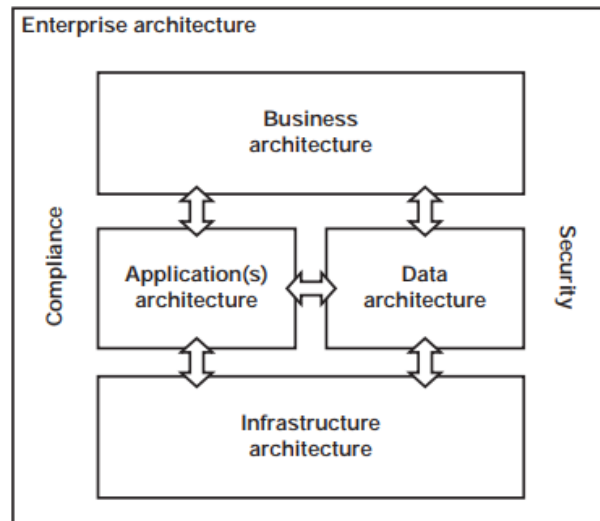


Figure 23 : composants d'une architecture d'entreprise (Paul & Cadle, 2020).

La Section 10.2 montre un exemple concret de standard d'architecture.

3.1.2 Environnement

Les sciences de gestion ont développé de multiple techniques et canevas de descriptions de l'environnement.

Par exemple, la chaîne de valeur centrée initialement sur les processus internes peut être augmentée comme dans la Figure 24, où sont représentés :

- à gauche : les fournisseurs des ressources nécessaires pour réaliser les processus, comme des personnes, des matières premières ou des sources de financement ;
- à droite : les clients qui sont les bénéficiaires finaux des processus d'affaires ;
- en dessous : les concurrents existants, voire les nouveaux entrants potentiels ;
- au-dessus : les facteurs externes susceptibles d'influencer l'organisation. Ces facteurs peuvent être détaillées selon l'approche PESTEL.

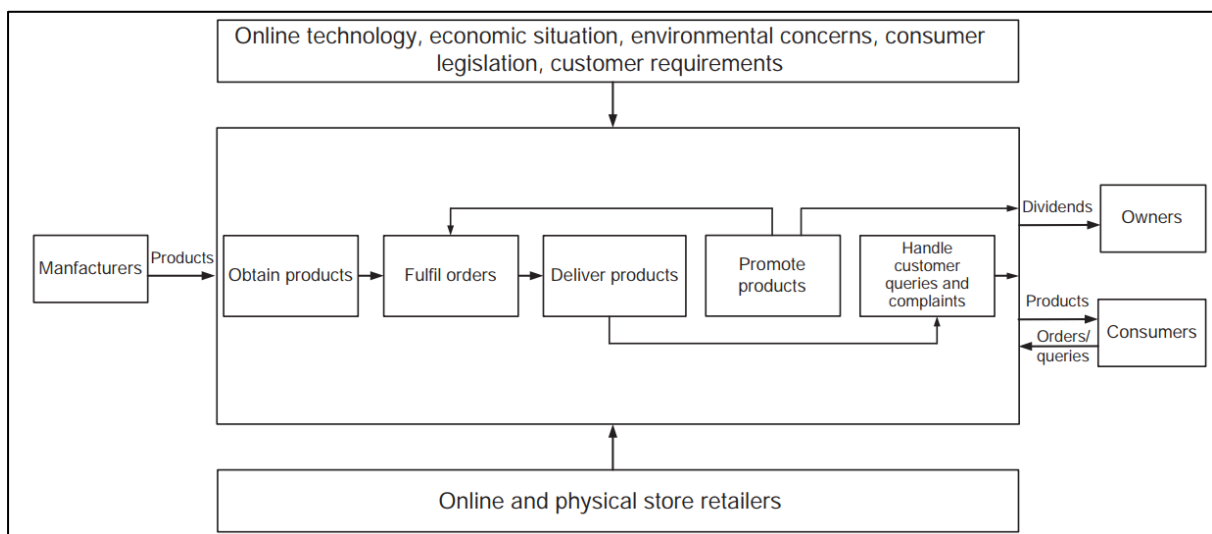


Figure 24 : exemple de chaîne de valeur dans son environnement (Paul & Cadle, 2020).

Le modèle PESTEL identifie les facteurs macro-environnementaux qui peuvent impacter l'organisation. PESTEL est l'acronyme des six catégories de facteurs visés par le modèle : Politique, Économique, Socioculturel, Technologique, Environnemental et Légal.

Par exemple d'analyse PESTEL pour Coca-Cola pourrait ressembler à ceci (<https://buzznessinfo.com/analyse-pestel-exemples/>) :

1. Politique : les réglementations sur la santé publique, telles que les taxes sur les boissons sucrées, peuvent affecter les ventes de Coca-Cola ;
2. Économique : les fluctuations économiques, comme les taux de change et le pouvoir d'achat des consommateurs, peuvent influencer les ventes. En période de récession, les consommateurs peuvent réduire leurs dépenses sur les produits non essentiels ;
3. Socioculturel : les tendances de consommation et les préférences des consommateurs jouent un rôle crucial. Il y a une demande croissante pour des boissons plus saines, ce qui pousse Coca-Cola à diversifier son portefeuille de produits ;
4. Technologique : l'innovation technologique dans les processus de production et de distribution est essentielle. Coca-Cola investit dans des technologies pour améliorer l'efficacité de sa chaîne d'approvisionnement et réduire son empreinte carbone ;
5. Environnemental : les préoccupations environnementales, telles que la gestion des déchets plastiques et la consommation d'eau, sont importantes. Coca-Cola est souvent critiqué pour son utilisation de plastique et travaille sur des initiatives de recyclage et de réduction des déchets ;
6. Légal : les lois et réglementations locales, telles que les normes de sécurité alimentaire et les restrictions publicitaires, peuvent varier d'un marché à l'autre, affectant ainsi les opérations de Coca-Cola.

3.1.3 Acteurs

Répertoire des acteurs. Le business analyst veillera à identifier et à connaître les différents acteurs avec lesquels il faudra instaurer une collaboration étroite. Selon (Laplante, 2018), un acteur, ou une partie prenante, désigne quelqu'un qui a un intérêt dans la réussite du projet de développement. Boehm suggère l'acronyme CRACK pour cerner la notion d'acteur, et qui signifie : "someone who is committed to the project, represents a group of stakeholders, is authorized to make decisions, will collaborate with team members and is knowledgeable".

Le BABOK recense de nombreux rôles possibles assumés par les acteurs, parmi lesquels :

- le business analyst : intermédiaire entre utilisateurs et développeurs ;
- le client : bénéficiaire des produits et services de l'organisation ;
- l'expert du domaine : référent disposant d'une connaissance approfondie de l'organisation et de ses besoins ;
- l'utilisateur final : acteur qui interagit avec le système et qui bénéficiera de sa valeur ;
- l'expert technique : spécialiste des techniques à mettre en œuvre pour la réalisation du système (développeur, administrateur de bases de données, ergonomes, etc.) ;
- le support opérationnel : groupe des responsables de la gestion opérationnelle et de la maintenance du système (help desk, release manager, web master, etc.) ;
- le chef de projet : responsable des différents aspects de la gestion du projet (respect des objectifs, gestion d'équipe, planification, etc.) ;
- le régulateur : responsable de la définition et du respect de standards (réglementation, réglementations métier, etc.) ;
- le sponsor : responsable de la mise à disposition des ressources du projet. Il désigne le chef de projet, puis encadre et soutient le projet ;
- le fournisseur : acteur qui délivre des produits et des services à l'organisation ;
- le testeur : responsable de la vérification de l'atteinte des objectifs par le système et de sa qualité.

Notons qu'un même acteur peut endosser plusieurs rôles, et que la liste ci-dessus n'est pas exhaustive !

Le tout est de bien identifier les acteurs en présence pour ne pas omettre certaines exigences. Pour ce faire, le business analyst peut exploiter différentes sources d'informations, comme un organigramme ou des descriptions de fonctions. Le format de documentation des acteurs n'est pas standardisé. Le Tableau 4 montre un exemple concret de répertoire des acteurs.

coordonnées	rôle	contacts	priorité	types d'exigences	techniques d'éllicitation
Jean Dupond	key user	réunions les 2/2 ; 8/2	élevé	fonctionnel	interview
Anne Durant	responsable sécurité	mail du 14/2, réunion du 4/3	moyenne	sécurité, régulation, données personnelles	échange de textes légaux par mail
Henry Golant	comptable		faible	interface avec le système comptable	échange de spécifications techniques

Tableau 4 : exemple de documentation des acteurs d'un projet.

Sur base des profils repris dans le répertoire des acteurs, le business analyst fixera :

- les modalités de collaboration les plus appropriées avec chacun, comme l'agenda et la localisation des échanges, ou encore les supports à la communication (mail, wiki, etc.). Par exemple, une séance de travail avec la direction sera brève, planifiée en fonction de l'agenda de la direction uniquement, et tenue en présentiel ;
- les besoins en informations, comme la fréquence des échanges ou le niveau de détail de la communication. Par exemple, une communication avec la direction reposera sur un support épuré (« executive summary »).

Une bonne pratique consiste à établir un plan de communication pour décider de ce qu'il faut communiquer, à qui, quand et comment.

Responsabilités des acteurs. Au-delà de ce répertoire, il est utile de documenter qui fait quoi dans le projet avec la technique RACI. Les responsabilités possibles par rapport à une tâche sont :

- “Responsible (R): the persons who will be performing the work on the task.
- Accountable (A): the person who is ultimately held accountable for successful completion of the task and is the decision maker. Only one stakeholder receives this assignment.
- Consulted (C): the stakeholder or stakeholder group who will be asked to provide an opinion or information about the task. This assignment is often provided to the subject matter experts (SMEs).
- Informed (I): a stakeholder or stakeholder group that is kept up to date on the task and notified of its outcome. Informed is different from Consulted as with Informed the communication is one-direction (business analyst to stakeholder) and with Consulted the communication is two-way.” (International Institute of Business Analysis, (IIBA), 2015)

La Figure 25 montre un exemple de matrice RACI pour les responsabilités liées à la tâche « Change Request Process ».

Change Request Process	RACI		
Executive Sponsor		A	
Business Analyst		R	
Project Manager		C	
Developer		C	
Tester		I	
Trainer		I	
Application Architect		C	
Data Modeller		C	
Database Analyst (DBA)		C	
Infrastructure Analyst		C	
Business Architect		R	
Information Architect		C	
Solution Owner		C	
Subject Matter Expert (SME)		C	
Other Stakeholders	R	C	I
	(varies)		

Figure 25 : exemple de responsabilités attribué pour la tâche Change Request Process (International Institute of Business Analysis, (IIBA), 2015).

Attitudes des acteurs. Dans la mesure du possible, il faudra aussi évaluer les attitudes pressenties des acteurs par rapport au projet. Certains se montreront très enthousiastes au projet (utilisateurs « champions ») et devront être impliqués tôt dans le projet, au même titre que les ceux qui ont une vue globale des besoins.

Malheureusement, certains acteurs peuvent se montrer réticents au projet quand ils ne perçoivent pas sa valeur attendue. Ils voient d'abord les difficultés plutôt que les solutions, et peuvent même se montrer hostiles au changement. Des actions ad hoc seront souvent nécessaires pour mettre en place une collaboration fructueuse avec ces acteurs et pour gagner leur adhésion qui, rappelons-le, est indispensable au succès du projet.

3.2 Élicitation des exigences

L'élicitation des exigences implique de rechercher, de découvrir et d'identifier les exigences *potentielles* relatives au futur système. Potentielles car les exigences seront ensuite analysées et confirmées par les phases en aval.

3.2.1 Bonnes pratiques de l'élicitation des exigences

Le travail d'élicitation paraît simple, mais il s'avère être plutôt ardu sur le terrain. Il ne faut pas croire que le business analyst « n'a qu'à » récolter des exigences présentes dans l'esprit des utilisateurs (Alexander & Beus-Dukic, 2009). En réalité, l'élicitation prend la forme *d'un apprentissage coopératif* entre le business analyst et les parties prenantes pour faire émerger progressivement les exigences.

Défis de l'élicitation. Nous avons déjà évoqué la difficulté des utilisateurs à imaginer un système immatériel qui reste à construire et à le décrire. D'autres obstacles se dressent sur le chemin de l'élicitation. Notamment :

- l'éparpillement de la connaissance, qui est souvent répartie entre plusieurs parties prenantes et/ou localisée dans plusieurs sources (intranet, rapports d'activités, etc.) ;
- les contradictions entre des exigences énoncées par des parties prenantes différentes ou provenant de sources différentes ;
- les problèmes d'accès à l'information confidentielle ;
- les obstacles à la communication, comme des problèmes de terminologie métier, des biais de communication (écart entre le discours des utilisateurs et la réalité), ou des informations jugées implicites ou évidentes par des utilisateurs qui omettent de les mentionner ;
- des difficultés liées aux parties prenantes elles-mêmes : manque de disponibilité, manque d'adhésion au projet voire résistance au changement, manque de pédagogie, changement dans les équipes, etc.
- l'instabilité des exigences dans un contexte d'affaires changeant.

Pour franchir certains de ces obstacles, le business analyst doit déployer des compétences communicationnelles et devenir le partenaire de confiance des parties prenantes (et surtout pas l'auditeur qui évalue leur travail). Par conséquent :

- il fera preuve d'une grande faculté d'écoute ;
- il adoptera des techniques de collaboration, un langage et un vocabulaire adaptés ;
- il reformulera la connaissance en la structurant et la validera formellement auprès des parties prenantes ;
- il veillera à bien identifier et impliquer toutes les parties prenantes ;
- il motivera sans relâche le projet en cours en rappelant sa valeur attendue.

La littérature prodigue de nombreux conseils pour une élicitation réussie (Laplante, 2018) :

- toujours bien comprendre le domaine d'application dans son ensemble avant l'élicitation ;
- partir du principe que les parties prenantes ne savent pas toujours ce dont elles ont besoin, et qu'elles ne le communiquent pas toujours efficacement ;
- accepter les changements dans les exigences à tout moment ;
- ne jamais faire d'hypothèse sur ce que les utilisateurs souhaitent ;
- éviter de donner davantage aux utilisateurs que ce qu'ils demandent.

Finalement, négocier les exigences avec les parties prenantes fait partie du travail du business analyst, en particulier quand il doit expliquer que telle fonctionnalité est trop coûteuse à développer ou tout simplement impossible à automatiser. Un climat de confiance et un esprit de collaboration facilitent les discussions, mais encore faut-il les instaurer, par exemple en visant des succès tôt dans le projet, même limités (« quick wins »).

Cadrage des exigences. Apporter une solution nouvelle à un besoin (problème ou opportunité) est l'occasion d'induire un progrès en profitant des potentialités des TIC. Les exigences devront porter cette perspective de progrès en intégrant les objectifs *d'amélioration* visés par le futur système et les *opportunités* des TIC les plus avancées.

Les exigences doivent cependant rester réalistes en intégrant les *limites* de l'automatisation, car il est illusoire de vouloir automatiser tous les processus d'affaires dans un contexte quasi permanent de rareté des ressources. De plus, les exigences respecteront les *contraintes* qui restreignent les solutions possibles, comme des réglementations ou autres normes.

Le travail sur les exigences se déroule donc dans un cadre reprenant les objectifs d'amélioration, les opportunités de la solution, ses limites et ses contraintes, comme l'illustre le Tableau 5 avec l'exemple Galion.

les objectifs d'amélioration	- disposer d'un site Web d'information sur l'entreprise et ses produits - ouvrir le système de prise de commandes au client final en proposant un e-shop - offrir des facilités logistiques pour le client comme le suivi de commande
les opportunités (en réponse à l'objectif de développement du volume d'affaires)	- automatisation de la vente - meilleur service au client (disponibilité 24/7, etc.) - présence sur le Web (information sur les produits et l'entreprise)
les limites de l'automatisation	la préparation de commande reste manuelle
les contraintes	certaines produits fragiles ne peuvent pas être expédiés au client et doivent être emportés aux points de ventes

Tableau 5 : exemple de cadrage des exigences pour Galion.

3.2.2 Techniques d'éllicitation

Le BABOK (International Institute of Business Analysis, (IIBA), 2015) distingue trois moyens principaux pour éliciter les exigences : l'études de sources existantes, la collaboration avec les parties prenantes et l'expérimentation. Il en existe bien d'autres, plus ou moins utilisées, comme le montre la Figure 26.

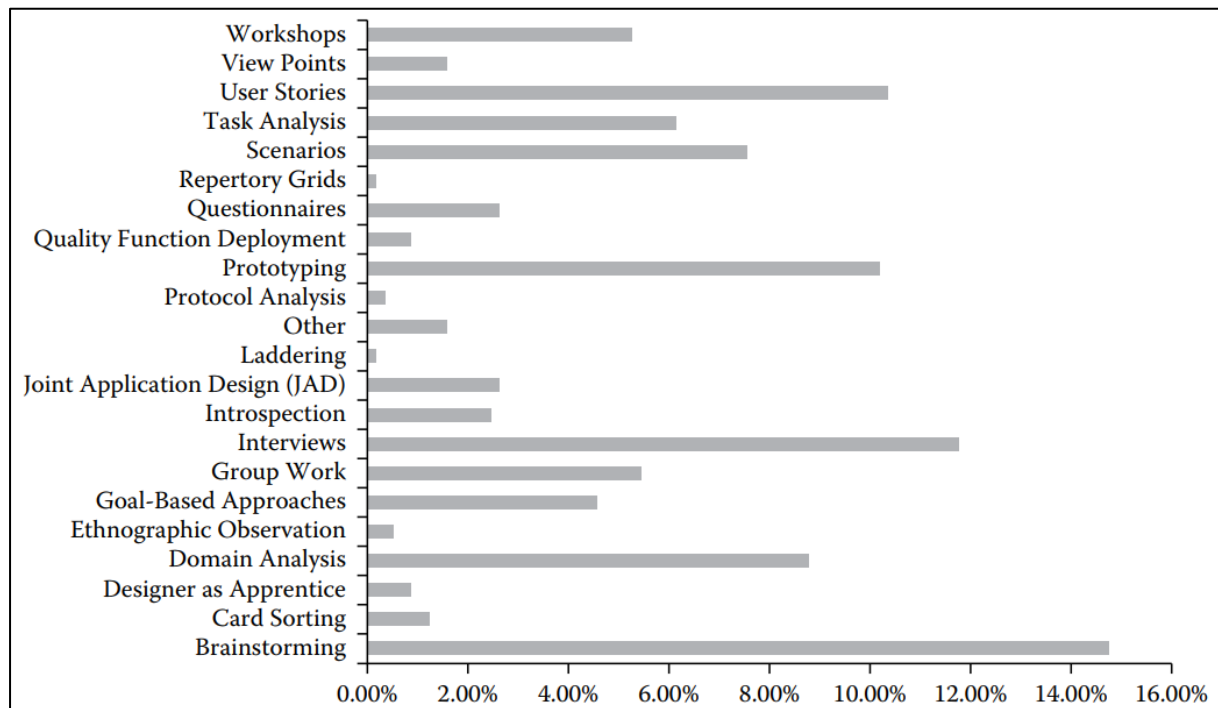


Figure 26 : popularité des techniques d'éllicitation des exigences (Laplante, 2018).

Sources existantes. L'étude des sources consiste à rechercher et à analyser des informations dans des supports existants sans l'intervention des parties prenantes.

Les sources existantes aident le business analyst à se faire une première idée du domaine, ce qui lui sera bien utile pour préparer les échanges futurs. Il évitera ainsi de solliciter les parties prenantes pour des obtenir des informations documentées par ailleurs ou pour disposer d'exemples concrets pour illustrer les discussions.

Ces sources sont très diversifiées. Citons par exemple :

- des descriptions existantes du domaine d'application, comme des rapports d'activités ou des catalogues de produits. Même si ces sources restent partielles et orientées vers un but autre l'énoncé d'exigences pour un système d'information, elles ont le mérite d'être précises et tangibles ;
- des descriptions du système d'information en place, comme des documents d'entrée et de sortie, des copies d'écran, etc. Ces sources sont très détaillées et donnent souvent l'occasion de critiquer l'existant, et de motiver indirectement le projet en cours. Cependant, elles sont obsolètes par essence, redondantes entre elles, et elles peuvent noyer le business analyst dans une masse d'instances de données plutôt que de mettre en évidence les concepts clés.

Collaboration. La collaboration entre les parties prenantes prend la forme d'interviews et workshops. L'interview (Section 3.2.3) est probablement la technique la plus répandue, car elle permet de collecter beaucoup d'informations rapidement sans trop de frais.

Les interviews se tiennent habituellement à deux (l'interviewer et l'interviewé), mais l'interview d'un groupe est aussi possible. Dans le contexte de la business analyse, les interviews visent souvent l'éllicitation des exigences. Ils facilitent le transfert de connaissances et la co-construction de solutions entre l'interviewer et l'interviewé.

Les workshops sont des séances de travail collectives qui poursuivent un but précis et regroupent des parties prenantes pour traiter d'un thème donné, comme la planification du projet, la modélisation, ...ou les exigences. Comparés aux interviews, les workshops peuvent traiter de questions plus ciblées mais plus diversifiées, i.e., au-delà de l'éllicitation des exigences.

Par exemple, un workshop pourrait avoir comme but :

- l'obtention d'un consensus sur un processus donné pour lequel les parties prenantes ont des vues divergentes ;
- la revue d'exigences pour améliorer leur qualité ;
- la génération de nouvelles idées sur des produits ou des processus.

Les workshops sont coordonnés par un facilitateur qui s'assure du respect de l'agenda, de la distribution du temps de parole entre les participants, de la gestion des conflits, du reporting, etc. A priori, le facilitateur a donc un rôle organisationnel, mais n'est pas nécessairement censé avoir un apport sur le fond. Le succès d'un workshop est donc lié à l'expertise du facilitateur, et aux connaissances des participants.

Le Tableau 6 présente les avantages et inconvénients des deux techniques.

	Interview	Workshop
Avantages	– focus sur un utilisateur (parfois plusieurs) et sur ses processus	– stimulation mutuelle des participants – vision plus globale des exigences que lors de l'interview d'un seul acteur – opportunité de résoudre des conflits directement pour arriver rapidement à un consensus
Inconvénients	– difficulté de comparer et d'intégrer les points de vue d'acteurs différents	– coût et difficultés logistiques de mobiliser plusieurs personnes – difficulté d'animer des discussions de groupes – équilibre des temps de parole – réduction de l'impact des conflits interpersonnels, etc.

Tableau 6 : comparaison entre interview et workshop.

L'annexe 10.3 propose des recommandations pratiques pour mener une réunion.

Observation. Cette technique suppose qu'il est plus simple de comprendre le déroulement d'une tâche en l'observant qu'en se la faisant expliquer. Par exemple, montrer comment lier les lacets d'une chaussure est plus simple à montrer qu'à expliquer.

L'observation et l'interview sont parfois combinés, quand le business analyst se met avec l'utilisateur derrière l'écran pour qu'il lui montre comment il effectue ses tâches tout en lui expliquant son travail.

Cette technique permet de capter les non-dits ou les exigences qui paraissent évidentes et acquises par les acteurs. Elle permet de mieux comprendre les problèmes auxquels sont confrontés les acteurs sur le terrain, et de confronter les solutions possibles au terrain.

L'observation est pertinente dans certaines situations spécifiques, mais se cantonne aux processus que l'on peut facilement observer sans interagir avec les acteurs. Elle est souvent assez coûteuse.

De plus, l'observation peut elle-même biaiser le comportement des acteurs, qui préfèrent montrer ce qu'on attend d'eux plutôt que leur manière de travailler au quotidien. Il faut donc expliquer aux acteurs que l'observation ne vise en aucun cas à évaluer leur performance, mais à comprendre les processus pour les améliorer.

Prototypage. Cette technique repose sur la construction de modèles provisoires ou éphémères du futur système en vue de découvrir de nouvelles exigences, en particulier des exigences d'usabilité. Un prototype peut être exécutable ou non, selon qu'il inclut ou non du code. Une maquette d'écran est un exemple de prototype non exécutable. Notons que dans le contexte de conception d'interface utilisateur, on distingue les concepts de wireframe, mockup et prototype, comme détaillé dans le Tableau 7.

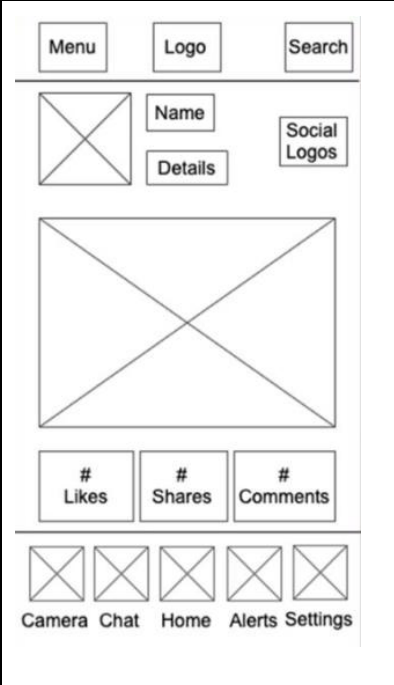
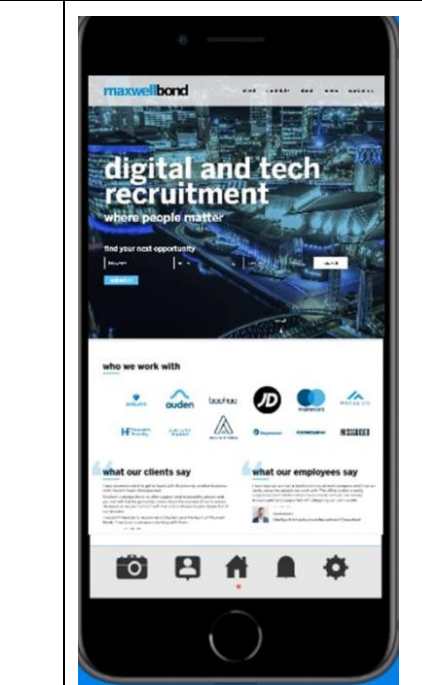
Wireframe : représentation schématique de l'interfac.	Mockup : représentation haute-fidélité de l'interface	Prototype : version partiellement fonctionnelle du système qui permettent de tester l'interactivité et l'usabilité du système
		

Tableau 7 : wireframe, mockup et prototype d'interface (inspiré de Delhaye M., *Stratégie d'intégration de l'approche UX design dans les pratiques du Business Analyst afin d'augmenter la compréhension des besoins utilisateurs, leur satisfaction et l'adoption des produits, mémoire de fin d'étude, ICHEC, 2024*).

Les techniques de développement rapides sont remises à l'avant de la scène avec les approches low/no code. Elles permettent de construire des prototypes à un coût raisonnable et sans une connaissance pointue des outils de développement. D'ailleurs, le business analyst est à même de construire lui-même des prototypes avec ces techniques.

Le prototypage est intéressant quand les exigences sont peu ou mal comprises au début de l'élucation. Se confronter à un prototype tangible permet de gagner de l'expérience et de découvrir/préciser les exigences. Le prototypage repose sur le principe que la compréhension des exigences évolue avec l'expérience, et que le prototypage permet d'acquérir cette expérience.

Attention cependant de passer un prototype inachevé en production sous la pression des délais... De plus, la construction d'un prototype demande un certain effort, pour peut-être ne jamais évoluer vers une version de production.

Technique d'élucation d'exigences non fonctionnelles. Certaines exigences non fonctionnelles émergent souvent pendant l'élucation des exigences fonctionnelles, mais pas systématiquement, et pas toutes. L'élucation des exigences non fonctionnelles nécessite donc des techniques spécifiques complémentaires, comme de questionnaires ciblés

Voici quelques exemples de questions à poser aux utilisateurs pour cerner les exigences non fonctionnelles (généré à partir par Bing Copilot le 6/2/24) :

- « Sécurité :

- Quelles sont les mesures de sécurité requises pour protéger les données sensibles ?
- Quels sont les niveaux d'accès et les autorisations nécessaires pour les utilisateurs ?
- Performance :
 - Quelles sont les attentes en termes de temps de réponse pour les différentes fonctionnalités ?
 - Quelle est la capacité attendue du système en termes d'utilisateurs simultanés ?
 - Quelles sont les exigences en matière de montée en charge (scalabilité) ?
- Fiabilité :
 - Quelles sont les exigences en matière de disponibilité et de temps de disponibilité ?
- Portabilité :
 - Le système doit-il être compatible avec certaines plateformes (par exemple, navigateurs, systèmes d'exploitation) ?
 - Existe-t-il des exigences spécifiques pour le déploiement sur des environnements cloud ou mobiles ?
- Usabilité :
 - Quelles sont les attentes en matière d'ergonomie et d'expérience utilisateur ?
 - Y a-t-il des exigences particulières concernant l'accessibilité ? »

Un guide plus complet est présenté dans l'annexe 10.4.

3.2.3 Zoom sur les interviews

L'interview est probablement la technique la plus répandue et souvent la plus efficace pour éliciter les exigences. Au niveau du contenu, une interview est :

- soit structurée quand les questions sont préparées à l'avance. Une interview structurée permet d'aller à l'essentiel sans discussion périphérique, mais elle limite l'investigation de nouvelles idées ;
- soit ouverte quand l'interview consiste en une discussion informelle sans question fixée au préalable, ce qui permet de traiter de sujets auxquels le business analyst n'a pas pensé, mais avec le risque de passer à côté de questions essentielles au profit de détails secondaires.

Un heureux compromis combinera interview structurée et discussion informelle et ouverte.

Avantages et risques. Les interviews sont souvent privilégiées car elles permettent :

- une discussion approfondie et aboutie sur les exigences ;
- le traitement d'exigences cachées ou implicites dans le chef des acteurs interviewés ;
- une validation en temps réel de la compréhension du problème par le business analyst ;
- une adaptation en temps réel de l'échange en fonction de l'interlocuteur et de son propos. Par exemple, la discussion peut s'engager sur une fonctionnalité que le business analyst n'avait prévu d'aborder si celle-ci s'avère critique ;
- la résolution conjointe de problèmes et l'élaboration de solutions adaptées : l'utilisateur fait souvent part de difficultés dans l'existant (as is). Le business travaille alors avec lui pour leur résolution (to be, gap analysis) ;
- la mise en place d'une relation de confiance déterminante pour l'adhésion de l'utilisateur au projet, qui se sentira écouté, concerné et impliqué dans le développement.

Malgré ces avantages, une interview n'est pas sans risque :

- les interviews peuvent s'avérer coûteuses et chronophages, voir répétitives quand on échange avec plusieurs acteurs dont les exigences se recouvrent ;

- des divergences peuvent apparaître lors d'interviews d'acteurs différents, si leurs points de vue sur le système sont partiels et divergents. Dans ce cas, un travail de consolidation de l'information et de résolution de conflits sera nécessaire. En cas de divergences importantes, le business analyst privilégiera la technique des workshops pour les aplanir collectivement ;
- mener une interview demande de l'expérience. Ce n'est pas toujours simple de mettre son interlocuteur en confiance, de respecter le timing, d'intégrer la communication non verbale, etc. ;
- une interview peut être biaisée :
 - par le business analyst, qui pourrait orienter ses questions par rapport à son expérience et son ressenti, sans considérer suffisamment les particularités du domaine d'application, et ainsi orienter les discussions, voire l'interviewé, vers des solutions qui ne répondent pas suffisamment aux besoins ;
 - par l'interviewé, qui orienterait ses réponses par exemple pour plaire à l'interviewer ou à sa hiérarchie, en minimisant ainsi certaines difficultés.

Compétences. Quoiqu'il en soit, l'intervieweur devrait disposer au minimum des compétences clés suivantes (selon <https://www.smartgecko.academy/interview-outil-essentiel-en-business-analyse/>) :

- *l'écoute active* : il faut non seulement entendre, mais aussi comprendre le message complet de son interlocuteur, demander des éclaircissements et des détails, capter la communication non verbale, et confirmer sa bonne compréhension par la reformulation ;
- une *communication claire et structurée*. Cela suppose une bonne diction, un bon niveau de langage, une proactivité dans le dialogue, mais aussi d'éviter les répétitions, les digressions inutiles, etc.
- un *professionnalisme* à tout épreuve pour gagner en crédibilité et en confiance. Cela implique notamment d'éviter les jugements de valeurs ou de personnes, de faire preuve d'objectivité, de respecter les parties prenantes ou encore de veiller à la confidentialité ;
- la *flexibilité*, en s'adaptant au profil de l'interviewé, notamment en adoptant un vocabulaire, des supports de communication et un timing adéquat ;
- un *esprit analytique*, indispensable pour décomposer un problème complexe.

Bonnes pratiques. Dans tous les cas, l'interview doit être soigneusement préparée et encadrée par un processus complet, tel que :

1. la sélection des parties prenantes à interviewer en fonction de l'information cherchée ;
2. la planification des interviews ;
3. la réalisation des interviews avec l'enregistrement des réponses ;
4. la retranscription des réponses dans des rapports d'interview ;
5. la validation ex post et la mise à jour des rapports par les parties prenantes interviewées.

La littérature abonde de recommandations pratiques pour bien mener une interview, comme celles du Tableau 8.

Recommandation	Exemple/détail
– préparer l'interview	– définir l'objectif des interviews et cibler les parties prenantes concernées – se renseigner sur l'interlocuteur, consulter les sources utiles – préparer une liste des questions les plus importantes, et les transmettre éventuellement au préalable – prendre en charge la logistique, en fixant lieu et le mode communication

– mener activement l'interview	– rappeler le but et l'agenda de l'interview, ainsi que le mode d'enregistrement et de diffusion des traces de l'interview – proposer un tour de table pour que chacun se présente et confirme son rôle dans le projet – commencer par adresser les questions les plus générales puis poursuivre avec des discussions plus ciblées – garder les questions ouvertes pour la fin – résumer l'interview quand elle est terminée et préciser la suite à y donner (rédaction et envoi de PV, agenda des prochaines réunions, etc.),
– garder le contrôle de l'interview	– limiter les digressions, – respecter le timing (typiquement 1 h) – couvrir les points prévus
– mettre l'interlocuteur à l'aise et en confiance	– commencer par des questions simples, s'adapter au profil et à la culture de l'interlocuteur, utiliser un vocabulaire accessible – motiver la démarche et rappeler la valeur espérée du projet pour l'interviewé – ne pas interviewer un collaborateur en présence de son supérieur hiérarchique
– se montrer (gentiment) impertinent mais créatif	– demander justifier des exigences (le pourquoi) – dégager des solutions plutôt que des problèmes
– préparer soigneusement les questions	– éviter des questions orientées exprimant une opinion – ex. : pourquoi votre entreprise s'obstine dans un marché peu porteur ? – éviter les questions qui n'ont pas de réponse – ex. : pourquoi un lecteur emprunte-t-il des livres ? – éviter les questions perçues comme stupides par l'interlocuteur, surtout si la réponse peut être facilement trouvée dans des sources connues – ex. : un professeur est-il un lecteur ?
– garder une trace de l'interview	– mener l'interview à deux (un meneur de la discussion et un rapporteur) – modéliser en temps réel avec l'utilisateur (en mode brouillon) – enregistrer l'interview pour une retranscription ultérieure (avec l'autorisation de l'interviewé)

Tableau 8 : échantillon de recommandations pour la conduite d'interview.

3.3 Analyse des exigences

L'élucitation met l'accent sur la récolte d'exigences, mais celles-ci sont assez brutes dans un premier temps. L'analyse des exigences complète l'élucitation en :

- résolvant les éventuels conflits entre les exigences ;
- évaluant les risques liés aux exigences ;
- choisissant la meilleure exigence parmi plusieurs alternatives possibles ;
- établissant des priorités entre les exigences.

Ce travail d'analyse vise à produire des exigences plus cohérentes et priorisées.

Résolution de conflits. Les conflits entre les exigences résultent souvent de formulations différentes des mêmes attentes par différentes parties prenantes. Les types de conflits les plus courants sont :

- les problèmes de *terminologie*, quand un même concept est désigné différemment ; ou inversement, quand un même terme recouvre plusieurs concepts. Par exemple, le client et le patient au sein d'un hôpital font référence à une même personne, et un cours peut être utilisé pour désigner une activité d'apprentissage ou une unité d'enseignement dans le contexte de l'enseignement supérieur belge francophone. Définir formellement les termes utilisés dans un glossaire permet de résoudre les problèmes de terminologie ;
- les *inconsistances*, quand il est impossible de satisfaire simultanément deux exigences ou plus. Par exemple, une exigence de facilité d'utilisation d'un système de paiement n'est pas compatible avec l'exigence de haute sécurité du même système ;
- les *divergences*, quand il est impossible de satisfaire simultanément deux exigences ou plus sous certaines conditions. Il s'agit d'une variation plus faible de l'inconsistance, mais plus difficile à identifier. Par exemple, l'exigence :

« un lecteur doit rendre un ouvrage après 2 semaines maximum »

sera divergente avec l'exigence :

« un lecteur rend un ouvrage dès qu'il n'en a plus besoin »

si un lecteur a besoin d'un ouvrage pendant plus de 2 semaines.

Les inconsistances et les divergences se résolvent par des discussions et des négociations entre les acteurs concernés.

Mesure du risque. Certaines exigences sont plus exposées aux risques que d'autres. Les facteurs de risques d'une exigence sont par exemple une mauvaise correspondance aux besoins des utilisateurs, ou la mise en œuvre de technologies complexes ou mal maîtrisées.

Définition 6 : un risque lié à une exigence est un facteur incertain dont la survenance peut impacter négativement l'objectif du futur système.

Un risque est caractérisé par une vraisemblance et un impact. Des contremesures peuvent être activées pour contrôler les risques, i.e., réduire sa vraisemblance et/ou son impact. Par exemple, pour mitiger un risque de faible correspondance aux besoins, une validation systématique par les utilisateurs peut être réalisée.

Le lecteur se référera à la littérature sur la gestion des risques pour davantage de détails.

Choix entre des alternatives. Des exigences alternatives diffèrent dans les moyens utilisés pour produire les mêmes effets. Les alternatives peuvent être fonction :

- de *variations fonctionnelles* quand différentes options ou modus operandi sont envisagées pour atteindre le même but. Par exemple,
 - un objectif de paiement préalable à l'expédition peut être atteint via un paiement par virement bancaire *ou* via un paiement en ligne ;
 - le calcul du bonus de fin d'année d'un collaborateur peut être réalisé par une IA *ou* par le DRH ;
 - pour prolonger un prêt, un lecteur pourrait envoyer une demande de prolongation au bibliothécaire pour qu'il modifie la date de fin du prêt, *ou* le lecteur pourrait modifier lui-même la date de fin du prêt que le bibliothécaire valide par la suite ;
- de *variations de coûts*. Un cas typique porte sur le choix entre garder l'exécution manuelle d'un processus sans coût de développement mais avec des coûts de main d'œuvre, *ou* automatiser le plus de traitements possibles, avec des coûts de développement et moins de coûts de main d'œuvre.

Plusieurs techniques simples permettent de choisir une alternative sur base d'une argumentation explicite. Ces techniques évaluent les alternatives selon différents critères et offrent un support à une discussion collégiale pour mener cette évaluation.

L'analyse multicritère pondérée évalue des alternatives par rapport à des critères pondérés, comme son nom l'indique ! Chaque alternative obtient un score calculé comme suit :

$$\text{scoreTotal}(\text{alternative}) = \sum (\text{score}(\text{alternative}, \text{critère}) * \text{poids}(\text{critère}))$$

Les échelles de poids et de score sont libres. L'exemple du Tableau 9 compare deux alternatives dans la manière de calculer un bonus de fin d'année d'un collaborateur : une effectuée par une IA, et une autre par le DRH. Ici, les poids des critères sont compris dans une fourchette de 1 à 5, et les évaluations sont dans une fourchette de 1 à 10.

		score des alternatives	
critères	poids	calcul par une IA	calcul par le DRH
coût de mise en œuvre	3	2	8
coût en production	4	8	2
fiabilité	5	5	7
temps de réponse	3	8	3
image de modernité	2	9	2
objectivité	4	7	5
score total		133	100

Tableau 9 : exemple d'analyse multicritère pondérée.

Priorité entre les exigences. Souvent, les exigences ne peuvent pas être toutes traitées car les ressources disponibles d'un projet (personnel, budget, délai) sont limitées par définition. Il faut choisir... et renoncer parfois à mettre en œuvre certaines exigences de faible priorité.

Le calcul de la priorité d'une exigence est fonction de plusieurs facteurs, comme (International Institute of Business Analysis, (IIBA), 2015) :

- sa valeur : mesure de sa contribution à la réalisation d'objectifs ;
- sa pénalité : conséquence de ne pas implémenter l'exigence, comme ignorer une obligation légale ;
- son coût de réalisation ;

- le niveau des risques qui lui sont associés : probabilité que l'exigence ne puisse pas apporter la valeur escomptée à cause de la survenance de risques, comme le risque de faisabilité technique ou le risque de rejet de la solution par l'utilisateur ;
- ses interdépendances avec d'autres exigences dont la réalisation peut être entravée si l'exigence n'est pas réalisée ;
- son urgence temporelle : une exigence peut perdre de sa valeur après une échéance donnée, quitte à relever son niveau de priorité plus tard. Par exemple, une exigence liée à un traitement cyclique peut être prioritaire à un certain moment du cycle uniquement, comme une clôture annuelle des comptes en fin d'exercice comptable ;
- stabilité : une exigence instable peut avoir une priorité plus faible en attendant sa stabilisation.

Plusieurs méthodes évaluent le niveau de priorité d'une exigence sur base soit de sa valeur, soit de son coût, soit des deux. Souvent, les utilisateurs ne sont pas à même d'évaluer le coût d'une exigence mais bien sa valeur, et inversement pour les développeurs.

La méthode *MoSCoW* propose une échelle de priorités sur base de la valeur. Les priorités possibles déterminent le nom acronyme de la méthode :

- **Must have** : exigence incontournable pour le succès du projet ;
- **Should have** : exigence importante mais pas nécessaire à court terme, à considérer plus tard ;
- **Could have** : exigence souhaitable mais pas nécessaire, souvent lié au confort, à considérer plus tard si les ressources le permettent ;
- **Won't have** : exigence n'ayant pas fait l'objet d'un consensus au sein des parties prenantes, et donc non retenue dans le cadre du projet.

La méthode *Planning Poker* vise une estimation collective et itérative de la complexité de réalisation d'exigences, d'où découle un coût estimé. Son déroulement est le suivant (source : WikiPédia) :

1. "les participants s'installent autour d'une table, placés de façon que tout le monde puisse se voir ;
2. le responsable de produit explique à l'équipe un scénario utilisateur ;
3. les participants posent des questions au responsable de produit, discutent du périmètre du scénario, évoquent les conditions de satisfaction qui permettront de le considérer comme "terminé" ;
4. chacun des participants évalue la complexité de ce scénario, choisit la carte qui correspond à son estimation et la dépose, face vers le bas, sur la table devant lui. La carte est choisie dans un jeu de cartes les valeurs correspondent à une suite de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.) qui sous-tend le principe que plus le scénario est gros, moins l'évaluation est précise ;
5. au signal du facilitateur, les cartes sont retournées en même temps ;
6. s'il n'y a pas unanimité, la discussion reprend et les participants qui proposent des valeurs extrêmes justifient leurs choix ;
7. on répète le processus d'estimation jusqu'à l'obtention de l'unanimité."

Finalement, la méthode *AHP* (*Analytic Hierarchy Process*) combine une estimation de coût et de valeur dans un résultat graphique comme celui de la Figure 27.

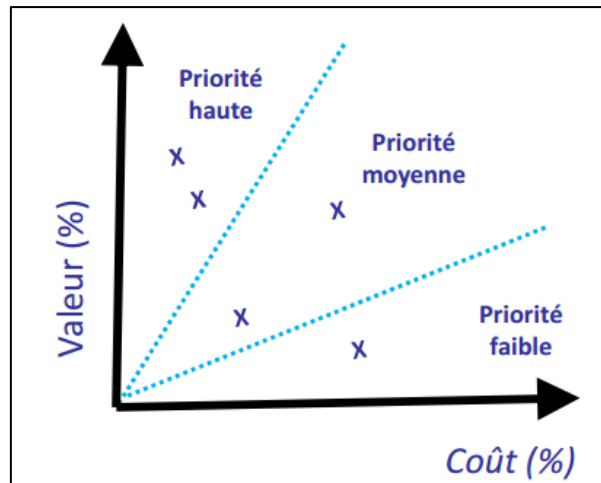


Figure 27 : présentation des priorités avec AHP.

AHP compare des exigences 2 à 2 selon leur valeur et leur coût relatifs selon l'échelle suivante :

- 1 : contribue à même hauteur
- 3 : contribue légèrement plus
- 5 : contribue plus
- 7 : contribue nettement plus
- 9 : contribue beaucoup plus

Les contributions relatives sont estimées par les parties prenantes puis présentées dans une matrice des contributions relatives, où l'élément E_{ij} est l'inverse de l'élément E_{ji} .

La Figure 28 donne l'exemple d'une matrice de contributions relatives de valeur pour les exigences d'un système de planification de réunions.

l'élément E_{ij} est l'inverse de l'élément E_{ji} trouver la meilleure date contribue nettement plus à la valeur que développer un Meeting Assistant					
Valeur	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant
Trouver la meilleure date	1	3	5	9	7
Trouver la meilleure localisation	1/3	1	3	7	7
Paramétrage	1/5	1/3	1	5	3
Multilingue	1/9	1/7	1/5	1	1/3
Meeting Assistant	1/7	1/7	1/3	3	1

Figure 28 : exemple de matrices de contributions relatives (valeur).

Les éléments de la matrice sont ensuite normalisés au niveau des colonnes ($E_{ij}^{normalisé} = E_{ij} / \sum_i E_{ij}$), et la valeur relative de l'exigence en % est calculée comme la moyenne des valeurs de la ligne correspondante, comme l'illustre la Figure 29.

$E_{ij}^{normalisé} = E_{ij} / \sum_i E_{ij}$ $1 / (1 + 1/3 + 1/5 + 1/9 + 1/7) = 0,56$						
Valeur relative	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant	Valeur relative
Trouver la meilleure date	0,56	0,65	0,52	0,36	0,38	0,50
Trouver la meilleure localisation	0,19	0,22	0,31	0,28	0,38	0,28
Paramétrage	0,11	0,07	0,10	0,20	0,16	0,13
Multilingue	0,06	0,03	0,02	0,04	0,02	0,03
Meeting Assistant	0,08	0,03	0,03	0,12	0,05	0,06

Figure 29 : calcul des valeurs relatives.

On procède de même pour estimer les coûts relatifs, puis les exigences sont positionnées dans des zones de priorités selon leur valeur et leur coût relatifs, comme le montrent le Tableau 10 et la Figure 30.

	Exigence	Valeur (%)	Coût (%)
Trouver la meilleure date	E1	49,51%	4,16%
Trouver la meilleure localisation	E2	27,59%	6,79%
Paramétrage	E3	13,05%	15,95%
Multilingue	E4	3,45%	24,49%
Meeting Assistant	E5	6,41%	48,61%

Tableau 10 : récapitulatif des valeurs et des des exigences.

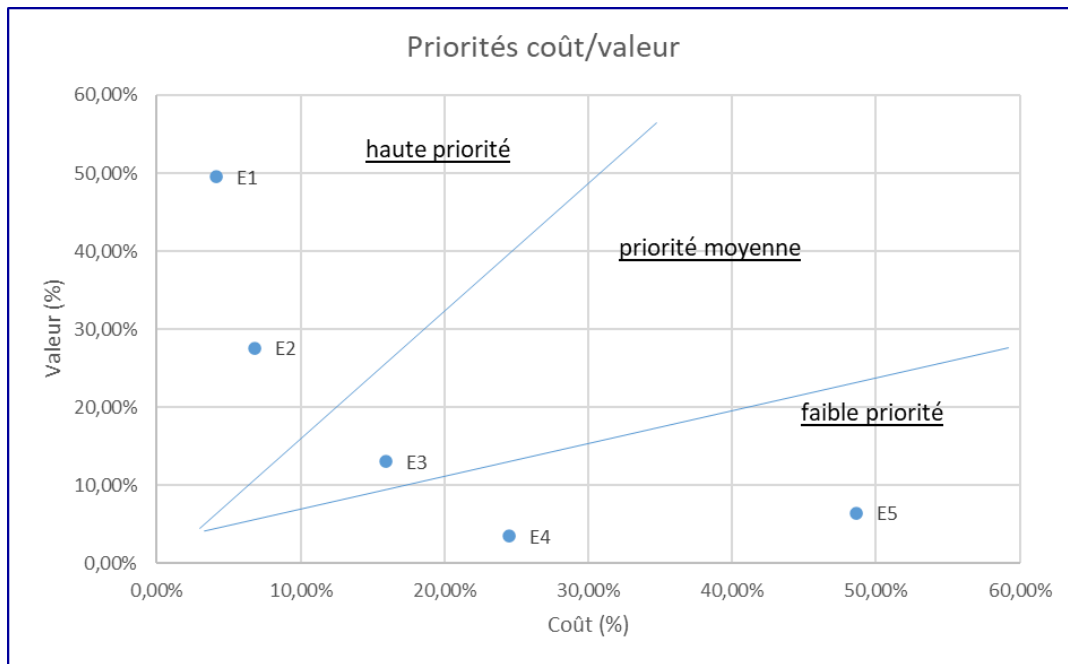


Figure 30 : exemple de priorités avec AHP.

La méthode AHP présente l'avantage de combiner valeur et coût dans l'établissement des priorités⁶. Cependant, AHP devient complexe quand il y a beaucoup d'exigences à prioriser (explosion combinatoire). Dans ce cas, on tentera de regrouper les exigences similaires ou proches dans des ensembles moins nombreux, qui serviront d'unités à prioriser.

En conclusion, les méthodes présentées dans cette section explicitent le raisonnement qui détermine les priorités avec plus ou moins de détails et de formalisme. Même si les estimations restent subjectives dans tous les cas, elles fournissent une bonne base de discussion sur les priorités, auxquelles il faudra encore intégrer d'autres facteurs comme l'interdépendance entre les exigences.

3.4 Représentation des exigences

Le livrable clé de la business analyse est le document qui reprend l'ensemble des exigences détaillées, mais aussi : le problème et objectifs adressés par le futur système, et les informations utiles pour la compréhension du domaine (glossaire, etc.). Ce document, appelé le **System Requirement Specification** (SRS ou "Spécification des Requis du Système"), alimente les phases en aval du développement. Sa structure SRS sera détaillée dans la Section 3.5, et concerne la formulation des exigences à un niveau macro.

Avant cela, cette section passe en revue les techniques de formulation usuelles des exigences au niveau micro, i.e., à l'échelle d'une exigence.

⁶ Des versions plus élaborées de la méthode intègrent des facteurs supplémentaires, comme la qualité, mais elles ne seront pas détaillées ici.

Langages pour les exigences. Les exigences peuvent être formulées avec différents langages, comme illustré dans le Tableau 11 :

- le langage naturel libre, comme le français ;
- le langage naturel structuré, qui consiste à suivre un canevas fixé pour exprimer les exigences. Par exemple, les approches Agile ont popularisé les exigences sous forme de *user stories*, dont une structure possible est : « en tant que <rôle>, je veux <fonctionnalité> afin de <valeur> » ;
- les langages graphiques, qui proposent des symboles, des connecteurs, etc., pour décrire les processus ou les données que le système devra prendre en charge ;
- les langages formels, inspirés des langages logiques.

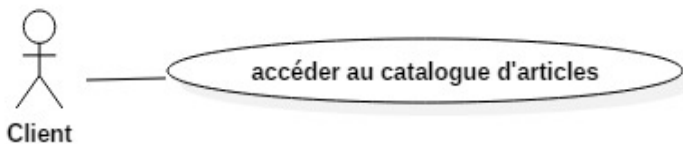
Exemple de langage	Exemple de formulation
Langage naturel libre	Les articles du catalogue sont accessibles aux clients.
Langage naturel structuré	En tant que client, je veux pouvoir accéder au catalogue des articles afin de choisir celui qui répond à mes attentes.
Langage graphique	 <pre> graph LR Client((Client)) --- UC([accéder au catalogue d'articles]) </pre>
Langage formel	• Nom: Réservoir ○ niveau: réel positif ○ capacité: réel positif ○ niveau <= capacité • Opération de remplissage avec succès ○ niveau + quantité <= capacité ○ niveau' = niveau + quantité

Tableau 11 : exemples de représentations des exigences.

Les performances de chaque langage varient selon les critères suivants :

- l'expressivité du langage, à savoir son aptitude à pouvoir formuler une grande variété d'exigences. Par exemple, un langage trop rudimentaire ne permettra pas de spécifier des exigences complexes, un peu comme le vocabulaire d'un jeune enfant qui n'est pas assez étoffé pour formuler des idées abstraites. Le langage naturel est très expressif, alors que les langages graphiques sont plus limités ;
- l'accessibilité, ou la facilité d'accès au langage pour les parties prenantes. Ainsi, certains langages demanderont une formation spécifique, et seront donc perçus comme moins accessibles que le langage naturel qui, lui, ne demande pas de formation ;
- le niveau de formalisme, qui s'exprime à travers des règles qui contraignent l'expression. Ce niveau conditionne l'aptitude du langage à être traité par des outils de génie logiciel, ce qui permet une certaine automatisation dans le traitement des exigences, comme des vérifications de qualité ou la génération automatique de spécifications techniques, voire de code.

Malheureusement, expressivité, accessibilité et formalisme ne sont pas très compatibles. Par exemple, un langage formel est peu accessible. Le business analyst choisira donc le meilleur compromis possible en fonction de la nature du problème à traiter, du profil des acteurs et de la méthodologie de projet utilisée.

La suite de cette section présente les trois premiers langages repris dans le Tableau 12, qui indique leur niveau respectif d'expressivité, d'accessibilité et de formalisme.

	expressivité	accessibilité	formalisme
langage naturel libre	***	***	-
langage naturel structuré	**	***	*
langage graphique	**	**	**
langage formel	**	*	***

Tableau 12 : niveau de qualité des langages.

En pratique, le langage naturel reste le support le plus populaire pour les exigences (60%), alors que les langages graphiques sont utilisés dans 35% des cas. Cependant, de nombreux projets mélangent différents langages. Typiquement, les méthodes Agile élicitent les exigences en langage naturel structuré, puis les détaillent avec des langages graphiques semi formels. En revanche, l'utilisation des langages formels reste marginale (moins de 2%) (Laplante, 2018).

Quel que soit leur format, chaque exigence est documentée typiquement avec les métadonnées suivantes (International Institute of Business Analysis, (IIBA), 2015) :

- identifiant unique ;
- auteur / propriété (utilisateur à la source de l'exigence) ;
- niveau de complexité ;
- priorité relative ;
- risques (liste d'événements incertains qui peuvent impacter la mise en œuvre de l'exigence) ;
- stabilité (niveau de maturité de l'exigence) ;
- statut (par exemple : proposé, accepté, vérifié, postposé, abandonné, réalisé) ;
- urgence (si un délai existe spécifiquement).

La Figure 31 illustre la manière MS Azure Devops documente une user story. Le lecteur se tournera vers des référentiels, comme www.volere.co.uk, pour d'autres templates.

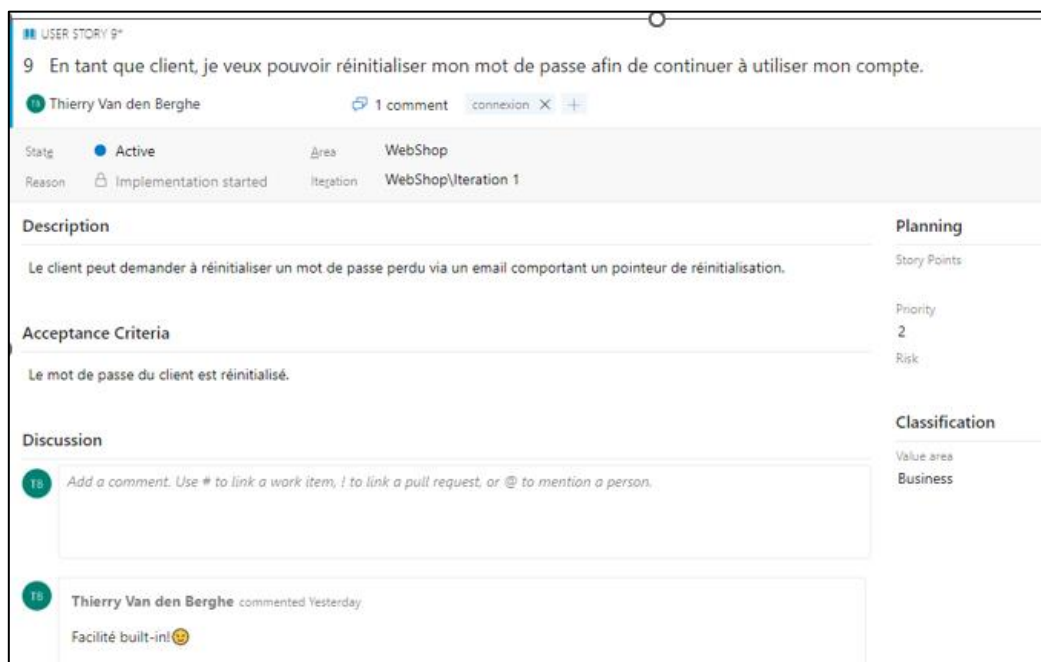


Figure 31 : exemples de métadonnées pour une exigence dans Azure Devops.

3.4.1 Qualité d'une exigence

Quelle que soit leur forme, une exigence doit répondre à des standards de qualités. La norme IEEE 29148 recommande les critères de qualité suivants :

- singularité : une exigence se limite à décrire une seule fonctionnalité ;
- faisabilité : une exigence doit pouvoir être raisonnablement mise en œuvre avec les TIC disponibles et dans le respect des contraintes de coût et de délai ;
- précision : une exigence doit être compréhensible sans la moindre ambiguïté, et ne pas être sujette à interprétation ;
- vérification : la satisfaction d'une exigence doit être mesurable avec des dispositifs clairs, comme des métriques ad hoc ;
- traçabilité : les liens de traçabilités d'une exigence vers d'autres doivent être explicités ;
- priorité : les exigences doivent être assorties de priorités.

Ces qualités génériques seront déclinées par la suite pour les différentes formulations d'exigences.

3.4.2 Exigences en langage naturel

Langage naturel libre. A priori, on serait tenté de spécifier les exigences avec le langage naturel, vu sa grande expressivité et son accessibilité. Cependant, son caractère libre et informel complique un traitement automatique, très pratique avec un grand nombre d'exigences, même si les IA génératives pourraient adresser ce problème. Par ailleurs, la richesse du langage naturel pose certains problèmes :

- ambiguïté : comment garantir qu'une exigence sera comprise de la même manière par toutes les parties prenantes ? Par exemple, lorsqu'on écrit : « le système doit prévoir un mécanisme de remise par quantité et par type de clients », peut-on cumuler les deux types de remises ou non ?
- silence : les utilisateurs supposent parfois que certains éléments sont connus de leurs interlocuteurs et omettent de les spécifier ;
- bruit : les exigences sont censées décrire principalement ce qui touche à l'information et à son traitement, mais elles reprennent parfois des éléments qui n'ont que très peu d'intérêt pour la construction du futur système, comme le client doit laisser son animal de compagnie à l'entrée du magasin ;
- contradictions : elles surgissent surtout lorsque les exigences sont rédigées par des parties prenantes différentes. Ces contradictions doivent être détectées puis aplanies via d'éventuels arbitrages ;
- sur-spécification : un utilisateur déborde parfois du cadre du projet, et communique des exigences concernant l'environnement du système ou des fonctionnalités qui ne doivent pas être prises en charge ;
- redondance : c'est un problème général dans la rédaction de contenus. Si on ne détecte pas qu'une même fonctionnalité est décrite plusieurs fois, on risque de dupliquer inutilement certains des développements ;
- références en avant : elles résultent de mention de notions (concepts, acronymes, etc.) qui seront expliquées en aval de leur mention.

Langage naturel structuré. Une solution pour mitiger certaines limites du langage naturel libre, notamment l'ambiguïté, consiste à imposer des règles de structure simples dans la formulation des exigences, tout en gardant l'expressivité du langage naturel.

Par exemple, le template IEEE830 impose de formuler chaque exigence selon la structure suivante :

- un sujet : système ou utilisateur ;
- un verbe ;
 - doit (« shall »), pour des exigences obligatoires (must have) ;
 - peut (« should »), pour des exigences optionnelles (ou de moindre priorité) ;
- un résultat attendu ;
- un éventuel critère de qualité.

Par exemple : « le système doit permettre au bibliothécaire de créer un nouvel ouvrage avec toutes les données utiles pour la classification ».

On espère ainsi faciliter la compréhension des exigences grâce à une formulation uniforme, même si certains éprouvent des difficultés à se plier à un template unique dans toutes les situations ou se sentent bridés dans leur créativité.

Storytelling. Les exigences telles que discutées ci-dessus peuvent paraître ardues à appréhender. La technique du storytelling consiste à mettre en récit les exigences pour leur donner du contexte et du lien, et ainsi faciliter leur traitement.

Notons que le storytelling est d'abord une méthode de communication populaire notamment en marketing. Mais cette technique peut être appliquée dans l'ingénierie des exigences. On met alors en scène des personnages (souvent fictifs) qui utilisent le système pour atteindre leurs objectifs. Le storytelling est intéressant notamment pour :

- l'élicitation des exigences. Par exemple, l'utilisateur peut décrire le déroulement d'un processus à travers le récit d'un utilisateur qui est impliqué dans ce processus des histoires concrètes. Ou encore, le business analyst peut narrer un scénario qui détaille un cas d'utilisation du système (cf. Section 4.4).
Par exemple : "Imaginez Sarah, une utilisatrice pressée qui souhaite accéder rapidement à son compte. Elle devrait pouvoir se connecter en quelques clics." Cette histoire correspond à une exigence comme "Le système doit permettre aux utilisateurs de se connecter", mais elle est susceptible de mieux capter l'attention de l'utilisateur, même si la formulation est assez verbeuse.
- la formulation de cas de test : le système est confronté à un scénario d'utilisation dont la forme s'apparente à une histoire, comme "Alice ouvre l'application, saisit ses informations d'identification, clique sur le bouton de recherche, etc."

Qualité des exigences en langage naturel (structuré ou non). On sait que la qualité et la clarté des exigences est une préoccupation clé des business analysts. Ils veilleront donc à éviter les pièges suivants dans la formulation des exigences :

- ambiguïtés
 - énoncés soumis à interprétation
 - par exemple, l'exigence « en tant qu'utilisateur, je veux une fonctionnalité de recherche. » est sujette à interprétation au niveau du rôle et de la fonctionnalité. Elle pourrait être reformulée comme suit : « en tant qu'acheteur en ligne, je souhaite pouvoir rechercher des produits par nom ou catégorie afin de trouver rapidement ce que je cherche »
- exigences multiples
 - on recommande une seule exigence par énoncé
 - l'utilisation de termes comme « et », « ou » et « ainsi que » est un signe de mélange d'exigences
 - par exemple, l'exigence « un client peut ajouter un article à son panier ou le supprimer » sera décomposée en deux exigences distinctes
- clauses échappatoires
 - énoncés couvrant plusieurs cas de figure

- l'utilisation de termes comme « si », « quand » ou « excepté » est un signe d'échappement. On tentera alors décomposer les cas dans des plusieurs exigences
- par exemple, l'exigence « un étudiant doit se présenter à l'examen excepté en cas de maladie », donnera : (1) « un étudiant doit se présenter à l'examen » et (2) « un étudiant malade doit justifier son absence à un examen »
- énoncés liés aux exigences techniques
 - relève des phases en aval
 - l'utilisation de termes techniques ou de références au matériel sont des signes de technicité excessive
 - ex. : les données comptables doivent être stockées dans le cloud
- spéculations et suggestions
 - exigences qui pourraient être demandées sans certitude
 - l'utilisation de termes comme « peut-être », « pourrait », « normalement » ou « souvent » sont des signes d'incertitude. Si une incertitude existe pour une exigence, on peut toujours caractériser son degré de volatilité dans ses métadonnées plutôt que dans sa formulation
 - ex. : normalement, un client doit régler sa facture avant expédition
- manque de précision, expression de résultats ou de critères de qualité difficilement mesurables
 - l'utilisation de termes comme « etc. », « ainsi de suite » ou « environ » est un indice d'ambiguïté potentielle
 - ex. : un employé souffrant doit prévenir son employeur *au plus vite*
 - ex. : un étudiant doit valider *un minimum de* cours pour réussir une année d'étude

3.4.3 Zoom sur les user stories

Une user story (US), ou récit utilisateur, est une exigence exprimée en langage naturel semi structuré. Le canevas associé étant fort répandu dans la pratique, les US méritent donc une section dédiée.

Définition 7 : une user story (US) est une description courte et simple d'une facilité exprimée du point de vue de l'utilisateur et qui apporte de la valeur à l'utilisateur. Elle est formulée selon le canevas de base : en tant que <rôle>, je veux <fonctionnalité> afin de <valeur>.

Par exemple :

- en tant qu'utilisateur, je veux pouvoir chercher mes clients par leur nom afin de les retrouver lorsque je reçois un appel de leur part
- en tant qu'administrateur, je veux visualiser les log files des utilisateurs afin de détecter d'éventuels problèmes de sécurité

Les US sont généralement facilement compréhensibles et orientées valeur et utilisateur. Ce sont d'ailleurs soit les utilisateurs, soit le business owner, ou soit le business analyst qui rédigent les US. Cependant, leur niveau de détail reste limité et elles devront être discutées plus en profondeur entre utilisateurs et développeurs. Les méthodes Agile, très friandes des US, préconisent d'ailleurs des échanges face à face pour développer les US.

Le Tableau 13 compare les deux formats de langage naturel structurés que sont les US et le template IEEE830.

User story	Template IEEE830
finalité	
orienté but/exigence domaine, guide le dialogue utilisateurs/informaticiens	orienté exigence utilisateur
niveau de détails	
limité à un relativement petit nombre de US de haut niveau d'abstraction (quelques dizaines) détaillé par la suite entre utilisateurs et informaticiens	reprend tout le détail des exigences parfois de nombreuses exigences de granularité fine
planification / gestion de projet	
conditionne une itération du cycle de développement (2-4 semaines dans le contexte Agile)	n'impose aucune planification

Tableau 13 : US versus template IEEE830.

Détail des US. La littérature suggère plusieurs pistes pour détailler les US, comme le modèle des 3C (Card – Conversation – Confirmation) de R. Jeffries :

- *card* : description écrite et tangible de l'US
- *conversation* : discussions orales pour expliciter les détails de l'US, afin de préciser son apport de valeur, les flux d'actions détaillés, des schéma partiels de données, des spécifications d'interface homme-machine, des contraintes à respecter, etc.
- *confirmation* : critère d'acceptation pour valider l'implémentation de la US. Le format usuel est la structure Given-When-Then (<http://referentiel.institut-agile.fr/gwt.html>) :

Given un contexte

When l'utilisateur effectue certaines actions

Then on doit pouvoir constater telles conséquences

Par exemple, pour l'US suivante : « en tant que client, je souhaite pouvoir ajouter des articles à mon panier d'achat pour pouvoir les acheter ultérieurement »,

le critère d'acceptation serait :

Given je me trouve sur la page du produit

When je clique sur le bouton "Ajouter au panier" pour un produit

Then le produit doit être ajouté à mon panier d'achat

And l'icône du panier dans la barre de navigation doit afficher le nombre d'articles mis à jour

And je dois voir un message de confirmation indiquant l'ajout du produit au panier

La spécification d'un critère d'acceptation est très intéressante pour faciliter la compréhension d'une US en décrivant de manière claire le comportement attendu. Ce principe peut s'étendre à d'autres formes d'exigences et contribue à augmenter la qualité des exigences en mitigeant les risques d'interprétation erronée. D'ailleurs, dans l'approche TDD (Test Driven Development), les critères d'acceptation d'une fonction sont définis avant le codage de cette fonction. La fonction est alors codée pour satisfaire à ces critères, puis testée et complétée itérativement.

Ce modèle des 3C n'est pas le seul disponible. Encore un fois, il reviendra au business analyst de fixer la meilleure manière d'expliciter une US en fonction du profil des parties prenantes et du problème à traiter.

Qualité des US. L'acronyme INVEST désigne les qualités attendues d'une US :

- **Independent** : les US sont indépendantes entre elles et peuvent être assorties de priorités. Cette indépendance permet de planifier la réalisation des différentes US de façon autonome, notamment lors d'itérations courtes préconisées par les méthodes Agile ;
- **Negotiable** : les US captent l'essence d'une exigence, mais pas ses détails, qui peuvent être précisés conjointement entre utilisateurs et développeurs plus tard. Une US peut même être négociée et voire adaptée suite à ces échanges ;
- **Valuable** : les US démontrent un apport de valeur pour les utilisateurs ;
- **Estimable** : les développeurs doivent être capables d'estimer l'effort de développement de chaque US ;
- **Small** : la fonctionnalité sous-jacente est relativement limitée pour être réalisée lors d'une itération courte. La formulation d'un US occupe typiquement 2 lignes ;
- **Testable** : les critères d'acceptation liés à l'US doivent être vérifiables.

Organisation des US. Les US telles quelles sont indépendantes et n'offrent pas de lien ni de perspective à plusieurs niveaux de détail des exigences. Mais les US peuvent être structurées et liées par différents mécanismes comme les initiatives, les épopées, les thèmes, les features, etc., un peu dans l'esprit du storytelling. Malheureusement, les compréhensions et les interprétations de ces mécanismes divergent dans la pratique, mais l'important est d'organiser les US pour mieux les cerner dans leur ensemble.

Dans l'exemple de la Figure 32, une épopée (epic) regroupe plusieurs US relatives à un même système, les thèmes reprennent les US d'un sous-système par ordre de priorité, et les thèmes sont liés par une relation d'ordre au sein de l'épopée.

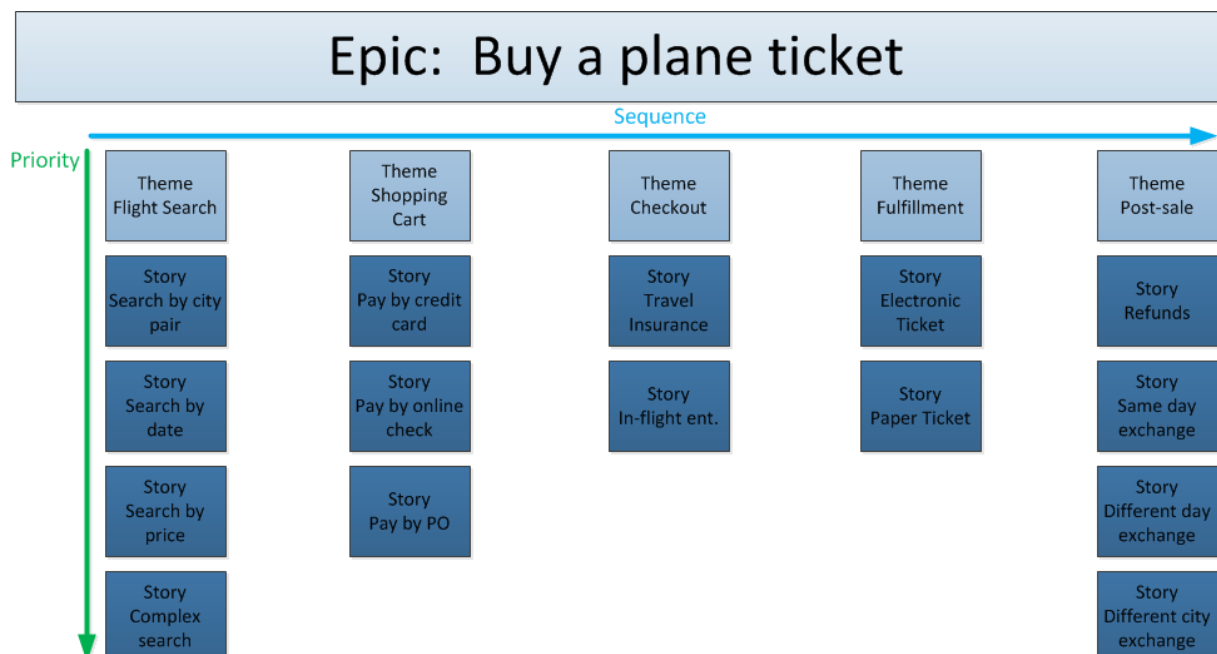


Figure 32 : exemple d'organisation de US (<https://www.slideshare.net/romanpichler/10-tips-for-creating-great-user-stories>)

D'autres mécanismes d'organisation sont disponibles, comme l'illustre le Tableau 14 :

Azure Devops :

"The epics and features that you create should reflect your business focus. As user stories or product backlog items roll up into features, and features roll up into epics—you'll want to name your features and epics with that in mind.

A feature typically represents a shippable component of software. An epic represents a business initiative to be accomplished."

<https://docs.microsoft.com/en-us/azure/devops/boards/backlogs/define-features-epics?view=azure-devops&tabs=agile-process>

The diagram illustrates the hierarchy of work items in Azure DevOps. It is divided into three main sections: Portfolio backlog, Backlog, and Issue & bug tracking. In the Portfolio backlog, an Epic (orange box) points to a Feature (purple box). In the Backlog, a Feature points to a User Story (blue box), which points to a Task (yellow box). A separate box labeled 'Configurable' contains a Bug (red box) and a Task (yellow box), with a dashed arrow pointing from the Bug to the Task. In the Issue & bug tracking section, an Issue (green box) is shown.

Kanban

Les US sont organisées en fonction de leur statut de réalisation et leur degré de priorité

The Kanban board is organized into four columns: À faire, En cours, À tester, and Terminé. Each column contains work items represented by colored boxes with a folded bottom-right corner. The colors indicate the priority of the work items: Must have (dark green), Should have (light green), Could have (yellow), and Won't have (red). The board shows the following distribution of work items:

À faire	En cours	À tester	Terminé
Must have, Must have	Must have, Must have	Must have	Must have
Must have, Should have	Must have		
Should have, Should have			
Could have, Could have			
Won't have, Won't have			

Tableau 14 : exemples d'organisation des US.

User stories non fonctionnelles. Les US captent également des exigences non fonctionnelles. La littérature propose des déclinaisons ad hoc du concept de US en fonction du type d'exigences non fonctionnelles à capter.

Par exemple, les Evil User Stories et les Security User Stories de la Figure 33 expriment les exigences non fonctionnelles liées à la sécurité.

 Les User Stories sont les exigences logicielles centrées sur la valeur exprimées en conversation par les utilisateurs	« En tant que (rôle utilisateur), je veux (activité), afin de (valeur métier) » « En tant qu'utilisateur, je veux renseigner mes identifiants, afin de me connecter à l'application »
 Les Evil User Stories mettent en évidence l'impact métier d'une activité malveillante ciblant le produit	« En tant que (utilisateur malveillant), je veux (activité malveillante), afin de (impact métier) » « En tant qu'attaquant, je veux essayer de deviner le mot de passe d'un utilisateur en envoyant de très nombreuses requêtes d'authentification en parallèle pour me connecter à sa session »
Les Security User Stories décrivent les mesures de sécurité à implémenter pour mitiger les risques.	« En tant que (rôle squad), je veux (activité), afin de (éviter qu'un evil user story se produise) » « En tant que développeur, je veux mettre en place un mécanisme de blocage des comptes utilisateurs après 5 tentatives pour éviter les attaques par bruteforce »

Figure 33 : Evil et Security User Stories (<https://www.riskinsight-wavestone.com/2019/12/cybersecurity-transformation-agile/>, consulté le 25/5/24).

3.4.4 Modélisation graphique des exigences

Exprimer les exigences en langage naturel est apprécié par les utilisateurs finaux car ce média est immédiatement accessible sans formation préalable. Cependant, le langage naturel devient vite trop lourd quand il s'agit d'énoncer les détails des chaînes d'actions d'un processus ou des schémas de données, comme il est illusoire de vouloir décrire en français les plans détaillés d'un bâtiment à construire.

Les langages de modélisation graphique complètent donc avantageusement la boîte à outils du business analyst pour détailler des exigences complexes.

Les avantages des représentations graphiques sont les suivants :

- leur caractère synthétique, ce qui réduit les incohérences, les ambiguïtés et les redondances ;
- leur lisibilité, ce qui facilite la compréhension de systèmes complexes ;
- le niveau intermédiaire de formalisme. Autrement dit, les exigences formulées avec ces langages peuvent faire l'objet de certaines validations automatiques au niveau de leur formulation, de la cohérence entre les exigences, ou de leur complétude.

Par contre :

- les langages de modélisation graphique nécessitent une formation pour les rédacteurs des exigences (business analyst, key user, etc.) et une information de base pour les lecteurs (utilisateurs finaux), ce qui réduit l'accessibilité à ces langages ;
- un langage particulier est souvent centré sur un aspect du système, comme la description d'une fonction ou la représentation des structures de données. Par conséquent, plusieurs représentations graphiques cohérentes sont nécessaires pour décrire toutes les facettes du futur système ;
- un langage est souvent bridé volontairement au niveau de sa syntaxe. Son expressivité est donc limitée, car il vise à représenter les types d'exigences les plus communs et à rester simple d'accès. Les concepteurs de ces langages doivent proposer un compromis entre la richesse du langage et sa facilité d'apprentissage...

Modèles graphiques (Visual Modeling). En informatique, un modèle est une représentation schématique, et donc simplifiée, du système. Par exemple, on établira des modèles pour représenter le déroulement dans le temps d'un processus d'affaires à automatiser, ou pour décrire les données que le système devra prendre en charge.

Définition 8 : un modèle est une représentation abstraite du futur système, dont les facilités clés sont mises en évidence, spécifiées et interconnectées (van Lamsweerde, 2009).

Un modèle est exprimé dans un langage de modélisation qui précise :

- une syntaxe : définition des types d'éléments du modèle et des combinaisons valides entre ces éléments ;
- une sémantique : définition de la signification d'un type d'éléments.

La notion de modèle est assez large, mais dans le domaine de l'ingénierie, un modèle préfigure ce que l'on souhaite construire et joue ainsi le rôle de prototype. Les avantages d'un modèle sont les suivants :

- il facilite le processus de création, car il peut être plus facilement modifié que l'objet réel qu'il représente. Par exemple, un architecte peut aisément mettre à jour le plan d'un bâtiment, ce qui est plus simple que de modifier un bâtiment construit ;
- il permet un échange d'idées et leur évaluation entre les acteurs du développement du système. Il aide à prendre des décisions de conception. Par exemple, l'architecte peut présenter à ses clients plusieurs plans illustrant des options possibles avant un choix définitif ;
- il aide à maîtriser la complexité du système par élimination des détails (abstraction) peu pertinents qui pourraient alourdir la communication et la validation du modèle ;
- le modèle guide la fabrication du système.

Dimensions de la modélisation. La complexité des systèmes informatiques impose des stratégies de décomposition pour faciliter leur spécification. En particulier, plusieurs modèles complémentaires sont requis pour circonscrire les différents aspects d'un logiciel (Figure 34) :

- les modèles de spécification des structures de données : ils identifient des types de données et leurs interrelations. Par exemple, un système devra gérer des données sur des clients, des articles et des commandes. Les modèles les plus répandus sont les diagrammes de classes et les diagrammes entité/association ;
- les modèles de spécification des traitements : ils décrivent la manière dont les données sont créées, transformées ou exploitées. Par exemple, on exposera en détail comment créer une commande ou comment mettre à jour un compte pour un client ;
- les modèles de spécification de la dynamique : ils précisent les séquences d'opérations permises au cours du temps et les événements qui déclenchent ces séquences. Par exemple, un événement Nouvelle commande induit la séquence suivante pour passer une nouvelle commande (1) créer un compte client, (2) choisir les articles et les quantités commandées, (3) calculer les frais de livraison, et (4) calculer le total de la commande ;
- les modèles de spécification d'interface : ils schématisent l'apparence et l'organisation de l'interface homme-machine, par exemple à travers des maquettes d'écrans de saisie (*mock-up*).

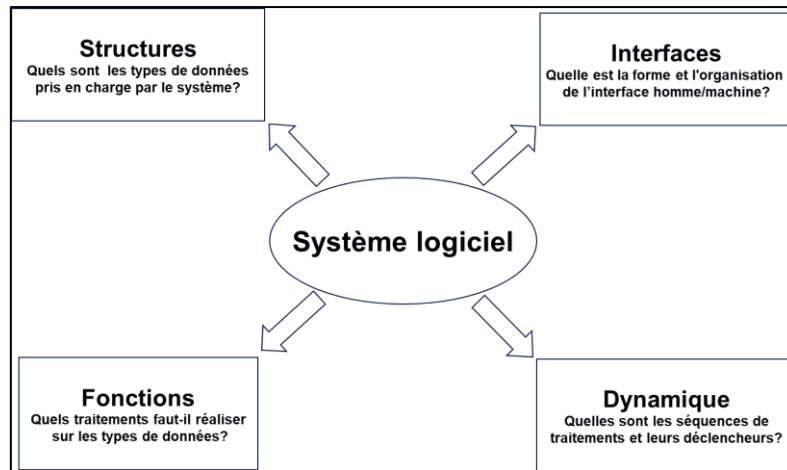


Figure 34 : dimensions du logiciel.

Nous nous limiterons par la suite aux modèles qui expriment les exigences utilisateurs. D'autres modèles ciblés sur les spécifications techniques dépassent le cadre de ces notes.

Unified Modeling Language (UML). UML reprend un ensemble de langages pour modéliser un système informatique selon ses différentes dimensions et pour les différentes phases de son développement.

La version 0.8 d'UML a été publiée en 1995 par la société Rational (qui est aussi à l'origine du UP, cf. Figure 14) avec une intention de rassembler plusieurs techniques de modélisation existantes dans un ensemble cohérent. Depuis lors, UML est devenu un standard de l'industrie informatique défini par l'Object Management Group (OMG, 2015). Notons que UML et UP partagent la même genèse, mais le premier propose des représentations uniquement, et le second un processus de développement. UML et UP sont donc complémentaires.

Par la suite, nous verrons comment les diagrammes de cas d'utilisation d'UML spécifient les exigences de haut niveau du système (Section 0), et comment les diagrammes d'activité d'UML détaillent les processus (Section 5). Le langage entité/association servira, lui, à spécifier les structures de données.

Le langage entité/association ne fait pas partie d'UML, mais est assez proche des diagrammes de classes d'UML. Le lecteur se référera à la littérature spécialisée (Hainaut, 2022) pour une présentation complète et comparée de ces deux langages.

La Figure 35 montre des exemples simplifiés de modèles dans le contexte d'une maison d'édition. Le diagramme de cas d'utilisation (en haut à gauche) montre les principales fonctions attendues d'un système et identifie ses utilisateurs génériques. Le diagramme entité/association (en bas à gauche) donne une idée des données qu'il faudra traiter et de leurs interconnexions. Le diagramme d'activité (à droite) montre le fonctionnement du processus d'une saisie de commande.

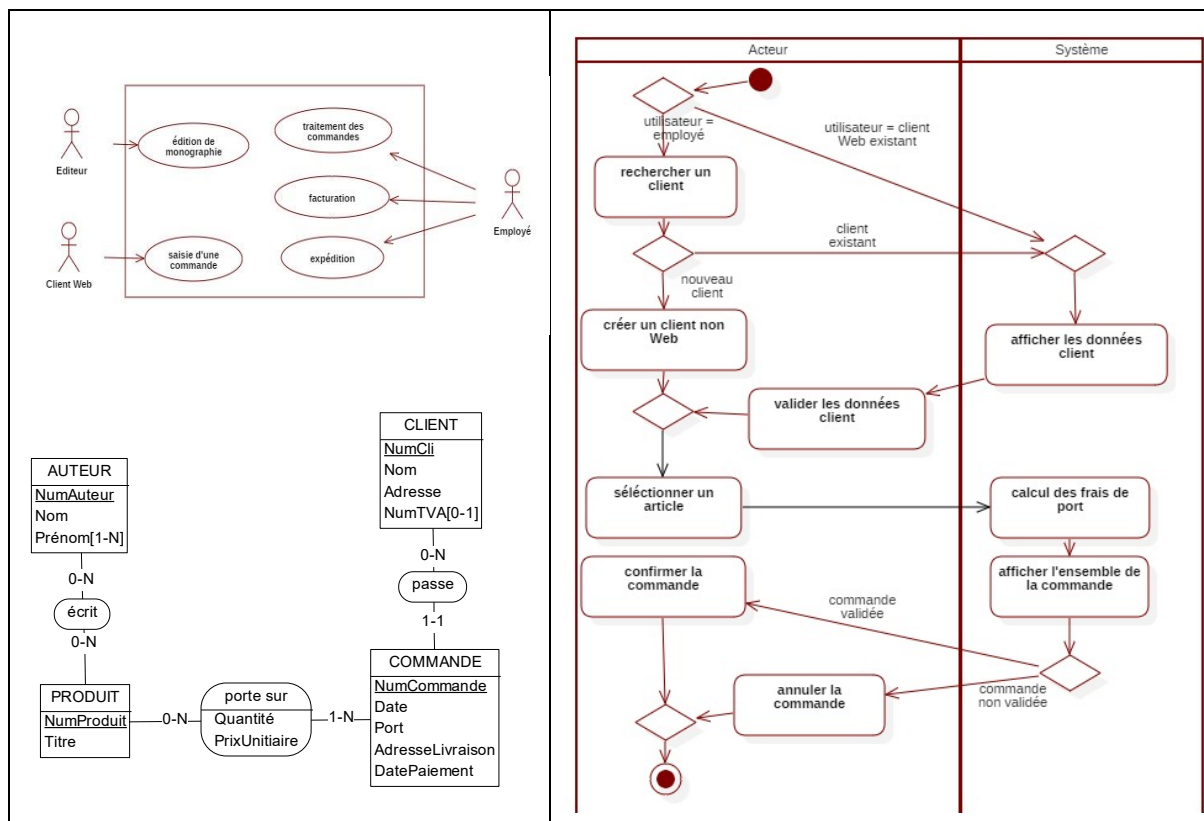


Figure 35 : exemples de diagramme de cas d'utilisation, de diagramme entité/association et de diagramme d'activité.

Qualités attendues d'un modèle. Un modèle doit d'abord être utile aux parties prenantes, par exemple en représentant clairement et fidèlement les états as is et to be.

Ensuite, un modèle doit respecter les règles de construction du langage (notation, syntaxe, etc.) pour garantir sa bonne interprétation.

3.5 Organisation des exigences

3.5.1 SRS (System Requirement Specification)

Au fur et à mesure que les exigences sont élicitées, analysées et formulées rigoureusement, elles s'accumulent et doivent être présentées dans un document structuré et cohérent : le SRS (System Requirement Specification ou Spécification des Requis du Système). Le SRS constitue une référence centrale pour les utilisateurs, l'équipe de développement, les testeurs et autres parties prenantes.

Un SRS poursuit plusieurs objectifs :

- transmettre les exigences à la communauté des développeurs. Le SRS est l'unique référence valide pour délimiter le système et déterminer son achèvement (ce qui n'est pas mentionné dans le SRS ne doit pas être développé) ;
- alimenter la phase de conception pour déduire les spécifications techniques du système, même si des allers-retours entre les phases sont fréquentes car les exigences doivent souvent être précisées ou complétées ;

- structurer les spécifications en sous-ensembles de taille plus maîtrisable qu'une liste unique d'un grand nombre d'exigences. Cette structuration peut alléger le travail de relecture et de validation, en permettant à une partie prenante de se focaliser uniquement sur les exigences qui la concerne. Cette structuration aide également à planifier le projet en différentes itérations ;
- servir de référence lors de la validation du système développé, puisque les tests vont essentiellement vérifier la bonne correspondance du système aux exigences.

La littérature propose plusieurs standards pour organiser un SRS, ce qui facilite l'échange d'information entre les parties prenantes car chacun s'attend à un document dont la structure est connue. Par exemple, si le développement est externalisé auprès d'une société de service en informatique, le SRS joue alors le rôle de cahier des charges, et suivre une structure standard facilite son analyse par les prestataires externes potentiels répondant à un appel d'offres. Et puis le rédacteur du SRS ne doivent pas réinventer la roue quand il suit un standard reconnu.

Naturellement, le choix d'un standard dépend du problème posé et des profils des parties prenantes. Un standard n'est qu'une proposition qui peut être adaptée en fonction des spécificités du projet.

Un standard de SRS définit des règles pour contraindre son organisation. Des exemples de standards sont : Volere (www.volere.co.uk) et IEEE – std 830, dont le Tableau 15 reprend les éléments clés (en plus du chapitre 3 qui constitue le cœur du document). On remarquera les sections dédiées aux exigences non fonctionnelles (section 3.3).

1.	Introduction
1.1	Objectif du document
1.2	Portée du document
1.3	Définitions, acronymes et abréviations
1.4	Références
1.5	Vue d'ensemble
2.	Description générale
2.1	Perspectives du produit
2.1.1	Interfaces système
2.1.2	Interfaces utilisateurs
2.1.3	Interfaces matérielles
2.1.4	Interfaces logicielles
2.1.5	Interfaces de communication
2.2	Fonctions du produit
2.3	Caractéristiques des utilisateurs
2.4	Contraintes
3.	Exigences spécifiques
3.1	Fonctionnalités
3.1.1	<Nom de l'exigence fonctionnelle 1>
3.2	Spécification des cas d'utilisation
3.3	Exigences supplémentaires
3.3.1	Utilisabilité
3.3.2	Fiabilité

	3.3.3	Performance
	3.3.4	Maintenabilité
4.		Contraintes de conception
	4.1	<Nom de la contrainte de conception 1 >
5.		Sécurité
6.		Exigences de documentation utilisateur et d'aide en ligne

Tableau 15 : structure type d'un SRS selon IEEE – std 830.

3.5.2 Validation de la qualité

L'expérience montre qu'en informatique, les défauts des phases hautes du cycle de développement ont des impacts importants sur les phases en amont et sur le livrable final. En effet, des défauts dans des logiciels mis en production ont parfois des conséquences désastreuses et coûteuses, comme la destruction d'une fusée (https://fr.wikipedia.org/wiki/Vol_501_d%27Ariane_5). On comprend donc pourquoi la recherche de qualité est permanente et mérite des tâches dédiées, planifiées et budgétées.

Sans surprise, la qualité globale d'un SRS est fonction de :

- sa complétude : couverture complète des besoins utilisateur, avec suffisamment de détails pour concevoir un système opérationnel. Attention cependant à se limiter au point de vue de l'utilisateur, sans autre considération technique ;
- la qualité des exigences : comme discuté plus haut, les exigences doivent être formulées clairement et précisément. Elles doivent être cohérentes et non conflictuelles, et leur mise en œuvre doit être réaliste dans le cadre des TIC disponibles, des ressources et du délai de réalisation prévus.

Une technique pour évaluer la qualité d'un SRS est la révision par des tiers, i.e., l'inspection du SRS par des acteurs indépendants autres que les rédacteurs, comme discuté dans la Section 8.4. Cette inspection peut être guidée par des listes de critères de qualité, comme celle de la Figure 36. À l'issue de l'inspection, soit le SRS doit être retravaillé, soit simplement amendé, soit il est validé et est alors considéré comme la référence pour les futures phases du projet. Le travail de modification d'un SRS déjà validé rentre dans le champ de la gestion des exigences, avec tout ce que cela implique comme processus ad hoc de révision.

Le groupe chargé de l'évaluation d'un SRS se compose généralement :

- du sponsor du projet, qui veille à l'alignement des exigences avec les objectifs business de l'organisation ;
- des utilisateurs ou de leurs représentants, qui vérifient que les exigences reflètent correctement et clairement leurs besoins ;
- des membres de l'équipe de développement, comme les développeurs qui vérifient la faisabilité des exigences ;
- des testeurs, qui s'assurent que les exigences peuvent être testées. La Section 8 discute plus en détails les techniques de test usuelles.

Functional requirement checklist for Requirement ID#:		
F1	Is written in clear, unambiguous, non-technical language.	
F2	Is reasonable and plausible.	
F3	Contains no imprecise terms such as <i>some, sometimes, often, usually, ordinarily, most, mostly</i> .	
F4	The use of absolute terms such as <i>always, every, all, none, and never</i> is realistic.	
F5	The person or system taking the action is clearly understood. (Watch for passive voice.)	
F6	Enumerated list items are stated or obvious. List endings such as <i>etc., and so forth, and so on</i> have obvious meaning to the reader.	
F7	Can be tested to determine whether the requirement is met.	
F8	Contains no implementation details.	
F9	Is consistent with other requirements in this document and with the stated goals of the system.	
F10	Is not redundant with another requirements in this document.	
F11	For and input requirement: The source, format, and range of input values are specified. Reasonableness and range of values edits are stated.	
F12	For and output requirement: The frequency, destination, sorted order, restrictions (filters), output data format, and calculations are specified.	
Exceptions (explain)		

Figure 36 : exemples de critères de qualité d'un SRS.

Une bonne pratique pour garantir la qualité des livrables d'un projet consiste à mettre en place une démarche d'assurance qualité qui s'installe tout au long du projet (le lecteur se référera à la littérature sur la gestion de projets pour davantage de détails).

L'assurance qualité reprend des tâches de détection, de documentation et d'analyse des défauts, et de correction. Elle inclut la gestion de la qualité à chaque étape d'un projet de développement, de la validation des exigences jusqu'aux tests du système final.

3.5.3 Contexte Agile

Les méthodes Agile préconisent elles aussi de regrouper les exigences dans un répertoire unique : le product backlog. Moins formel que le SRS, il doit néanmoins reprendre des exigences correctement validées (par exemple selon les critères INVEST des user stories).

Cependant, la validation des exigences se déroulent en deux temps pour les méthodes Agile :

- lorsque le product backlog est achevé : les exigences souvent assez générales doivent être vérifiées :
- lors de la gestion du product backlog, i.e., quand les exigences générales sont détaillées. Par exemple, si une user story est raffinée par un diagramme détaillant le processus sous-jacent, ce diagramme sera lui aussi validé avant que l'exigence générale puisse activer un développement IT.

Hormis ces spécificités, les processus et exigences de qualité des exigences dans une contexte Agile (product backlog) sont comparables à celles de processus plus linéaires (SRS).

3.5.4 Cahier des charges

Le SRS peut servir de support à un appel d'offres de service lorsque le développement est confié à un prestataire externe à l'organisation. Dans ce cas, le SRS est augmenté pour produire un cahier des charges qui sera transmis à une liste de prestataires potentiels qui, sur cette base, pourront remettre une offre budgétisée pour le développement du système. La Figure 37 illustre les étapes habituelles d'un appel d'offres.

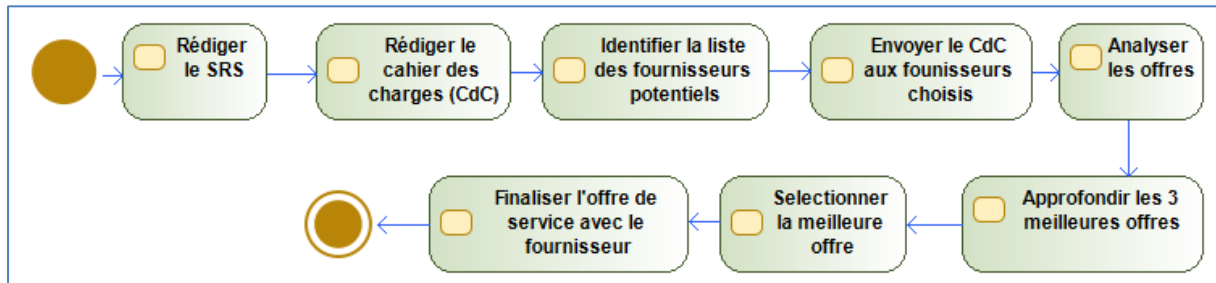


Figure 37 : processus d'appel d'offres.

Habituellement, le cahier des charges contient, en plus des exigences du SRS :

- une présentation détaillée de l'entreprise et de son métier ;
- les modalités de l'appel d'offres, comme des critères d'éligibilité du fournisseur, les échéances à respecter par le fournisseur, les modalités et critères d'évaluation des offres, des clauses contractuelles portant sur les responsabilités, la confidentialité, etc.
- un canevas de réponse, i.e., des rubriques types à compléter par le fournisseur, comme l'exemple du Tableau 16.

Contenu de l'offre

- une brève présentation du fournisseur et de ses compétences ;
- l'architecture du système proposé et une présentation des technologies mises en œuvre ;
- une description du processus de développement pressenti, y compris la gestion des risques et de la qualité ;
- un phasage du projet ;
- une estimation complète des coûts de réalisation du système ;
- les modalités de garantie et de maintenance ;
- la liste des éventuelles références dans des domaines similaires à celui du présent cahier des charges ;
- les CV des membres de l'équipe affectée du projet.

Tableau 16 : exemple de canevas de réponse.

Le fournisseur qui souhaite soumissionner à un appel d'offres suit également un processus défini pour y répondre, comme :

1. l'analyse de faisabilité, qui consiste à évaluer la capacité à répondre à l'appel d'offres sur base des exigences du cahier des charges, des produits disponibles et de l'expérience du domaine d'application ;
2. la prise de décision de remettre une offre ou non en considérant : l'étude de faisabilité, les options stratégiques, la disponibilité des équipes, etc.
3. la rédaction d'une offre, comprenant une réponse technique aux exigences et une estimation des coûts ;
4. la négociation avec le client.

Finalement, voici quelques acronymes usuels liés aux appels d'offres :

- RFI (Request for Information), est une demande d'informations auprès de fournisseurs en amont de l'appel d'offres, dans le cadre d'une première analyse des fournisseurs susceptibles (et capables) de répondre aux besoins ;
- RFQ (Request for Quotation), est une demande limitée à un devis chiffré ;
- RFP (Request for Proposal), est à une demande d'offre suite à un appel d'offres ;
- SLA (Service-level agreement), est un document contractuel qui engage le prestataire à fournir un service qui satisfait à des exigences bien établies à l'avance, comme des délais d'intervention en cas de défaillance.

3.6 Gestion des exigences

Changement des systèmes d'information. Le développement de système informatique est vraiment particulier : contrairement à l'ingénierie des produits matériels, comme une maison ou une voiture, les exigences utilisateurs sont très mouvantes et peuvent évoluer même pendant le développement. Imaginez qu'un propriétaire demande à son architecte de rajouter un étage alors que l'entrepreneur achève les finitions !

En réalité, les organisations évoluent dans un environnement très changeant et très concurrentiel (Figure 38). Il faut donc s'adapter et évoluer en permanence, et les systèmes d'information doivent suivre. Le changement est inhérent aux projets TIC : les systèmes doivent être flexibles (faciles à adapter), et les équipes doivent être agiles (intégrer le changement et adapter rapidement les systèmes).

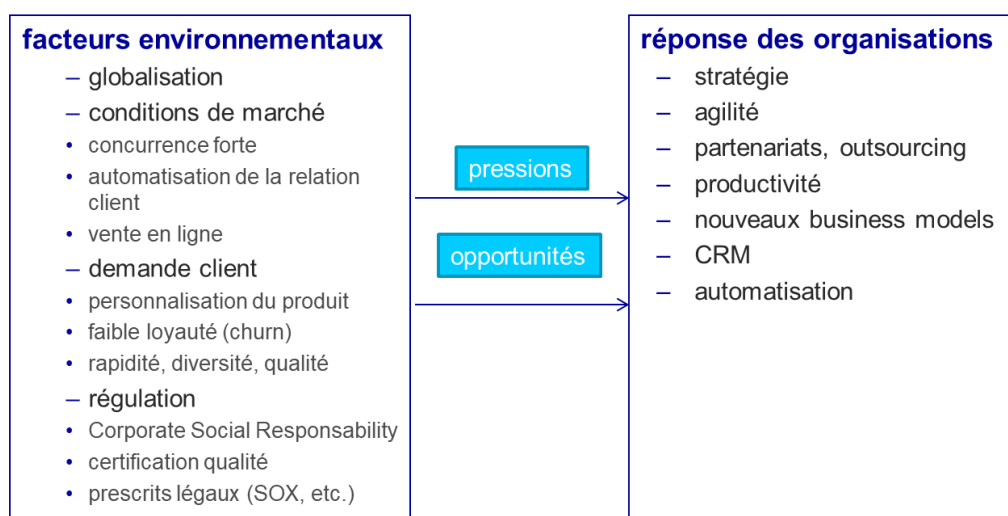


Figure 38 : pressions et opportunités liées à un environnement changeant.

On distingue plusieurs catégories de changements des systèmes d'information :

- optimisation du système :
 - correction : changement pour éliminer un défaut, comme les erreurs de programmation ;
 - amélioration : changement pour perfectionner un système déjà fonctionnel, comme améliorer la navigation dans des sites Web en fonction des usages fréquents ;
- mise à jour du système :
 - adaptation : changement pour répondre aux évolutions des modifications des exigences, comme adapter le calcul de la TVA d'une commande en fonction de nouveaux prescrits légaux ;

- extension : changement pour répondre à des exigences nouvelles, comme la prise en charge d'un nouveau moyen de paiement d'un e-shop, ou l'ajout d'un système de recommandation automatique avec une IA.

Changement des exigences. Une fois les exigences finalisées dans un SRS, il faut gérer le changement continu des exigences. Cette gestion implique de :

- documenter les interdépendances entre les exigences de tous niveaux (cf. la traçabilité ci-après) ;
- répondre aux changements en actualisant les exigences, ce qui implique des tâches supplémentaires d'élicitation, d'analyse, de formulation et d'organisation des exigences ;
- analyser les répercussions des changements d'exigences à tous les niveaux sur base de leurs interdépendances.

La Figure 39 illustre la propagation d'un changement d'exigence utilisateur dans un réseau de spécifications interdépendantes.

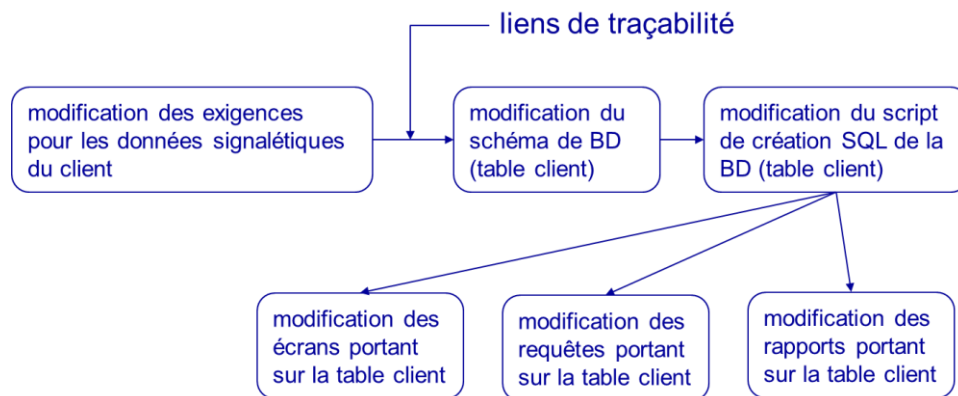


Figure 39 : exemple de propagation d'un changement d'exigences.

Bonnes pratiques. En réalité, gérer le changement n'est pas si simple pour différentes raisons :

- modifier un système a un coût souvent non budgété ;
- les informaticiens ont parfois du mal à modifier un logiciel stable sur lequel ils ont déjà passé beaucoup de temps ;
- les utilisateurs ne sont pas toujours disposés à changer leurs habitudes ;
- etc.

Voici quelques recommandations :

- mettre en place un processus du changement des exigences, comme adopter un cycle de développement adapté (cf. Agile) ou définir des procédures de modification d'exigence, comme (1) documenter la proposition de changement, (2) analyser la proposition de changement, (3) consulter les parties prenantes et (4) décider du changement ;
- instaurer une culture du changement claire et partagée qui engage toutes les parties prenantes, et lutter contre la résistance naturelle au changement ;
- anticiper les réponses aux changements dans les exigences mêmes, et penser en termes de flexibilité très en amont du projet. Par exemple, on pourrait transformer l'exigence suivante :

le président du jury de délibération doit pouvoir valider des crédits d'une UE pour un étudiant qui a obtenu 9/20 à cette UE

en deux exigences plus paramétrables et donc plus flexibles :

le président du jury de délibération doit pouvoir valider les crédits d'une UE pour un étudiant qui a obtenu la cote minimale à cette UE

l'administrateur doit pouvoir paramétrer la cote minimale à obtenir pour valider les crédits d'une UE

Gestion de la traçabilité. Nous avons vu que les exigences sont interdépendantes entre elles. La traçabilité concerne justement la gestion de ces interdépendances. On distingue :

- la traçabilité horizontale, qui trace la progression d'une exigence à travers les différentes phases du cycle de vie du projet, de sa source à sa réalisation et inversement ;
- la traçabilité verticale, qui lie les différents niveaux d'exigences. Une exigence utilisateur sera liée à des objectifs business amont, et à des spécifications techniques aval.

Selon la norme IEEE 830: "An SRS is traceable if the origin of each of its requirements is clear and if it facilitates the referencing of each requirement in future development or enhancement documentation. "

Définition 9 : la gestion de la traçabilité est le processus de définition, d'exploitation et de maintenance des liens de traçabilité entre les exigences.

Les objectifs de la gestion de la traçabilité sont :

- pouvoir identifier l'impact d'un changement d'une exigence sur le reste du système ;
- mieux détecter les inconsistances et manquements, comme des exigences non liées à des spécifications techniques ;
- mieux comprendre la motivation des exigences à travers leurs interdépendances.

Notons finalement que la gestion de la traçabilité a un coût potentiellement important si le niveau de détail est fin, et qu'un éventuel retour vient assez tard dans le projet.

3.7 Estimation des coûts

La question qui fâche dans les projets informatiques est : combien ça va coûter ?

Et très rapidement, le business analyst va être sollicité pour estimer le coût de la solution qu'il imagine, même assez tôt dans le projet alors que les exigences sont à peine formulées. Cependant, il devra faire preuve de la plus grande prudence, voire de retenue, ne fut-ce que pour donner des ordres de grandeur de coût. En effet, plusieurs facteurs compliquent la tâche :

- le coût d'un système ne peut être précisément estimé que s'il est complètement spécifié au niveau fonctionnel et technique. Avant cela, les estimations de coûts sont incertaines, sans parler des changements potentiels dans les exigences qui impactent les coûts. D'ailleurs, une estimation précoce est assortie d'une incertitude d'un facteur 8 (!), comme le montre la Figure 40 ;
- les exigences non fonctionnelles impactent les coûts, parfois très significativement, par exemple quand le système doit être très fiable (aéronautique), très sécurisé (médical) ou très performant (militaire). La mise en œuvre des exigences non fonctionnelles peuvent nécessiter des technologies logicielles et

matérielles sophistiquées et coûteuses, dont les caractéristiques n'émergent qu'assez tardivement dans le projet ;

- les estimations devraient intégrer des facteurs externes liés au contexte du développement, comme la compétence et l'expérience des business analysts et des développeurs, ou la maturité de l'organisation en termes de méthodologie de développement.

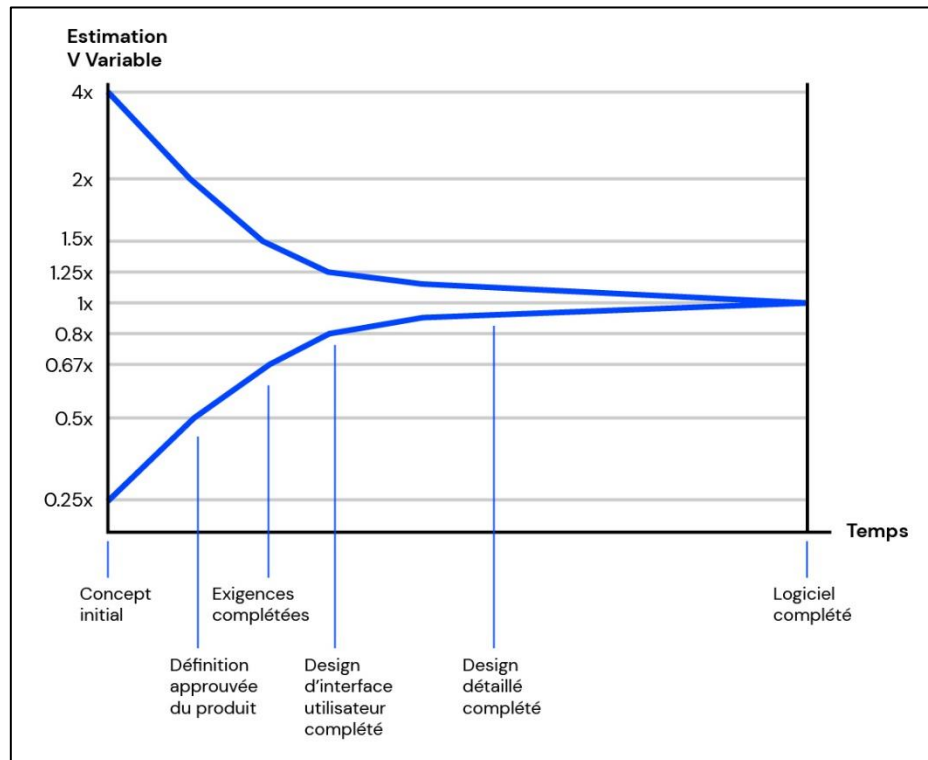


Figure 40 : cône d'incertitude de l'estimation des coûts de développement (adapté de McConnell, S (1997) Software Project Survival Guide, Microsoft Press, 1997).

Pourtant, des estimations de coût sont indispensables pour établir des priorités dans un contexte universel de budget limité. Comme discuté dans la Section 3.2.3, le coût et la valeur apportée sont les principaux déterminants des priorités des exigences. Et pour rentrer dans un budget disponible, il vaut mieux renoncer aux exigences les moins prioritaires plutôt que de transiger sur la qualité du produit fini.

3.7.1 Modèles d'estimation paramétriques

La communauté de l'ingénierie logicielle a proposé assez tôt des approches quantitatives pour estimer l'ampleur d'un logiciel à développer sur base de ses caractéristiques. En connaissant la productivité moyenne des équipes, on peut alors déduire un effort de développement à produire, et donc une estimation du coût de développement.

Les modèles d'estimation sont généralement paramétriques, i.e., ils proposent des équations dont les variables d'entrée expriment les caractéristiques du logiciel et du contexte de développement. Ils sont établis empiriquement par régression sur des projets antérieurs.

Les modèles qui sont présentés dans cette section estiment l'effort à partir d'unités de volume :

- le modèle COCOMO se base sur un volume exprimé en nombre de lignes de code pour évaluer l'effort à produire pour le développement ;
- le modèle des points de fonction estime un volume sur base d'une description assez précise du futur système (écrans, tables, etc.), et qui n'est pas directement disponible en début de projet. Ce modèle

n'estime pas l'effort, mais un nombre de points de fonction qui est convertible en nombre de lignes de codes, que le COCOMO peut exploiter pour estimer l'effort ;

- d'autres modèles se basent des volumes d'exigences de haut niveau pour estimer l'effort. Cette section présente plus loin un modèle basé sur les cas d'utilisation.

Modèle générique d'estimation de l'effort. La forme générale de l'effort à produire pour développer un logiciel est la suivante :

$$E = (a + b \text{ KLOC}^c) f(x_1, \dots, x_n)$$

où

- E représente l'effort à produire en personne/mois ;
- KLOC représente la taille du code du logiciel à développer en Kilo Lines Of Code ;
- a, b, et c sont des constantes empiriques ;
- $f(x_1, \dots, x_n)$ représente une fonction d'ajustement dont les variables évaluent des caractéristiques globales du système à construire et du contexte de développement, comme la taille de la base de données ou l'expérience des développeurs.

L'effort à produire évolue de manière légèrement exponentielle en fonction du volume de code à produire (c est proche de 1). Ces efforts incluent le travail de toutes les tâches de développement, comme l'ingénierie des exigences, le développement proprement dit ou la gestion de projet.

La Figure 41 montre quelques applications de ce modèle général.

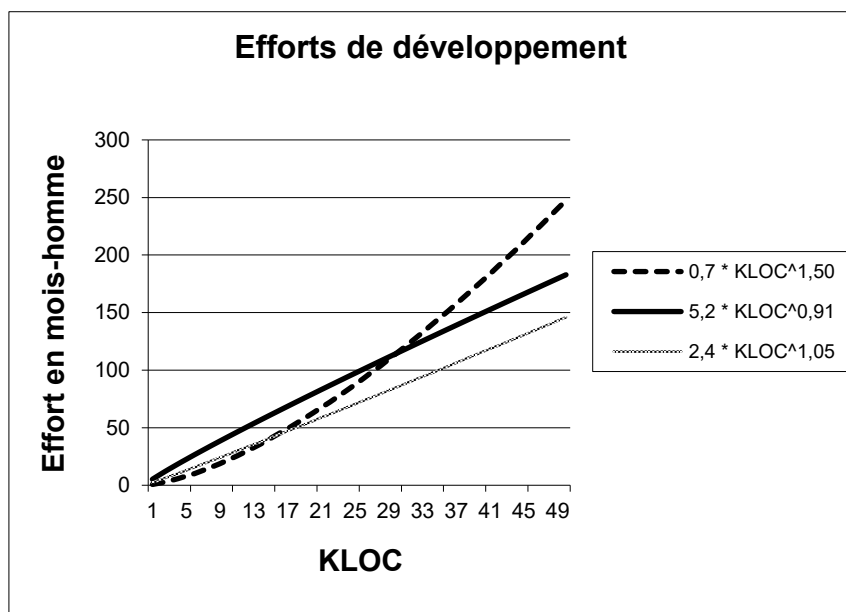


Figure 41: variations de l'effort de développement.

Constructive Cost Model (COCOMO). Ce modèle particulier est une déclinaison du modèle générale qui considère trois catégories de projets :

- *organique* : petites équipes de développement avec une bonne expérience du domaine d'application, projets d'importance moyenne, comme des systèmes de gestion commerciale ;
- *semi-détaché* : complexité intermédiaire, projet de dimension variable, personnel d'expériences diverses ;

- *étendu* : complexité importante, domaine avec beaucoup de contraintes précises concernant notamment le matériel, comme un système de contrôle de trafic aérien.

Le détail du modèle est le suivant :

$$E = a \text{ KLOC}^b \text{ EAF}$$

$$D = c \text{ E}^d$$

$$N = E / D$$

où :

- E représente l'effort en personne/mois ;
- D représente le temps de développement en mois ;
- N représente le nombre de personnes à affecter au projet ;
- a, b, c et d varient en fonction du type de projet selon la Table 3.1;
- EAF (Effort Adjustment Factor) représente le produit des facteurs d'ajustement repris dans la Table 3.2.

Type de projet	a	b	c	d
organique	2.4	1.05	2.5	0.38
semi-détaché	3.0	1.12	2.5	0.35
étendu	3.6	1.20	2.5	0.32

Table 3.1: Paramètres COCOMO.

Facteurs coût	très bas	bas	nominal	élevé	fort élevé	très élevé
Application						
fiabilité	0.75	0.88	1.00	1.15	1.40	
taille de la base de données		0.94	1.00	1.08	1.16	
complexité	0.70	0.85	1.00	1.15	1.30	1.65
Matériel (exemple)						
Temps de réponse			1.00	1.11	1.30	1.66
Personnel						
compétence des analystes	1.46	1.19	1.00	0.86	0.71	
expérience du domaine	1.29	1.13	1.00	0.91	0.82	
compétence des programmeurs	1.42	1.17	1.00	0.86	0.70	

expérience du langage de programmation	1.14	1.07	1.00	0.95		
Projet						
modernité des techniques	1.24	1.10	1.00	0.91	0.82	
outils CASE	1.24	1.10	1.00	0.91	0.83	
méthode de développement	1.23	1.08	1.00	1.04	1.10	

Table 3.2: Exemples de facteurs d'ajustement d'effort dans COCOMO.

Par exemple, le développement d'un programme de 32 KLOC nécessite, dans l'hypothèse d'un projet organique dont les facteurs EAF ont tous une valeur nominale :

$$E = 2.4 \cdot 32^{1.05} = 91 \text{ personne/mois}$$

$$D = 2.5 \cdot 91^{0.38} = 14 \text{ mois}$$

$$N = 91 / 15 = 6.5 \text{ personnes}$$

Dans le cas organique, la productivité journalière implicite P d'une personne est estimée comme suit :

$$P = \text{KLOC} / (D * N * \text{Nbr jours ouvrables dans un mois})$$

$$= 32000 / (14 * 6,5 * 22) = 16 \text{ LOC/jour}$$

Cette valeur peut paraître modeste, mais l'équipe de développement a bien d'autres tâches que l'écriture de programmes, comme des discussions détaillées sur les exigences.

Le modèle COCOMO est assez ancien et a été pensé pour des cycles de développement « classiques » (waterfall). Il a cependant évolué pour intégrer des cycles de développement itératifs, la réutilisation, l'assemblage par composants et le déploiement de packages comme des ERP. Le lecteur se référera à la littérature spécifique pour davantage de précisions (voir par exemple les modèles COCOMO II pour intégrer le phasage du projet, COCOTS pour les logiciels sur étagère (packages), ou WEBMO pour les projets Web).

Modèle basé sur les points de fonction (FP). Cette approche estime le volume relatif d'une application, exprimé en *points de fonction* calculés à partir de différents *paramètres* du futur système, comme des entrées de données ou des requêtes. Le nombre de points de fonction est ajusté en considérant certains facteurs comme les contraintes imposées par l'environnement, comme des conversions de données.

La Table 3.3 illustre cette approche [Press05] :

- la colonne *paramètres* reprend les paramètres à considérer ;
- la colonne *estimation* indique les valeurs estimées des paramètres ventilées selon une difficulté estimée ;
- les colonnes *simple*, *moyenne* et *difficile* affichent les pondérations des paramètres selon une difficulté estimée ;
- la colonne *total* multiplie les estimations par la pondération correspondante.

Paramètres	Estimations (simple, moyenne, difficile)	Simple	Moyenne	Difficile	Total
------------	---	--------	---------	-----------	-------

nombre d'entrées	_____	poids: 3	poids: 4	poids: 6	_____
nombre de sorties	_____	poids: 4	poids: 5	poids: 7	_____
nombre de requêtes	_____	poids: 3	poids: 4	poids: 6	_____
nombre de fichiers ou tables	_____	poids: 7	poids: 10	poids: 15	_____
total t					_____

Table 3.3: Pondération pour l'ampleur d'un développement.

Le *nombre d'entrées* reprend le nombre de lots d'informations rentrant dans le système (saisie d'un signalétique, introduction d'un bon de commande, scanning d'un plan, etc.). Une unité possible est l'écran de saisie. La complexité d'un écran dépend de nombreux facteurs, comme nombre de zones de saisie/affichage, l'importance des contrôles de validité des données ou le degré d'automatisme dans la saisie. Le *nombre de sorties* reprend le nombre de lots d'informations produits par le système (impression de facture, envoi de message, etc.). Une unité possible est le rapport. On évalue ici la complexité liée à la mise en forme des données.

Le *nombre de requêtes* reprend les interrogations qui sont suivies d'une réponse immédiate du système (nombre de passagers sur un vol, chiffre d'affaires ventilé et par pays et par familles de produits, etc.). On évalue ici la difficulté liée à la complexité des résultats à produire et des calculs intermédiaires.

Le *nombre de fichiers ou tables* reprend les principales grappes de données persistantes, comme des clients, des produits ou des fournisseurs. La complexité d'une table de données est liée au nombre de zones, aux contrôles de validation des données, etc.

Le calcul basé sur la Table 3.3 va produire une valeur t qui correspond au "volume" du logiciel à produire exprimé en points de fonction. Cette valeur sera encore pondérée avec des facteurs qualitatifs F_i appelés *valeurs d'ajustement de complexité*. La valeur de ces facteurs est estimée sur une échelle de 0 à 5 (0-pas d'influence, 1-occasionnel, 2-moderé, 3-moyen, 4-significatif et 5-essentiel). Chacun des F_i recouvre une des dimensions suivantes :

- Le système requiert-il des fonctions de sauvegarde et de récupération sur erreur ?
- Faut-il développer un système de communication de données ?
- Faut-il prévoir des fonctions de distribution des traitements (par exemple, client/serveur) ?
- La performance est-elle critique ?
- Le système sera-t-il utilisé dans un environnement existant très actif (utilisation intensive) ?
- Faut-il des saisies en temps réels ?
- La saisie des données est-elle multi-utilisateur ?
- Les entrées, sorties, fichiers et requêtes sont-ils complexes (de manière générale) ?
- Le code produit doit-il être réutilisable ?
- Faut-il prévoir des conversions de données et l'installation du système ?
- Le système doit-il être portable vers d'autres organisations ?
- L'application doit-elle facilement évoluer ?
- L'application doit-elle être simple d'utilisation ?

On calcule alors une valeur *relative* selon l'expression empirique suivante :

$$v = t * [0.65 + 0.01 * (\text{somme des } F_i)]$$

v représente l'ampleur ajustée du développement exprimée en points de fonction. Cette valeur peut être convertie, sur base de l'expérience des projets antérieurs, en nombre de personne/jours de travail ou en coûts.

Il est également possible de convertir v en nombre de lignes de code selon des tables de conversion qui considèrent le langage de programmation cible, et d'ainsi appliquer COCOMO pour estimer l'effort.

Modèles basés sur les cas d'utilisation. Les variables entrantes de ces modèles sont les nombres de cas d'utilisation et leur complexité, ainsi que des facteurs d'ajustement liés au contexte du développement.

Par exemple, la Figure 42 et la Figure 43 montrent un modèle basé sur les Use Case Points introduit par Kerner en 1993, où PH_{perUCP} (Person/Hours per Use Case Points) exprime la productivité d'un développeur en heures de travail à prévoir pour développer un point de cas d'utilisation.

La complexité d'un acteur peut raisonnablement être évaluée par exemple en fonction du profil moyen des utilisateurs qui endosseront ce rôle d'acteur.

Step	Rule	Output
1	Classify actors: a) Simple, WF (Weight Factor) = 1 b) Average, WF = 2 c) Complex, WF = 3	Unadjusted Actor Weights (UAW) = $\text{Sum}(\#Actors * WF)$
2	Classify use cases: a) Simple- 3 or fewer transactions, WF = 5 b) Average- 4 to 7 transactions, WF = 10 c) Complex- more than 7 transactions, WF = 15	Unadjusted Use Case Weights (UUCW) = $\text{Sum}(\#Use Cases * WF)$
3	Calculate the Unadjusted Use Case Point (UUCP).	$UUCP = UAW + UUCW$
4	Assign values to the technical and environmental factors [0..5], multiply by their weights [-1..2], and calculate the weighted sums (TFactor and EFactor). Calculate TCF and EF as shown.	Technical Complexity Factor (TCF) = $0.6 + (0.01 * TFactor)$ Environmental Factor (EF) = $1.4 + (-0.03 * EFactor)$
5	Calculate the adjusted Use Case Points (UCP).	$UCP = UUCP * TCF * EF$
6	Estimate effort (E) in person-hours.	$E = UCP * PH_{perUCP}$ where $PH_{perUCP} \rightarrow [20-36]$

Figure 42 : modèle d'estimation basé sur les cas d'utilisation (selon Beng Kuan Tan, Yuan Zhao, and Hongyu Zhang. *Estimating LOC for information systems from their conceptual data models. Proceeding of the 28th international conference on Software engineering*, pg.: 321 – 330, Shanghai, China, 2006).

Factor	Description	Weight
T1	Distributed system	2.0
T2	Response time/performance objectives	1.0
T3	End-user efficiency	1.0
T4	Internal processing complexity	1.0
T5	Code reusability	1.0
T6	Easy to install	0.5
T7	Easy to use	0.5
T8	Portability to other platforms	2.0
T9	System maintenance	1.0
T10	Concurrent/parallel processing	1.0
T11	Security features	1.0
T12	Access for third parties	1.0
T13	End user training	1.0

Factor	Description	Weight
E1	Familiarity with development process used	1.5
E2	Application experience	0.5
E3	Object-oriented experience of team	1.0
E4	Lead analyst capability	0.5
E5	Motivation of the team	1.0
E6	Stability of requirements	2.0
E7	Part-time staff	-1.0
E8	Difficult programming language	-1.0

Figure 43 : facteurs de complexité technique (à gauche) et environnementale (à droite).

De tels modèles sont implémentés dans certains outils de génie logiciel, comme Enterprise Architect (Figure 44).

QA Reports
QA Reports - Use Case Metrics

Testing Details Maintenance Details Dependency Details Implementation Details **Use Case Metrics**

Use Cases
Root Package: Structured Use Case Model [Reload]
Phase like: * Bookmarked: All
Keyword like: Use Cases: 6 [Include Actors]

Package	Name	Type	Complexity	Phase
Use Cases	Use Case B	UseCase	5	1.0
Use Cases	Use Case E	UseCase	5	1.0
Use Cases	Use Case F	UseCase	5	1.0
Use Cases	Use Case D	UseCase	5	1.0
Use Cases	Use Case A	UseCase	5	1.0
Use Cases	Use Case C	UseCase	5	1.0

Unadjusted Use Case Points (UUCP) = Sum of Complexity: 30
Ave Hours per Use Case: Easy: 40 Med: 80 Diff: 120

Technical Complexity Factor
Unadjusted TCF Value (UTV): 47
TCF Weight Factor (TWF): 0.01
TCF Constant (TC): 0.6
TCF = TC + (TWF x UTV): 1.07

Environment Complexity Factor
Unadjusted ECF Value (UEV): 21
ECF Weight Factor (EWF): -0.03
ECF Constant (EC): 1.4
ECF = EC + (EWF x UEV): 0.77

Total Estimate
Use Case Points (UCP) = UUCP * TCF * ECF = 30 * 1.07 * 0.77 = 24 UCP
Estimated Work Effort (hours) = 10 * 24 = 240 Hours
Estimated Cost = EWE * Default hourly Rate = 240 * 40 = 9600 Cost

[Re-Calculate] [Report] [View Report] [Default Rate] [Help]

Figure 44 : copie d'écran d'Enterprise Architect liée à l'évaluation du coût à partir des cas d'utilisation (<https://www.sparxsystems.fr/resources/tutorials/use-case-metrics.html>).

Pour illustrer ce modèle somme toute assez simple, la Figure 45 montre le digramme de cas d'utilisation pour un problème simple de vente d'article pour animaux discuté en détails dans la Section 4.5.

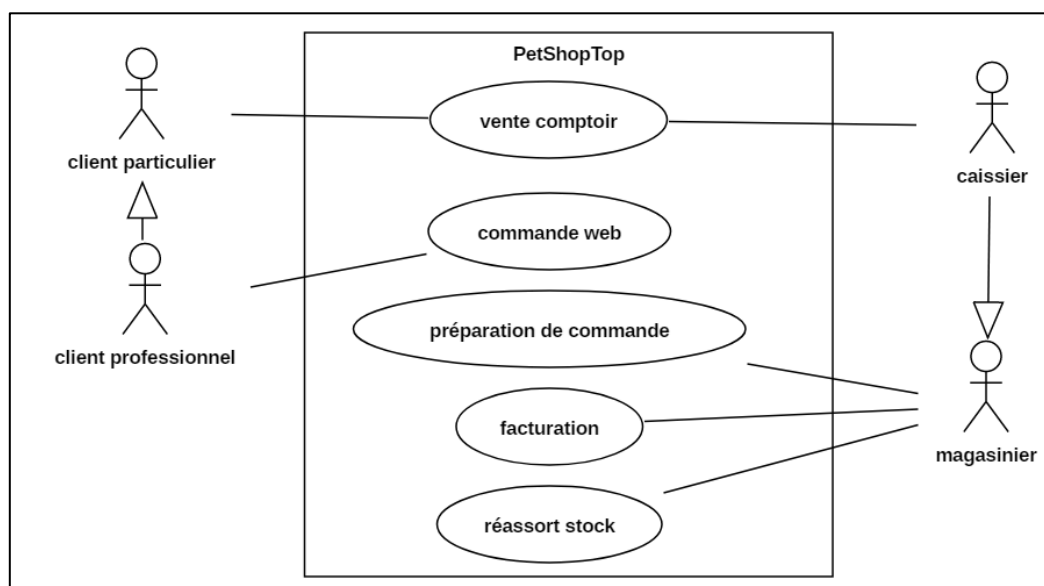


Figure 45 : cas d'utilisation principaux chez PetShopTop.

Le Tableau 17 calcule une estimation d'effort sans considérer les facteurs d'ajustement, dont le total est laissé à la valeur nominale de 1. L'effort estimé est d'un peu plus de 203 personne/jours.

	Acteurs		
	simple	average	complex
		client professionnel, caissier	client particulier, magasinier
nombre	0	2	2
poids	1	2	3
UAW	10		
	Use cases		
	simple	average	complex
	réassort stock	préparation de commande, facturation	vente comptoir, commande Web
nombre	1	2	2
poids	5	10	15
UUCW	55		
UUCP	65		
TCF	1		
EF	1		
UCP	65		
PHperUCP	25		
E(personne/heure)	1625		
E(personne/jour)	203,125		

Tableau 17 : estimation d'effort pour le diagramme d'utilisation PetShopPro.

3.7.2 Estimations d'experts

Les approches Agile en particulier préconisent des approches plus directes et plus intuitives pour estimer les coûts de développement. Ces estimations reposent directement sur des avis d'experts ou de groupes d'experts. Mais que l'on ne s'y trompe pas : les modèles paramétriques eux aussi reposent sur des évaluations des acteurs, avec toute la subjectivité et l'incertitude qui en découlent.

La suite de cette section porte sur l'estimation de coûts selon la méthode Scrum (Layton & Morrow, 2018).

Les *story points* d'une user story sont une mesure relative de l'effort à produire pour la mettre en œuvre. Ils sont définis pour chaque US par un nombre de la suite de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.) et n'ont pas d'unité.

Différentes cérémoniaux empiriques permettent à l'équipe de développement d'estimer le nombre de story points d'une user story, comme Planning Poker (Section 3.2.3).

Chaque sprint va traiter d'un certain nombre d'exigences totalisant d'un certain nombre de story points.

La *vélocité* de l'équipe de développement exprime le nombre de story points que l'équipe peut traiter pendant un sprint. Il s'agit d'une mesure qui vise à faciliter la planification des sprints en adressant le volume correct de user stories à chaque sprint, mais la vélocité n'est pas un indicateur de productivité.

La vélocité est établie après un premier sprint, puis affinée lors des sprints suivants.

Connaissant les story points et la vélocité de l'équipe de développement, on peut facilement dériver le coût et la durée du développement. Par exemple, supposons que :

- la taille du product backlog = 500 story points
- la vélocité moyenne de l'équipe = 20 story points par sprint
- la durée d'un sprint = une semaine
- le salaire annuel d'un développeur = 100 000 €
- nombre de développeurs = 7, donc coût hebdomadaire = 13 462€

On déduit immédiatement :

- le nombre de sprints nécessaires = $500 / 20 = 25$
- la durée = 25 semaines
- le coût du développement = $25 * 13\,462 = 336\,550$ €

Les story points constituent l'unité de l'axe vertical d'un Burn down chart, qui montre la progression du travail réalisé au cours du temps, comme dans l'exemple de la Figure 46.

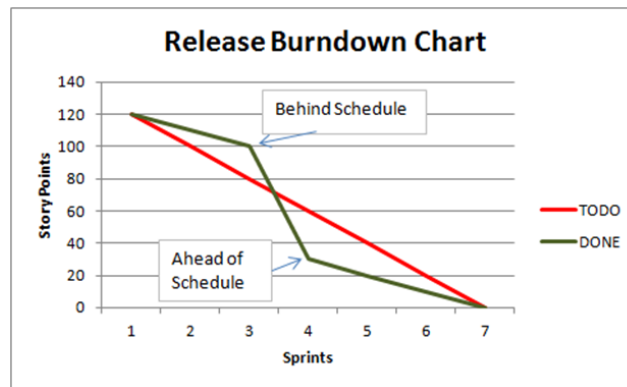


Figure 46: exemple de Burn down Chart (<https://worldofagile.com/blog/burn-down-chart/>)

3.8 Exercices

EXI01 : jeu de rôle bibliothèque

Réaliser un jeu de rôle visant à éliciter les exigences liées à un système de gestion d'une bibliothèque universitaire.

Rôles :

- 1 directeur de l'établissement
- 1 bibliothécaire qui accompagne le directeur
- 2 business analyst

Contexte utilisateur :

- Vous êtes le directeur d'une école supérieure et vous recevez pour la première fois une équipe de consultants informatiques qu'un ami vous a recommandée. Ceux-ci vont vous aider à formuler les exigences pour une nouvelle application pour la bibliothèque de votre école.
- Vous vous apprêtez à recevoir les consultants pour une première discussion. Vous avez convié le bibliothécaire à la réunion, même si vous savez qu'il n'est pas très enthousiaste à l'idée d'employer une nouvelle application...
- La bibliothèque est gérée par une application obsolète, non responsive et non interfacée avec la gestion administrative des étudiants et des professeurs.
- En gros, le bibliothécaire doit maintenir un catalogue d'ouvrages relatifs à différentes thématiques, et doit gérer le prêt d'ouvrage à un étudiant ou à un professeur. Un lecteur doit pouvoir rechercher en ligne un ouvrage dans le catalogue et le réserver pour un prêt.
- Vous disposez de 10 minutes pour vous préparer aux questions des consultants en imaginant les exigences complètes de la bibliothèque.

Contexte BA :

- Vous êtes un business analyst expérimenté. Votre patron joue au golf avec un directeur d'une école supérieure réputée. Il vous a organisé un RV avec ce dernier car il souhaite renouveler le système informatique de sa bibliothèque.
- Vous vous êtes brièvement renseigné sur l'école : on y forme des gestionnaires et des business analysts de renom, mais vous ne disposez pas d'information sur la bibliothèque ni sur son système d'information.
- Lorsque vous arrivez au RV, le directeur vous reçoit avec le bibliothécaire. Celui-ci semble d'emblée opposé à la démarche... Heureusement, vous vous êtes fait accompagner d'un collègue pour vous aider dans l'interview du directeur, et de deux stagiaires pour prendre note des échanges de l'interview.

- Votre objectif est de cerner le domaine d'application et d'identifier les exigences clés du système.
- Vous gèrerez la réunion en 3 temps comme suit :
 - Introduction (1-2 minutes)
 - suggérez un tour de table pour se présenter
 - précisez le but de l'interview (i.e., objectif du projet, cerner le domaine d'application, identifier les exigences clés sans rentrer dans tous les détails)
 - Élicitation des exigences (15-16 minutes)
 - couvrez toutes les questions que vous avez préparées
 - ...mais restez à l'écoute d'informations pertinentes qui sont en dehors de vos questions
 - contrôlez le timing
 - respectez le temps de parole de chacun
 - Conclusion (1-2 minutes)
 - résumez les points clés
 - demandez si quelqu'un veut ajouter quelque chose
 - annoncez la suite du travail de business analyse

EXI02 : jeu de rôle site de location d'objets

Réaliser un jeu de rôle visant à éliciter les exigences liées au développement d'un site de location d'outils entre particuliers.

Rôles

2 business analyst

1 utilisateur final

EXI03 : conflits entre exigences

Détectez les éventuels conflits dans les exigences suivantes liée au fonctionnement d'une entreprise de formations continues. Caractérisez le conflit et proposez des solutions pour les éliminer.

- E1 : le système doit permettre à un participant de s'inscrire à n'importe quelle formation
- E2 : le système doit permettre à la secrétaire d'inscrire un maximum de 20 participants à une formation
- E3 : le système doit permettre au responsable pédagogique de créer une nouvelle formation
- E4 : le système doit permettre à la secrétaire de facturer l'inscription à un séminaire à un client
- E5 : le système doit vérifier qu'un formateur n'anime pas plusieurs formations qui ont lieu en même temps
- E6 : le système doit regrouper toutes les inscriptions non facturées d'un même client sur une seule facture

EXI04 : Choix de vie. Vous sortez de vos humanités et vous ne prévoyez aucun héritage. Votre but est de gagner beaucoup d'argent. Identifiez les différentes solutions et comparez-les selon une analyse multicritère.

EXI05 : Choix de processus. Comparez les 2 processus suivants pour prolonger un prêt dans une bibliothèque selon une analyse multicritères :

- un lecteur envoie une demande de prolongation par email ; le bibliothécaire modifie la date de fin

- un lecteur modifie la date de fin du prêt par le système ; le système valide la date de fin si la prolongation est inférieure à 5 jours ; le bibliothécaire valide cette date de fin si la prolongation est supérieure ou égale à 5 jours

EXI06 : Choix de délégation. Vous êtes à la tête d’une entreprise d’import-export de produits assez spécifiques. Vous disposez d’une équipe d’informaticiens en interne mais ceux-ci sont assez occupés à maintenir les systèmes existants et ils ne sont pas au fait des dernières évolutions technologiques. Vous souhaitez malgré tout disposer d’une plate-forme de vente en ligne. Identifiez les différentes solutions et comparez-les selon une analyse multicritère.

EXI07 : établissez les priorités des exigences suivantes relatives à une plate-forme e-learning selon la méthode AHP. Adoptez le point de vue de l’étudiant.

- télécharger des fichiers postés par l’enseignant
- envoyer un mail à l’enseignant
- communiquer au sein d’un groupe d’étudiants
- consulter un agenda des échanges liées aux cours
- utiliser le système sur mobile

EXI08 : en tant que membre d’une association d’étudiants de votre HE, vous souhaitez fixer des priorités dans les projets à réaliser sur base :

- des efforts à produire pour chaque projet selon Planning poker en termes de jours-hommes
- de la valeur des projets selon MoSCoW

Les projets sont :

- P1 : organiser une soirée de rentrée
- P2 : organiser des élections des délégués étudiants
- P3 : mettre en place une plateforme de ventes en ligne d’ordinateurs de seconde main
- P4 : développer un système de covoiturage pour étudiants

Constituez des groupes de 5 étudiants et désignez un facilitateur pour :

- préparer et animer le workshop,
- préparer la logistique (réunir le groupe autour d’une table, outiller chaque participant avec des cartes ou <https://planningpokeronline.com>, etc.)
- coordonner les discussions Planning Poker
- coordonner les discussions d’estimation de valeur des projets avec MoSCoW
 - balancer les arguments des uns et des autres
 - chacun doit pouvoir exprimer ses arguments
 - essayer de trouver une position commune

EXI09 : détectez les problèmes de formulation dans les exigences suivantes relatives à un restaurant. Qualifiez les problèmes et proposer des solutions.

- E1 : le système doit permettre à un serveur d’enregistrer une commande mais aussi de la compléter
- E2 : le système doit permettre au cuisinier de définir un maximum de menus différents
- E3 : le système doit permettre au cuisinier et au serveur d’annuler une commande
- E4 : le système doit éventuellement permettre une gestion des stocks
- E5 : le système doit proposer un dessert alternatif si le dessert souhaité n’est plus disponible
- E6 : le système doit calculer la TVA de 21% sur la plupart des boissons

- E7 : le système doit permettre au serveur d'éditer la note
- E8 : le système doit permettre de connecter le terminal du serveur par Bluetooth
- E9 : le système doit permettre au barman de consulter les boissons à préparer pour un client
- E10 : le système doit permettre client de réserver une table ou d'annuler sa réservation

EXI10 : réaliser un jeu de rôle visant à éliciter les exigences liées à un système de gestion d'un hôpital. Les exigences seront formulées sous la forme de user stories et seront organisées selon un canevas aux choix.

Rôles :

- 1 patient
- 1 responsable administratif
- 1 médecin
- 2 business analysts

EXI11 : imaginez un SRS pour un hôpital selon le standard IEEE 830 en dehors des exigences fonctionnelles

4 Diagramme de cas d'utilisation

Les diagrammes de cas d'utilisation aident les utilisateurs et le business analyst à exposer aux informaticiens ce que doit faire le système sans se préoccuper de comment il doit le faire. Leur formalisme est suffisamment simple pour qu'ils puissent être facilement compris et interprétés par toutes les parties prenantes du projet.

Définition 10 : un diagramme de cas d'utilisation recense et décrit les scénarii d'utilisation du système. Il précise quels sont les acteurs qui interagissent avec le système et ce qu'ils peuvent réaliser avec le système pour atteindre leurs buts.

Les diagrammes de cas d'utilisation fixent les choix des utilisateurs au niveau des fonctions à prévoir ou à exclure, et conditionnent l'ensemble du projet de développement. Ils méritent donc une attention toute particulière, car des omissions ou des imprécisions à ce stade auront des conséquences importantes par la suite.

Un cas d'utilisation est une forme d'exigence utilisateur, car il décrit la manière dont l'utilisateur perçoit le système.

Le diagramme de cas d'utilisation vient parfois préciser des exigences en langage naturel, comme les US. Le Tableau 18 compare ces deux techniques complémentaires.

	user stories	use cases
intention	décrire le comportement du système d'un point de vue fonctionnel technique privilégiée par les méthodes Agile	décrire le comportement du système d'un point de vue fonctionnel technique privilégiée par le Unified Process
niveau de détail	faible dans un premier temps annonce des discussions complémentaires entre utilisateurs et informaticiens qui déboucheront sur une documentation détaillée par la suite	élevé si chaque cas d'utilisation est détaillé par des scénarios
forme	langage naturel structuré + détails sous différentes formes	graphique + scénarios en langage naturel ou graphique
modalité	encourage le travail collaboratif (Agile) et la communication synchrone	encourage la formalisation écrite et la communication asynchrone rarement rédigé par les utilisateurs, plutôt produit par le Business Analyst

Tableau 18 : comparaison entre les diagrammes de cas d'utilisation et les user stories.

4.1 Cas d'utilisation

Un cas d'utilisation (*use case*) décrit le comportement du système du point de vue de l'utilisateur : le système est vu comme une boîte noire dont on ignore le fonctionnement interne. L'exécution d'un cas d'utilisation est généralement déclenchée par un acteur qui sollicite le système.

Définition 11 : un cas d'utilisation représente une fonctionnalité qui apporte de la valeur à l'utilisateur.

Un cas d'utilisation est représenté par un ovale, comme dans les exemples de la Figure 47.

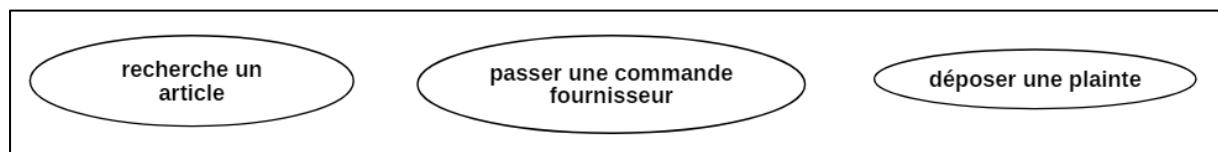


Figure 47 : exemples de cas d'utilisation.

4.2 Acteur

Un acteur peut désigner un utilisateur humain ou un autre système, tel qu'un utilisateur, un internaute, un modérateur du forum, ou un système extérieur. Un acteur représente plutôt un rôle qu'une personne particulière, ce qui évite de mentionner nominativement tous les utilisateurs du système. Par contre, un même utilisateur peut assumer plusieurs rôles, et par exemple interagir avec le système tantôt en tant qu'acteur internaute et tantôt en tant qu'acteur modérateur du forum.

Définition 12 : un acteur représente une entité extérieure au système et interagit avec le système.

Un acteur peut soit stimuler le système en vue de la réalisation d'un traitement, comme l'acteur *client* qui utilise une application e-banking pour passer un ordre de virement, soit simplement participer à la fonctionnalité du système, comme l'acteur *système bancaire* qui reçoit le virement de la même application.

Un acteur est connecté à un cas d'utilisation par une association qui représente le fait que l'acteur interagit avec le système dans le cadre du cas d'utilisation. Une association est symbolisée par un segment de droite qui relie un acteur et un cas.

Un diagramme de cas d'utilisation reprend donc les acteurs, les cas d'utilisation et les associations entre les acteurs et les cas. Il mentionne également une désignation du système et indique la limite entre le système et son environnement, comme dans la Figure 48.



Figure 48 : notation des diagrammes de cas d'utilisation.

Hierarchie d'acteurs. Les acteurs peuvent être structurés par des relations de généralisation : un acteur peut être défini comme un cas particulier d'un acteur plus général. Par exemple, l'acteur *modérateur* peut être vu comme un cas particulier de l'acteur *internaute*. Dans ce cas, les associations de l'acteur le plus général sont implicitement héritées par l'acteur particulier. La Figure 49 montre la notation d'une généralisation d'acteurs, symbolisée par une flèche partant de l'acteur particulier et pointant vers l'acteur le plus général. Ici, le *modérateur* peut implicitement utiliser le système pour poster un message sur le forum, car il hérite de ce cas d'utilisation en tant que cas particulier d'*internaute*.

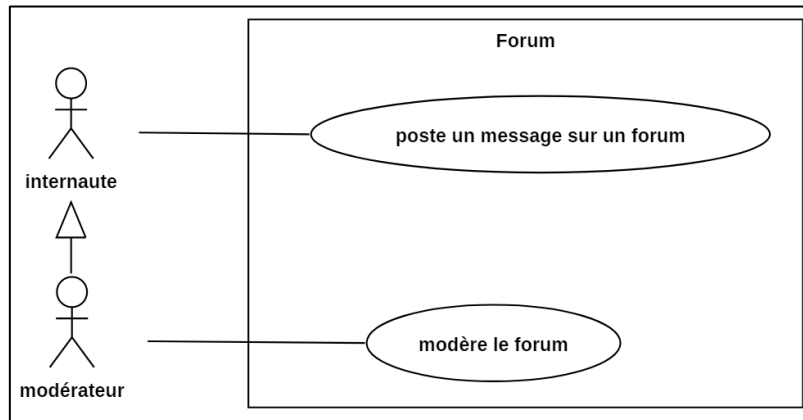


Figure 49 : généralisation d'acteurs.

Si plusieurs acteurs sont connectés à un même cas d'utilisation, ils sont alors tous obligatoirement impliqués dans son exécution. Dans l'exemple de gauche de la Figure 50, un *client* **et** un *caissier* participent tous deux à une vente au comptoir. Par contre, dans l'exemple de droite, le *client* et le *caissier* héritent de l'association au cas *vendre au comptoir*, et donc **soit** le vendeur seul **soit** le client seul (self scanning) peut individuellement réaliser ce cas.

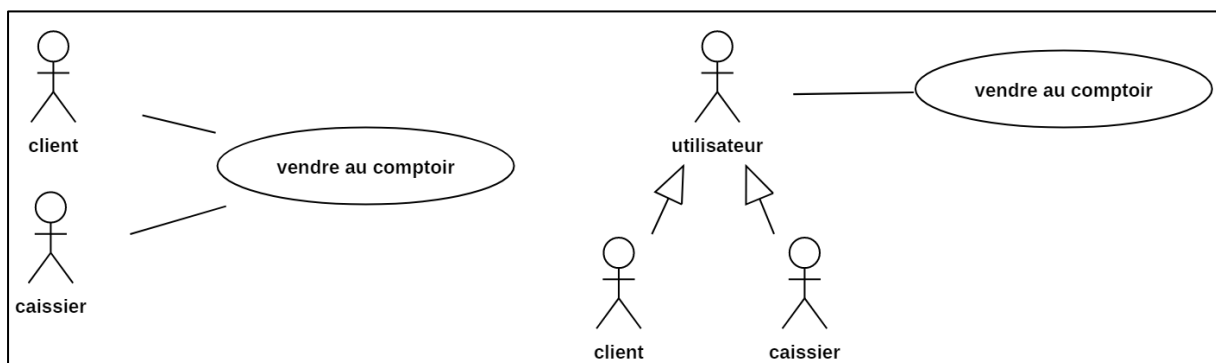


Figure 50 : participation multiple et généralisation.

4.3 Structuration des cas d'utilisation

Parfois, des comportements identiques sont communs à plusieurs cas d'utilisation. Pour éviter de décrire ces comportements plusieurs fois, UML propose un mécanisme de factorisation de cas d'utilisation à travers des appels entre cas.

Deux types de relations permettent de relier les cas entre eux (Figure 51) :

- La relation `include`, qui signifie que le comportement d'un cas d'utilisation appelé est systématiquement intégré dans un cas d'utilisation appelant. Cette relation est représentée par une flèche pointillée dirigée vers le cas appelé.
- La relation `extend`, qui signifie que le comportement d'un cas d'utilisation peut éventuellement être intégré au comportement d'un cas d'utilisation appelant si une certaine condition est vérifiée. Cette relation est représentée par une flèche pointillée dirigée vers le cas appelant. La condition est notée entre accolades et est préfixée du mot clé `condition`.

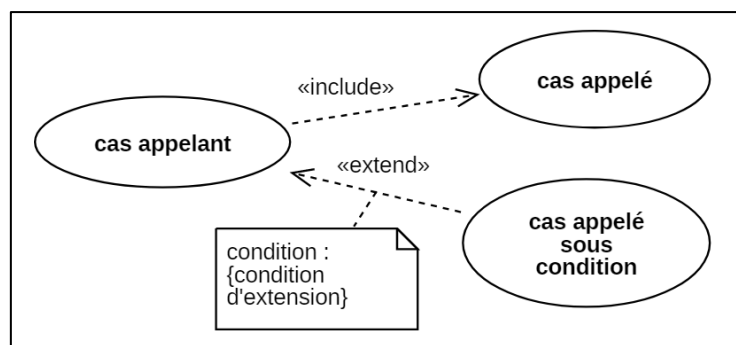


Figure 51 : notation des relations entre cas d'utilisation.

Dans l'exemple de la Figure 52, le cas d'utilisation `passer une commande` inclut systématiquement le cas d'utilisation `payer une commande`, ce qui signifie qu'à un moment donné de la saisie d'une commande, le client devra inévitablement la payer. Par contre, Le cas d'utilisation `changer d'adresse livraison` peut étendre le cas d'utilisation `passer une commande` si la condition `si autre adresse` est vérifiée.

Notons qu'un cas d'utilisation appelé peut être invoqué directement par un acteur, indépendamment des cas d'utilisation appelants, et un même cas d'utilisation peut être appelé par plusieurs autres cas d'utilisation et vice versa.

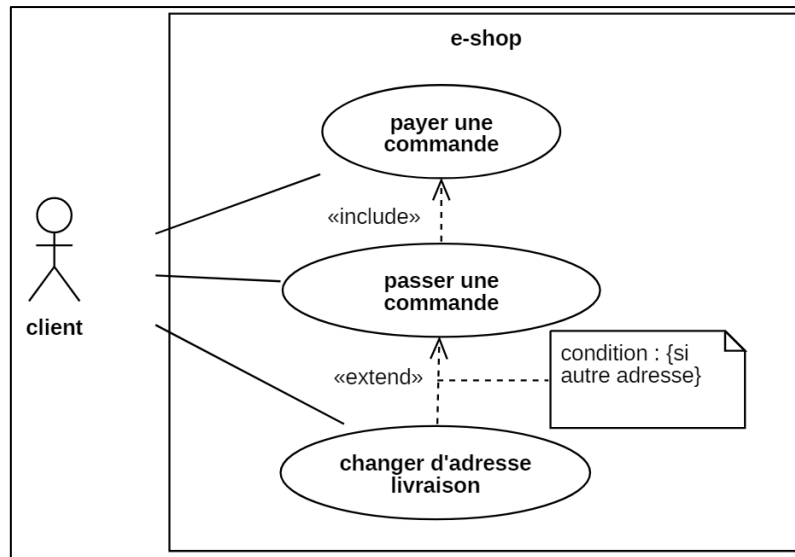


Figure 52 : exemples de relations entre des cas d'utilisation.

4.4 Spécification des cas d'utilisation

Identifier les cas d'utilisation et les nommer constitue une première étape pour fixer les fonctionnalités et les limites du système. Une description plus complète de chaque cas d'utilisation est cependant nécessaire pour bien appréhender le comportement correspondant, comme pour les user stories. Par exemple, un cas d'utilisation peut être documenté en précisant les rubriques suivantes :

- son nom ;
- les acteurs impliqués dans son déroulement ;
- son objectif principal ;
- une description informelle du comportement ;
- un scénario qui décrit les interactions entre le système et son environnement selon un mode conversationnel, c'est-à-dire un échange de stimuli émis par l'environnement et de réponses du système.

Voici par exemple le détail d'un cas d'utilisation Acheter des articles.

Nom du cas : Acheter des articles

Acteurs : Client, caissier

Objectif : Saisir une vente et son paiement

Description : Un client se présente à la caisse pour régler ses achats. Le caissier enregistre les articles choisis et encaisse le paiement du client. Finalement, le client quitte la caisse avec ses articles.

Scénario :

Action de l'utilisateur	Réponse du système
1. Le cas est initié lorsqu'un client arrive à la caisse avec ses articles.	

2. Le caissier enregistre l'identifiant de chaque article.	3. Le système détermine le prix de l'article et rajoute l'information relative à l'article dans la transaction de la vente en cours.
4. Le caissier signale que la saisie des articles est terminée.	5. Le système calcule et affiche le total de la vente.
6. Le caissier réclame le montant au client.	
7. Le client donne les espèces au caissier.	
8. Le caissier enregistre le montant payé.	9. Le système affiche le solde à rendre au client.
	Un ticket de caisse est imprimé.
10. Le caissier encaisse les espèces et rend la monnaie éventuelle.	11. La vente est clôturée.

Parfois, un scénario mentionnera des interactions qui ne concernent pas directement le système, mais qui permettent de bien comprendre le processus d'affaires sous-jacent. Dans l'exemple précédent, l'action 7. Le client donne les espèces au caissier ne concerne pas la spécification du système, mais documente utilement le processus d'achat des articles.

UML n'impose pas une manière de décrire les cas d'utilisation. Le lecteur trouvera des recommandations complémentaires dans la littérature dédiée (Kulak & Guiney, 2000). Par exemple, un cas d'utilisation peut être détaillé avec les rubriques : nom, résumé, scénario de base, chemins alternatifs, points d'extension, déclenchement, hypothèse, pré-conditions, post-conditions, règles métier correspondantes, auteur et date.

Scénarios alternatifs. Une première version du scénario va décrire un comportement standard du cas d'utilisation (« happy flow »), c'est-à-dire un processus au cours duquel les choses se déroulent de manière prévisible. Mais très vite, il faudra considérer des situations plus élaborées qui dévient de ce comportement standard. Dans l'exemple précédent, que se passe-t-il si un client achète plusieurs exemplaires du même article ? Ou encore, comment gérer les différents modes de paiement ? Ces exceptions au déroulement standard sont spécifiées à l'aide de séquences alternatives ou conditionnelles.

Par exemple :

- séquence conditionnelle complémentaire à l'action 2 :
 - si plusieurs exemplaires du même article sont présentés :
 - le caissier peut introduire une quantité achetée.
- séquence alternative à l'action 7 :
 - si le client paye en espèce :
 - le client donne les espèces au caissier.
 - si le client paye par carte :
 - le client introduit son code.
 - le système communique avec l'organisme de paiement et débite le compte du client.

Un système est robuste s'il fonctionne correctement dans des conditions non standard. Le système sera donc robuste quand toutes les exceptions seront identifiées et traitées.

4.5 Démarche de construction

Un diagramme de cas d'utilisation se construit progressivement et itérativement (en cohérence avec la plupart des processus de développement). Une fois la liste des cas d'utilisation stabilisée, ils sont détaillés à travers des scénarii de plus en plus robustes, puis structurés avec des relations *include* et *extend*.

Voici un exemple de démarche de construction possible d'un diagramme de cas d'utilisation qui est présentée à travers un exemple de magasin vendant des articles pour animaux, et dont voici une brève description :

- PetShopTop vend de la nourriture et des accessoires pour animaux à des clients privés ou professionnels. Deux modes de vente sont proposés : une vente au comptoir et une commande Web pour les professionnels uniquement.
- Les magasiniers se chargent de la préparation des commandes et de la facturation. Ils s'occupent du réassortiment du stock et tiennent la caisse.
- De temps en temps, des délégués commerciaux viennent en magasin pour organiser les rayons.

1. Identifier les acteurs principaux. Un acteur principal fait partie de la cible première du système, c'est-à-dire que c'est à son intention que le système est construit. Dans l'exemple PetShopTop (Figure 53), ces acteurs principaux sont :

- les clients, qui comptent une catégorie particulière : les clients professionnels ;
- les magasiniers qui, eux, peuvent se spécialiser en caissiers.

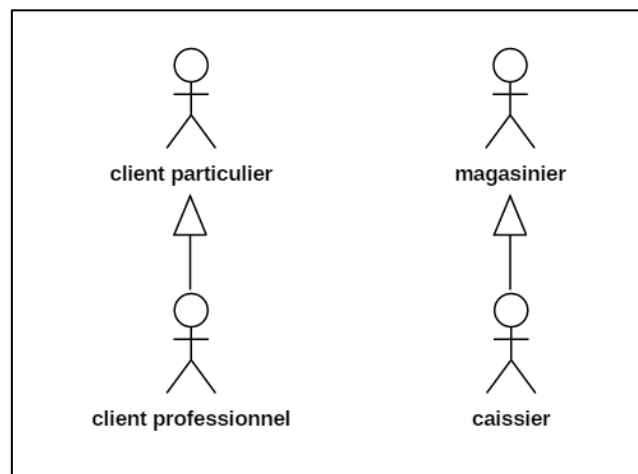


Figure 53 : hiérarchie d'acteurs chez PetShopTop.

2. Identifier les cas d'utilisation principaux. Un cas d'utilisation est qualifié de principal quand il est associé à un acteur principal. La Figure 54 montre les cas d'utilisation principaux pour le cas PetShopTop.

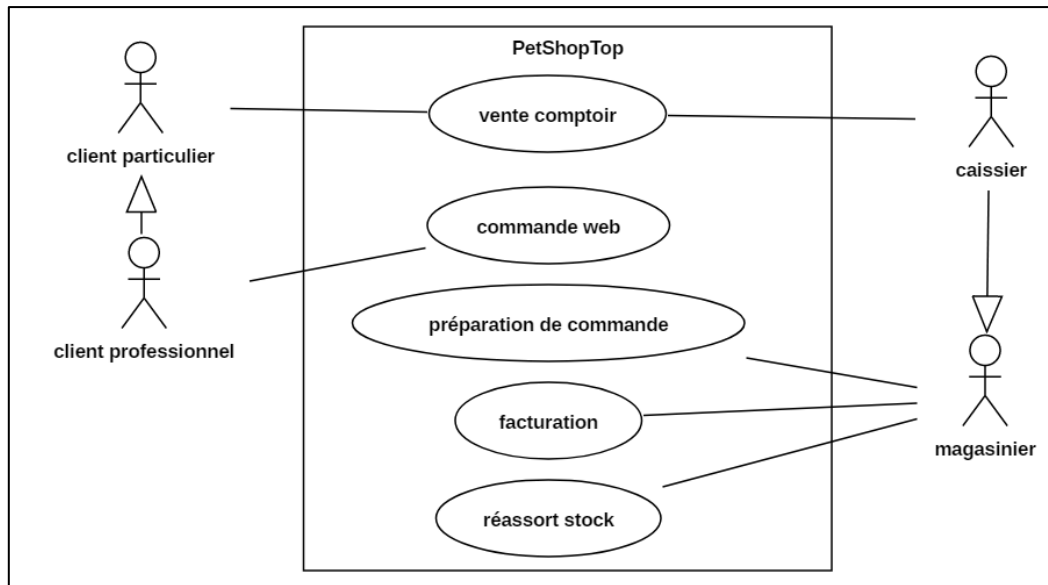


Figure 54 : cas d'utilisation principaux chez PetShopTop.

3. Identifier les acteurs et cas secondaires. Un acteur est secondaire s'il n'est pas dans la cible première du système, et donc s'il n'utilise le système que ponctuellement, comme l'administrateur d'un site Web. Chez PetShopTop, le délégué commercial utilise le système sans en être le bénéficiaire principal (Figure 55).

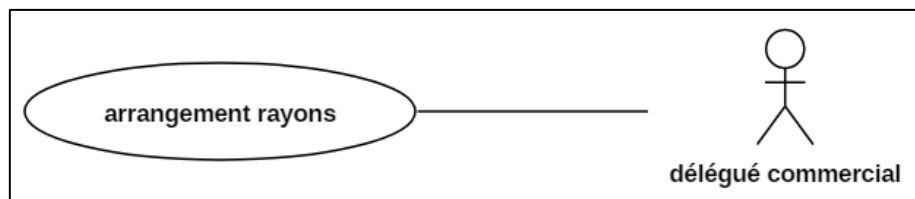


Figure 55 : acteur et cas d'utilisation secondaires chez PetShopTop.

4. Détailler les scénarii standard et les alternatives. Si les variations à un scénario standard sont trop nombreuses ou trop complexes, plusieurs scénarii sont développés pour assurer meilleure lisibilité.

5. Structurer les cas d'utilisation. Le détail des cas d'utilisation peut faire apparaître certaines répétitions d'un cas à l'autre, comme des actions liées à l'identification d'un utilisateur. Plutôt que de répéter ces comportements communs à plusieurs cas, on les factorise dans des cas appelés par des relations *include* ou *extend*.

Dans l'exemple de la Figure 56, le paiement par carte peut avoir lieu lors de la vente comptoir et a toujours lieu lors de l'enregistrement d'une commande Web.

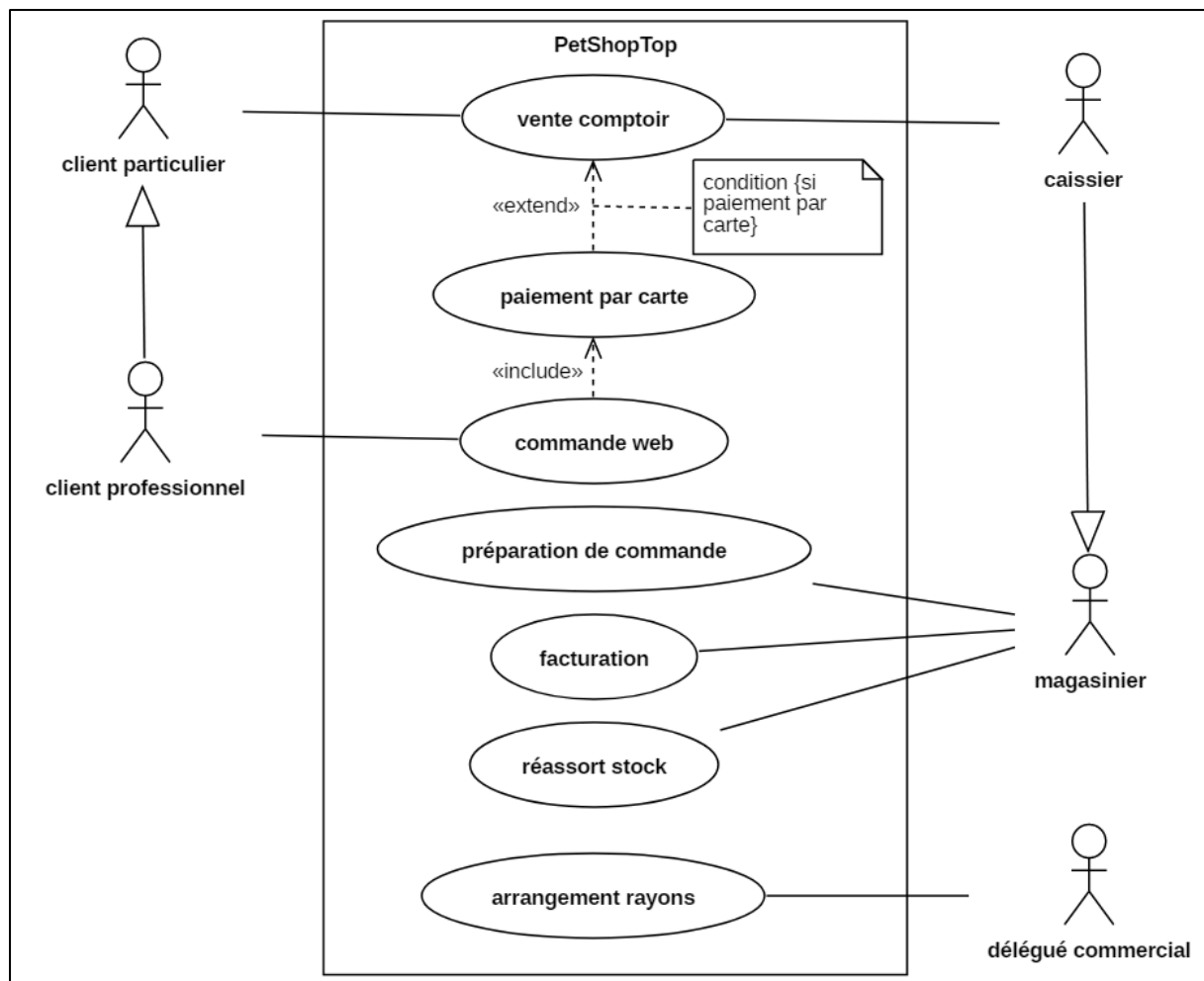


Figure 56 : diagramme de cas d'utilisation pour PetShopTop.

4.6 Erreurs fréquentes

Même s'ils paraissent simples, les diagrammes de cas d'utilisation comportent souvent des erreurs de conception et l'effort à produire pour leur conception est parfois sous-estimé.

Le comportement décrit par un cas d'utilisation ne doit être ni trop limité, ni trop large. Par exemple, des cas d'utilisation comme cliquer sur le bouton OK ou gérer le back office n'ont pas une granularité idéale. Typiquement, un cas d'utilisation est détaillé en une à deux pages, mais il n'y a pas de règle absolue en la matière. Le Tableau 19 montre quelques pièges à éviter (Seidl, Scholz, Huemer, & Kappel, 2015).

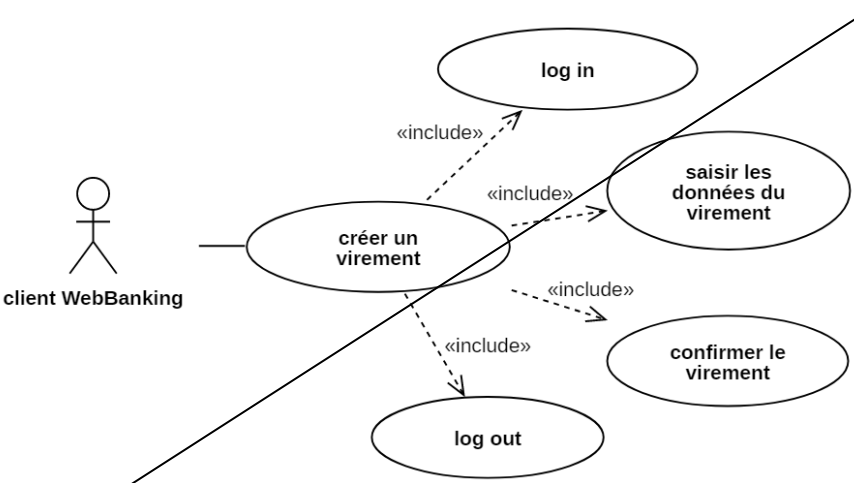
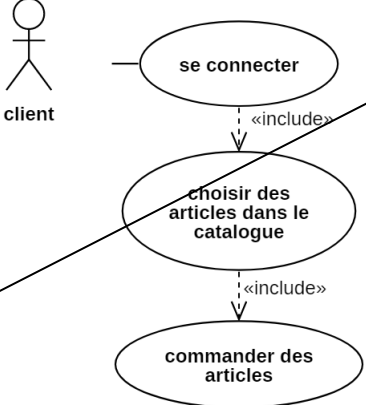
Éviter la décomposition fonctionnelle : les différentes étapes d'un cas d'utilisation sont décrites dans un scénario et ne justifient pas la création d'autres cas d'utilisation dont la granularité serait faible. Dans l'exemple ci-contre, seul le cas créer un virement est pertinent.	
Éviter la modélisation de workflows : un diagramme de cas d'utilisation ne doit pas mentionner de relations d'ordre entre les cas ni représenter l'enchaînement des opérations dans le temps.	

Tableau 19 : exemples d'erreurs à éviter au niveau des diagrammes de cas d'utilisation.

4.7 Exemple

Imaginons un projet de développement d'une plateforme de location d'objets. Son objectif est de mettre en relation des locataires et des propriétaires qui souhaitent louer des objets, comme des outils, des vêtements, ou des appareils photo.

On demande de déduire un diagramme de cas d'utilisation à partir d'exigences rédigées préalablement en langage naturel libre.

Vu la disponibilité des exigences, le diagramme peut être construit itérativement en traitant successivement chaque exigence (la démarche de construction exposée dans la Section 4.5 s'applique plutôt lorsque le diagramme de cas d'utilisation est établi sans disposer d'exigences préalables sous une autre forme).

Voici chaque exigence et sa formulation sous la forme d'un diagramme de cas d'utilisation.

Exigence 1 : un internaute peut consulter le catalogue des objets disponibles et éventuellement lancer une recherche sur ce catalogue.

La Figure 57 montre un diagramme de départ avec un premier acteur principal et une première utilisation du système.

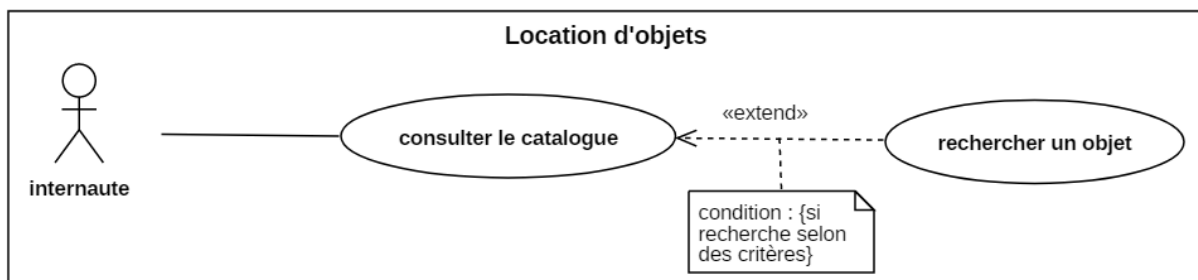


Figure 57 : exigence 1 pour la location d'objets.

Exigence 2 : quand un internaute souhaite louer un objet, il doit s'enregistrer sur la plateforme avant sa première location.

Cette seconde exigence complète le diagramme avec un cas d'utilisation s'enregistrer, comme dans la Figure 58.

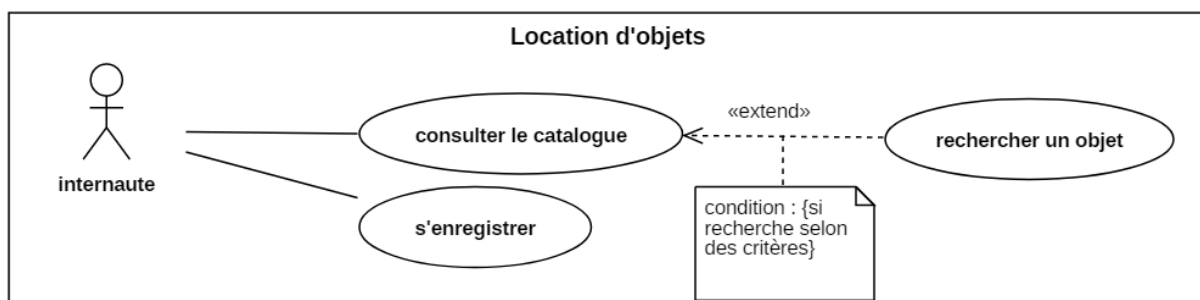


Figure 58 : exigence 2 pour la location d'objets.

Exigence 3 : une fois enregistré, l'internaute peut, après s'être identifié, réserver un objet et le louer. Il peut aussi laisser un commentaire sur un objet qu'il a recherché. Il devra procéder à un paiement lors de la réservation (acompte) et de la location (solde).

Cette exigence suggère une catégorie particulière d'internaute : le locataire, qui hérite des utilisations de l'internaute, mais qui accède à d'autres facilités.

L'identification, le paiement et la recherche d'objet sont des comportements communs à certains cas d'utilisation et sont donc représentés par de nouveaux cas appelés par des relations *include* dans la Figure 59.

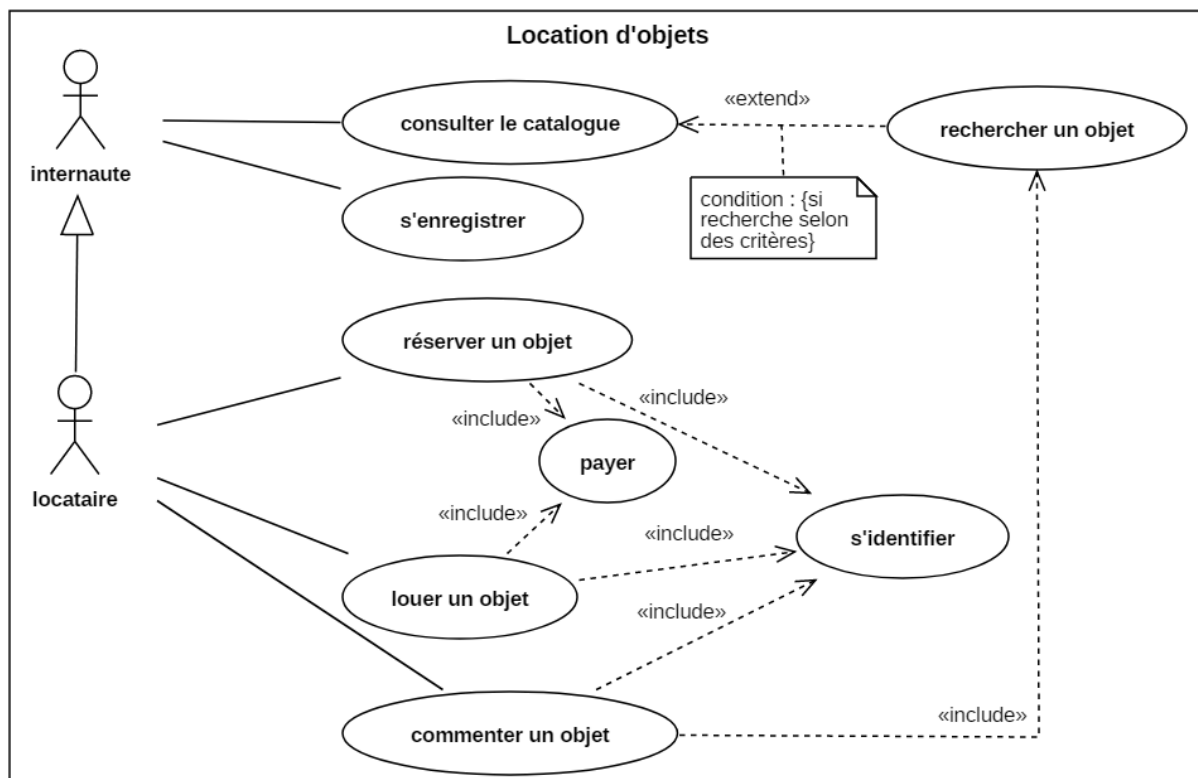


Figure 59 : exigence 3 pour la location d'objets.

Exigence 4 : le système permet à un propriétaire de poster les données d'un objet qu'il met en location et, à cette occasion, d'ajouter éventuellement des commentaires sur ses objets. Un propriétaire peut laisser un commentaire sur un objet après l'avoir posté sur la plateforme.

La Figure 60 fait apparaître le propriétaire et son cas d'utilisation poster un objet. Le cas laisser un commentaire sur un objet a été associé à un nouvel acteur (utilisateur identifié) qui généralise le propriétaire et le locataire (dont les cas d'utilisation spécifiques ne sont pas représentés dans la Figure 60).

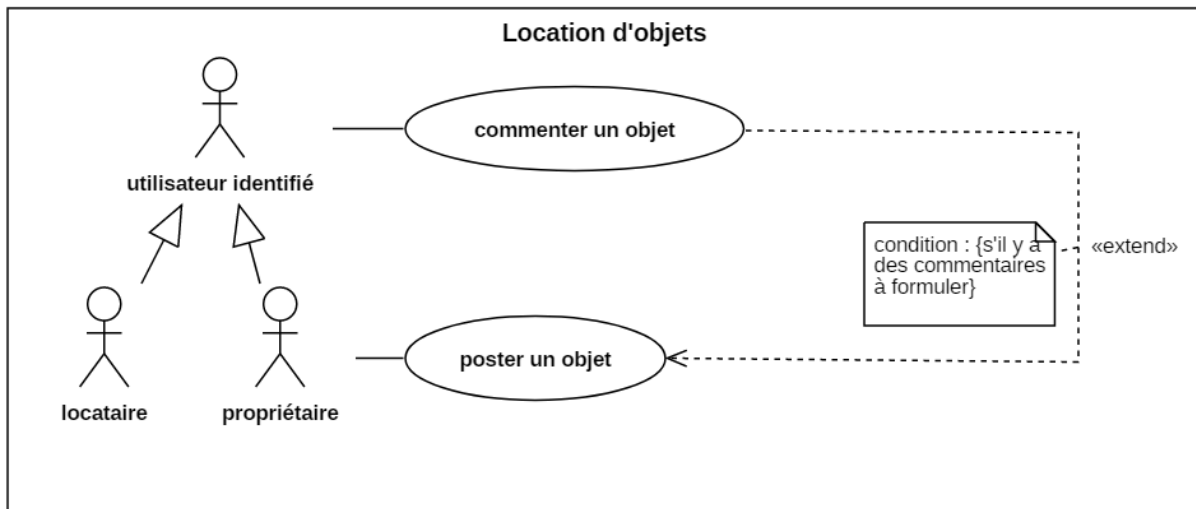


Figure 60 : exigence 4 pour la location d'objets.

Exigence 5 : tout comme le locataire, le propriétaire doit s'enregistrer sur la plateforme et s'identifier pour accéder à toutes ses fonctions.

L'utilisateur identifié hérite assez naturellement de l'internaute, et le cas d'utilisation lié au propriétaire inclut le cas d'utilisation s'identifier (Figure 61).

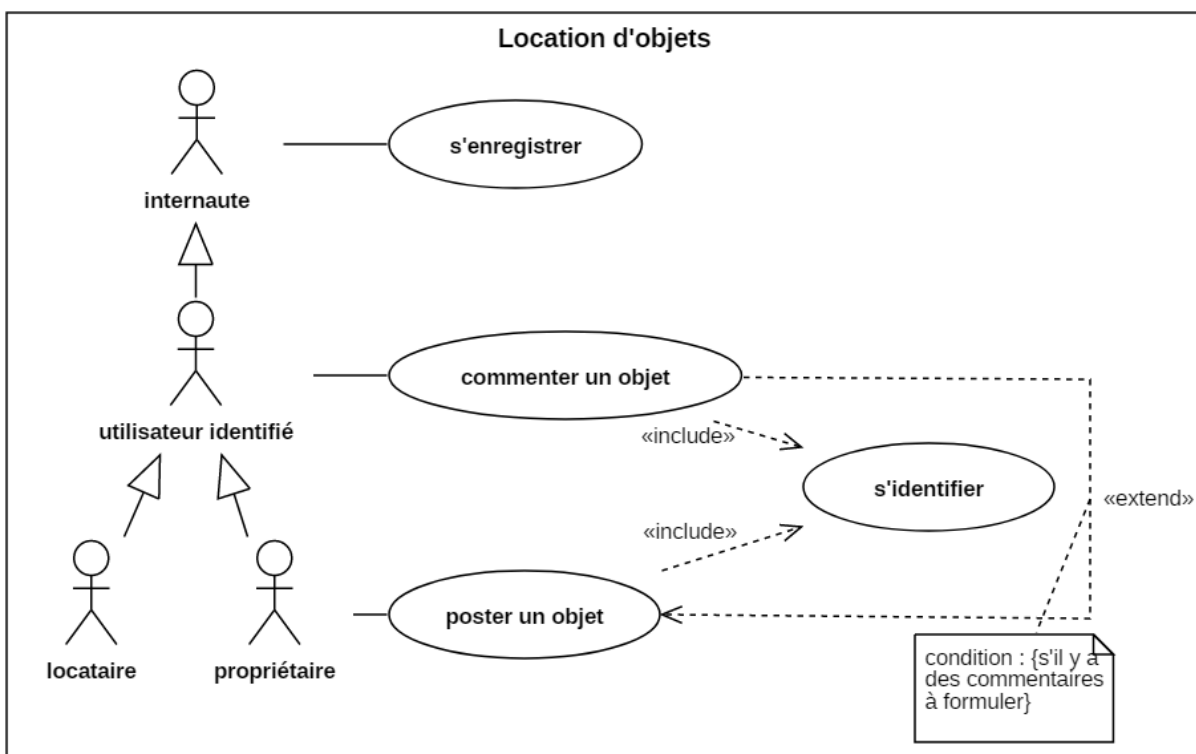


Figure 61 : exigence 5 pour la location d'objets.

Exigence 6 : l'administrateur de la plateforme gère les catégories servant à classifier les objets et modère les commentaires sur les objets.

Finalement, le diagramme de cas d'utilisation de la Figure 62 rassemble les constructions précédentes et ajoute l'acteur administrateur et ses deux cas d'utilisation.

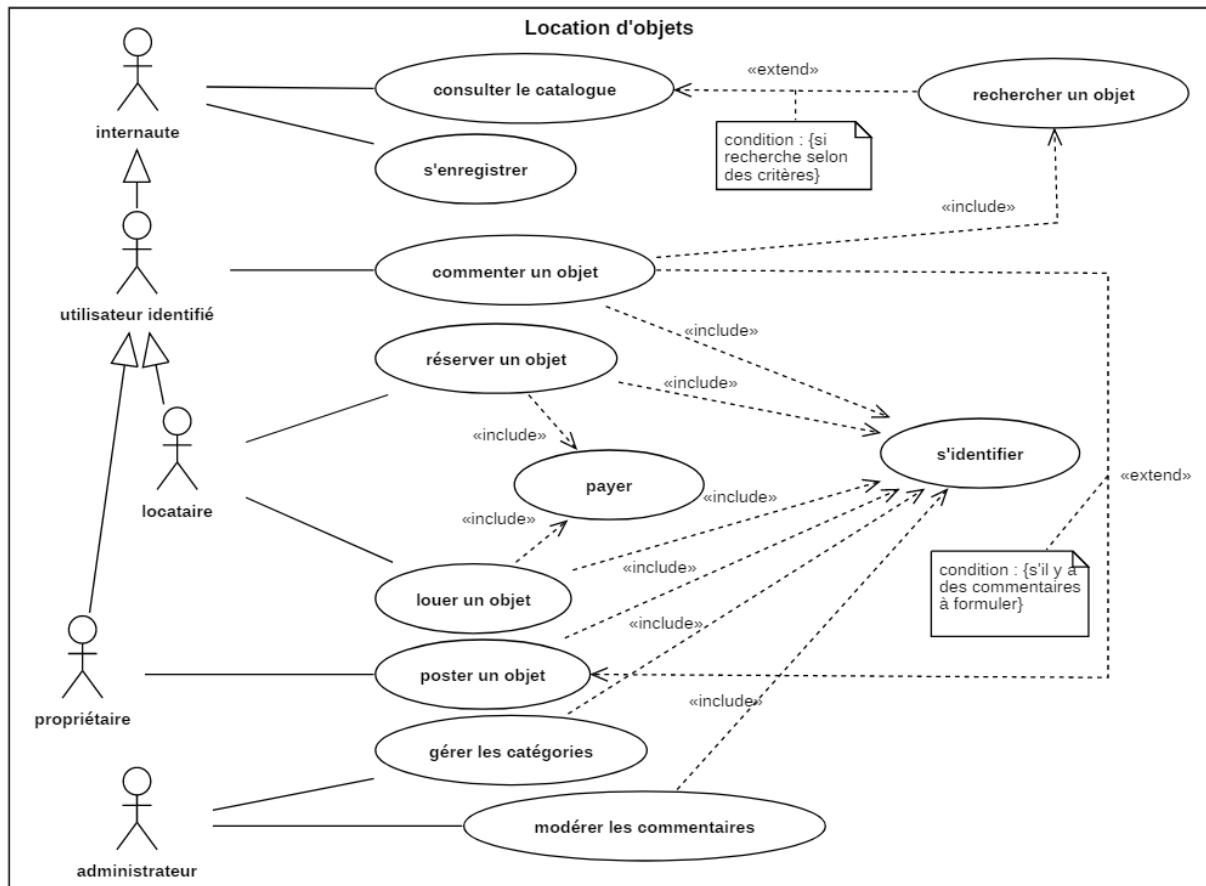


Figure 62 : diagramme de cas d'utilisation pour la location d'objets.

4.8 Exercices

EUC01 : compagnie d'assurance. Proposez un diagramme de cas d'utilisation (sans détailler les séquences) pour le système suivant.

Une compagnie d'assurance souhaite une solution pour la gestion de sa branche "assurance auto" à partir d'un site Internet.

Le site à développer doit permettre à un client de réaliser des devis en ligne. Le client doit aussi pouvoir souscrire une nouvelle assurance pour son véhicule et saisir une déclaration de sinistre. Chaque client doit être identifié par un nom d'utilisateur et un mot de passe pour accéder à ces deux dernières fonctionnalités.

La demande de souscription est examinée par un employé du service contrats. Ce dernier vérifie que la demande est bien formulée. Le cas échéant, il vérifie le bonus/malus du client et la bonne immatriculation du véhicule avec la collaboration d'une centrale des assurances.

Les employés de tous les services peuvent visualiser l'ensemble des données liées à un client. Naturellement, les employés doivent s'identifier pour accéder à une des fonctions du système.

Formulez d'éventuelles hypothèses qui interprètent ou précisent l'énoncé si vous estimez que ce dernier est ambigu.

Que proposeriez-vous comme fonctions supplémentaires ou comme améliorations ?

EUC02 : streaming. Une société de développement Web souhaite créer une plateforme d'échange de films en ligne. Proposez un diagramme de cas d'utilisation (sans détailler les séquences) pour le système suivant.

Le système permet à un internaute non identifié de rechercher des films dans le catalogue. Un utilisateur identifié pourra télécharger des films dans le cadre d'un certain quota de volume. Si le quota est atteint pendant qu'il télécharge des films, il peut acheter du quota supplémentaire. Un utilisateur peut aussi acheter du quota avant ou après le téléchargement. Dans tous les cas, il doit s'identifier lors de ces opérations.

Un contributeur, identifié lui aussi, peut poster des films sur la plateforme et rechercher des contenus. S'il poste une nouvelle version d'un film existant, alors il peut remplacer l'ancienne version par la nouvelle.

Un robot logiciel examine les films de la plateforme. En cas de contenus à censurer, le robot supprime le film.

Formulez d'éventuelles hypothèses qui interprètent ou précisent l'énoncé si vous estimez que ce dernier est ambigu.

Imaginez un scénario pour la recherche de films.

EUC03 : Tax on Web. La plateforme Tax on Web est un système de déclaration fiscale en ligne.

Elle permet à un déclarant d'encoder un montant dans une rubrique de sa déclaration, mais aussi de corriger ce montant. Lors de l'encodage ou de la modification d'une rubrique, le déclarant peut poster solliciter l'aide d'un assistant digital à l'encodage si un tel assistant est disponible pour la rubrique en question.

Le déclarant utilisera la plateforme pour poster une annexe à sa déclaration, calculer l'impôt à payer et clôturer sa déclaration. Au cours de ces deux derniers traitements, la déclaration est vérifiée.

Un déclarant est soit un citoyen, soit un comptable.

Seul un comptable peut utiliser le système pour demander un délai pour clôturer une déclaration.

Quant au citoyen, il peut demander en ligne un RV avec un fonctionnaire de l'administration fiscale.

Que ce soit lors de la demande de délai ou la demande de RV, un fonctionnaire de l'administration utilise le système pour accepter la demande.

Proposer un diagramme de cas d'utilisation pour ce problème. Formulez d'éventuelles hypothèses qui interprètent ou précisent l'énoncé si vous estimez que ce dernier est ambigu.

EUC04 : musique en ligne.

Une plateforme de vente de musique en ligne permet à tout internaute de rechercher un album de musique, et éventuellement de voir les détails de l'album recherché. Un internaute peut se créer un nouveau compte sur la plateforme.

Les clients identifiés peuvent créditer leur compte d'un certain montant et télécharger un album après l'avoir recherché. Les internautes peuvent laisser un commentaire d'évaluation sur un album qu'ils ont téléchargé.

Proposez un diagramme de cas d'utilisation sans détailler les scénarii.

Solution. La Figure 63 donne une solution possible.

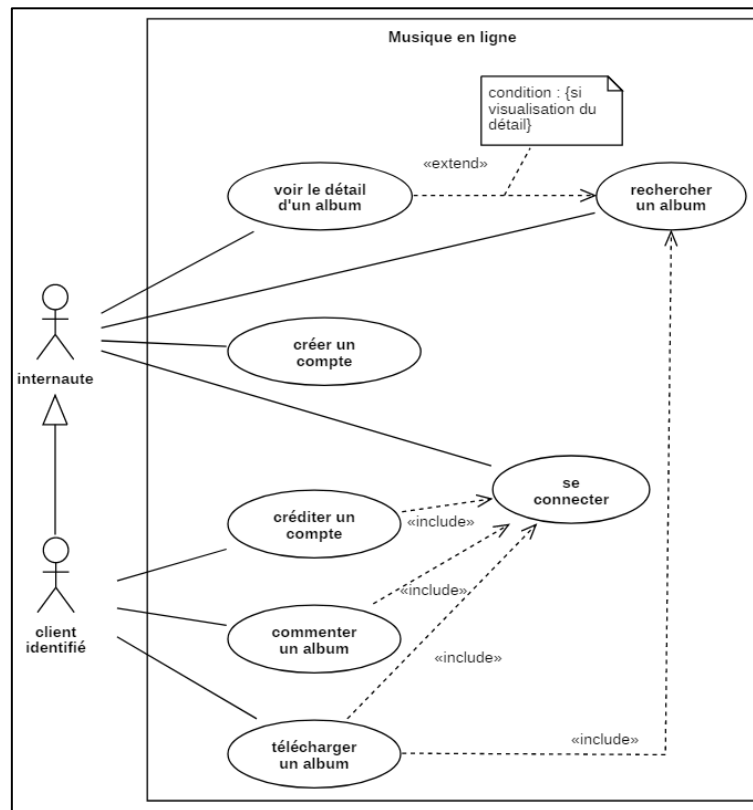


Figure 63 : diagramme de cas d'utilisation pour l'exercice de musique en ligne.

EUC05 : réservation de terrains de tennis.

On vous demande un diagramme de cas d'utilisation sans détailler les scénarii pour un club de tennis qui souhaite développer un système de réservation des terrains en ligne.

Le système permet aux membres de visualiser les réservations déjà effectuées et de réserver un terrain après s'être identifiés. Si un membre ne peut enregistrer qu'une seule réservation, les professeurs de tennis du club peuvent, eux, réserver plusieurs terrains à la fois.

Le secrétaire administre le système en définissant les terrains disponibles et en gérant les comptes des membres qui sont en ordre de cotisation. Il peut aussi consulter les statistiques de fréquentation des terrains.

La consultation des réservations est accessible à tout internaute afin de stimuler l'inscription de nouveaux membres. D'ailleurs, un internaute non-membre peut remplir en ligne un formulaire d'inscription.

Solution. La Figure 64 montre une solution possible. Le cas s'identifier est appelé par de nombreux autres cas qui nécessitent une identification. Cette organisation est assez commune.

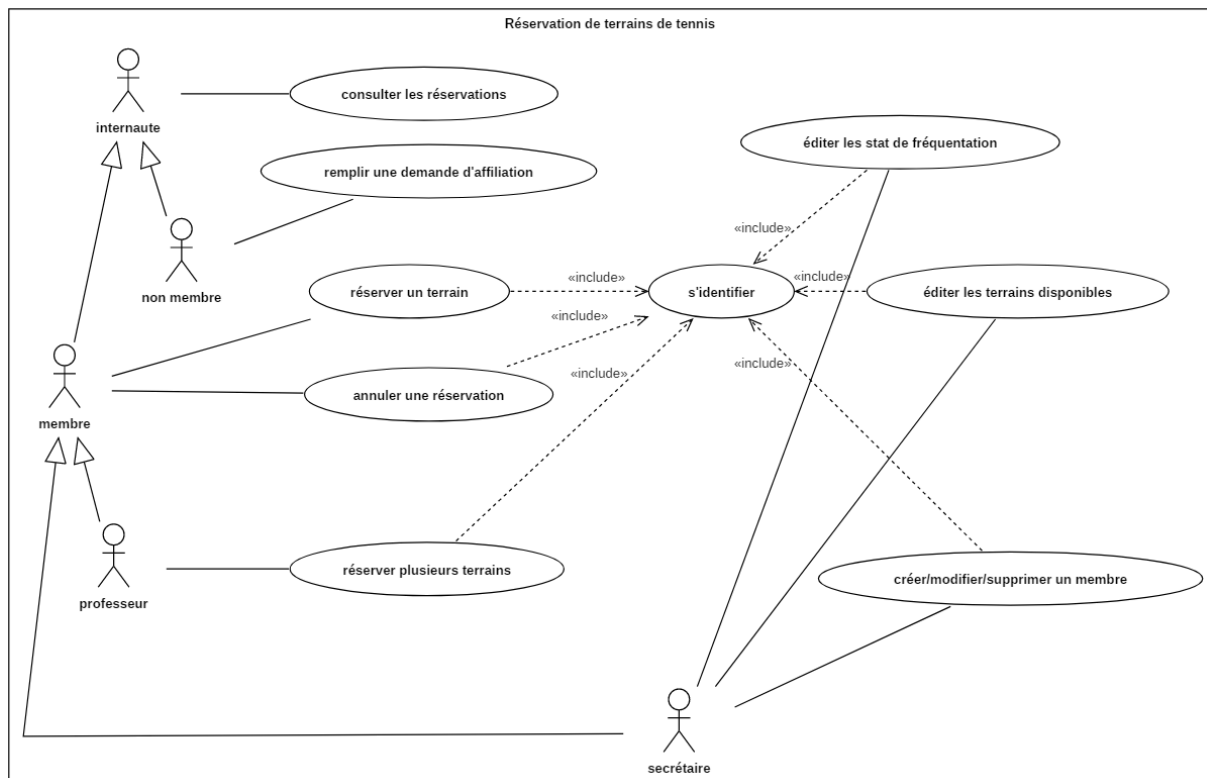


Figure 64 : diagramme de cas d'utilisation pour l'exercice de réservation de terrains de tennis.

EUC05 : check-in dans un aéroport.

Un aéroport régional souhaite déployer des bornes automatiques à l'intention de ses passagers.

Le système permet à un passager de réaliser son check-in en toute autonomie. Lors de cette opération, le passager peut procéder au chargement de ses éventuels bagages en soute.

Un passager peut imprimer sa carte d'embarquement. Il devra obligatoirement s'identifier pendant cette opération.

Certains passagers voyagent au sein d'un groupe qui est géré par un guide. Ce dernier peut procéder à une opération de check-in collectif pour l'ensemble du groupe, avec un éventuel chargement de bagages. Cependant, le guide doit au préalable réaliser son propre check-in.

D'autres passagers sont handicapés et peuvent utiliser les bornes pour rentrer en contact avec un employé de l'aéroport afin de demander de l'assistance. L'employé utilise alors lui aussi le système.

Tout passager peut émettre une plainte via une borne moyennant identification.

Proposez un diagramme de cas d'utilisation sans détailler les scénarii, mais en suggérant des améliorations au dispositif décrit.

Solution. Une solution possible est présentée par la Figure 65.

Sur le terrain, l'élaboration de diagrammes, et plus généralement la spécification d'exigences, suit un processus itératif à l'occasion duquel la solution est complétée de version en version à la suite des appréciations successives des utilisateurs. Cette pratique est difficile à reproduire dans un ouvrage comme celui-ci, mais le lecteur doit garder à l'esprit que la solution émerge progressivement des interactions entre les acteurs du système.

Pour l'exercice qui nous occupe, les pistes d'amélioration à soumettre à d'hypothétiques utilisateurs sont nombreuses et sont laissées au soin du lecteur. Mentionnons cependant le besoin très probable de s'identifier lors des opérations de check-in.

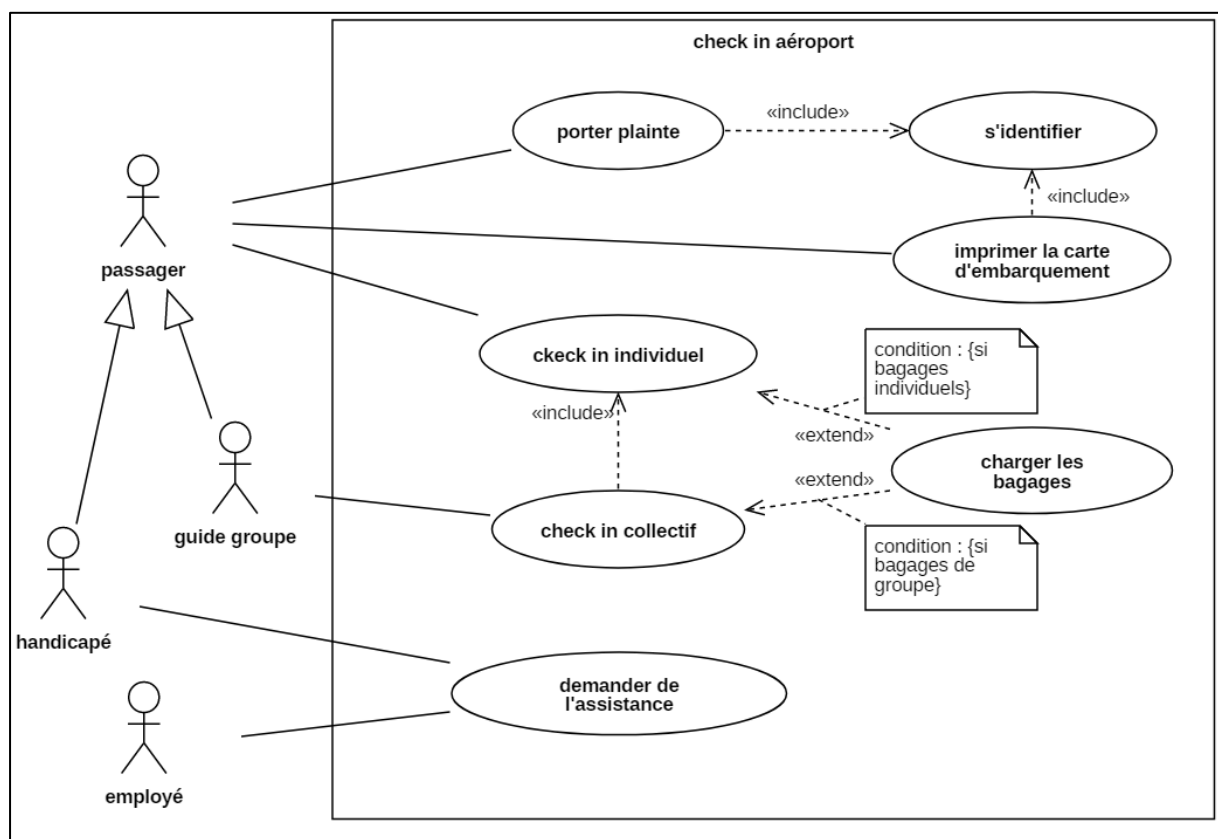


Figure 65 : diagramme de cas d'utilisation pour l'exercice de check-in dans un aéroport.

5 Diagramme d'activité

Les diagrammes d'activité décrivent les fonctions et la dynamique du futur système, c'est-à-dire l'ordre dans lequel les traitements sont réalisés. En pratique, les diagrammes d'activité sont souvent utilisés pour détailler ou reformuler le scénario d'un cas d'utilisation ou pour décrire un processus d'affaires.

Définition 13 : un diagramme d'activité décompose une fonction attendue d'un système informatique en actions. Il montre comment ces actions s'enchaînent dans le temps et comment elles se transmettent de l'information.

Les principaux composants de base d'un diagramme d'activité sont :

- des actions, qui décomposent la fonction (ou l'activité) visée par le diagramme (Section 5.1) ;
- des flux de contrôle, qui connectent les actions dans le temps selon une logique de prédécesseur-successeur (Section 5.2) ;
- des flux d'objets, qui matérialisent les échanges d'information entre les actions. En UML, l'information qui transite d'une action à l'autre prend la forme d'un objet (Section 5.5) ;
- des nœuds de contrôle, comme des branchements conditionnels (Section 5.3) ;
- une localisation des actions pour préciser quelle entité (acteur, département, etc.) est censée exécuter l'action (Section 5.4).

Définition 14 : une activité est un graphe orienté dont les nœuds représentent ses composants, comme des actions ou des nœuds de contrôle. Ces nœuds sont connectés par des flux de contrôle ou des flux d'objets.

Un diagramme d'activité représente une activité.

Les diagrammes d'activité restent au niveau d'une description du système vu par l'utilisateur, en ignorant son organisation interne. Notons que les diagrammes d'activité s'inspirent largement des notations BPMN (**B**usiness **P**rocess **M**odel and **N**otation présentées dans la Section 7 (BPMN, 2015).

5.1 Action

Définition 15 : une action représente une étape indivisible d'une activité.

La Figure 66 montre la notation d'une action et quelques exemples. Par essence, une action n'est pas décomposable en unités plus fines. Par contre, elle détaille une activité de plus haut niveau, comme celle correspondant à un cas d'utilisation.

Une action peut désigner une opération exécutée par le système. Dans ce cas, elle modifie l'état du système, par exemple en mettant à jour des données, ou elle renvoie de l'information sur l'état du système, par exemple en produisant la liste de toutes les commandes ouvertes. Mais une action peut également décrire un traitement pris en charge par un acteur humain ou un autre système.

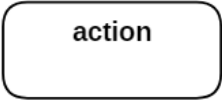
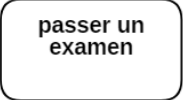
<u>Notation</u>	<u>Exemples</u>		
			

Figure 66 : notation et exemples d'actions.

Spécifier uniquement le nom des actions n'est pas suffisant pour permettre aux informaticiens de les programmer. Une technique usuelle consiste à décrire l'effet d'une action en termes de postconditions, c'est-à-dire des affirmations qui doivent être vérifiées après l'exécution de l'action. Par exemple, les postconditions d'une action enregistrer la commande seraient :

- une nouvelle commande est créée dans le système et est reliée au client ;
- les quantités disponibles en stocks sont réservées à concurrence des quantités commandées ;
- la liste des commandes à préparer est mise à jour.

5.2 Arc entre des actions

Les actions, voir les activités, peuvent être connectées par des arcs de flux de contrôle ou de flux d'objets.

Définition 16 : un flux de contrôle représente une transition entre deux actions. La fin de l'exécution d'une action à l'origine du flux de contrôle déclenche l'exécution d'une seconde action située à l'extrémité du flux.

Les flux de contrôle montrent donc les séquences d'enchaînements des actions dans le temps (Figure 67). Il est préférable de se limiter à un seul flux entrant et un seul flux sortant par action pour des motifs de clarté⁷⁷.

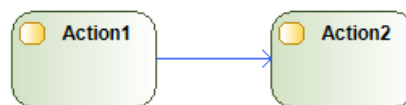


Figure 67 : notation des flux de contrôle.

Finalement, une condition de garde peut compléter un flux (de contrôle ou d'objet). Cette condition devra être vérifiée pour que le flux soit activé, comme dans la Figure 68.

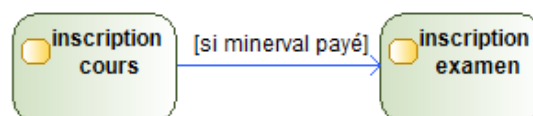


Figure 68 : exemple de condition de garde pour un flux.

La dynamique induite par les flux se comprend parfois mieux avec la métaphore des jetons (Figure 69). Il s'agit d'un mécanisme virtuel de coordination pour décrire l'ordre d'exécution :

- une action qui reçoit un jeton peut s'exécuter ;
- une action qui a terminé son exécution passe le jeton à l'action suivante, ce qui déclenche son exécution ;

⁷⁷ Si plusieurs flux arrivent à la même action, tous les prédécesseurs à l'origine de ces flux doivent être terminés pour enclencher cette action, selon la logique de la jointure (voir plus loin). Si plusieurs flux sortent de la même action, alors tous les successeurs seront exécutés, selon la logique de la bifurcation (voir plus loin).

- les conditions de garde peuvent empêcher le passage du jeton.

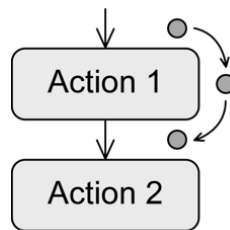


Figure 69 : métaphore du jeton (Seidl, Scholz, Huemer, & Kappel, 2015).

5.3 Nœud

Un diagramme d'activité composé uniquement d'actions et de flux de contrôle n'est pas suffisamment expressif pour représenter des enchaînements complexes d'actions, comme des répétitions ou des branchements conditionnels. UML prévoit donc des nœuds spéciaux pour représenter de tels enchaînements.

Définition 17 : un nœud de contrôle organise des flux d'exécution d'une activité au-delà des enchaînements séquentiels.

Début et fin d'activité (Figure 70). Le nœud de début d'activité marque le démarrage de l'activité et fournit un jeton au flux sortant du nœud. Plusieurs nœuds initiaux dans le même diagramme modélisent des chaînes de traitement exécutés en parallèle, ...ce qui peut engendrer des diagrammes complexes.

Le nœud de fin d'activité marque sa terminaison et termine tous les flux de l'activité en cours. Le premier jeton qui atteint un nœud de fin termine toute l'activité, sous-branches concurrentes incluses, et les autres jetons en circulation sont détruits.

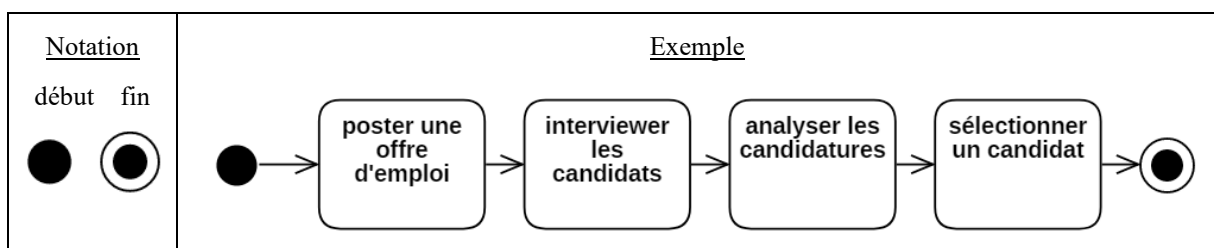


Figure 70 : notation et exemple de nœuds de début et de fin d'activité.

Fin de flot. Le nœud de fin de flot termine uniquement la sous-branche de l'activité où il est placé, et n'affecte pas les autres jetons.

Il se note :

Décision. Le nœud de décision (*choice*) exprime plusieurs alternatives possibles, dont une seule sera choisie en fonction de conditions appelées conditions de garde. Dans la partie gauche de la Figure 71, après la terminaison

de l'action 1, soit l'action 2 est exécutée si la condition garde 2 est vraie, soit l'action 3 est exécutée si la condition de garde 3 est vraie. Un nœud de choix peut comporter autant de flux sortants que l'on veut, du moment qu'ils sont assortis d'autant de conditions de garde mutuellement exclusives.

Le nœud de décision agit comme un aiguillage qui détermine une (seule) voie de sortie pour un jeton entrant dans le nœud en fonction des conditions de garde.

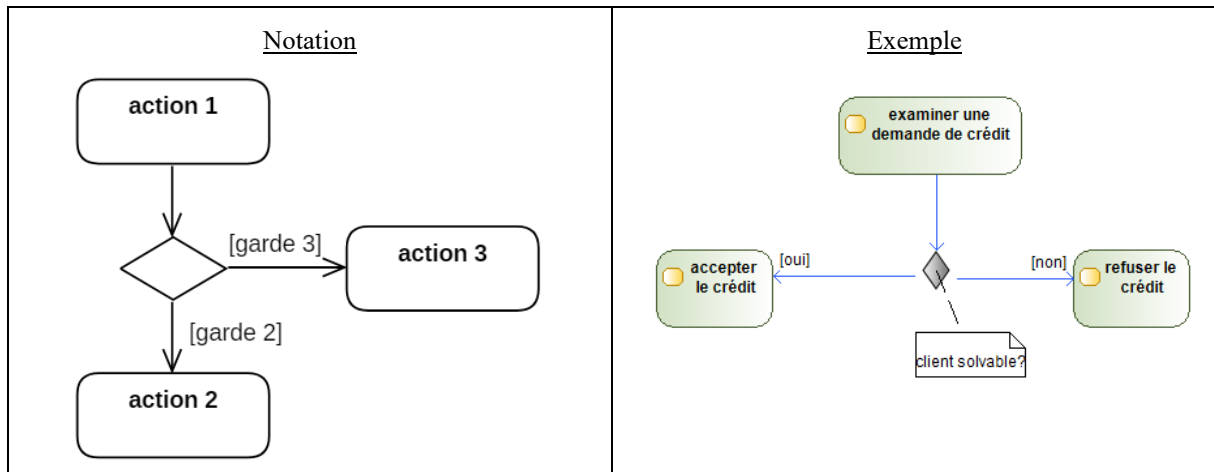


Figure 71 : notation et exemple de nœud de choix.

Fusion. Le nœud de fusion (*merge*) enclenche un flux sortant dès qu'une action liée à un des flux entrants dans le nœud est terminée. Dans la partie gauche de la Figure 72, l'action 3 est exécutée après la terminaison de l'action 1 *ou* la terminaison de l'action 2. Un nœud de fusion peut comporter autant de flux entrants que l'on veut.

Le nœud de fusion transmet le premier jeton entrant vers le nœud suivant. Décision et fusion peuvent être combinées dans le même nœud (losange avec plusieurs flux entrants – fusion- et plusieurs flux sortants – choix).

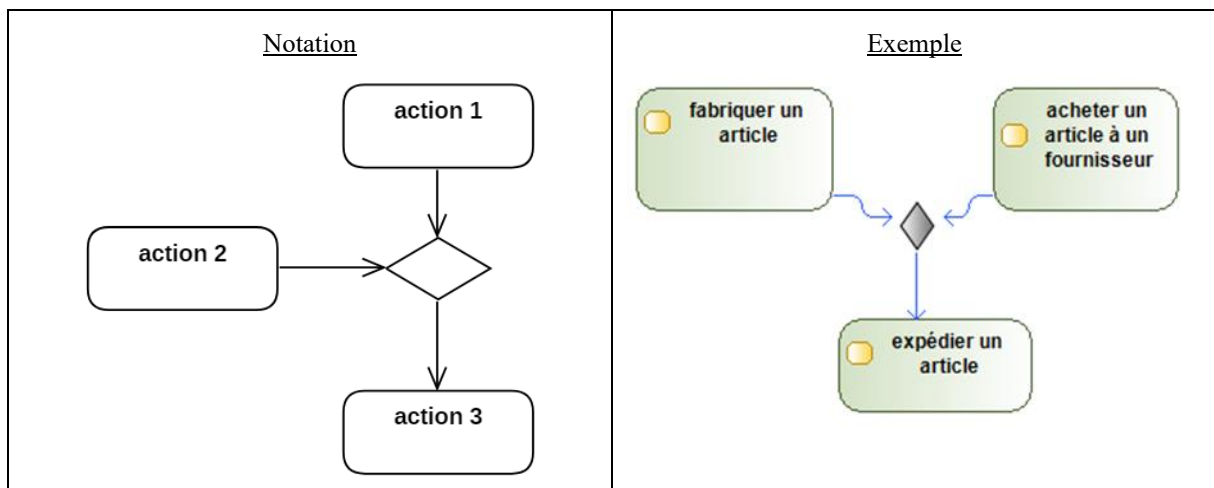


Figure 72 : notation et exemple de nœud de fusion.

Jointure. Le nœud de jointure (*join*) représente une conjonction synchronisée de flux. Dans la partie gauche de la Figure 73, l'action 3 est exécutée après la terminaison de l'action 1 *et* la terminaison de l'action 2 (les deux actions doivent être terminées). Un nœud de fusion peut comporter autant de flux entrants que l'on veut.

Ce nœud attend tous les jetons des sous-branches entrantes, puis les fusionne en un seul jeton sortant.

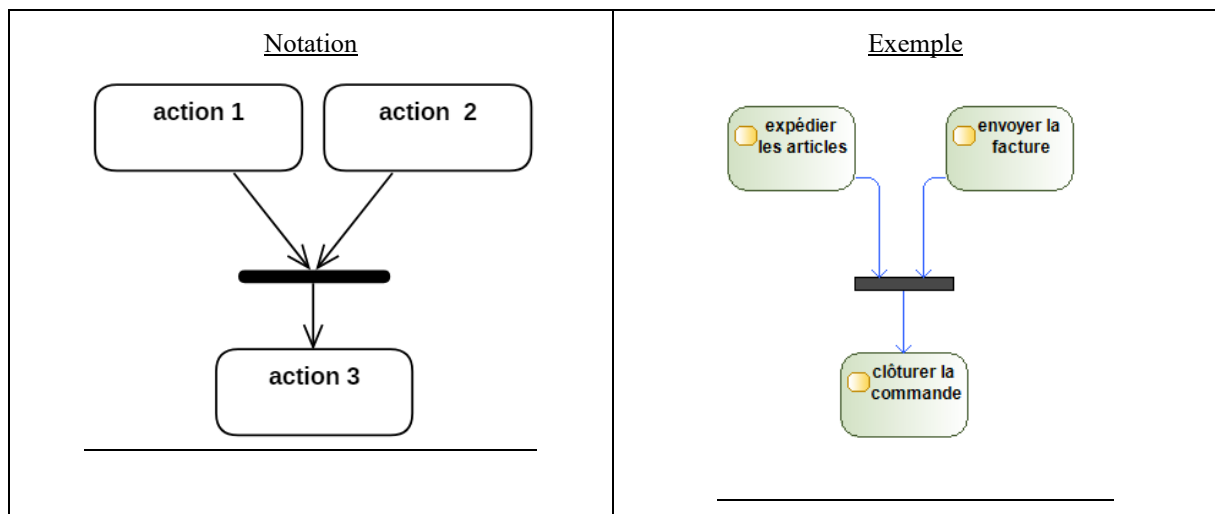


Figure 73 : notation et exemple de nœud de jointure.

Bifurcation. Le nœud de bifurcation (*fork*) représente un déclenchement de flux de contrôle concurrents. Dans la partie gauche de la Figure 74, l’action 2 et l’action 3 sont exécutées après la terminaison de l’action 1. Un nœud de bifurcation peut comporter autant de flux sortants que l’on veut. Il duplique un jeton entrant pour chaque sous-branche sortante.

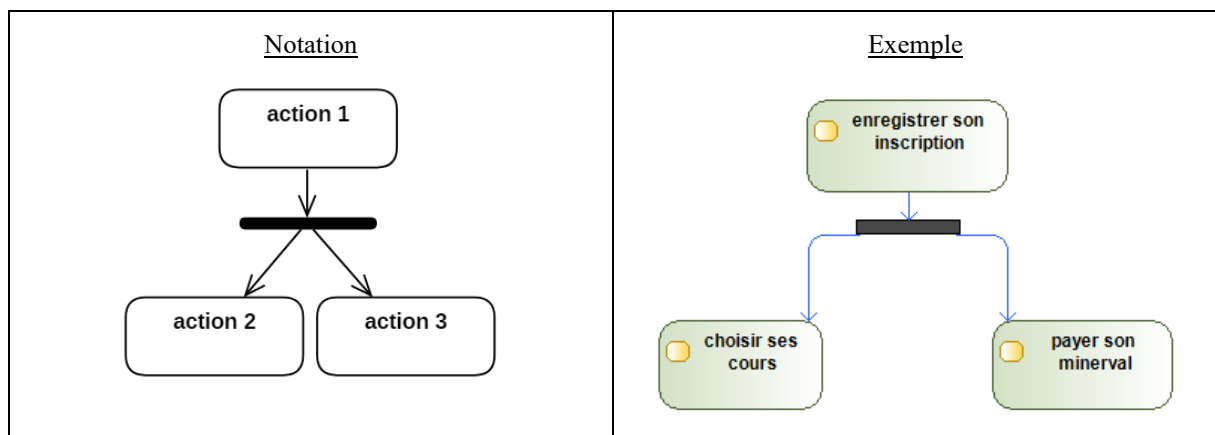
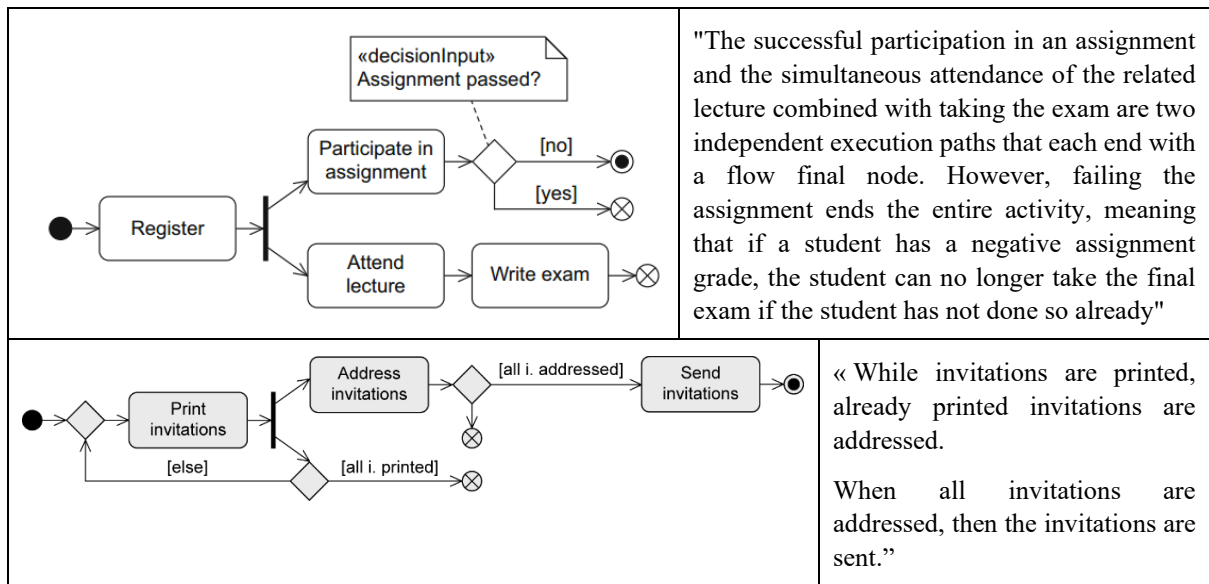


Figure 74 : notation et exemple de nœud de bifurcation.

Notons que bifurcation et jointure peuvent être combinées dans un seul nœud (barre avec plusieurs flux entrants – jointure- et plusieurs flux sortants – bifurcation).

Parallélisme. Un digramme d’activité peut combiner des branches d’actions qui se déroulent en parallèle, notamment avec des nœuds de bifurcation qui agissent comme des multiplicateurs de jetons. Il s’agit alors d’avoir les idées bien claires et de bien gérer la terminaison de chaque branche, comme dans les deux exemples du tableau suivant tirés de (Seidl, Scholz, Huemer, & Kappel, 2015).



Interprétation de notations implicites. Il est conseillé d’expliciter tous les nœuds d’un diagramme d’activité pour éviter de devoir interpréter des notations par défaut. Ainsi, deux flux sortants d’une même action sous-entendent une logique de bifurcation, alors que deux flux entrants dans une même action sous-entendent une logique de fusion (attention, BPMN diffère dans son interprétation par défaut). Ainsi, la Figure 75 montre deux diagramme parfaitement équivalents. La version du dessus sera préférée car plus explicite.

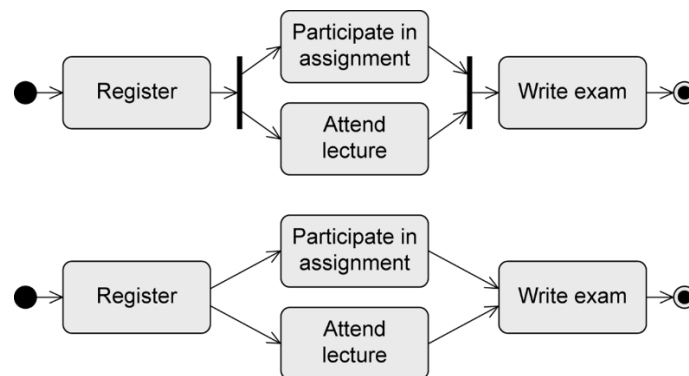


Figure 75 : interprétation de nœuds non explicites.

5.4 Partition

Une partition est un mécanisme de groupage des composants d’un diagramme d’activité. Les partitions servent à indiquer l’entité qui est en charge d’un groupe d’actions, comme un acteur ou un département. Les partitions peuvent être dessinées horizontalement et/ou verticalement, comme dans la Figure 76.

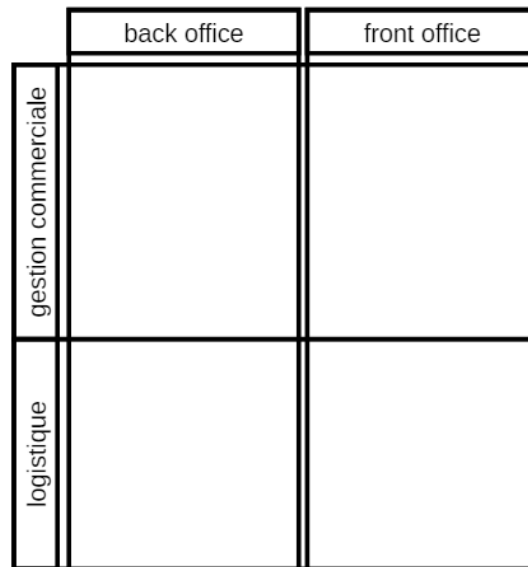


Figure 76 : notation de partitions.

La Figure 77 montre un diagramme d'activité décrivant un processus de traitement d'une commande. Les différentes actions sont réalisées par le client, par le département vente ou par le département logistique. Après l'envoi d'une commande par le client, deux flux d'exécution parallèles sont enclenchés : le paiement du client et le traitement de la commande par les deux départements. En cas de rupture de stock, le département logistique réassort le stock. La commande ne peut être expédiée qu'après son paiement et sa préparation.

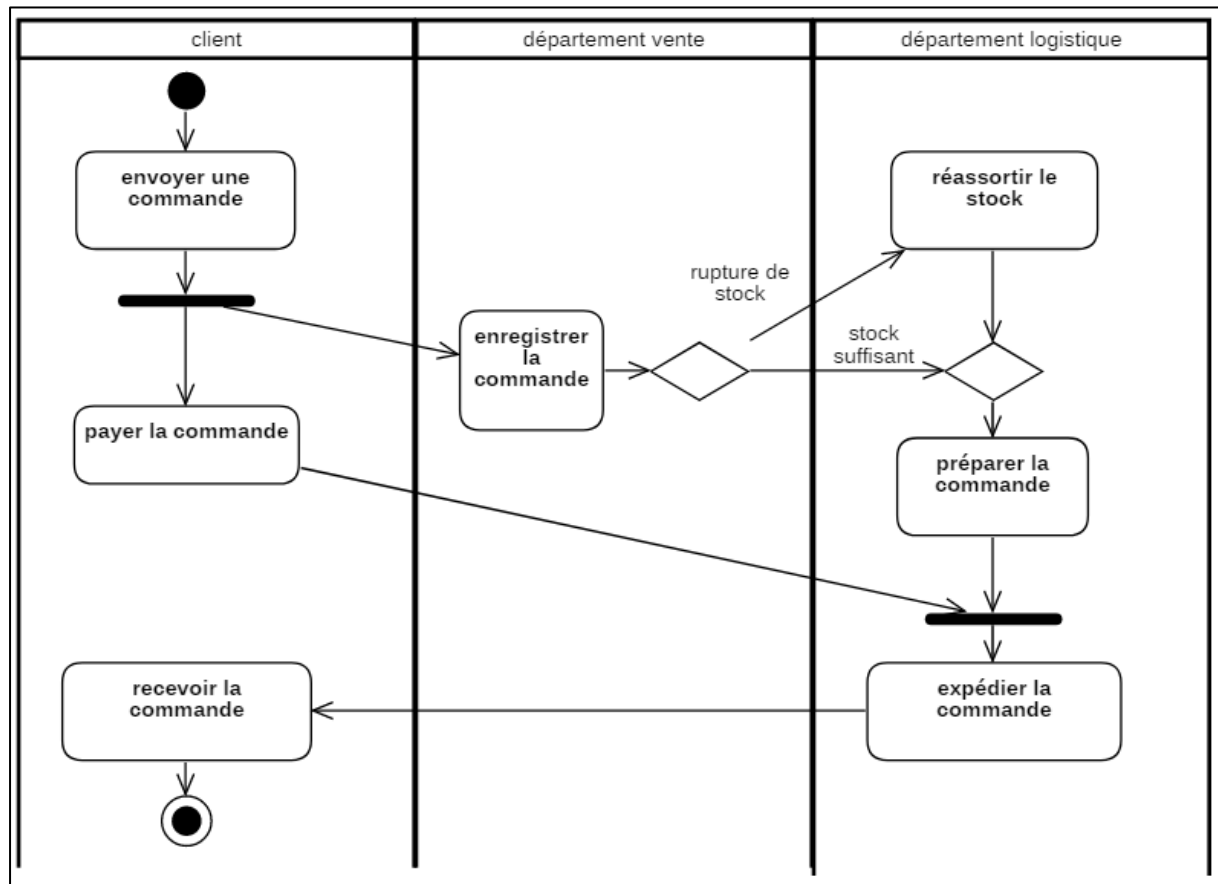


Figure 77 : exemple de diagramme d'activité pour traiter une commande.

Les constructions présentées jusqu'ici constituent un ensemble de base qui permet de décrire assez simplement des activités courantes dans le domaine de la gestion. UML en prévoit cependant bien d'autres dont certaines sont présentées dans la suite. Le lecteur se tournera vers la littérature spécialisée pour appréhender l'ensemble des notations (Bock, 2003-2004).

5.5 Données et persistance

Même si l'intention première d'un diagramme d'activité est de décrire des processus avec un focus sur les dimensions fonctionnelle et dynamique du système, il inclut des notations pour les données qui transitent entre les actions ainsi que des mécanismes de stockage temporaires ou persistants. Et si représenter des transports de données ici a un sens pour l'utilisateur, les langages dédiés à la dimension structurelle, comme l'entité/association, sont bien plus expressifs pour modéliser des schémas de données élaborés. Tant tous les cas, on veillera à ce que les différentes descriptions complémentaires du même système restent cohérentes entre elles.

Nœud et flux objet. Un nœud objet matérialise l'échange de données ou d'objets entre des actions et s'intègre naturellement à un flux d'objet qui connecte une action source de l'objet à une action cible qui se déclenche avec le flux d'objet. Ce dernier cumule donc flux contrôle et transport de données sous forme d'un objet, comme dans l'exemple de la Figure 78. En option, le statut de l'objet peut être mentionné (ici, le statut *préparée* pour l'objet *commande*).

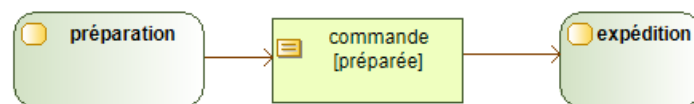


Figure 78 : exemple de flux d'objet.

Les flux d'objets servent, entre autres, à décrire le cycle de traitement d'une entité, comme une commande, un article ou une plainte. Dans l'exemple de la Figure 79, un objet est représenté par un rectangle dont le libellé mentionne son nom et son état (entre crochets). Ici, un objet *commande* est transmis entre différentes actions qui s'enchaînent en modifiant successivement son état de placée à facturée.

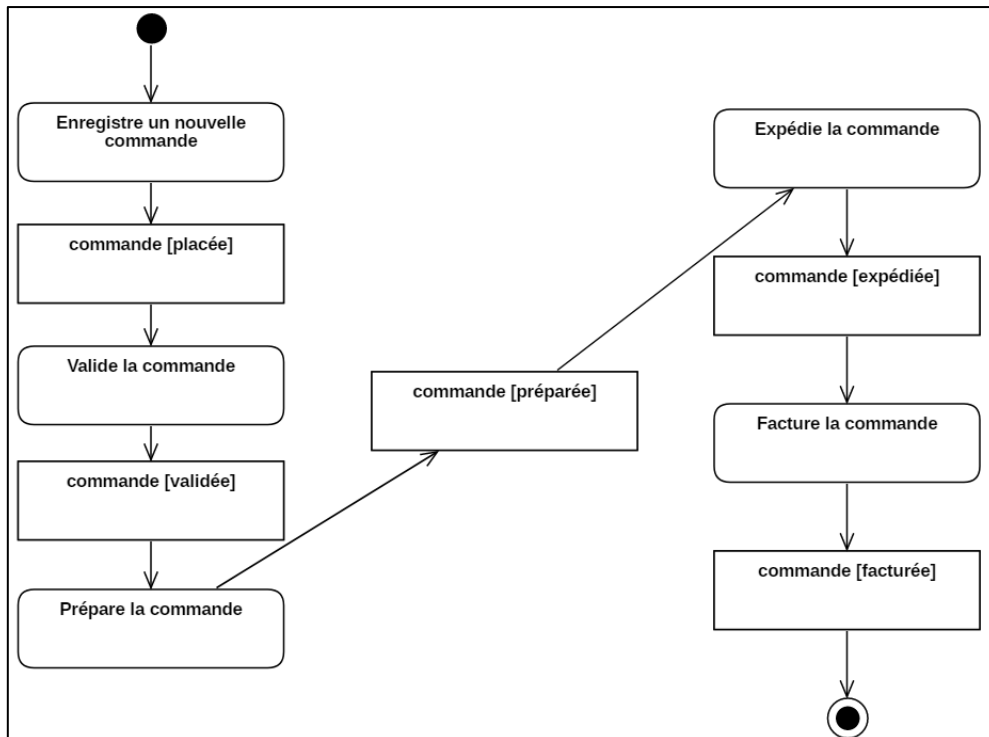


Figure 79 : exemple de cycle de transformation d'un d'objet.

Buffer central et datastore. Ces deux constructions représentent un dispositif de stockage et de restitution des objets.

Le *buffer central* permet une transmission asynchrone d'objets entre des actions. Il est volatile car un flux d'objet sortant implique sa suppression du buffer. La Figure 80 montre un exemple de buffer central avec un flux d'objet entrant (qui ajoute un objet dans le buffer) et un flux d'objet sortant (qui retire un objet du buffer). Une notation alternative des objets comme paramètre entrant ou sortant d'une action est utilisée ici.

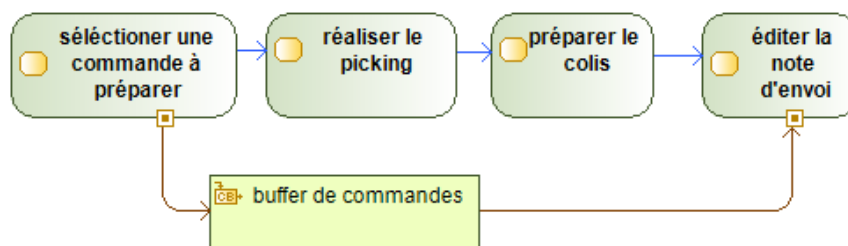


Figure 80 : exemple de buffer central.

Le *datastore* matérialise une mémoire persistante comparable à une base de données, dans laquelle les objets sont sauvegardés de manière permanente, indépendamment de leur consommation (flux sortants) du datastore (Figure 81).

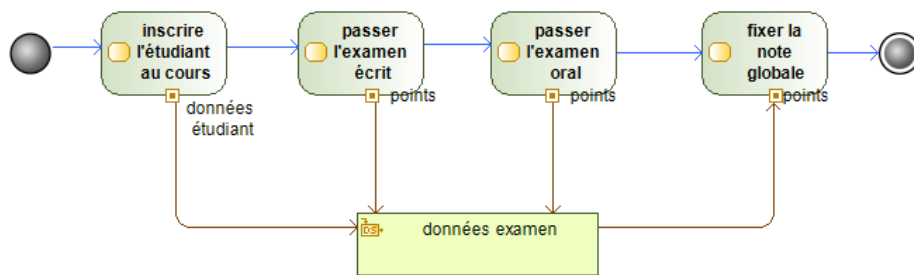


Figure 81 : exemple de datastore.

5.6 Événement, poids de flux et décomposition

Actions événements (event-based actions). Il s'agit de types spécifiques d'actions pour envoyer ou recevoir des signaux vers/de l'environnement du système, ou pour matérialiser des événements dans le temps.

Ainsi, une action d'acceptation représente l'attente de l'occurrence d'un événement de réception d'un signal, comme livraison fournisseur dans l'exemple de la Figure 82.

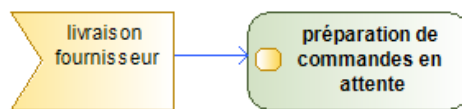


Figure 82 : exemple de réception d'un signal.

Une autre notation représente l'envoi d'un signal vers l'environnement, comme fin de livraison fournisseur dans la Figure 83.

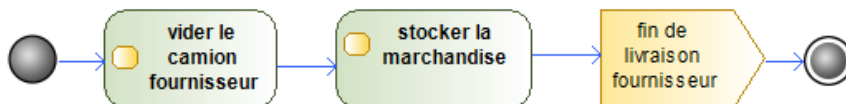


Figure 83 : exemple d'émission d'un signal.

Ces deux dernières constructions sont fréquemment combinées pour représenter un enchaînement asynchrone lié à l'envoi d'un signal à l'environnement et à l'attente d'une réponse correspondante, comme dans la Figure 84 où :

- dès qu'une commande est préparée, un signal de demande de paiement est émis ;
- l'activité attend de recevoir le signal de confirmation de paiement pour continuer son exécution (enchaînement asynchrone lié à un événement externe) ;
- l'arc entre demande de paiement et confirmation du paiement signifie que l'activité est mise en pause jusqu'au moment où le signal confirmation du paiement est reçu. Ce signal ne peut être capté qu'après que le signal demande de paiement ait été émis (sans flux entrant, une action d'acceptation s'exécute sans préalable).

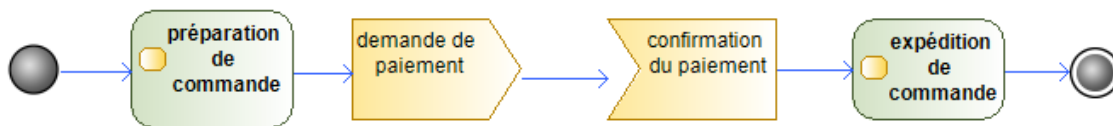


Figure 84 : exemple d'enchaînement asynchrone.

Événement temporel. Le *nœud sablier* représente une action d'acceptation d'un événement temporel, dont la nature est décrite en langage naturel (point dans le temps, échéances répétitives, par exemple tous les mardis à 8h, ou comme dans la Figure 85).

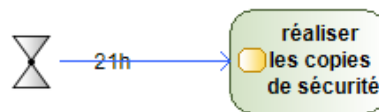


Figure 85 : exemple d'événement temporel.

Poids de flux. Cette notation indique qu'un nombre minimal de jetons doivent être présents pour exécuter l'action cible (par défaut : poids = 1). Dans la Figure 86, la création d'un groupe s'exécutera quand 30 occurrences de l'événement nouvelle inscription se seront produites.

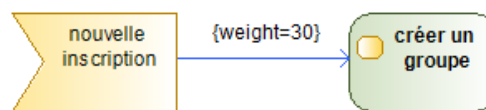


Figure 86 : exemple de poids de flux.

Call behavior action. Finalement, UML dispose d'un mécanisme de décomposition hiérarchique bien utile pour gérer la complexité de diagrammes volumineux : l'appel d'une activité indépendante au sein d'un diagramme de haut niveau. Cette construction permet aussi l'appel de la même fonctionnalité à partir de plusieurs diagrammes appelants, et donc une forme de réutilisation. Dans l'exemple de la Figure 87, réassort stock représente l'appel d'une activité du même nom spécifiée par ailleurs.

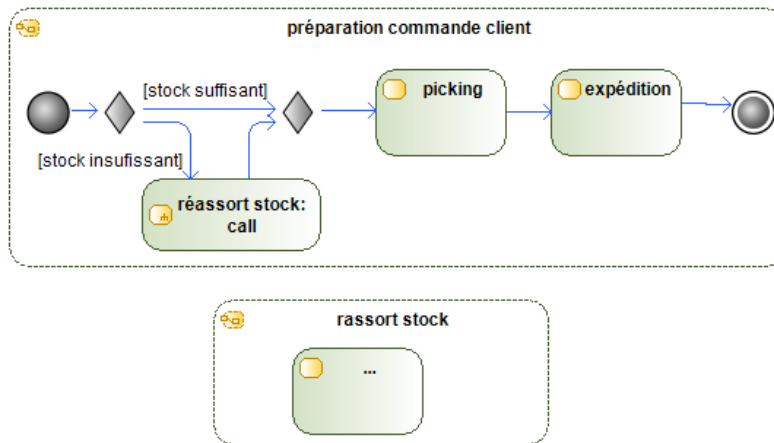


Figure 87 : exemple d'appel d'une activité.

5.7 Exception et zone interruptible

Un système est robuste s'il se comporte raisonnablement même lors de circonstances inhabituelles (en dehors du « happy flow of events »). Les diagrammes d'activité proposent des mécanismes de gestion d'exception pour adopter un comportement adéquat en cas de problème inattendu. Ces mécanismes sont comparables à la gestion des exceptions dans les langages de programmation.

Gestionnaire d'exception. Il définit le comportement du système en cas d'erreur en exécutant une autre action ou un groupe d'actions à la place de celle qui provoque l'erreur, comme le montre la Figure 88. Le flux d'exécution continue régulièrement après l'exécution de l'action ou le groupe d'actions alternatifs.

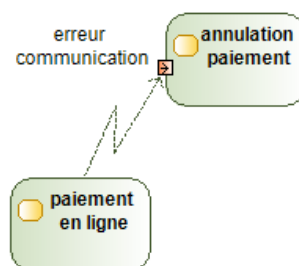


Figure 88 : notation du gestionnaire d'exception.

Plusieurs exceptions peuvent être rattachées à la même action. Dans l'exemple de la Figure 89, l'action `Print exam instructions` peut être remplacée par `Resolve paper jam` si l'exception `Paper jam` se produit, ou par un groupe `Add paper` si l'exception `paper tray empty` se produit.

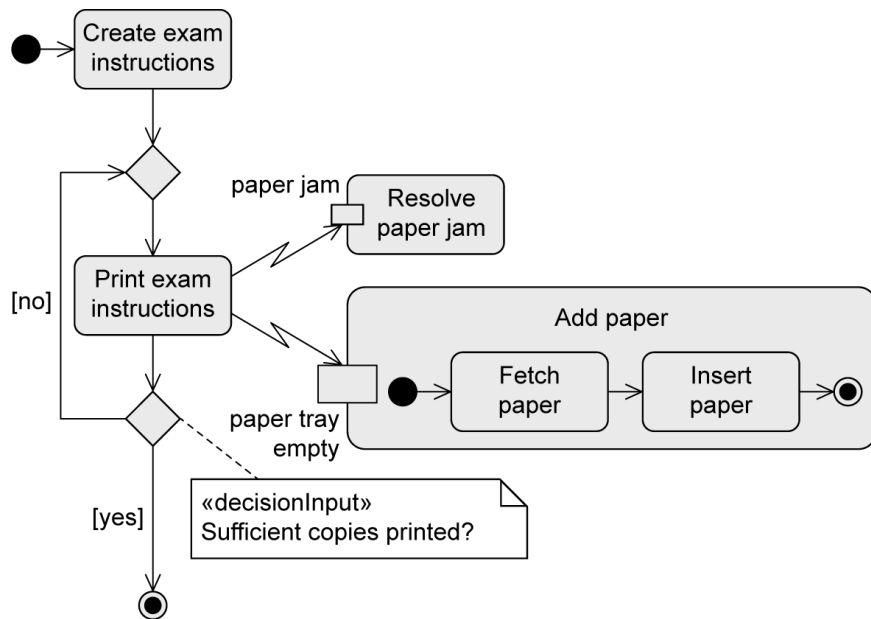


Figure 89 : exemples d'exceptions multiples (Seidl, Scholz, Huemer, & Kappel, 2015).

Interruption. Une *zone interruptible* est une partie délimitée du diagramme d'activité qui englobe un groupe d'actions. L'exécution d'une zone interruptible peut être arrêtée lorsqu'un événement donné se produit. Après interruption, un flux de contrôle est dirigé en dehors de la zone interruptible pour exécuter des actions ad hoc. Le fonctionnement est illustré par la Figure 90 : si l'événement *Erreur* se produit lors de l'exécution de B ou C :

- tous les jetons de la région interruptible sont supprimés ;
- E est exécuté et D n'est pas exécuté (pas de retour à la fin de l'exécution de l'alternative en cas d'erreur).

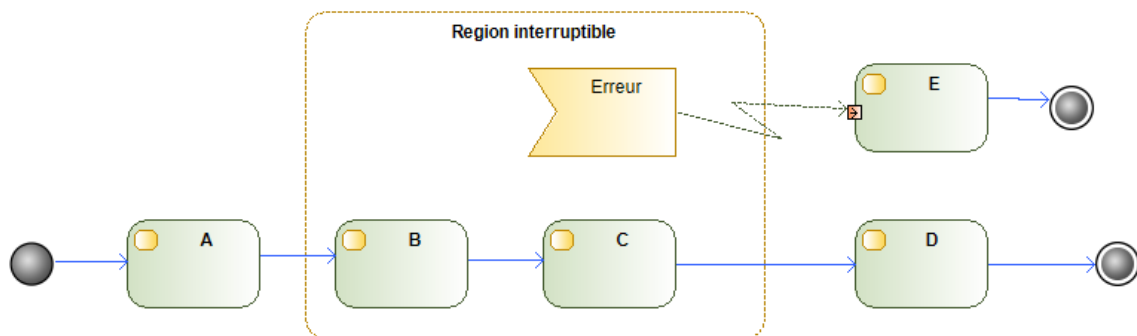


Figure 90 : notation d'une zone interruptible.

Par exemple, une activité de commande en ligne peut être abandonnée à tout moment par le client s'il quitte le site de vente. Dans ce cas, un traitement spécifique est exécuté, par exemple pour annuler la commande qui était en cours avant l'interruption.

5.8 Zone d'expansion

Une zone d'expansion est une partie délimitée du diagramme d'activité qui peut être exécutée plusieurs fois en séquences itératives ou parallèles. L'exécution d'une zone d'expansion peut consommer des objets entrants et produire des objets sortants localisés dans des nœuds d'expansion. Il y aura alors autant d'itérations de la zone d'expansion que d'objets entrants.

Dans l'exemple de la Figure 91, une zone d'expansion englobe des actions pour fabriquer un produit fini à partir d'un objet de matière première. Les actions de cette zone seront itérées pour chaque objet de matière première entrant et produiront, en cas de happy flow, un objet de produit fini. En cas de défaut de fabrication et de mise au rebut, le flux de la zone itérative est interrompu sans délivrer de produit fini, et une nouvelle itération peut commencer.

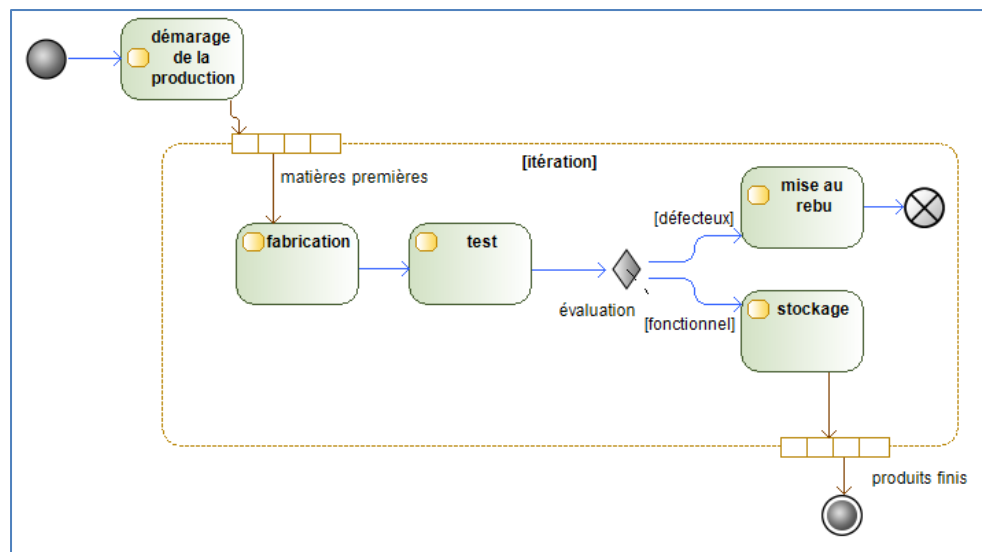


Figure 91 : exemple de zone d'expansion itérative.

5.9 Erreurs fréquentes

La Figure 92 évoque quelques erreurs communément commises dans les diagrammes d'activité :

- des acteurs sont mentionnés en tant qu'actions, comme l'action `vendeur` qui n'en est pas une ;
- des conditions de garde sont formulées en tant qu'actions, comme les actions `si montant supérieur à 1000` et `si montant inférieur à 1000` ;
- des conditions de garde liées aux flux sortants d'un même nœud de choix ne sont pas mutuellement exclusives. Par exemple, `si montant supérieur à 1000` et `si montant inférieur à 1200` se recouvrent ;
- des actions restent sans flux sortant, comme `accorder une réduction`.

Mentionnons encore l'omission du sens des flux, qui rend vite le diagramme incompréhensible.

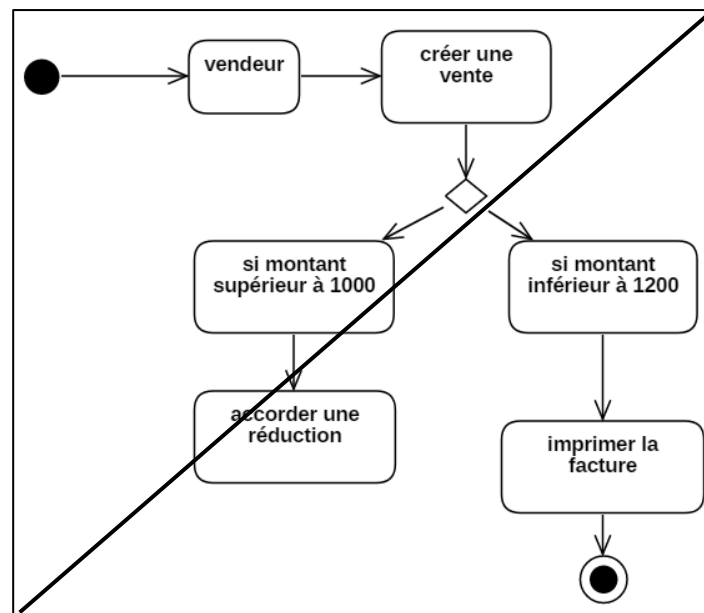


Figure 92 : exemple d'erreurs dans un diagramme d'activité.

5.10 Exemple

Pour illustrer les digrammes d'activité, imaginons le cas d'un détaillant d'articles de sport qui souhaite représenter un processus de prise de commande sur le Web. Le diagramme d'activité correspondant va être construit en traitant progressivement chaque exigence d'un énoncé en langage naturel.

Exigence 1 : le système affiche une page d'accueil à partir de laquelle un internaute peut choisir une rubrique de produits en s'identifiant éventuellement au préalable. Une rubrique correspond à un sport, comme une rubrique tennis ou une rubrique jogging.

Pour commencer, l'internaute peut naviguer vers une rubrique de son choix après une éventuelle identification. On a décidé de ne pas indiquer de partition vu la nature du problème, comme le montre la Figure 93.

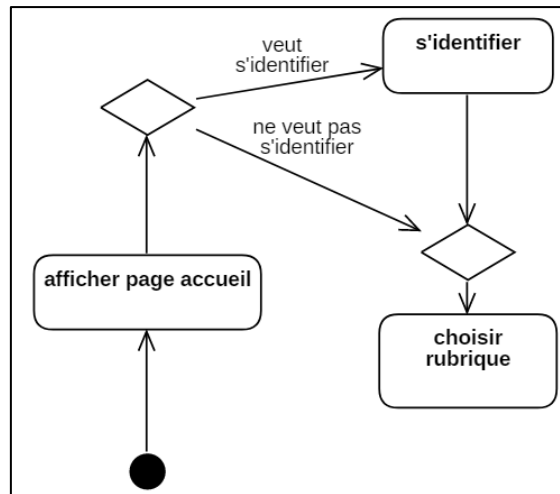


Figure 93 : commande Web – identification.

Exigence 2 : au sein d’une rubrique, l’internaute sélectionne un ou plusieurs articles à commander. Naturellement, les articles proposés sont ceux de la rubrique.

Après avoir précisé une rubrique, l’internaute peut choisir autant d’articles à commander au sein de cette rubrique qu’il le souhaite. La Figure 94 indique donc une boucle sur l’action `choisir article` (la boucle est indiquée sur la figure mais est hors formalisme UML). À ce stade, cette boucle reste incomplète car il n’y a pas encore de moyen d’en sortir⁸.

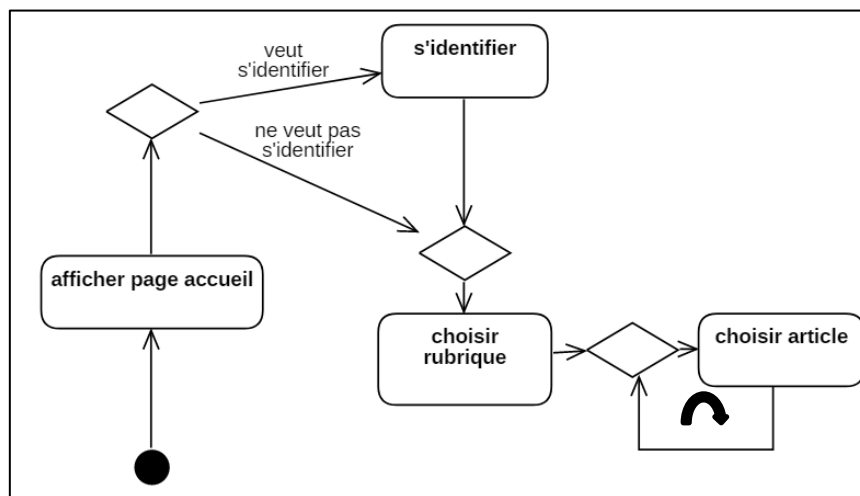


Figure 94 : commande Web – choix d’articles.

Exigence 3 : l’internaute peut naviguer vers d’autres rubriques pour sélectionner d’autres articles à commander. La Figure 95 introduit une seconde boucle pour répéter l’action `choisir rubrique`.

⁸ Les versions intermédiaires du diagramme d’activité sont partielles et certaines constructions peuvent manquer temporairement, comme un nœud de fin.

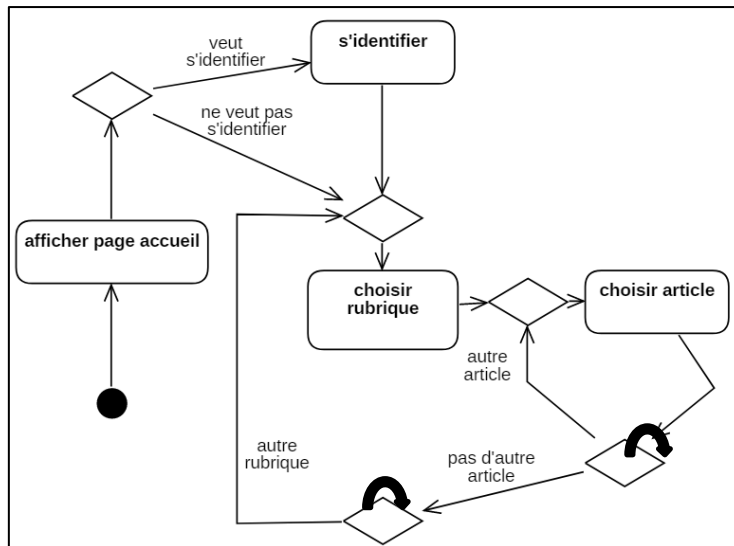


Figure 95 : commande Web – changement de rubrique.

Exigence 4 : quand l'internaute a sélectionné tous les articles qu'il veut commander, et s'il ne souhaite plus consulter d'autres rubriques, le système lui montre alors le récapitulatif de sa commande et l'invite à s'identifier si ce n'est déjà fait. L'internaute clôture la transaction en confirmant la commande.

La sortie de la seconde boucle enchaîne sur le récapitulatif de la commande. Dans la Figure 96, l'internaute clôture la transaction en cas d'identification en début de commande et l'activité prend fin.

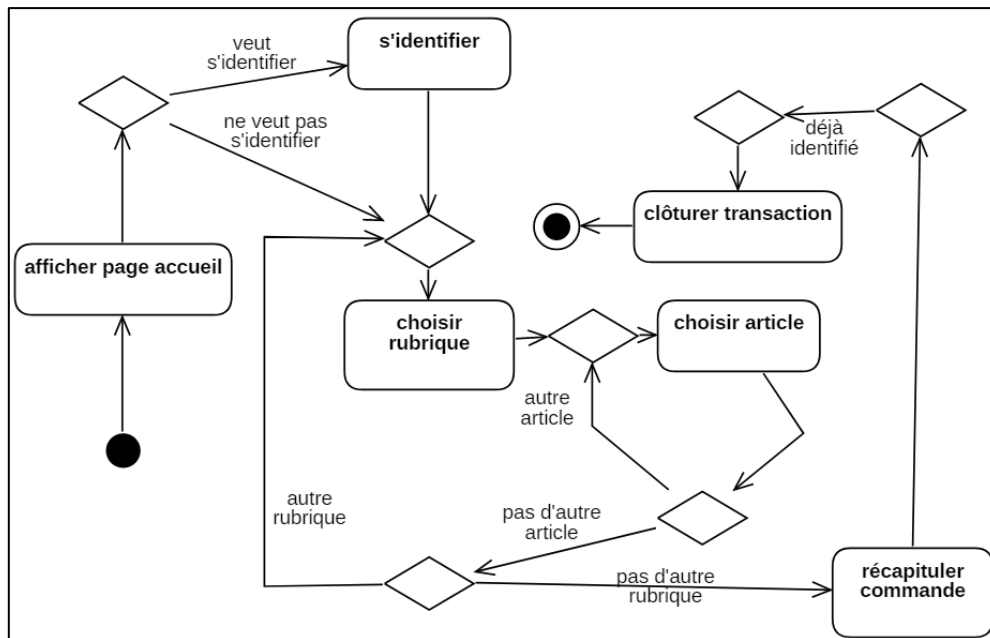


Figure 96 : commande Web – clôture de transaction.

Par contre, si l'internaute ne s'est pas identifié en début d'activité, il doit le faire avant de clôturer la transaction. La Figure 97 indique qu'après l'identification :

- soit la transaction peut être clôturée si une commande est en cours de réalisation ;
- soit on est au début de l'activité et le flux d'exécution est dirigé vers les choix de rubriques et d'articles.

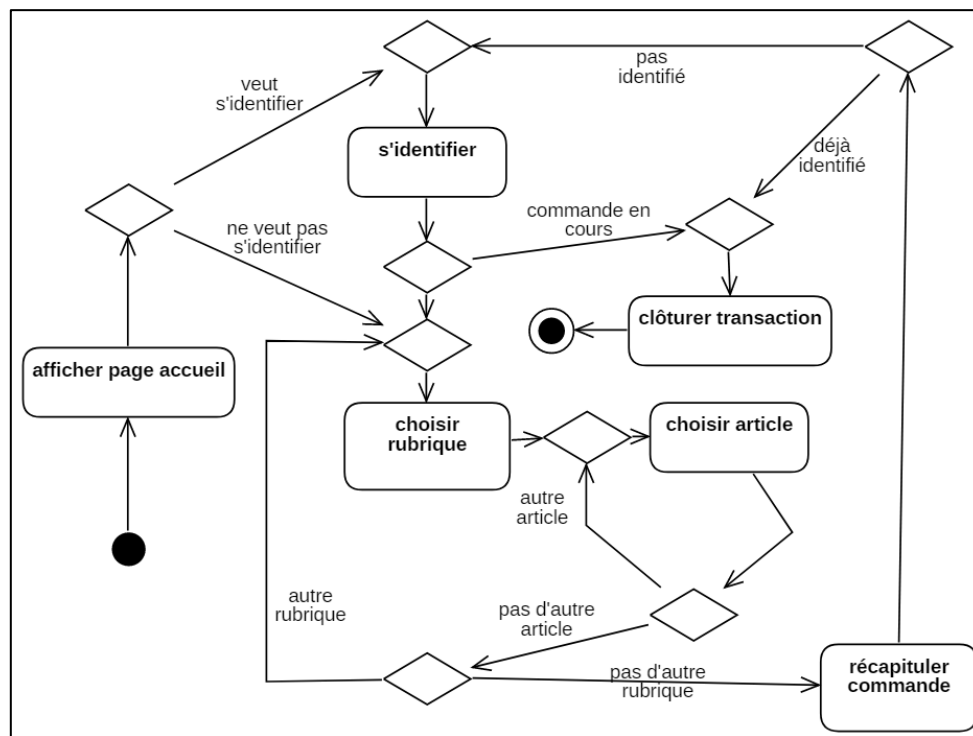


Figure 97 : commande Web – clôture avec identification.

5.11 Exercices

ACT01 : livraison de commande.

Une entreprise de vente de matériel informatique gère le traitement de ses commandes comme suit.

Un client enregistre une commande sur le site de l'entreprise. Le département ventes vérifie alors la commande. Si le solde des paiements en cours du client est supérieur à 1 000, alors la commande est annulée. Sinon le traitement de la commande se poursuit en vérifiant les stocks disponibles pour les articles de la commande.

Si les stocks sont suffisants, le département ventes édite une facture pendant que le service expédition prépare la commande. Quand ces deux actions sont terminées, ce même service expédie la commande en joignant la facture au colis.

Par contre, si les stocks sont insuffisants, le département ventes met la commande en attente et, simultanément, enregistre une nouvelle commande fournisseur pour les articles concernés.

Développez un diagramme d'activités pour formaliser le processus décrit ci-dessus en précisant une partition qui indique les acteurs qui réalisent les différentes actions.

ACT02 : prime énergie

Une administration régionale souhaite communiquer une procédure d'octroi de primes à l'investissement lié à la réduction de la consommation énergétique des ménages. La procédure est la suivante.

Un citoyen demandeur dépose un dossier de demande de prime via le site de l'administration. Le dossier est examiné par l'administration et, s'il est incomplet, le demandeur doit réintroduire son dossier complété. Plusieurs allers-retours de ce genre sont possibles.

Ensuite, l'administration vérifie que le montant total des éventuelles primes déjà accordées au demandeur n'excède pas un montant de 10 000 € pour les 3 dernières années. Si c'est le cas, la procédure est arrêtée. Sinon, on calcule le montant maximal de la prime liée à la demande en cours. Si ce montant dépasse 2000 €, l'administration désigne un expert pour juger de la validité de la demande au domicile du demandeur. Cet expert rédige ensuite un rapport à l'intention de l'administration.

Ensuite, l'administration décide du montant de la prime (après réception de l'éventuel rapport d'expert si le montant de la prime peut dépasser 2000 €). Finalement, la décision est envoyée au demandeur et le dossier de demande est clôturé – ces deux dernières opérations se déroulent en parallèle.

Développez un diagramme d'activités pour formaliser le processus décrit ci-dessus en précisant une partition (swimlanes) qui indique les acteurs qui réalisent les différentes actions. Précisez les éventuelles hypothèses sur lesquelles vous basez votre proposition.

ACT03 : Top Pizza

Top Pizza livre des pizzas à domicile. Le processus est le suivant :

- le client compose sa pizza sur le site puis la commande ;
- une fois la commande postée, Top Pizza reçoit un signal lui indiquant qu'une nouvelle commande est postée ;
- Top Pizza prépare alors la pizza commandée puis la livre ;
- Top Pizza informe le client quand la pizza est en chemin vers son domicile. Après un laps de temps variable, le client reçoit un signal de l'arrivée du livreur sur son GSM. Il paie alors la pizza livrée.

ACT04 : ATM

Les étapes de vérification d'une carte bancaire dans un ATM sont :

- l'ATM lit la carte
- le client saisit son code PIN
- l'ATM vérifie le code PIN

Le client peut appuyer sur le bouton abandon lors de la saisie du code PIN. L'ATM lui demande alors de confirmer l'abandon de l'opération et la vérification s'arrête.

Si le client ne fait rien pendant une minute après avoir introduit sa carte, le système affiche un message de dépassement et la vérification s'arrête

Suggestion : pas de partition

ACT05 : Top Pizza V2

Modifier le processus de Top Pizza comme suit :

- après avoir commandé sa pizza, le client regarde la TV
- si la pizza lui est livrée, il arrête de regarder la TV et paie sa pizza
- si la pizza n'est pas livrée après 1 heure d'attente, il arrête de regarder la TV et va préparer un sandwich

suggestion : définissez une zone interruptible pour représenter l'arrêt de l'action de regarder la TV

ACT06 : Call-center

Un call center fonctionne comme suit. L'opérateur se connecte au système puis sélectionne une liste de contacts à appeler.

Pour chaque contact sélectionné :

- l'opérateur appelle le contact. Si le contact ne répond pas à l'appel, l'opérateur passe au contact suivant ;
- en cas de réponse, l'opérateur propose une offre commerciale ;
- si le contact refuse l'offre, l'opérateur passe au contact suivant ;
- si le contact accepte l'offre, l'opérateur fixe un RV, le contact devient un prospect et l'opérateur passe au contact suivant ;
- à tout moment, si le contact raccroche, alors le contact est supprimé du système et l'opérateur passe au contact suivant.

Après avoir appelé tous les contacts sélectionnés, l'opérateur édite un rapport de performance

Remarque : ne pas utiliser de partition

ACT07 : édition d'un ouvrage.

EdiPlus, une maison d'édition spécialisée dans des ouvrages sur le jardinage, fait appel à vos services pour formaliser son processus de production.

Ce processus commence lorsqu'un auteur travaille sur un manuscrit. L'éditeur évalue ensuite ce manuscrit et peut soit l'accepter, soit demander à l'auteur de le retravailler, soit le refuser définitivement (ce qui met fin au processus de production).

Dès qu'un manuscrit est accepté, l'éditeur réalise une campagne d'information auprès des librairies et, en même temps, un metteur en page met le manuscrit en bonne forme pour l'imprimer. Lorsque le metteur en page a terminé son travail, un imprimeur réalise le tirage numéro 1.

Une fois que la campagne d'information est terminée et que les ouvrages sont imprimés, l'éditeur procède à la vente du tirage numéro 1.

Ensuite, l'éditeur gère les retours d'ouvrages s'il y a des invendus. Si les librairies demandent des exemplaires supplémentaires, alors l'imprimeur procède à un tirage numéro 2 qui est ensuite vendu par l'éditeur.

Développez un diagramme d'activités pour formaliser le processus décrit ci-dessus en précisant une partition qui indique les acteurs qui réalisent les différentes actions.

La Figure 99 montre une solution possible.

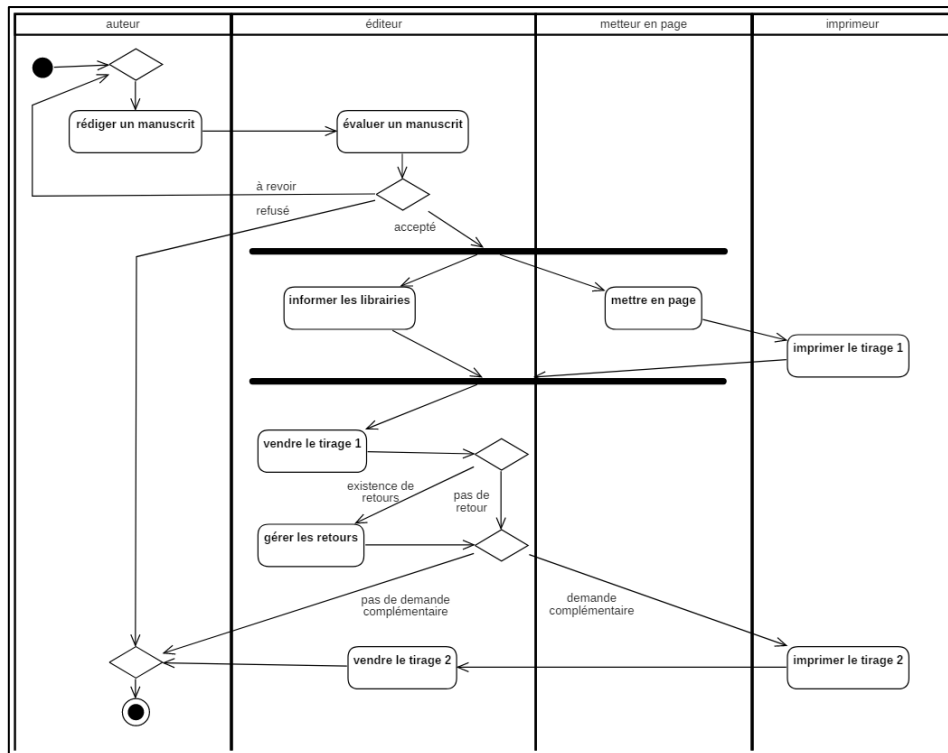


Figure 98 : diagramme d'activité pour l'exercice d'édition d'un ouvrage.

ACT08 : : procédure de création d'un compte de base de données (BD).

La création d'un compte utilisateur dans une entreprise doit suivre la procédure suivante.

Un employé qui souhaite devenir un nouvel utilisateur de la BD doit d'abord remplir une demande papier reprenant ses coordonnées. Il doit ensuite obtenir l'accord de son supérieur hiérarchique, qui cosigne la demande papier. En cas d'accord du supérieur, l'employé finalise sa demande en indiquant l'identifiant du compte et en l'expédiant à l'administrateur de la base de données (DBA, pour **D**ata **B**ase **A**ddministrator). Dans le cas contraire, la procédure est abandonnée.

Lorsqu'il reçoit une demande de création de compte, le DBA vérifie d'abord si l'identifiant du compte existe déjà. Si c'est le cas, il renvoie la demande à l'expéditeur pour obtenir un autre identifiant. Sinon, il crée le compte dans le système de gestion de base de données (SGBD) et, en même temps, il met à jour la liste des utilisateurs. Quand ces deux opérations sont terminées, le SGBD génère automatiquement un mot de passe, qui est ensuite envoyé à l'employé-utilisateur.

Ce dernier doit alors modifier ce mot de passe dans les cinq jours qui suivent. Si le mot de passe est bien modifié dans les délais, le compte est confirmé, sinon il est annulé.

Développez un diagramme d'activités pour formaliser le processus décrit ci-dessus en précisant une partition qui indique les acteurs qui réalisent les différentes actions.

La Figure 99 montre une solution possible.

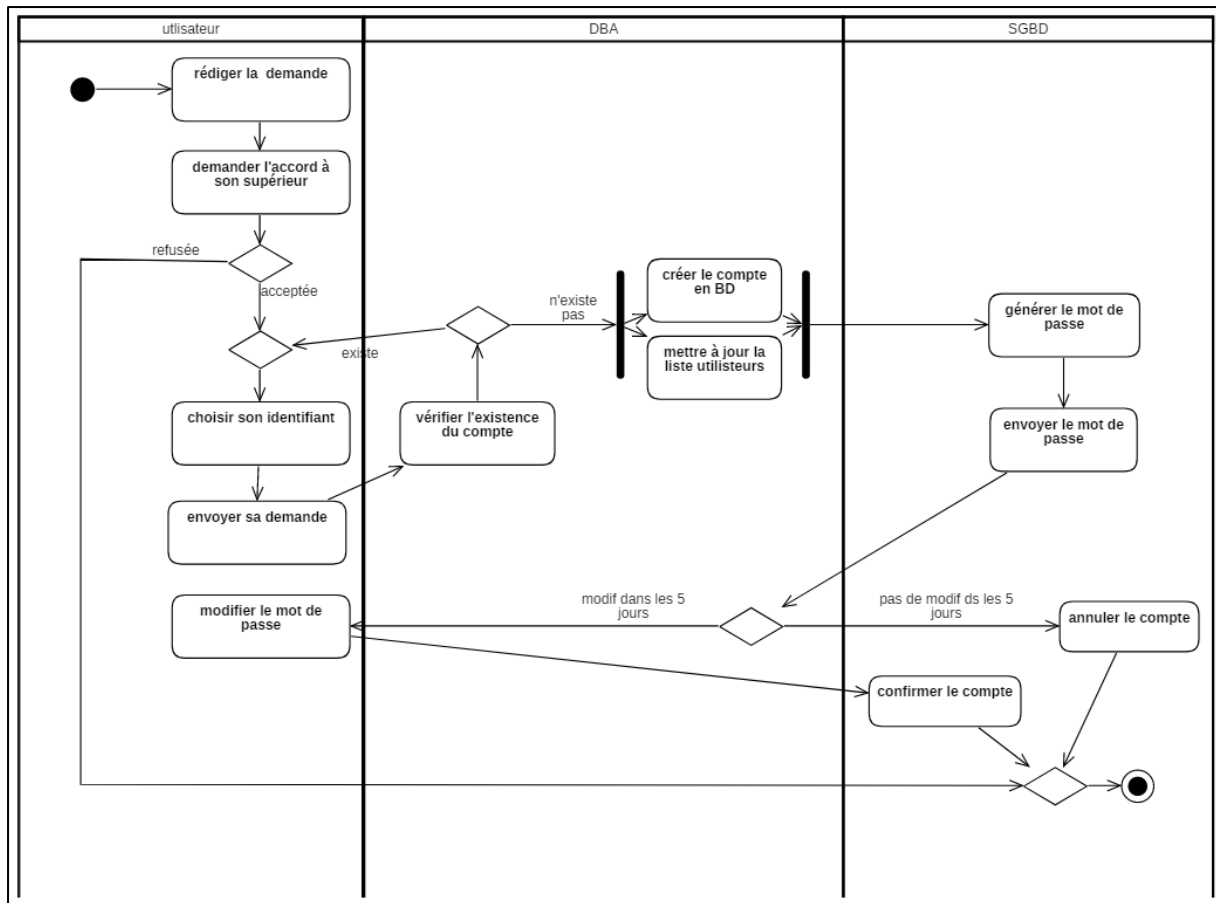
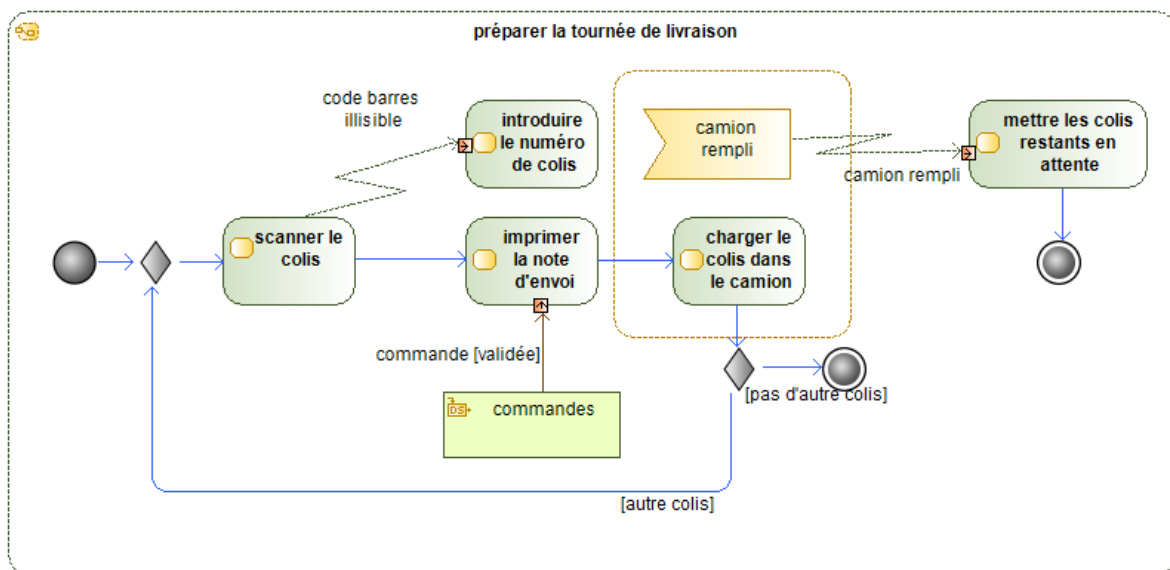
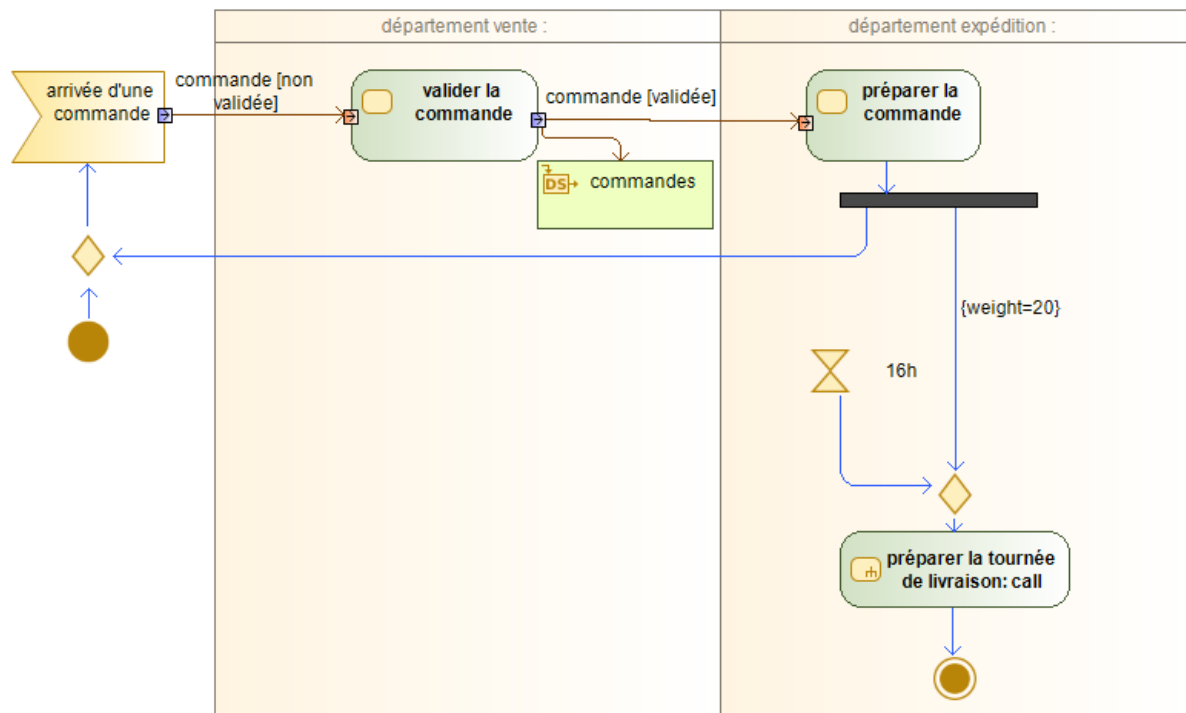


Figure 99 : diagramme d'activité pour l'exercice de création d'un compte en BD.

ACT09 : traitement de commandes revisité

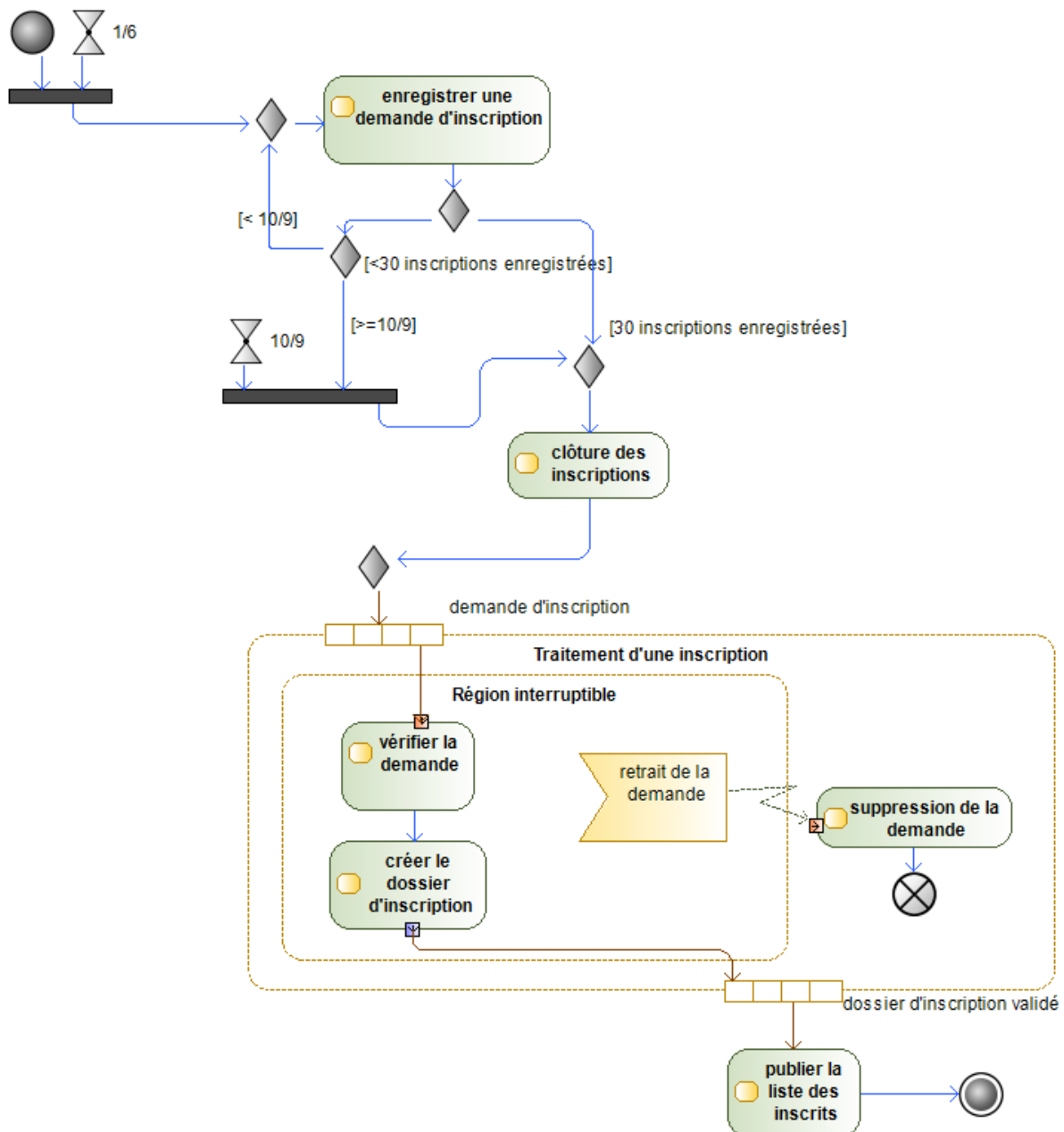
L'activité démarre lorsque l'arrivée d'une commande est signalée au département vente. Cette commande est dans un état "non validé" avant d'être validée par le département vente. À l'issue de cette action, la commande est dans un état "validée". La commande est à la fois stockée dans la base de données des commandes et transmise vers le département expédition qui va alors la préparer. Ce processus se répète soit jusqu'au moment où 20 commandes sont préparées, soit jusqu'à 16h. Finalement, l'activité de préparation d'une tournée de livraison est réalisée.

La préparation d'une tournée consiste, pour chaque colis, à scanner son code-barres, à imprimer la note d'envoi correspondante en interrogrant la base de données des commandes, puis le chargeant dans le camion. Si le code-barres est illisible, alors le numéro du colis est introduit manuellement. Si le camion est rempli, alors la préparation est arrêtée et les colis restants sont mis en attente.



ACT10 : inscription à un master

- L'enregistrement d'une demande d'inscription à un programme démarre le 1/6 et se clôture
 - soit dès que 30 demandes ont été enregistrées ;
 - soit le 10/9.
- Les demandes d'inscription sont ensuite vérifiées une à une. Pour chacune d'entre elles, on crée un dossier d'inscription valide. Si pendant ce temps, un étudiant signale le retrait de sa demande, alors le traitement de sa demande est interrompu et la demande est supprimée.
- Finalement, on publie la liste des inscrits au programme.
- Développez un diagramme d'activité sans partition, en restant attentif à la logique temporelle.
- Proposez ensuite des améliorations à ce processus.



6 Réingénierie des processus avec BPM

Jusqu'ici, ces notes ont abordé les démarches et outils clés de l'ingénierie des exigences, en présentant la nature de la business analyse et le cycle détaillé de l'ingénierie des exigences, puis les langages de représentation graphique des exigences.

En complément, les deux sections suivantes présentent deux outils populaires dans la communauté des business analysts :

- l'approche Business Process Management (BPM), centrée sur la réingénierie des processus d'affaires et qui est discuté dans la suite de cette section ;
- le langage Business Process Management Notation (BPMN), centré sur la modélisation des processus d'affaires et qui est introduit par la Section 7.

6.1 Processus d'affaires (business process ou processus métier)

Cette notion est largement discutée par les sciences de gestion. Selon (Deffray & Raymont, 2018) :

- un processus d'affaires est « un ensemble corrélé d'activités produisant une valeur ajoutée tangible à partir d'une sollicitation initiale (événement déclencheur).
- Les activités (ou tâches, ou étapes) sont réalisées par des acteurs (humains ou automatiques) avec l'aide de moyens adaptés.
- Le caractère "métier" du processus s'exprime par la nature du résultat, qui doit avoir un sens pour un client (interne ou externe) et si possible être mesurable.
- Le terme "activité" désigne ici une unité de travail, apportant une valeur par transformation ou production d'information ou de matériel. »

Définition 18 : un processus d'affaires est ensemble d'événements, d'activités et de décisions connectés qui impliquent des acteurs et des ressources, et qui apportent ensemble de la valeur aux parties prenantes liées au processus.

Les organisations mettent en œuvre de nombreux processus d'affaires comparables, comme des processus de commande client (*order-to-cash*), d'appel d'offres (*quote-to order*), de résolution de problèmes (*fault-to-resolution*) ou de règlement de plaintes (*claim-to-settlement*).

Comme le montre la Figure 100, les processus peuvent être classés en :

- processus de management : ils relèvent de la définition d'une stratégie d'entreprise et de politiques de gestion, ou encore de la prise de décision, etc. ;
- processus du cœur de métier (core business) : ils concernent réalisation de produits et de services ;
- processus de support : ils visent à mettre à disposition des ressources et des services qui soutiennent les autres processus.

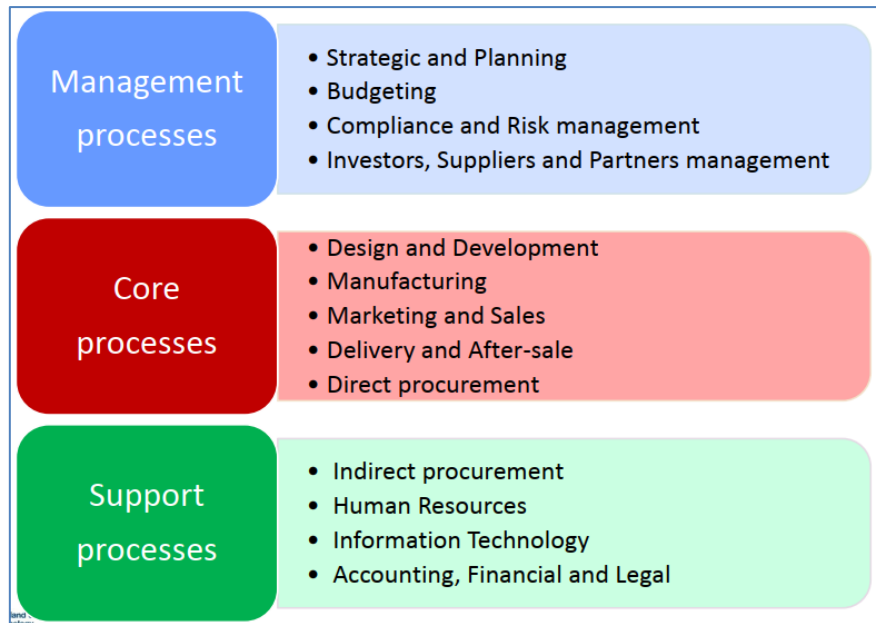


Figure 100 : exemples de processus génériques.

Par exemple, les processus classiques dans une entreprise de fast-food pourraient être les suivants :

- management : stratégie, veille concurrentielle, gestion des implantations, etc.
- cœur de métier : approvisionnement, préparation des hamburgers, préparation d'une commande, marketing, vente comptoir, gestion de salle, etc.
- support : GRH, IT, comptabilité, etc.

Un processus d'affaires a un début et une fin. Il s'exécute dans un délai donné entre son déclenchement et l'obtention du résultat. Un processus d'affaires se distingue d'une fonction qui, elle, peut regrouper plusieurs processus.

6.2 Cycle BPM

Le BPM suggère une démarche pour la réingénierie des processus. La démarche incorpore des principes directeurs, comme l'amélioration continue, des méthodes, comme un phasage précis, et des outils, comme des instruments de mesure de la performance.

Définition 19 : la démarche BPM est un corpus de principes, de méthodes et d'outils pour concevoir, analyser et monitorer les processus d'affaires en vue de les améliorer (Dumas, La Rosa, Mendling Hajo, & Reijers, 2013).

La démarche BPM repose sur le phasage de la Figure 101. On y retrouve les notions de « as-is » et « to-be ».

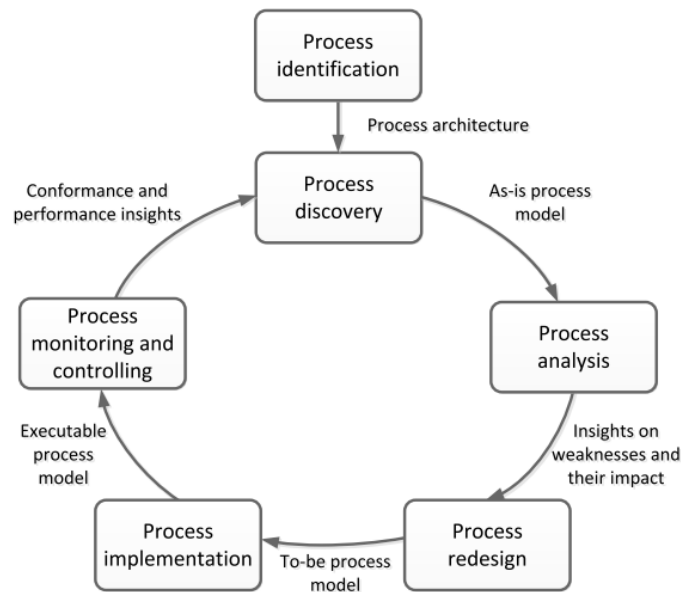


Figure 101 : cycle BPM.

Process identification. Cette première phase du BPM débouche sur une architecture des processus qui va identifier les processus et leurs interrelations, comme des liens d'entrée - sortie.

On identifie typiquement 10 à 20 processus dans une organisation, mais parfois bien plus dans les très grands comptes (~100's). Le lecteur se tournera vers la littérature spécialisée en sciences de gestion pour dresser une architecture des processus, comme ICOM (**i**nputs, **c**ontrols, **o**utputs, **m**echanisms/resources used) proposé par la méthode TOGAF (**T**he **O**pen **G**roup **A**rchitecture **F**ramework), ou la technique de spécification des processus SIPOC (**s**upplier, **i**nput, **p**rocess, **o**utput, **c**ustomer) amenée par la méthode Six Sigma.

Les processus sont priorisés afin à décider de ceux à optimiser dans un contexte (quasi universel) de rareté des ressources. Plusieurs critères interviennent dans l'établissement de ces priorités, comme :

- l'apport de valeur : quels sont les processus qui ont contribuent le plus à la réalisation des objectifs ? Quels sont ceux qui apportent le plus de valeur ?
- le niveau de dysfonctionnement : quels sont les processus qui posent le plus de problèmes ?
- le degré de faisabilité : quels sont les processus qui sont les plus à même d'opérationnaliser les décisions de gestion en tenant compte des contraintes et des difficultés, comme l'adhésion des acteurs ?

Process discovery. Cette seconde phase collecte les informations utiles pour représenter en détails les processus existants ("as is") qui rentrent dans le périmètre de la réingénierie. Les processus sont modélisés avec des langages usuels comme les diagrammes d'activité (Section 5) ou BPMN (Section 7).

Process analysis. L'intention de cette phase est de détecter et d'analyser des faiblesses des processus existants en vue de les améliorer.

On va d'abord réaliser une analyse *quantitative*, i.e., calculer des indicateurs de performance du processus (grandeurs mesurables) notamment liés au temps d'exécution, aux coûts et à la qualité des sorties du processus. Par exemple, pour quantifier la performance d'un processus de préparation des hamburgers, on établit les mesures suivantes, qui malheureusement ici révèlent un dysfonctionnement :

- nombre maximum de hamburgers/heure : 256
- demande maximum de hamburgers/heure : 300
- temp d'attente moyen en heure de pointe : 5,43 min

- nombre moyen mensuel de plaintes liées au temps d'attentes : 24

Ensuite, on va entreprendre une analyse *qualitative*, i.e., une réflexion sur le fonctionnement du processus, sur les causes des dysfonctionnements et sur les opportunités d'améliorations. Ici encore, les sciences de gestion proposent plusieurs outils, comme le Lean Management, la Root Cause Analysis ou les diagrammes de causes à effets d'Ishikawa. Cette dernière technique, illustrée par la Figure 102, identifie des causes qui ont un effet négatif sur le résultat d'un processus selon différents facteurs :

- technologie (Machine) : manque de fonctionnalité, mauvaise interface utilisateur, etc.
- processus (Method) : responsabilités mal établies, mauvaise communication, etc.
- fournitures (Material) : mauvaise qualité des matières premières, etc.
- personnel (Man) : manque de qualifications, démotivation, etc.
- mesures (Measurement) : sous-estimation de la demande, etc.
- environnement (Milieu) : mesures de confinement, etc.

Par exemple, la Figure 102 identifie plusieurs facteurs qui expliquent pourquoi un processus de préparation des hamburgers ne répond pas à la demande aux heures de pointe.

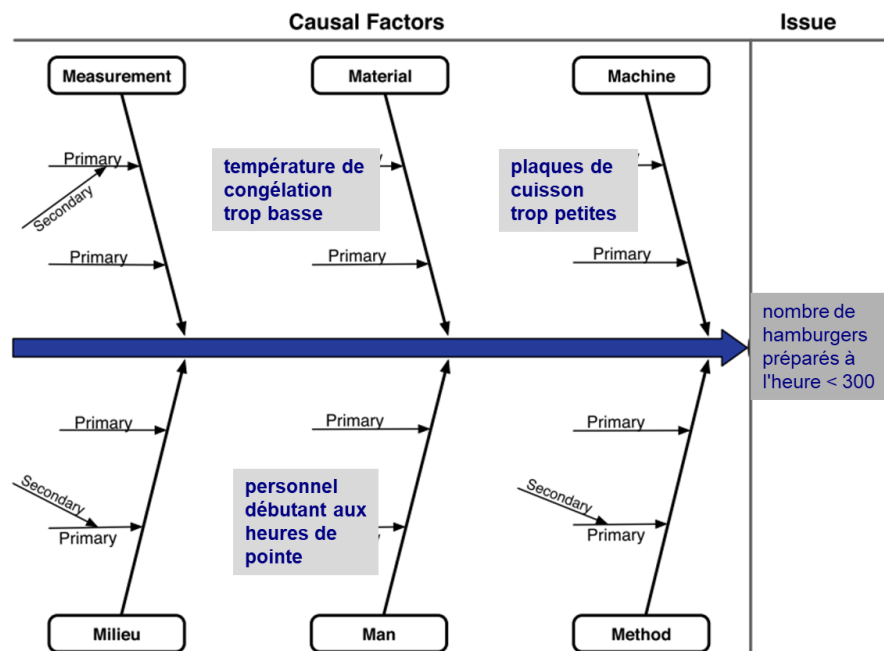


Figure 102 : exemple de digramme de causes à effets.

Process redesign. Cette étape clé propose des améliorations potentielles pour optimiser les processus. Elle débouche sur une proposition d'une version améliorée des processus : le modèle de processus "to be".

Le plus souvent, un processus en place est adapté pour mieux fonctionner. Mais parfois, on préfère reconstruire un nouveau processus à partir d'une page blanche, et donc en ignorant ce qui est déjà en place. C'est le cas par exemple quand un processus existant est obsolète ou trop défectueux pour pouvoir être amélioré sans être repensé complètement.

Plusieurs directions sont envisageables pour améliorer un processus :

- réduire ses coûts ;
- réduire son temps d'exécution ;

- augmenter la qualité de ses résultats ;
- augmenter sa flexibilité (i.e., son aptitude à la variation).

Il s'agira de trouver des compromis entre ces directions en fonction du problème posé. Par exemple, augmenter la qualité implique souvent d'augmenter les coûts.

(Dumas, La Rosa, Mendling Hajo, & Reijers, 2013) présente de nombreux exemples de facteurs d'amélioration des processus qui peuvent inspirer le business analyst, comme :

- affectation aux acteurs internes ou externes du processus
 - ex. : localiser le contrôle au niveau du client (↓qualité, ↑coût)
 - ex. : le client d'une banque enregistre lui-même un prêt
- décomposition du processus en activités
 - ex. : éliminer les activités inutiles ou peu porteuses de valeur (↓qualité, ↑coût, ↑temps)
 - ex. : réduire les tests intermédiaires de qualité
- séquençage en activités
 - ex. : exécuter des activités en parallèle (↓coût, ↑temps)
- gestion des ressources
 - ex. : attribuer les activités de manière flexible (↓coût, ↑temps, ↑flexibilité)
 - ex. : assurer la polyvalence des collaborateurs pour plusieurs activités différentes
- gestion de l'information créée ou consommée par le processus
 - ex. : implémenter des contrôles d'intégrité de l'information (↑qualité, ↓temps, ↓coût)
 - ex. : faire contrôler un contrat de prêt par le directeur de l'agence bancaire
- activation des technologies pour exécuter le processus
 - ex. : automatiser les activités (↑qualité, ↓coût, ↑temps, ↓flexibilité)
 - ex. : le client encode lui-même sa commande de repas sur des bornes interactives
- gestion de l'environnement
 - ex. : déléguer des activités à des tiers de confiance (↑qualité, ↑coût, ↓flexibilité)

Process implementation. Cette phase consiste à mettre en œuvre les modèles des processus "to be" et à adapter concrètement les processus. Ce travail implique habituellement des développements IT (adaptations de logiciels ou création de nouveaux logiciels), mais pas seulement : il peut porter sur des changements de procédure, une redistribution des tâches, etc.

Généralement, cette phase demande de gros efforts et s'accompagne d'une gestion du changement lors du déploiement des nouveaux processus. En effet, il est déterminant d'expliquer, de motiver, d'impliquer d'accompagner et de former les parties prenantes pour gagner leur adhésion.

À ce stade, la réingénierie des processus proprement dite est finalisée. BPM suggère cependant une phase complémentaire qui s'inscrit dans la gestion continue de la qualité.

Process monitoring and controlling. Cette phase prévoit une activité continue de mesure et de surveillance des performances des processus. Les sciences de gestion discutent abondamment les mécanismes de mesures de la performance (Business Activity Monitoring (BAM), la définition d'innombrables KPI - Key Performance Indicator, etc.).

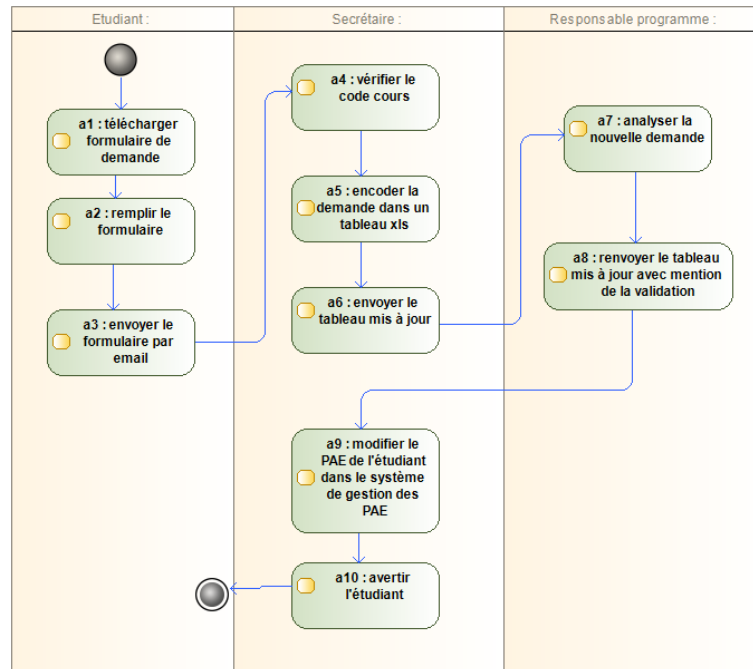
Si les performances mesurées ne sont pas suffisantes, des actions correctrices seront à prévoir. En cas de dysfonctionnements importants, un nouveau cycle BPM peut être initié.

6.3 Exercice

BPM01. Le digramme suivant explique le processus « as is » pour traiter une demande de changement de PAE d'un étudiant. Ce processus implique des échanges de mails et de documents (formulaire de demande de changement de PAE et tableau récapitulatif des demandes réalisés en Excel).

Proposez des améliorations concrètes et indiquez les facteurs d'amélioration et les impacts attendus.

Proposez finalement une version « to-be ».



7 Modélisation des processus avec BPMN

Le langage BPMN a été développé en 2002 par le Business Process Management Initiative puis normalisé en 2008 par l'OMG (<http://www.omg.org/>). Antérieur à UML, il a largement inspiré les diagrammes d'activité.

Depuis lors, BPMN s'est enrichi de nombreux éléments et sous-langages, comme les chorégraphies ou les diagrammes de collaboration⁹ qui ne seront pas détaillés dans la suite. Cette section se concentre donc sur les *process diagrams*, le sous-langage historique de BPMN pour représenter les processus d'affaires.

Un process diagram (communément appelé diagramme BPMN) combine trois perspectives complémentaires :

- la perspective des flux de contrôle (« what needs to be done and when »), qui exprime les relations de prédécesseurs/successeurs pour enchaîner activités et événements, comme les suites d'activités de l'exemple de la Figure 103 ;
- la perspective des données (« what do we need to work on »), qui représente les relations d'entrée/sortie de données des activités, comme la circulation de commandes et de rapports d'erreur de la Figure 103 ;
- la perspective des ressources (« who's doing the work »), qui introduit les acteurs/systèmes qui réalisent les activités et qui génèrent des événements, comme les couloirs d'exécution de la Figure 103 .

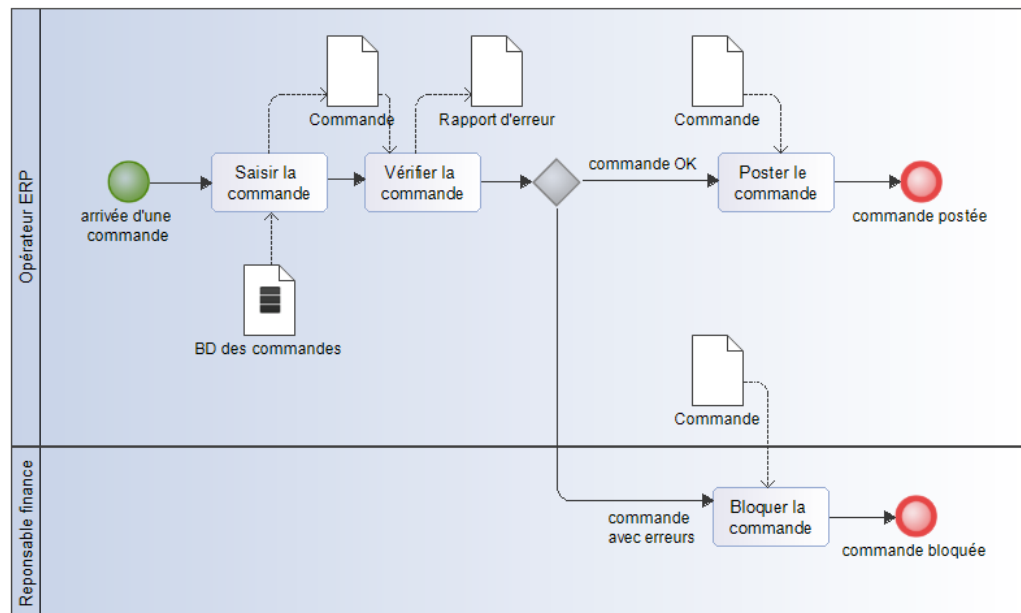


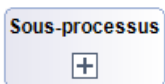
Figure 103 : illustration des 3 perspectives d'un diagramme BPMN.

⁹ Choreography diagrams : modélisation de haut niveau des interactions entre plusieurs parties, avec un accent mis sur les échanges de messages.

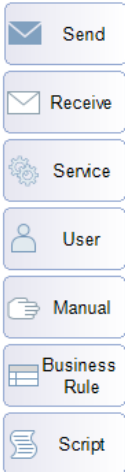
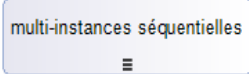
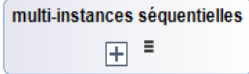
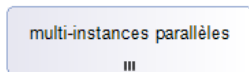
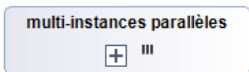
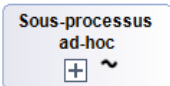
Collaboration diagrams : représentation des interactions entre plusieurs processus synchronisés.

7.1 Activité

La brique de base de BPMN est l'activité qui représente une unité de travail à accomplir. Trois types d'activités sont supportés :

<i>tâches</i> : activité atomique (non décomposable)	
<i>sous-processus</i> : activité composée de tâches ou de sous-processus	
<i>appel</i> : activité qui fait référence à une activité externe au processus courant, et qui est appelée par le processus courant	

Les tâches, qui sont les activités les plus usuelles, peuvent être elles-mêmes typées en fonction de leur nature :

<i>send</i> : envoie un message à un autre couloir (lane) <i>receive</i> : reçoit un message <i>service</i> : utilise un Web service ou similaire <i>user tasks</i> : est exécutée à l'initiative d'un acteur humain <i>manual task</i> : est exécutée sans application <i>business rule</i> : interagit avec un moteur de règles pour analyser des données <i>script task</i> : provoque l'exécution d'un script d'instructions	
répétition de tâches ou de sous-processus en séquence	 
répétition de tâches ou de sous-processus en parallèle	 
sous-processus ad-hoc, dont les activités peuvent être exécutées ou non, dans n'importe ordre, et éventuellement plusieurs fois (spécification partielle)	

7.2 Séquencement

L'ordonnancement des tâches est très comparable à celui des diagrammes d'activité, à quelques différences de notations près. À nouveau, la métaphore du jeton est bien utile pour comprendre la logique de BPMN.

<i>flux de contrôle</i>	
<i>start event</i> : signale quand une instance du processus démarre (création de jetons)	
<i>intermediate event</i> : intervient pendant le déroulement du processus, et bloque le flux de contrôle tant que l'événement ne s'est pas produit	
<i>end event</i> : signale quand une branche du processus se termine (destruction des jetons de la branche correspondante)	
<i>terminate end event</i> : signale quand une instance du processus se termine complètement (destruction de tous les jetons)	
branchement exclusif (<i>XOR</i>) - division (sortie) : un seul flux sortant est activé - convergence (entrée) : l'arrivée d'un seul flux entrant suffit à déclencher le flux sortant	 Exclusive Gateway
branchement parallèle (<i>AND</i>) - division : tous les flux sortants sont activés - convergence : l'arrivée de tous les flux entrants est requise pour déclencher le flux sortant	 Parallel Gateway
branchement inclusif (<i>OR</i>) - division : un ou plusieurs flux sortants sont activés - convergence : l'arrivée de tous les flux des branches actives (celles où circule un jeton) est requise pour déclencher le flux sortant	 Inclusive Gateway

Des conditions de garde sont requises pour la division des branchements XOR et OR, afin de déterminer la ou les branches sortantes à activer. La Figure 104 donne un exemple du *ou* inclusif, ou OR.

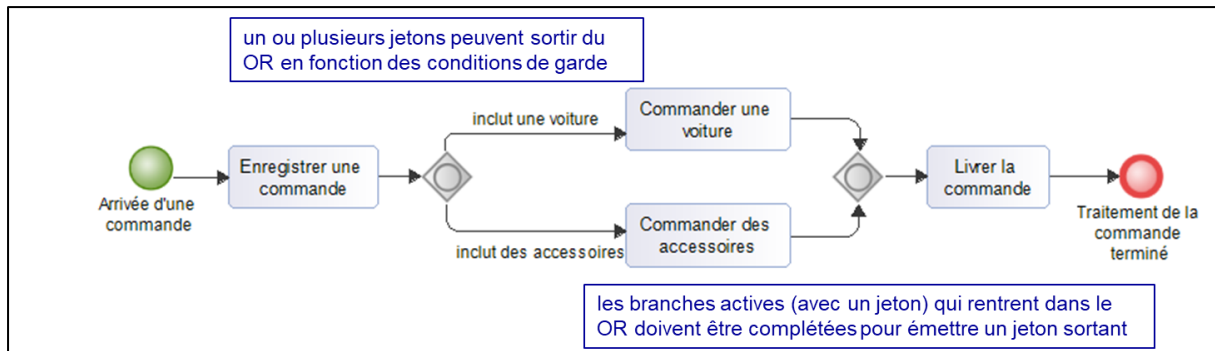


Figure 104 : exemple d'application du nœud OR.

Remarque : il faut rester attentif à la sémantique de notations implicites. Ainsi :

- 2 flux (ou plus) entrant dans une activité = XOR (différent des diagrammes d'activité en UML : AND) ;
- 2 flux (ou plus) sortant d'une activité = AND (idem diagrammes d'activité en UML).

7.3 Données

Plusieurs artéfacts sont proposés pour la perspective sur les données :

<i>objet de données</i> : représente de l'information qui transite à travers le processus	 Object
<i>association</i> : connecte un artéfact de données à une activité	Association 
<i>dépôt de données</i> : représente un endroit de stockage persistant	 Store
<i>données d'entrée</i> : représente des données entrantes en provenance de l'environnement du processus	 Input
<i>données sortantes</i> : représente des données résultant du processus	 Output

7.4 Ressources

Les pistes et corridors indiquent la localisation des activités, comme des acteurs, des organisations, des départements, etc.

Piste	Piste	
	Corridor	Corridor

7.5 Compléments sur les événements

Un événement peut conditionner le début ou la fin d'un processus comme mentionné plus haut (*start*, *end*, *terminate*). Mais BPMN propose une palette très complète d'événements qui peuvent se rattacher à un flux ou à une activité. Voici quelques exemples qui illustrent la puissance de BPMN pour ces événements.

événement attaché à un flux	
événement attaché à une activité et <i>interrupting</i> : si l'événement se produit, le flux normal est interrompu et seul le flux d'exception lié à l'événement se produit	
événement attaché à une activité et <i>non interrupting</i> : si l'événement se produit, le flux normal et le flux d'exception lié à l'événement se produisent	
un événement peut être : • lanceur (<i>throwing</i>) : génération d'un signal déclencheur • capteur (<i>catching</i>) : arrêt du processus, attente d'un signal déclencheur puis redémarrage à l'arrivée du signal	
un événement peut traiter une exception	

7.6 Exercices

BPM02 : renting de voiture.

Une société de renting met en place un processus de location de voitures à ses clients.

Le processus est déclenché par l'arrivée de demande de location de voiture. Un client peut louer une voiture s'il possède un permis de conduire valide et s'il peut payer la location. Un employé sélectionne le type de véhicule, le type de transmission puis établit un contrat de location qui est enregistré dans le système.

La location peut alors démarrer. Le contrat est résilié une fois la location terminée, ce qui clôture la procédure.

Représentez ce processus avec un diagramme BPMN sans pistes/corridors.

BPM03 : SuperExpress

SuperExpress est un transporteur de colis qui imagine le processus ci-dessous pour organiser la tournée quotidienne de livraison de colis de chacun de ses camions.

Chaque jour à 9h, les colis à livrer sont chargés dans un camion. En cas de problème de chargement (à représenter avec événement d'erreur), on redistribue les colis dans d'autres camions et on reprend le chargement en cours.

À 10h, le camion démarre sa tournée et, en même temps, les mails de tracking sont envoyés aux clients destinataires d'un colis.

Une fois arrivé chez un client, le livreur sonne à la porte et livre le colis. Mais si le client n'ouvre pas après 2 minutes, le livreur notifie l'absence du client (et le colis n'est pas livré). Ces actions sont répétées jusqu'au dernier colis à livrer.

Représentez ce processus avec un diagramme BPMN sans préciser les pistes/corridors et proposez des améliorations possibles.

BPMN04 : réservation de vol

Une agence de voyage traite les demandes de ses clients pour réserver un vol comme suit.

Un client envoie une demande de réservation pour une destination et une date donnée. Un employé recherche alors les meilleures propositions de vols puis envoie au client différentes propositions de vols. Le client peut alors :

- soit avertir l'agence qu'il refuse les propositions et avorte sa demande. À ce moment, l'employé archive la demande et le processus se termine
- soit indiquer son choix de vol à l'agence. Il sera alors invité à payer le montant du billet
-

S'il ne réagit pas dans les 24h, l'agence notifie au client que le délai est dépassé et l'invite à recommencer sa demande. L'employé archive alors la demande et le processus prend fin.

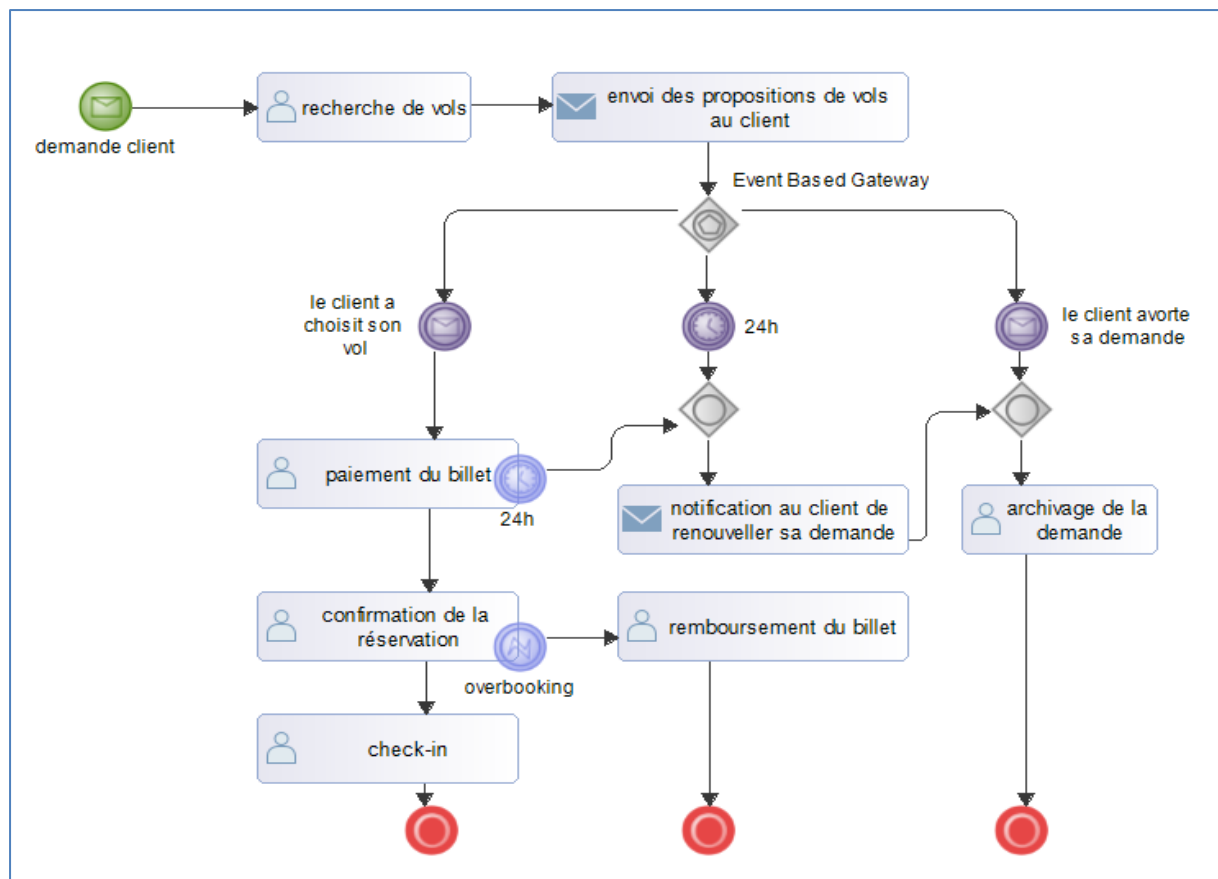
Le client doit payer son billet dans les 24 heures. Passé ce délai, l'agence notifie au client que le délai est dépassé et l'invite à recommencer sa demande. L'employé archive alors la demande et le processus prend fin.

Si le paiement intervient dans le délai, l'employé confirme la réservation puis finalement le client procède au check in du vol.

Dans des cas exceptionnels d'overbooking, une erreur peut se produire lors de la confirmation de la réservation. Dans ce cas, l'employé rembourse le client.

Représentez ce processus avec un diagramme BPMN sans préciser les pistes/corridors et proposez des améliorations possibles.

Suggestion : utilisez un Event Based Gateway, pour permettre de prendre une décision d'enchaînement en fonction d'occurrences d'événements.



BPM05 : PieceAuto 3000

PieceAuto 3000 livre des pièces de rechange pour l'automobile partout en Europe.

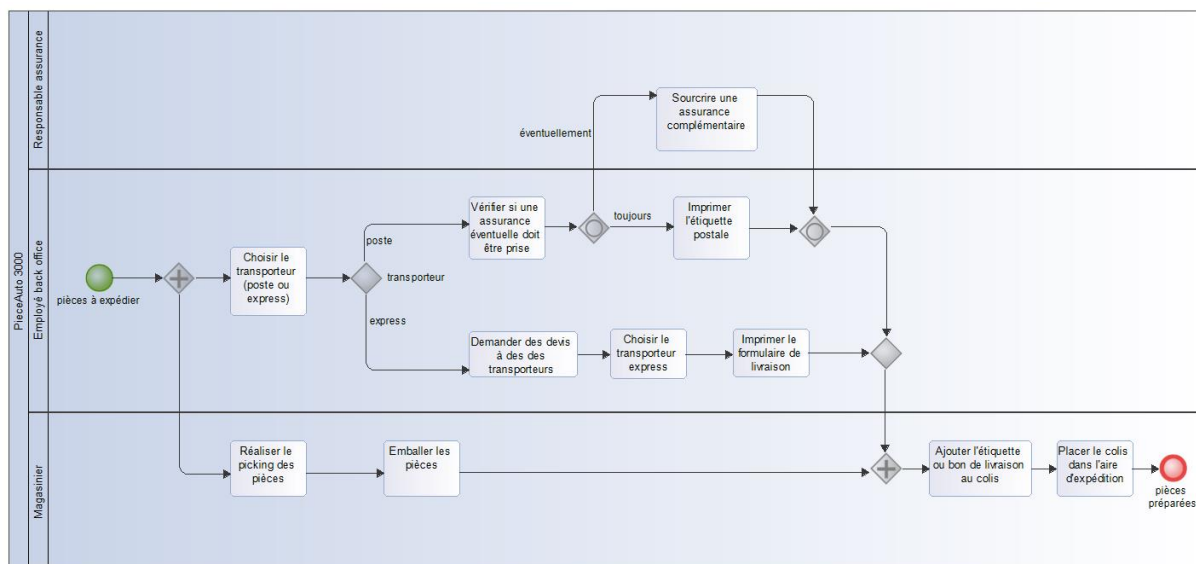
Le processus de préparation de commande s'enclenche quand il y a des pièces à expédier pour un client. L'employé du back office choisit d'expédier les pièces soit par la poste nationale, soit par un transporteur express.

Dans le premier cas (poste), il vérifie si une assurance éventuelle doit être prise. Si c'est le cas, alors le responsable assurance souscrit une assurance complémentaire pour le transport. Quoi qu'il en soit, l'employé du back office imprime une étiquette postale à apposer sur le colis.

Dans le second cas (transporteur express), l'employé demande des devis à différents transporteurs, puis choisit le plus approprié avant d'imprimer un formulaire de livraison qui accompagnera le colis.

Parallèlement aux tâches de l'employé du back office, le magasinier réalise le picking des pièces dans le stock puis les emballe. Il ajoute ensuite au colis soit l'étiquette postale soit le formulaire de livraison qu'a imprimé l'employé du back office. Finalement, le magasinier place le colis dans l'aire d'expédition.

Représentez ce processus avec un diagramme BPMN.



8 Tests

Toute démarche d'ingénierie intègre des vérifications systématiques de la qualité des livrables à chaque phase du processus de développement. En particulier, nous avons déjà discuté les critères de qualité des exigences.

En marge de l'ingénierie logicielle, le business analyst est fréquemment impliqué dans le test du produit fini, i.e., du logiciel, car il connaît parfaitement les exigences du business et est donc à même de vérifier si le logiciel y répond. Mais plus généralement, les tests logiciels impliquent aussi l'équipe IT, le management et l'utilisateur final.

8.1 Définition et enjeux

Définition 20 : "Le test est l'exécution ou l'évaluation d'un système ou d'un composant du système, par des moyens automatiques ou manuels pour : (1) vérifier qu'il répond à ses spécifications, ou (2) identifier les différences entre les résultats attendus et les résultats obtenus" (IEEE STD 729 - Standard glossary of software engineering)".

Cette définition reflète ce qui est d'abord attendu d'un logiciel : faire ce qu'il est censé faire, et le faire correctement. Si ce n'est pas le cas, alors le logiciel comporte des défauts qu'il convient d'identifier.

Un *défaut*, ou un bug, est imperfection présente dans un logiciel, et plus généralement dans n'importe quel livrable du projet, comme des exigences erronées dans un SRS.

Un défaut est provoqué par une *erreur* humaine, comme une mauvaise compréhension des besoins ou une erreur de codage.

L'exécution d'un logiciel comportant un défaut provoque une *défaillance*, i.e., une déviation par rapport à un comportement requis et attendu, comme un résultat de calcul erroné.

Une défaillance peut avoir des *effets* indésirables. Parfois, ces effets sont très importants (cf. Ariane 5).

Le *débogage* désigne le travail d'analyse et correction des défauts.

Importance des tests. Souvent perçus comme rébarbatifs et peu productifs, les tests contribuent pourtant grandement au succès d'un projet de développement. Ils permettent :

- de réduire les risques liés à la solution ;
- d'augmenter la satisfaction, l'adhésion et la confiance des utilisateurs dans la solution ;
- de réduire les coûts liés aux effets des défaillances.

Les tests conditionnent parfois la réception ou le refus des livrables, tel que prévu dans un contrat de service entre un client et un prestataire. Attention, un client qui teste un logiciel et qui marque son acceptation aura des difficultés à se retourner contre son fournisseur si des défauts sont détectés plus tard.

Finalement, les tests certifient la conformité du système par rapport à une norme ou un règlement.

Principes généraux. Les tests révèlent la présence de défauts, mais ils ne peuvent pas malheureusement pas prouver l'absence complète de défauts car des tests exhaustifs sont impossibles à réaliser en pratique. Par exemple, pour tester complètement un logiciel qui transforme par calcul des données numériques d'entrée en résultats de sortie, il faudrait réaliser des tests avec toutes les combinaisons de données numériques d'entrée possibles, ce qui est illusoire.

La dure réalité du logiciel est qu'il subsiste quasi toujours des erreurs dans un logiciel, même après de nombreux tests, corrections et utilisations. Tout ce qu'on peut espérer, c'est réduire le nombre d'erreurs au maximum. Par exemple, un logiciel qui comporte 2 erreurs par 1000 lignes de code est considéré comme un logiciel de qualité. Ainsi, plusieurs dizaines de milliers d'erreurs subsistent dans un système d'exploitation de quelques dizaines de millions de lignes de code...

Plus on teste tard, plus le coût de correction est élevé, comme le montre la Figure 105 (extraite de Basili and Boehm, Software Defect Reduction Top 10 List, IEEE - Computer Society, vol. 34, (1), Jan. 2001, pp. 135–137). Les tests doivent donc être planifiés au plus tôt dans le projet et les livrables intermédiaires en amont doivent être systématiquement testés et validés.

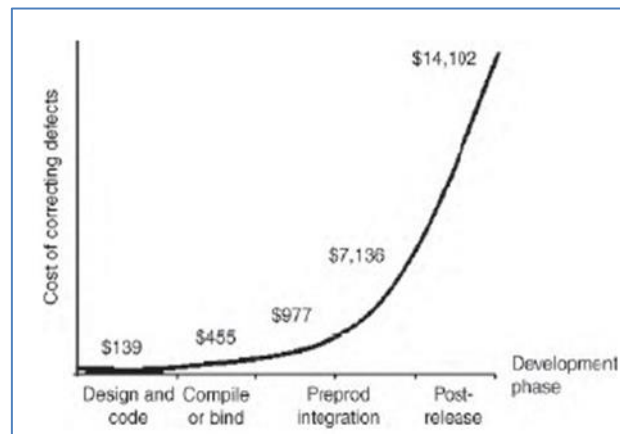


Figure 105 : coût de correction en fonction de la phase de développement.

Finalement, les tests doivent être replacés dans un contexte humain parfois sensible : identifier des défauts d'un livrable peut être perçu comme une critique par les auteurs du livrable. Certains ont parfois du mal à accepter ou à reconnaître leurs erreurs, et il faut toujours bien distinguer les qualités de la personne et celles de son travail. Les quelques bonnes pratiques suivantes aideront à effectuer les tests dans un climat serein :

- insister sur la collaboration, en opposition à la confrontation ;
- rappeler l'objectif commun de toutes les parties : augmenter la qualité ;
- énoncer les bénéfices et les enjeux des tests ;
- communiquer sur le livrable, pas sur l'auteur ;
- dépersonnaliser les défauts, i.e., présenter les défauts comme générés par le processus de développement dans son ensemble, et pas par un acteur particulier.

8.2 Phasage des tests

Les tests impliquent de mobiliser des ressources et de planifier un ensemble d'activités au même titre que les autres activités du projet de développement. Une démarche complète de test est présentée dans (ISTQB) et résumée dans la Figure 106 (S. Salmons, Ingénierie du test logiciel, Slideshare, consulté le 18/12/23).

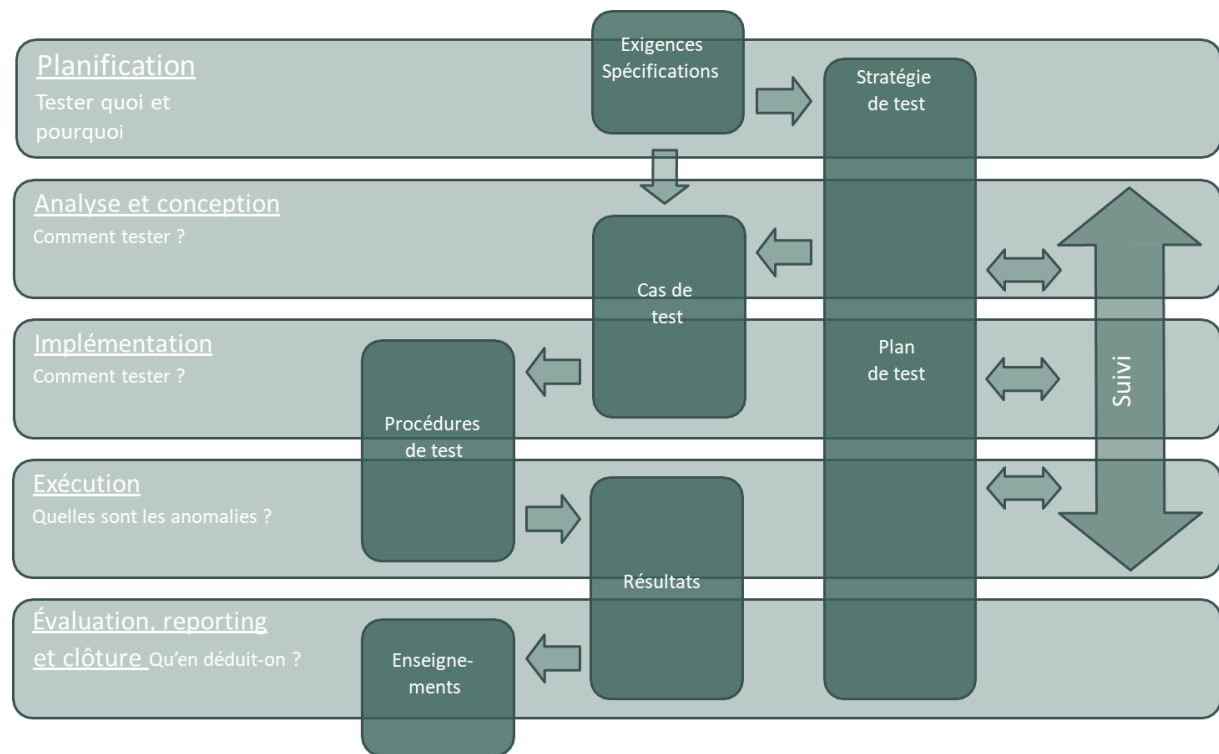


Figure 106 : phases typiques des tests.

Un test doit être conçu avant sa réalisation, par exemple à partir d'un cas de test qui précise le contenu du test, par exemple à l'aide :

- des données descriptives sur la nature du test et sur son déroulement ;
- d'un répertoire des données d'entrées à utiliser pour le test ;
- de préconditions : conditions nécessaires au démarrage du test ;
- de postconditions : conditions qui doivent être vérifiées après l'exécution du test ;
- d'un scénario reprenant les étapes du test.

La Figure 107 montre un exemple de cas de test et le résultat de test correspondant.

ID	13409		
Titre	réception des produits d'une commande fournisseur		
Auteur du test	Lara Cordy		
Utilisateur en charge du test	Annie Fabian		
Date d'exécution du test	1/6/22		
Etape	Action utilisateur	Réponse attendue	Résultat
1	Se connecter au module « inventaire »	L'écran d'accueil « inventaire » apparaît	pass
2	Cliquer sur la ligne correspondant à la commande à réceptionner (et donc le statut est « à réceptionner »)	la liste des articles de la commande à réceptionner est affichée avec la quantité à recevoir (zone « qqté »)	pass
3	Encoder la quantité réceptionnée dans la zone « réceptionnée » (à répéter pour chaque ligne article)	La quantité restante (à recevoir - réceptionnée) est affichée dans la zone « différence »	fail : possibilité de quantité réceptionnée > quantité à recevoir
4	Cliquer sur « Valider »	L'écran d'accueil « inventaire » apparaît avec un statut « livrée » pour la commande réceptionnée	pass

Figure 107 : exemple de cas de test.

Place des tests dans le cycle de développement. Les activités de tests peuvent être vues comme un sous-projet du projet de développement. Naturellement, les étapes du développement et celles des tests sont intimement mêlées.

8.3 Types de tests

On distingue plusieurs types de tests en fonction de l'activité dont il faut valider le livrable, comme le montre la Figure 108.

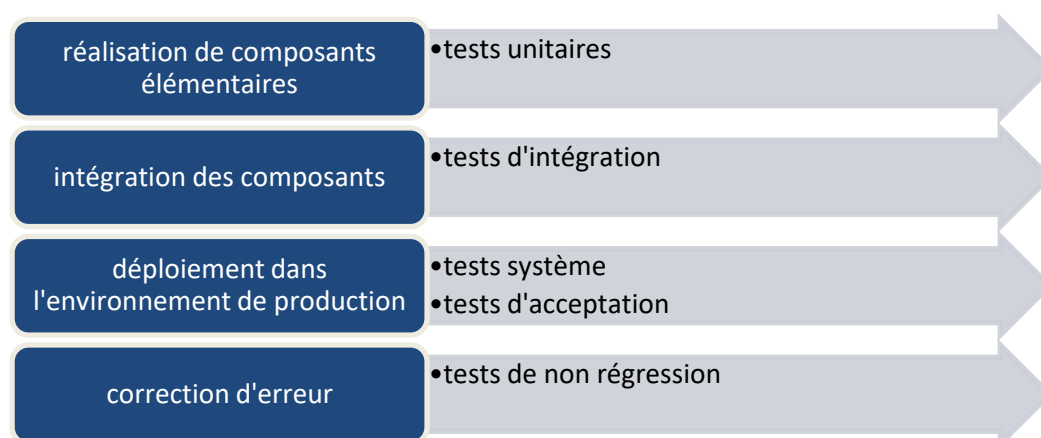


Figure 108 : types de tests.

Le Tableau 20 compare les différents types de test.

	objectifs	cible	défauts recherchés	responsabilités
Test unitaire	détecter les défauts et défaillances d'un composant sans considérer les autres composants vérifier si le comportement du composant correspond aux spécifications	composant isolable et testable, si possible indépendant des autres composants ex. : modèle de données, classe d'objets, module	erreurs de codage variables non initialisées, pointeur nul, condition inversée, etc.	généralement, le développeur
Test d'intégration	détecter les défauts et défaillances dans les interfaces et les interactions entre composants ou entre systèmes ¹⁰ vérifier si le comportement des interfaces correspond aux spécifications	interactions et interfaces entre des composants intégrés interactions et interfaces entre systèmes, progiciels et micro-services	cohérence des arguments dans les appels de composants, etc.	intégration de composants : généralement, le développeur intégration de systèmes : généralement, le testeur
Test système	détecter les défaillances dans le logiciel dans son ensemble s'assurer qu'il correspond aux exigences	logiciel entièrement intégré, installé et configuré	défaut par rapport aux exigences fonctionnelles et non fonctionnelles	généralement, testeur indépendant business analyst
Test d'acceptation	vérifier que le système est complet et qu'il fonctionne comme attendu, afin de marquer l'acceptation du système pour sa mise en production acquérir la confiance dans la qualité et la capacité opérationnelle du système	système tel que dans l'environnement de production	la recherche de défauts n'est pas un objectif ici mais bien la conformité par rapport aux exigences	client utilisateur ou son délégué, testeurs indépendants
Test de non-régression	s'assurer que les corrections et les évolutions du logiciel n'ont pas introduit de nouveaux défauts	logiciel dans sa version courante	erreurs de codage	généralement, le développeur

Tableau 20 : comparaison entre les types de tests.

Notons encore la distinction suivante au niveau des tests d'acceptation :

¹⁰ On ne teste pas le résultat de l'ensemble mais uniquement les échanges au niveau des interfaces

- les tests d'acceptation utilisateur (*UAT - User Acceptance Testing*). Ils valident la bonne utilisation du système par les utilisateurs dans un environnement opérationnel réel ou simulé. Ces tests sont importants car ils renforcent la confiance de l'utilisateur final dans le système ;
- les tests d'acceptation opérationnelle (*OAT - Operational Acceptance Testing*), qui valident l'aptitude du système à être maintenu et administré par les opérateurs et administrateurs prévus. Ils renforcent la confiance des administrateurs système dans les aspects opérationnels, comme la sauvegarde et restauration, la gestion des comptes utilisateurs, la migration de données, etc.

Les tests d'acceptation peuvent également valider la conformité aux critères d'acceptation d'un contrat ou d'un règlement pour la production d'un logiciel spécifique. Leurs résultats peuvent déboucher sur des certifications délivrées par des organismes spécialisés.

Pour être complet, nous devons encore évoquer les test alpha et bêta qui évaluent un logiciel commercial COTS¹¹ avant sa mise sur le marché. Ces tests sont effectués par les utilisateurs clients potentiels. Les tests alpha sont réalisés sur le site des développeurs, tandis que les tests bêta sont réalisés sur le site des clients.

8.4 Techniques de test

La Figure 109 répertorie différentes manières de réaliser des tests. Comme souvent, le choix d'une technique est lié à plusieurs facteurs, comme le type de test à réaliser, les outils disponibles, l'expérience antérieure des testeurs, etc.

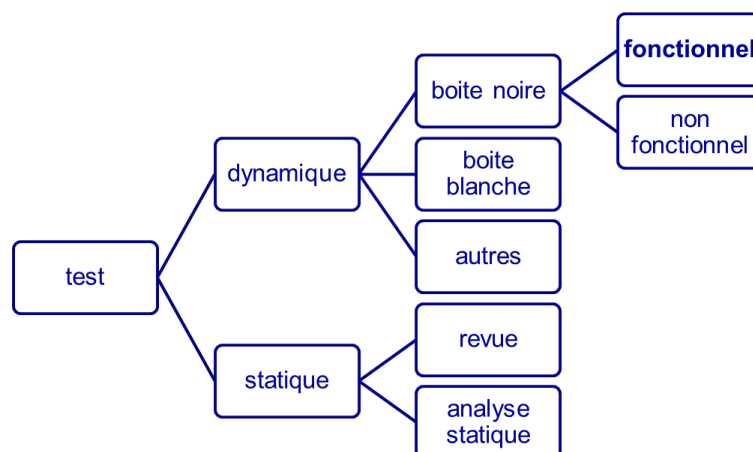


Figure 109 : techniques de test.

Tests statiques. Ici, le logiciel est testé sans qu'il soit exécuté, voire ni développé si l'évaluation porte uniquement sur les exigences.

Un test statique consiste à examiner le code pour détecter directement d'éventuels défauts. Les avantages de cette technique sont les suivants :

- les défauts sont détectés sans devoir observer les défaillances causées par ces défauts. Observer une défaillance nécessite un logiciel opérationnel, des données réalistes, un environnement d'exécution, etc., ce qui demande un certain effort de mise en œuvre ;

¹¹ COTS, pour Commercial Off-The-Shelf, qui signifie : vendu sur l'étagère.

- un test statique est possible sur un logiciel partiellement développé, sans déploiement, et donc il peut être effectué assez tôt dans le cycle de développement et à un coût limité ;
- des défauts de cohérence, comme des variables déclarées mais inutilisées, et des problèmes de qualité interne du code, comme sa lisibilité ou sa modularité, ne pas identifiables par d'autres techniques.

L'*analyse statique* consiste à réaliser les tests statiques avec des outils automatiques qui examinent le code pour détecter des défauts comme des variables non initialisées, de mauvaises conventions de nommage, la présence de code mort (i.e., parties jamais exécutées), etc.

La *revue*, elle, implique des acteurs humains pour tester un livrable comme des listes d'exigences, du code, ou autre. La norme ANSI/IEEE 1028-1997 la définit comme « une technique d'évaluation en détail, d'un document, d'un module de code source, etc. par un groupe de personnes pour détecter des défauts, les violations des standards et des conventions, ainsi que tout autre problème ».

L'organisation d'une revue est variable. Elle peut prendre la forme de :

- discussions informelles entre développeurs ;
- programmation en binôme («Over the shoulder review») ;
- d'inspection formelle réalisée par une équipe externe, selon un processus établi.

Tests dynamiques. Il s'agit d'examiner le comportement du logiciel lors de son exécution afin de détecter des défaillances, ce qui suppose un déploiement minimum.

Plus particulièrement, les tests en *boîte noire* vérifient la satisfaction des exigences fonctionnelles ou non fonctionnelles, sans considérer la structure interne du logiciel. Ils peuvent être réalisés sans maîtriser le codage, et donc par des utilisateurs finaux ou des business analysts, qui ont l'avantage de connaître les exigences.

Par contre, les tests en *boîte blanche* sont menés par les développeurs, qui recherchent des défauts de fonctionnement.

Finalement, d'autres techniques de tests dynamiques complètent la panoplie, comme l'analyse dynamique qui consiste à évaluer le fonctionnement d'un logiciel avec des outils automatiques.

Certaines techniques sont clairement appropriées pour mener certains types de test. Typiquement :

- les tests unitaires seront réalisés majoritairement avec des techniques de tests statiques et/ou en boîte blanche ;
- les tests d'acceptation utiliseront la technique en boîte noire.

9 IA pour l'ingénierie des exigences

Impossible de ne pas évoquer le sujet brûlant de l'intelligence artificielle au service du business analyst : quelles sont les opportunités et les risques de l'IA ? comment déployer progressivement l'IA dans les projets ? etc., sont autant de questions qui se posent pour quasi toutes les activités de l'organisation.

Cette section a l'ambition de donner un bref aperçu de l'utilisation potentielle de l'IA pour la gestion des exigences. Au moment de la rédaction de ces notes (juin 2024), nous en sommes aux prémices. Mais vu les progrès fulgurants de l'IA et son adoption massive par toutes les parties prenantes, cette section ne peut être qu'obsolète...

Rappelons d'abord que l'IA doit être utilisée comme un super assistant qui doit être prudemment supervisé. Au début de son utilisation, les acteurs humains doivent commencer par un apprentissage mutuel, qui consiste (1) à alimenter l'IA avec des données exploitables par ses modèles et (2) à se former à l'utilisation des IA.

Ensuite, les acteurs humains vont être amenés à superviser les tâches de l'IA et à valider les résultats qu'elle produit, un peu comme ils le feraient avec un assistant humain. Comme le pense Scott Bennett de l'IIBA : “If you were to hire an intern into your organization, you'd give them the easier work. You would certainly check their work when it comes back to you, and you may give them some research work to do as well. Think of AI as the same thing. Think of AI as your assistant.” (Info-Tech Research Group, 2024).

En fin de compte, c'est bien l'humain qui est toujours responsable des livrables, qu'il se fasse assister ou non par une machine. Il doit donc garder le contrôle de bout en bout des processus. L'IA n'est jamais utilisée que pour automatiser des tâches répétitives et consommatrices de temps. N'oublions pas que, malgré son nom, l'IA n'est pas intelligente et sa créativité reste limitée.

On peut d'ailleurs voir l'IA comme un outil supplémentaire pour l'automatisation. Elle peut augmenter la productivité de l'acteur humain, mais pas le remplacer intégralement, du moins dans le contexte de la business analyse. D'ailleurs, selon (Info-Tech Research Group, 2024), “AI won't replace the business analyst, but an analyst who knows how to leverage AI will replace one who doesn't”.

Par conséquent, les BA juniors (et les étudiants !) doivent impérativement se former complètement aux outils et méthodes de la business analyse ... avant d'espérer tirer profit de l'IA pour améliorer leurs performances ! En aucun cas, ils ne peuvent se décharger d'un apprentissage en espérant se faire remplacer par une IA. Au contraire, ils doivent développer des compétences supplémentaires liées à la maîtrise de l'IA, comme la compréhension des modèles de l'IA, le prompt engineering, etc.

9.1 Opportunités de l'IA

Les possibilités de l'IA sont très larges. Le Tableau 21 donne quelques exemples d'utilisation possible pour l'ingénierie des exigences.

Élicitation des exigences	
Interview des parties prenantes	Une IA peut retranscrire une interview pour un usage ultérieur. Elle peut donner des indications sur le comportement et le sentiment des interviewés.
Documentation du domaine	L'IA peut aider le business analyst à appréhender le domaine d'application en compilant des informations existantes, comme des sites d'entreprise ou des rapports d'activités. Cette aide peut faire gagner un temps précieux aux parties prenantes qui ne doivent alors plus présenter le domaine aux business analysts.

Gestion de contenus	
Traduction automatique	Les IA facilitent la communication dans des projets multilingues.
Création d'exigences	Une IA peut analyser des contenus (interviews et autres sources) pour proposer une première liste d'exigences, éventuellement complétée par des cas de test pour les valider ultérieurement.
Rédaction de documents	Les IA peuvent améliorer la forme d'un livrable (grammaire, orthographe, etc.), voire valider la formulation d'exigences par rapport à des standards de qualité établis, comme le modèle INVEST pour les user stories.
Communication et reporting	Une IA peut produire des rapports sous différentes formes, comme des tableaux de bord, des minutes de réunions ou des présentations, et avec différents niveaux de détails. Une IA peut être utilisée pour personnaliser un contenu en fonction des attentes d'un public cible, et par exemple construire un executive summary pour une direction et un rapport détaillé pour l'équipe du projet.
Gestion des risques	
Identification des risques	Des risques peuvent être déduits par une IA à partir d'un historique des risques concernant des projets similaires. La faisabilité et l'impact sur le projet d'une exigence donnée pourrait être aussi prédite selon le même mécanisme.

Tableau 21 : exemples d'utilisation de l'IA pour l'ingénierie des exigences.

A titre d'exemple, voyons comme MS- Copilot (interrogée le 1/7/24) peut déduire une première liste de US à partir du rapport d'interview suivant :

BA. Bonjour. Comment voyez-vous l'application idéale pour vous assister dans le recrutement ?

Responsable. Hé bien beaucoup de tâches liées à la publication d'offres d'emploi sur notre site Web, à la collecte des candidatures et à leur évaluation restent manuelles. Par exemple, lorsqu'un manager de département souhaite recruter un nouveau collaborateur, il envoie le descriptif de l'offre d'emploi au Web master, qui doit ensuite la mettre en ligne. Nous souhaiterions que les managers puissent créer directement une offre dans le système et ensuite la rendre visible ou la supprimer. Certains managers voudraient aussi pouvoir dupliquer une offre existante et l'adapter avant de la publier, mais ce n'est pas prioritaire.

BA. Je vois. Souhaitez-vous pouvoir valider personnellement une offre avant sa mise en ligne ?

Responsable : Disons que pour le moment, les managers me contactent souvent lorsqu'ils rédigent le texte de l'offre. Je leur donne alors des conseils et, en quelque sorte, je valide leur idée. Mais un processus plus formel serait peut-être utile.

BA. Que se passe-t-il lorsqu'une offre d'emploi est en ligne ?

Responsable. Il faudrait que l'application permette aux candidats de postuler, en remplissant un formulaire de candidature en ligne et en chargeant leur CV. On pourrait même imaginer que le manager reçoive une alerte lorsqu'une candidature concernant une de ses offres est enregistrée.

BA. Le formulaire est-il le même pour toutes les offres ?

Responsable. Non, le manager doit pouvoir l'adapter pour chaque offre d'emploi, éventuellement à partir de modèles préexistants.

BA. J'imagine que l'application devra faciliter l'évaluation des candidatures ?

Responsable. Tout à fait. Il faudrait que les managers puissent accéder aux candidatures relatives à une offre (i.e., les formulaires de candidatures et les CV) et enregistrer leur évaluation de chaque candidature.

Si possible, l'application devrait générer un courrier de refus aux candidats non retenus.

Mais pour un candidat intéressant, le manager devrait pouvoir l'inviter à un RV, qui devra être noté dans son calendrier. On pourrait même imaginer de demander aux candidats de préciser leurs disponibilités pour un éventuel interview quand ils déposent leur candidature.

BA. D'accord. Voyez-vous d'autres points ?

Responsable. Oui, un dernier, mais en option. On gagnerait beaucoup de temps si l'application était capable de vérifier que les candidats satisfont à certains critères de l'offre pour laquelle ils postulent, comme le niveau d'étude ou le nombre d'années d'expérience. Mais je me doute que ça risque d'être difficile à automatiser.

BA. Peut-être, mais nous verrons ce qu'il est possible de faire.

La réponse de l'IA constitue une base intéressante pour l'élicitation :

- « Épopée 1 : Gestion des Offres d'Emploi
 - Thème 1.1 : Création et Publication des Offres
 - US1 : En tant que manager, je veux pouvoir créer une offre d'emploi dans le système afin de la rendre visible sur le site Web.
 - US2 : En tant que manager, je veux pouvoir dupliquer une offre existante afin de l'adapter et la publier rapidement.
 - US3 : En tant que responsable, je veux pouvoir valider une offre avant sa mise en ligne afin d'assurer sa qualité.
- Épopée 2 : Gestion des Candidatures
 - Thème 2.1 : Soumission des Candidatures
 - US4 : En tant que candidat, je veux pouvoir postuler en ligne en remplissant un formulaire et en chargeant mon CV afin de soumettre ma candidature facilement.
 - US5 : En tant que manager, je veux recevoir une alerte lorsqu'une candidature est enregistrée afin de réagir rapidement.
 - Thème 2.2 : Évaluation des Candidatures
 - US6 : En tant que manager, je veux pouvoir accéder aux candidatures relatives à une offre afin de les évaluer.
 - US7 : En tant que manager, je veux pouvoir enregistrer mon évaluation de chaque candidature afin de suivre le processus de sélection.
 - US8 : En tant que manager, je veux que l'application génère automatiquement un courrier de refus pour les candidats non retenus afin de les informer rapidement.
 - US9 : En tant que manager, je veux pouvoir inviter un candidat intéressant à un RV et noter ce RV dans mon calendrier afin de planifier les interviews.
 - Thème 2.3 : Vérification des Critères
 - US10 : En tant que manager, je veux que l'application vérifie automatiquement que les candidats satisfont à certains critères de l'offre afin de gagner du temps dans le processus de sélection. »

9.2 Risques de l'IA

Comme toutes technologie, l'IA comporte des risques. Identifions d'abord quelques risques clés liés aux données.

Risque de fiabilité. Les modèles de l'IA sont entraînés avec des données d'apprentissage dont la qualité est déterminante pour la fiabilité des modèles et de leurs outputs. On connaît les biais induits par les contenus du Web sur les IA génératives comme, comme des biais de genre ou de race.

Dans le contexte de la business analyse, les données d'apprentissage sont liées au domaine d'application. Ces données reprennent des rapports d'interview, des descriptions de l'existant, des exigences sous différentes forme, etc. Elles doivent être les plus complètes et précises possible. Par exemple, si un groupe d'utilisateurs est sous-représenté dans les données d'apprentissage, leurs exigences pourraient être mal comprises voire oubliées.

La qualité des données externes que l'on injecterait dans les modèles doit aussi être contrôlée. Par exemple, des attaquants malveillants voient les systèmes d'IA comme des opportunités de perturber leur fonctionnement. De nouvelles techniques d'attaques émergent, comme l'injection de données contradictoires (« adversarial data ») qui sont pensées pour biaiser le résultat des IA. Par exemple, un attaquant pourrait ajouter des exemples incorrects dans les données d'apprentissage pour dégrader les performances du modèle.

Risque de sécurité. Les IA sont souvent entraînées avec des données du domaine qui sont confidentielles, comme des descriptions de processus originaux qui confèrent un avantage concurrentiel. C'est donc tout l'écosystème IA d'une entreprise (infrastructure, modèle, données, etc.) qui doit être sécurisé avec des mécanismes de contrôle strict d'accès, de cryptage des données, d'audits réguliers, etc.

Risque de confiance. D'autres risques sont liés aux modèles. Par exemple, quand un modèle d'IA est incapable d'expliquer comment il a généré ses résultats, les utilisateurs éprouvent des difficultés à faire confiance à une IA dont le raisonnement leur paraît trop opaque.

9.3 Démarche de mise en œuvre de l'IA

Malgré le développement impressionnant de l'IA, la plupart des entreprises en sont encore à la phase d'adoption qui précède une mise en œuvre effective. Il reste du chemin à parcourir pour comprendre comment l'IA peut transformer les processus existants pour les rendre plus performants.

En particulier, activer l'IA pour le business analyse exige de mettre en place :

- un support du management, une vision et une gouvernance explicite pour soutenir et cadrer le développement de l'IA ;
- une équipe de business analysts formés au fonctionnement et aux outils l'IA. Ces derniers doivent être compétents et disponibles pour gérer l'IA dans un environnement de travail réel ;
- des processus de business analyse et des outils optimisés pour l'IA ;
- un support opérationnel pour l'IA, incluant une infrastructure IT pour les outils et les données, une gestion de la sécurité, etc.

La Figure 110 montre un cheminement parmi d'autres pour démarrer un premier projet de business analyse avec l'IA.

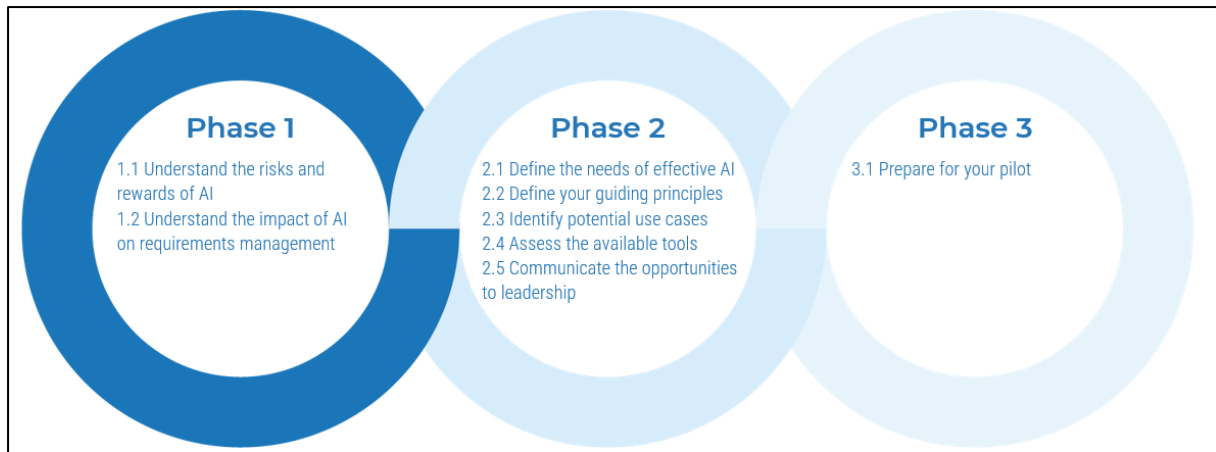


Figure 110 : phasage d'un projet de business analyse avec l'IA (Info-Tech Research Group, 2024)

10 Annexes

10.1 Exemple de Solution Design Document

Contenu type :

1. Introduction
 - Description de la mission du projet
 - Parties prenantes
2. Exigences
 - Exigences métier
 - État actuel (as is)
 - Problèmes (as is)
3. Solution
 - Description détaillée
 - Portée
 - Jalons temporels
4. Tests à réaliser

Exemple de contenu :

1. Introduction

- Parties prenantes
 - consultées pour le projet :

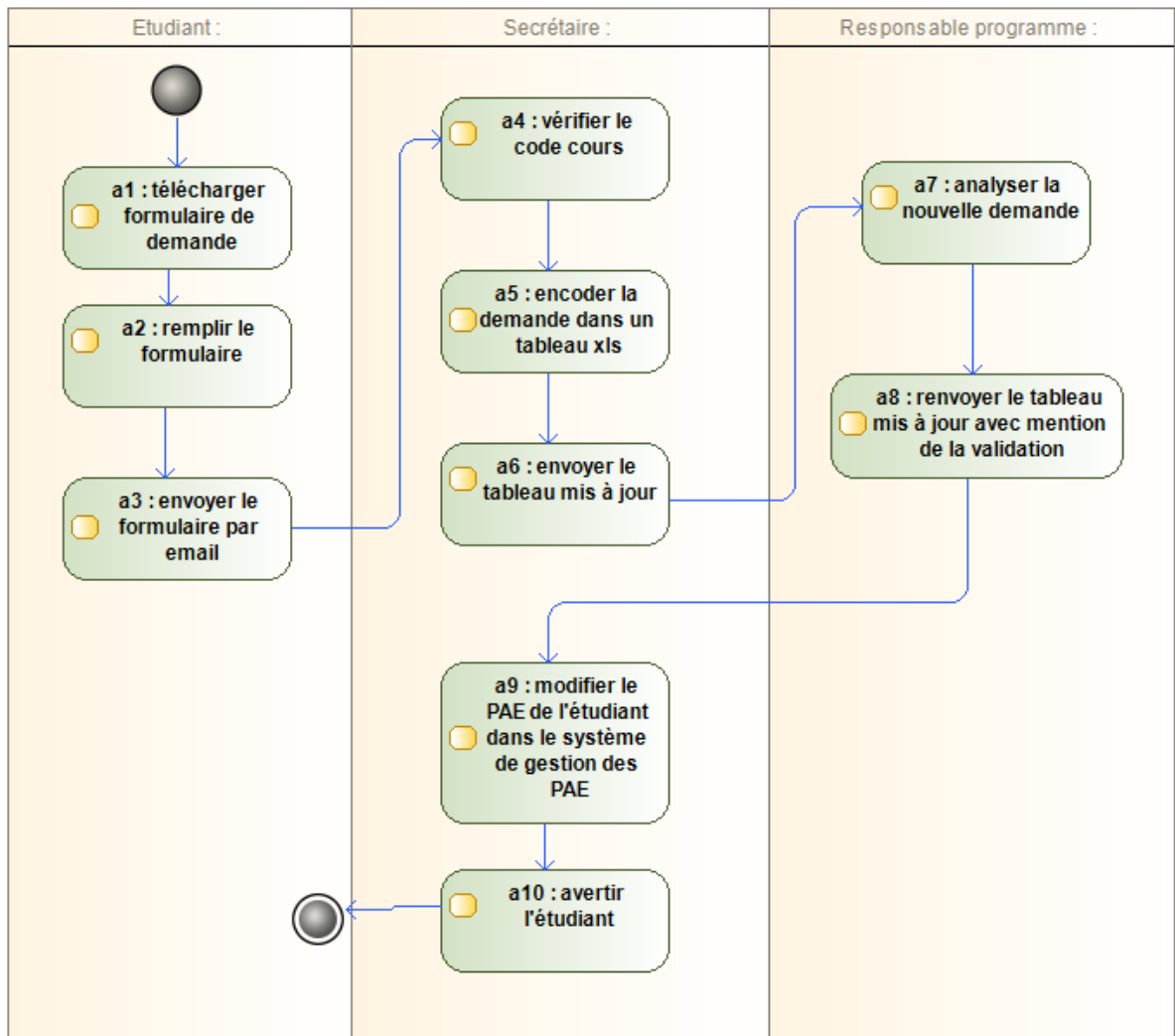
Nom	Prénom	Département	Fonction
Durant	Nathalie	Secrétariat des étudiants	Secrétaire
Mahieu	Martine	Secrétariat des étudiants	Responsable programme
Pornel	Yannick	IT	Développeur
Van den zande	Alban	IT	Business Analyste

- utilisateurs de la solution :
 - étudiants
 - responsables des programmes
 - secrétaires

- 2. Exigences métier

- en tant qu'étudiant, je veux introduire une demande de modification de mon PAE pour adapter mon programme à mon profil
- en tant que responsable de programme, je veux examiner une demande de modification de PAE pour statuer sur celle-ci
- en tant que responsable de programme, je veux accepter ou rejeter une demande de modification de PAE pour répondre à la demande de l'étudiant
- etc.

As is :



- Problèmes (as is)

- le processus en place implique des échanges de mails et de documents
 - formulaire de demande de changement de PAE
 - tableau récapitulatif des demandes réalisé en Excel
- les problèmes sont les suivants :
 - encodage multiple de l'information avec des risques d'erreur

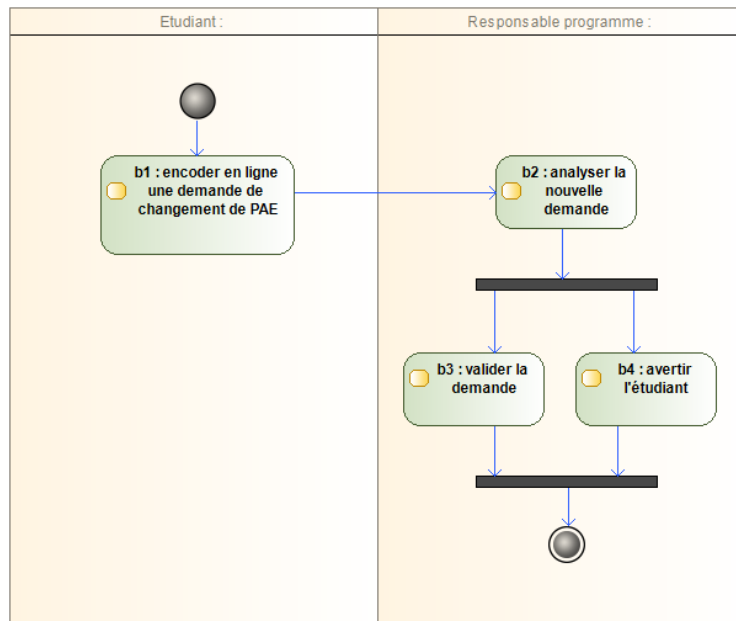
- un intermédiaire est nécessaire (secrétaire) mais n'apporte pas de plus-value à l'information
- les délais de traitement sont longs

3. Solution

- la solution préconisée consiste à revoir le processus en place et à l'automatiser
- le processus sera intégré au flux d'information au sein du système central de gestion administrative afin de centraliser toute l'information utile au même endroit et sans redondance
- cette centralisation permet d'automatiser de nombreuses tâches chronophages, comme la vérification des codes cours de la demande, et facilite l'accès et la circulation d'information
- la solution permet de supprimer un intermédiaire (secrétaire)
- la solution fonctionnera de la sorte :
 - i. encodage de la demande de changement via un formulaire électronique contenant les champs suivants moyennant identification :

Nom du champ	Type	Choix pré-enregistré	Contrainte
Matricule	Liste	Liste des étudiants	Obligatoire par identification
Code cours	Liste	Liste des UE accessibles à l'étudiant	Obligatoire
Nature du changement	Liste	Ajout, suppression	Obligatoire

- ii. analyse de la demande
- iii. validation ou refus de la demande Signature électronique via la plateforme
- iv. communication de la décision



- Portée
 - le processus s’applique aux étudiants full time et au cours de l’institution
 - hors périmètre : étudiant en échange, cours suivis en dehors de l’institution (Erasmus, etc.)
 - Jalons

Tâche	Responsable
T + 1 mois : Analyse business	IT -Alban
T + 3 mois : Développement	IT - Yannick
T + 4 mois : Tests	Secrétariat des étudiants - Nathalie

4. Tests à réaliser

Tâche	Réponse système attendue
Encodage du formulaire de demande de changement du PAE	Création d’une demande de changement de PAE sauvegardée dans un statut « ouvert »
Analyse de la nouvelle demande	Affichage des données étudiant, de son PAE complet et des UE à modifier
Validation d’une demande	Marquage la demande comme acceptée ou refusée. Modification du PAE en cas d’acceptation. Sauvegarde de la demande dans un statut « traitée »

Avertissement de l'étudiant	Génération d'un mail de décision à destination de l'étudiant
-----------------------------	--

10.2 Introduction à ArchiMate

Le business analyst doit aligner les systèmes informatiques sur les processus d'affaires et la stratégie de l'entreprise. Pour cela, il peut montrer comment les processus de l'entreprise s'articulent avec l'infrastructure IT et ainsi produire ainsi une architecture d'entreprise¹².

Le consortium Open Group (www.opengroup.org) propose deux standards conjoints pour les architectures d'entreprise :

- TOGAF (The Open Group Architecture Framework), qui décrit une approche de gestion des architectures ;
- ArchiMate, qui est un langage de modélisation graphique pour représenter des architectures.

Dans sa version native, le cadre ArchiMate s'organise en 3 couches principales et 3 aspects complémentaires (Figure 111).

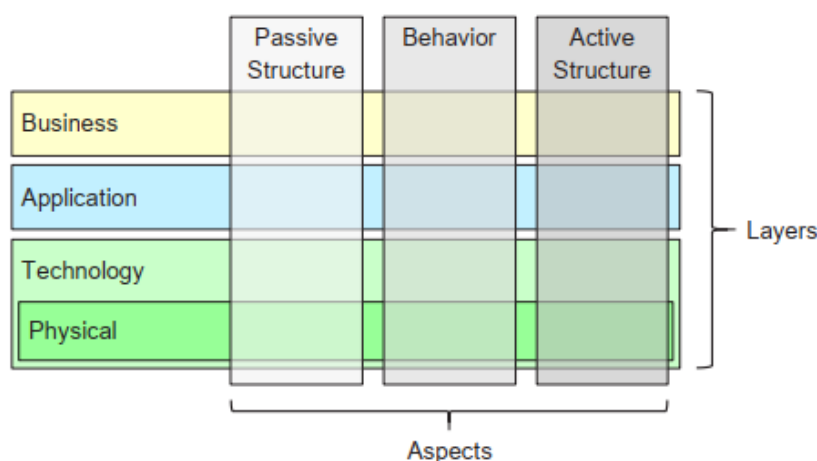


Figure 111 : framework natif d'ArchiMate.

L'organisation en couche matérialise un mécanisme d'abstraction qui facilite l'alignement entre le business et l'IT. Chacune des couches est servie la couche inférieure, à savoir que les services d'une couche sont opérationnalisés par la couche inférieure :

-
- ¹² "Architecture descriptions are formal descriptions of a system, organized in a way that supports reasoning about the structural and behavioral properties of the system and its evolution. They define the components or building blocks that make up the overall system and provide a plan from which products can be procured, and subsystems developed, that will work together to implement the overall system. It thus enables you to manage your overall IT investment in a way that meets the needs of your business." (<https://archimatetool.gitbook.io/project/what-is-architecture>)"

- la couche *Business* décrit les services offerts aux clients et qui sont réalisés par des processus d'affaires, eux-mêmes pris en charge par des acteurs de l'entreprise ;
- la couche *Application* décrit les services applicatifs qui supportent les processus d'affaires, ainsi que les applications qui les réalisent ;
- la couche *Technologie* décrit les services techniques pour exécuter les applications et l'infrastructure matérielle qui supporte ces services. La sous-couche *Physique* spécialise certains composants.

L'organisation en « aspects » classe les composants de l'architecture selon leur nature :

- *Active Structure Aspect* : reprend des éléments de structures dits actifs, i.e., des entités capables d'assumer un comportement, comme des acteurs business, des composants applicatifs, etc. ;
- *Behavior Aspect* : reprend les éléments de comportement, i.e., des unités d'activité qui peut être réalisées par un élément de structure actif, comme des processus, des fonctions, etc. ;
- *Passive Structure Aspect* : reprend les objets sur lesquels les comportements agissent.

Des versions récentes complètent le cadre initial avec une couche de *Strategy* et une couche d'*Implementation & Migration*, ainsi qu'avec un aspect *Motivation*.

La Figure 112 montre un exemple d'architecture, de composants et d'interrelations. Le lecteur se référera à la littérature pour découvrir les nombreux composants et interrelations disponibles ((The Open Group, 2019).

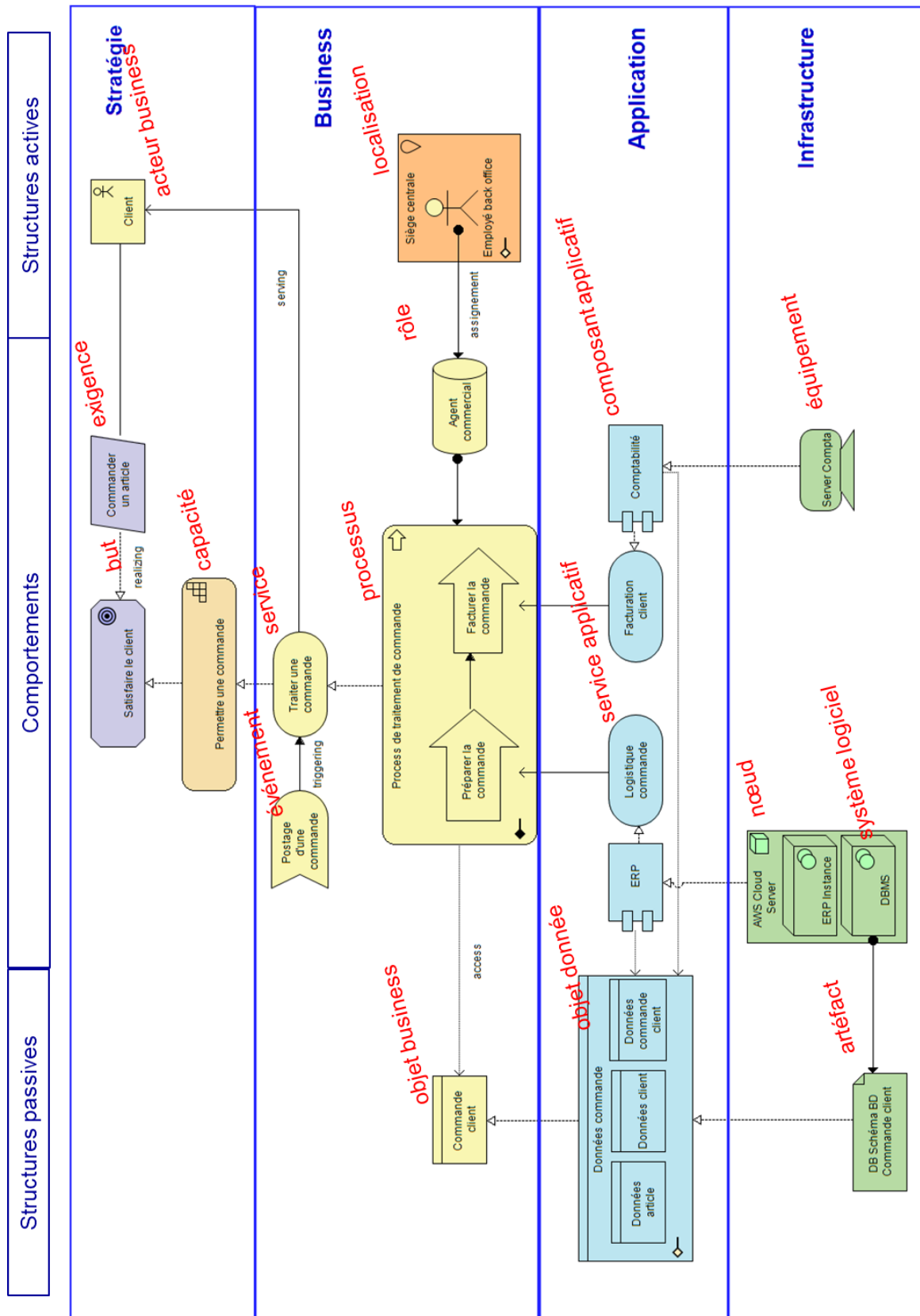
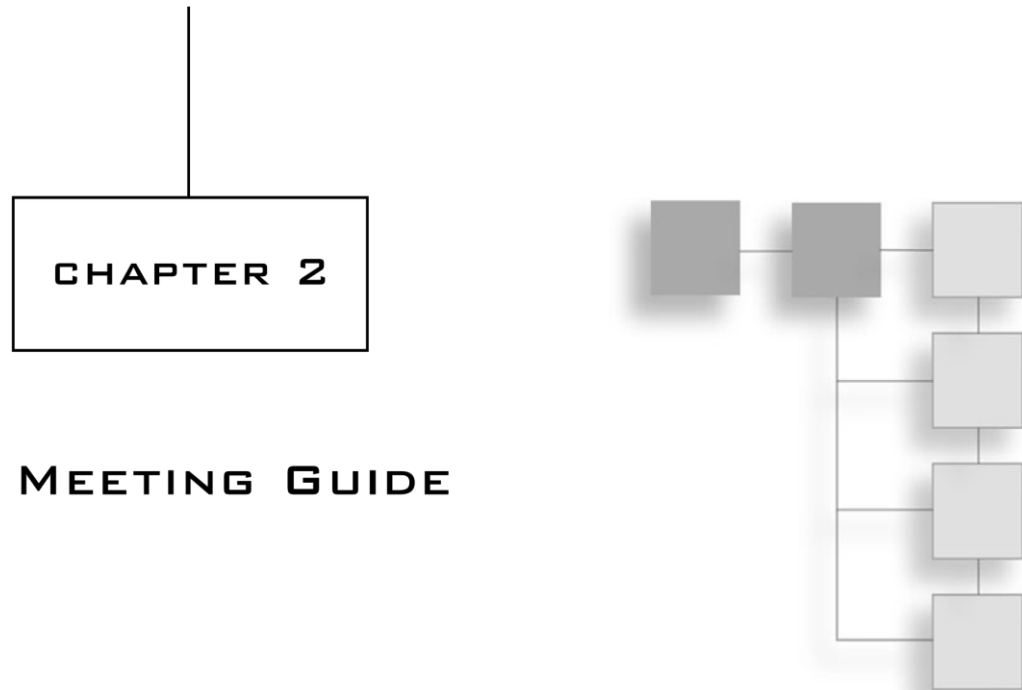


Figure 112 : exemple d'architecture d'entreprise représentée avec ArchiMate.



This chapter provides guidelines for meetings that a BA may be required to facilitate or participate in over the course of an IT project. For an overview of where each type of meeting fits into the life cycle, please see the Overview of BA Activities Throughout the Life Cycle chapter in this handbook.

Please note that not every requirements-gathering and verification event needs to be in the form of a group meeting. See the “Requirements Investigation Methods” checklist in the Tips and Checklists chapter for alternatives. The stakeholders, questions, and deliverables that appear in these guidelines apply regardless of which investigation method—or combination of methods—is chosen.

Planning for the Meeting

The following sections contain guidance for the BA regarding the planning of and preparation for meetings, including suggestions on the types of BA meetings to plan for, who to invite, and a standard meeting agenda.

Checklist: Who to Invite to Requirements Workshops

For more information about those listed in the following Roles and Participant checklists, see Tables 2.1 and 2.2 in this chapter.

Roles

- Facilitator
- Executive Sponsor

- Scribe
- Primary Participants
- Advisory Participants
- Observers

Participants

- Customers
- Users
- High-Level Management
- Line Management
- Product Champion
- Service Desk Agents
- Executive Sponsor and Approval Boards
- Business Process Owners
- Service Owner
- Service Manager
- Service Level Manager
- Product Manager
- Business Relationship Manager
- Project Manager
- Subject Matter Experts (SMEs)
- Business Architect
- Standards and Guidelines Organizations
- Internal Solution Providers (systems analysts/system architects/developers/vendors/IT service providers)
- External Service Provider organizations
- Testers
- Maintenance Programmers

Contribution to Meeting by Role and Stakeholder Type

This section describes the roles assigned for the meeting and the contributions sought from each type of stakeholder invited.

Responsibility by Role

Table 2.1 describes the responsibilities of each role in the meeting.

Table 2.1 Responsibility by Role

Role	Responsibility
Facilitator	Runs the meeting. Ensures meeting stays close to the agenda and meets its objectives. Must be seen as neutral.
Scribe	Documents results of the meeting.
Primary Participants	Have speaking and voting rights.
Advisory Participants	Have speaking rights but no voting rights.
Observers	Do not have speaking or voting rights.

Contribution by Stakeholder Type

Table 2.2 describes the contributions made by each type of stakeholder invited to the meeting.

Types of Meetings That a BA May Be Asked to Participate In¹

- **Scope initiation:** Brainstorm opportunities, risks, features.
- **Requirements definition.**
- **Problem Log (PLOG) Resolution:** Resolve a problem or issue.
- **Structured walkthrough:** Review a project artifact, such as a section of the BRD.
- **Process modeling, workflow analysis:** Analyze the workflow of a business process.
- **Design reviews:** Review external design definitions, screen prototypes, etc.
- **Business process management:** Manage, improve a business process.
- **Gate review:** Review and sign off on deliverables at specified points in the project.
- **Confirmation of agreements.**
- **Post-implementation review:** Assess results of a change.

Facilitated Meeting Work Plan

A Facilitated Meeting Work Plan is a plan of activities to be carried out by the facilitator in order to prepare for and execute a facilitated meeting. The purpose of the Facilitated Meeting Work Plan is to identify all of the activities that a facilitator will need to engage in, in order to be able to estimate effort and resources required for the meeting and consider all the steps necessary for a successful outcome. The purpose of a *standard* (but customizable) Work Plan is to institutionalize lessons learned from previous engagements and entrench best practices—such as proper preparation for the meeting—within an organization.

¹Thanks to Bob Smolkin for the initial draft of this list.

Table 2.2 Contribution by Stakeholder Type

Stakeholder Type	Contribution
Customers	Provide insight into problems with current service offerings. Know what new services or changes are desirable from the perspective of the person paying for the service.
Users	Ensure that requirements meet the needs of end users of the IT system. Ensure system supports all user tasks (goals) and workflow requirements and is easy to use. Understand day-to-day problems with current system.
High-Level Management	Ensures that business objectives (such as increased capacity, reduced costs, and improved efficiencies) are met and that management control, tracking, and reporting requirements are included.
Line Management (internal users)	Ensures that solution improves line operations for internal users; concerned with increasing efficiency, performance, throughput, turnaround, and so on of users under their control.
Product Champion	Has broad vision for the change. An agent of change, motivator.
Service Desk Agents	Have first-hand knowledge of customer complaints, interruptions of service, and other problems with current system. Have own requirements whenever a change occurs: new procedures, training, and so on.
Executive sponsor and steering group	Point of escalation for resolving conflicts that arise as service design progresses. Early and continuing involvement of sponsors and the steering group (approval boards) promotes buy-in.
Business Process Owners	Understand problems and issues with current business process. Ensure that IT changes are consistent with the end-to-end business process. Ensure that the proposed process is fit for purpose. Ensure continual improvement of the process and its metrics.
Service Owner	An executive position with hiring and firing authority. Ensures that business objectives (financial, etc.) for the service are addressed and realized.
Service Manager	Ensures tactical, operational needs for the service are met. Responsible for continual service improvement and for evaluating emerging needs of customers.
Service Level Manager	Ensures that Service Level Agreements (SLAs) are defined, agreed on, and met. Works with customers and suppliers of services.

(continued)

Stakeholder Type	Contribution
Product Manager	Provides input regarding the impact on the business if the proposed changes are made or not made. Provides insight regarding the impact of the change on existing services. Defines the overall risk profile and costs across lines of service.
Business Relationship Manager	Provides a consolidated view of costs and risks across customers and contracts. Provides insight into how proposed changes will impact other services currently supplied to customers.
Change Manager	Responsible for a final step in the approval process—ensuring that the Change Management process is followed. Primarily concerned with protecting the production environment and ensuring that the change does no harm (i.e., that change-related incidents are minimized). Reviews a checklist of items to ensure that the sponsor has agreed to the changes, that funding is available, and that resources have been allocated.
Project Manager	Ensures that requirements are within scope of project and well-managed and that the System Development Life Cycle (SDLC) is being properly followed.
SMEs	Provide deep knowledge in their area of expertise (business, technical, or other).
Business Architects	Ensure that business standards, guidelines, and so on are being followed and that the changes are consistent with the business model.
Standards and guidelines organizations	Representatives of (or experts in) standards and guidelines that constrain the project.
Internal Solution Providers (systems analysts/system architects/developers/vendors/IT service providers)	Provide a reality check to ensure that the requirements and requested warranties are realistic and that the requirements are of sufficient quality to be used for design and coding purposes.
External Service Provider organizations	Provide a reality check to ensure that the requirements and requested warranties are realistic and that the requirements are of sufficient quality to be used for design and coding and/or for the selection and customization of an off-the-shelf solution.

Supplier Manager	Ensures that all contracts with suppliers support the needs of the business and that all suppliers meet their contractual commitments. Ensures that suppliers have acceptable plans for responding to failures of their components.
------------------	---

Table 2.2 Contribution by Stakeholder Type (continued)

Stakeholder Type	Contribution
Testers	Ensure that requirements are testable. Report on test plans and results.
Maintenance Programmers	Have first-hand knowledge of bugs in current system. Ensure that the changes will be maintainable in the future (proper documentation, standards followed, and so on).

Prerequisites, Timing Considerations

Create a Facilitated Meeting Work Plan once a need for a facilitated meeting has been identified. (See the previous section, “Types of Meetings,” for objectives that might justify a facilitated meeting.) Be aware there may be guidelines, standards, and/or templates in use on the project that constrain the plan.

Standard Facilitated Meeting Work Plan

Following is a generic Facilitated Meeting Work Plan. If there is no standard or template for a plan on your project or organization, use this as a basis and customize according to your needs. If your plan uses an existing standard, use the following as a checklist to ensure all activities have been covered.

1. Prepare for the meeting.
 - 1.1 Assess meeting readiness.²
 - 1.2 Determine the purpose of the meeting.
 - 1.3 Determine the deliverables.
 - 1.4 Create agenda, meeting rules, and expectations.³
 - 1.5 Hold briefing sessions.
 - 1.6 Prepare and distribute materials: agenda, ground rules and expectations, input artifacts.
 - 1.7 Set up room.
2. Facilitate the meeting.
 - 2.1 Open meeting.
 - 2.2 Conduct meeting.
 - 2.3 Close meeting.
3. Follow up.

²See the next section, “Meeting Readiness Checklist.”

³See “Standard Meeting Agenda,” “Facilitated Meeting Rules and Guidelines,” and “Facilitated Meeting Expectations” sections in this chapter.

Meeting Readiness Checklist

- | | |
|---|--|
| ■ Is there a vision for this project? | ☐ Yes ☐ No |
| ■ Do the sponsor and users agree on scope? Is consensus possible? | ☐ Yes ☐ No |
| ■ Will all impacted stakeholders be included in the process? | ☐ Yes ☐ No |
| ■ Are they committed to the process? | ☐ Yes ☐ No |
| ■ Is the deadline feasible? | ☐ Yes ☐ No |
| ■ Will the participants work well with each other and with the facilitator? | ☐ Yes ☐ No |
| ■ Will the participants include people who have long-term experience with the business area and with the IT system? | ☐ Yes ☐ No |
| ■ Are they open to new ideas? | ☐ Yes ☐ No |
| ■ Are they prepared to transfer their knowledge? | ☐ Yes ☐ No |
| ■ Will a suitable location for the sessions be made available? | ☐ Yes ☐ No |

Standard Meeting Agenda

1. Open the session.
 - 1.1 Review administrative items: Schedule, rest rooms, phone calls, ground rules, introductions, etc.
 - 1.2 Review session objectives.
 - 1.3 Review agenda.
 - 1.4 Overview: Explain how we got here, overview of the business area, services, processes impacted by the project.
 - 1.5 Review action items from previous meetings that were due for this session.
2. Discuss assumptions.

Review and verify assumptions.
3. Discuss requirements.

Elicit and/or verify requirements. Use meeting deliverables to guide and structure the interview or review. For guidelines specific to each meeting type, see the following sections in this chapter: “Review Meeting (Structured Walkthrough and Gate Review),” “Meeting Objective: (Kick-Off Meeting) Identify Opportunities and Challenges,” and so on.
4. Additional items.

Open the floor to additional items not on the agenda.

5. Close the session.
 - 5.1 Review meeting minutes and deliverables.
 - 5.2 Review open and outstanding issues. Assign course of action, date, and responsible person for each issue.
 - 5.3 Approval and sign-off.
 - 5.4 Meeting follow-up procedures. Describe how stakeholders will be kept informed after the meeting and how they can contribute after the meeting.
 - 5.6 Next step(s).

Facilitated Meeting Rules and Guidelines

- One speaker at a time.
- One topic at a time.
- Stick to the agenda.
- No side conversations.
- Primary participants or substitutes must be present.
- Open to all ideas (as long as they are on topic).
- All persons are treated equally, regardless of job title.
- No complaint raised without a constructive suggestion for addressing it.
- Participants understand that the users who do the job have the best understanding of that job; the developers have the best understanding of the technology.

Facilitated Meeting Expectations

Participants agree to the following:

- Support the facilitator's role in the project.
- Be honest and open regarding the successes, problems, and risks associated with the current system and the project.
- Do preparatory work, as required, before the meeting, including reading the meeting agenda and input documents distributed prior to the event.
- If unable to attend, to provide a competent substitute.
- Be respectful of others and treat them as peers.
- Obey the rules of the meeting.
- Provide feedback to the facilitator in a timely manner.

10.4 Exemples de questions relatives à l'éllicitation des exigences non fonctionnelles (Podeswa)

Table 2.15 Define Non-Functional SLRs

Questions	Where to Document the Answer
☐ What types of reports and records are required by auditors?	☐ Auditing and reporting requirements (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ What types of activity logs (histories) are required?	☐ Activity logging requirements (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ Will the business or system be required to manage licenses? If so, what are the requirements for installing, tracking, and monitoring of licenses?	☐ Licensing requirements (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ Describe any security requirements related to access to data, privacy restrictions, homeland security, and so on.	☐ Security requirements (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ Describe any precedence and concurrency rules regarding the performing of services and processes, the movement of work items, approvals, and so on. Do some processes have to occur in a specific sequence with respect to each other? Can some processes occur at the same time?	☐ Dependencies and Rules of precedence (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ What is the maximum number of users that must be able to be engaged in the same operation at the same time?	☐ Concurrency Requirements (This item is listed in the "System-Wide Capabilities" section of the SLR template in the Templates chapter.)
☐ Describe any requirements regarding the nature of the user interface. ☐ Are there requirements related to ease of use of the service? How will compliance be measured? ☐ Are there any standards and guidelines that the user interface must comply with? ☐ Describe accessibility requirements for users with special needs, such as those with disabilities.	☐ Usability requirements (This item is listed as a section of the SLR template in the Templates chapter.)
☐ What level of fault tolerance must the business or system guarantee? (See the rest of this table for more specific reliability questions.)	☐ Reliability requirements (This item is listed as a section of the SLR template in the Templates chapter.)
☐ How correct must the metrics generated by the services covered in this project be? 100% accurate? 90%?	☐ Accuracy requirements (This item is listed in the "Reliability Requirements" section of the SLR template in the Templates chapter.)
☐ How precise must amounts be? Dollar amounts to the nearest cent? 1/10 of a cent?	☐ Precision requirements (This item is listed in the "Reliability Requirements" section of the SLR template in the Templates chapter.)

(continued)

☐ What is the minimum allowable mean time between an occurrence of a service failure and a second failure of the same service?	<i>ITIL:</i> ☐ Mean Time Between Failures (MTBF) (This item is listed under the "Availability Requirements" section of the SLR template in the Templates chapter.)
☐ What is the minimum allowable mean time between an occurrence of a system or service failure and an occurrence of the next failure?	<i>ITIL:</i> ☐ Mean Time Between System/Service Incidents (MTBSI) (This item is listed under the "Availability Requirements" section of the SLR template in the Templates chapter.)
☐ What is the maximum allowable mean elapsed time to fix and restore a service, from the time an incident occurs until it is available to the customer?	<i>ITIL:</i> ☐ Mean Time Between System/Service Incidents (MTBSI) (This item is listed under the "Availability Requirements" section of the SLR template in the Templates chapter.)
☐ What is the maximum allowable mean time between the occurrence of an incident and its detection by the business/system?	☐ Detection and recording (This item is listed under the "Availability Requirements" section of the SLR template in the Templates chapter.)
☐ What is the maximum allowable mean time to repair a Configuration Item or IT service after a failure, from when the CI or IT service fails until it is repaired (not including the time required to recover or restore)?	<i>ITIL:</i> ☐ Mean Time to Repair (MTTR) (This item is listed under the "Availability Requirements" section of the SLR template in the Templates chapter.)
☐ What extra assets (redundancies) are required to support reliability and sustainability requirements? (See the rest of this table for more directed questions regarding various types of redundancy.)	☐ Redundancy (This item is listed under the "Reliability Requirements" section of the SLR template in the Templates chapter.)
☐ What extra assets are required to support continuous operation of non-interruptible services?	☐ Active redundancy (See the "Redundancy" section of the SLR template in the Templates chapter.)
☐ What duplicated assets are required to operate simultaneously and always be ready to replace their counterparts?	☐ Active redundancy (See the "Redundancy" section of the SLR template in the Templates chapter.)
☐ What duplication of assets is required to support reliability of interruptible services? ☐ What redundant assets will be kept off-line (on standby) until required?	☐ Passive redundancy (See the "Redundancy" section of the SLR template in the Templates chapter.)
☐ For which services and assets, should the risk be spread by using different types of assets to provide the same service (for example, using different service delivery mechanisms)?	☐ Heterogeneous redundancy (See the "Redundancy" section of the SLR template in the Templates chapter.)

Bibliographie

- Alexander, I., & Beus-Dukic, L. (2009). *Discovering Requirements: How to Specify Products and Services*. Wiley Publishing.
- Bock, C. (2003-2004). UML 2 Activity and Action Models (Part 1 - 4). *Journal of Object Technology*, pp. vol.2, no. 4, pg 43-53; vol.2, no. 5, pg 41-56; vol.2, no. 6, pg 7-23; vol.3, no. 1, pg 27-41. Récupéré sur www.jot.fm
- BPMN. (2015). *bpmn.org*. Récupéré sur BPMN: <http://www.bpmn.org/>
- Deffray, P., & Raimont, G. (2018). *Togaf, Archimate, UML et BPMN* (éd. 3e). Dunod.
- Dumas, M., La Rosa, M., Mendling Hajo, J., & Reijers, A. (2013). *Fundamentals of Business Process Management*. Springer.
- Gartner. (2010, 1 12). *www.gartner.com*. Consulté le 1 12, 2010, sur Gartner Group: www.gartner.com
- Hainaut, J.-L. (2022). *Bases de données* (éd. 5e). Dunod.
- Info-Tech Research Group. (2024, 06 17). *Get Started With AI in Requirements Management*. Récupéré sur [www.infotech.com: https://www.infotech.com/research/ss/get-started-with-ai-in-requirements-management](https://www.infotech.com/research/ss/get-started-with-ai-in-requirements-management)
- International Institute of Business Analysis, (IIBA). (2015). *A Guide to the Business Analysis Body of Knowledge*. ISTQB. (s.d.). *Syllabus Niveau Fondation*. 2018.
- Kulak, D., & Guiney, E. (2000). *Use Cases, Requirements in Context* (éd. 2e). ACM Press.
- Laplante, P. A. (2018). *Requirement Engineering for Software and Systems* (éd. 3e). Taylor & Francis GRoupe.
- Layton, M. C., & Morrow, D. (2018). *Scrum for Dummies* (éd. 2e). Wiley.
- Milani, F. (2019). *Digital Business Analysis*. Springer.
- OMG. (2015). *OMG*. Récupéré sur OMG: <http://www.omg.org>
- Paul, D., & Cadle, J. (2020). *Business Analysis* (éd. 4ième). BCS Learning and Development.
- Podeswa, H. (s.d.). *The Business Analyst's Handbook*. Courwise Technology, Cengage Learning.
- Project Management Institute. (2013). *A Guide to the Project Management Body of Knowledge: (Pmbok Guide)* (éd. 5e). Project Management Institute.
- Seidl, M., Scholz, M., Huemer, C., & Kappel, G. (2015). *UML @ Classroom: An Introduction to Object-Oriented Modeling*. Springer.
- The Open Group. (2019). *ArchiMate® 3.1 Specification*. Consulté le 04 14, 2024, sur <https://pubs.opengroup.org/architecture/archimate31-doc/toc.html>
- van Lamsweerde, A. (2009). *Requirements Engineering: From System*. Wiley Publishing.